Lightning Book of BOLTs

The Lightning BOLT authors

Compiled by Adam Shannon

Abstract

The Lightning Book of BOLTs is a readable compilation of the Lightning Network Specifications. The BOLTs themselves are unchanged. They are grouped here in the order a node actually uses them — connect, open a channel, route a payment, settle on-chain — with short chapter introductions so you know why you're looking at a given document.

Credit for the protocol belongs to the original BOLT authors. This book is a reading order, not a rewrite.

Contents

Introduction
How to Read This Book
Source snapshot
Overview
  BOLT #0: Introduction and Index
Connecting Nodes
  BOLT #8: Encrypted and Authenticated Transport
  BOLT #1: Base Protocol
  BOLT #9: Assigned Feature Flags
Opening and Running Channels
  BOLT #2: Peer Protocol for Channel Management
  BOLT #3: Bitcoin Transaction and Script Formats
Routing Payments
  BOLT #4: Onion Routing Protocol
  BOLT #7: P2P Node and Channel Discovery
Paying People
  BOLT #11: Invoice Protocol for Lightning Payments
  BOLT #12: Negotiation Protocol for Lightning Payments
When Channels Close
  BOLT #5: Recommendations for On-chain Transaction Handling
Finding the Network
  BOLT #10: DNS Bootstrap and Assisted Node Location
Newer Work
  Extension BOLT XX: Simple Taproot Channels
Conclusion

Introduction

Welcome to the Lightning Book of BOLTs. This is a reading companion to the Lightning Network Specifications (BOLTs) — the documents that Lightning implementations actually implement.

I didn't write the BOLTs. Credit belongs to the original authors and the many contributors who have argued, patched, and shipped this protocol over the years. My job here is to put those specs in an order you can actually read, and to add a little context at the start of each chapter so you know why you're looking at a given BOLT.

The official repository numbers the BOLTs 00, 01, 02, and so on. That's a fine catalog. It's a rough way to learn the protocol. This book groups them the way a node actually uses them: connect, open a channel, route a payment, settle on-chain.

Each chapter starts with a short introduction, then the BOLT text itself, unchanged. If something in a spec looks dense, that's because it is — these documents are written for implementers. The wrapping text is here so you don't have to guess which document to open next.

Thanks for picking this up. I hope it makes Lightning a little less intimidating, and that it sends you back to the real BOLTs (and to contributing upstream) when you're ready.

How to Read This Book

You don't have to read this cover to cover. The BOLTs are a specification, not a novel. Still, there is a path through them that matches how Lightning actually works, and that's the path this book follows.

Start with the overview and BOLT 00. That document is the table of contents, the glossary, and the design goals. If a later BOLT uses a word you don't recognize — `htlc_id`, `cltv_expiry`, `channel_ready` — BOLT 00 is where it was defined.

Then walk the stack. First you connect two nodes (transport, messages, features). Then you open a channel and learn the Bitcoin transactions that back it. Then you route a payment through other people's channels. Then you look at invoices and offers, which are how humans actually pay each other. Closures and on-chain recovery come after you understand a live channel. DNS bootstrap and Simple Taproot Channels are extras: useful, but they aren't the core loop.

Skip freely. If you only care about invoice format, jump to the payments chapter. If you're writing a gossip crawler, start at routing. Come back to the earlier chapters when a type or a message doesn't make sense.

One more thing: this book does not edit the BOLTs. Typos, ambiguities, and proposed changes belong in lightning/bolts, not here. The git commit page near the front tells you which snapshot of the specs you're reading.

Source snapshot

This book was built from the following commit of lightning/bolts. If something here disagrees with upstream, upstream wins.

```
commit 1aadb719b4007c4cea0ba6e36b08c4fb53788dee
Merge: 1528972 807aaf8
Author: Olaoluwa Osuntokun <laolu32@gmail.com>
Date:   Mon Sep 21 10:24:56 2026 -0700

    Merge pull request #1355 from SomberNight/202608_bolt04_total_msat_assumed

    BOLT 4: clarify final onion payload must always contain total_msat
```

Overview

Lightning is a network of payment channels sitting on top of Bitcoin. Two nodes lock coins into a shared output, then update a balance off-chain by exchanging signatures. When they are done, they broadcast the latest agreed state and walk away. Payments to people you don't have a channel with hop through other nodes, each hop wrapped in an onion.

The BOLTs — Basis of Lightning Technology — are the documents that make this interoperable. If your node speaks the BOLTs, it can open channels, route payments, and gossip with any other implementation. They are not a tutorial and they are not a whitepaper. They are the contract between implementations.

BOLT 00 is the front door. It explains the goals of the protocol, the keywords (`MUST`, `SHOULD`, `MAY`), and a glossary of terms you'll see everywhere else. Read it once for orientation, then keep it as a dictionary.

After that we'll stop going in numerical order. A node doesn't "do BOLT 01" in isolation. It first sets up an encrypted connection, then it talks, then it negotiates features. That's the next chapter.

In this chapter

- BOLT 00 — Introduction, glossary, and design goals

BOLT #0: Introduction and Index

Welcome, friend! These Basis of Lightning Technology (BOLT) documents describe a layer-2 protocol for off-chain bitcoin transfer by mutual cooperation, relying on on-chain transactions for enforcement if necessary.

Some requirements are subtle; we have tried to highlight motivations and reasoning behind the results you see here. I'm sure we've fallen short; if you find any part confusing or wrong, please contact us and help us improve.

This is version 0.

1. BOLT #1: Base Protocol
2. BOLT #2: Peer Protocol for Channel Management
3. BOLT #3: Bitcoin Transaction and Script Formats

4. BOLT #4: Onion Routing Protocol
5. BOLT #5: Recommendations for On-chain Transaction Handling
6. BOLT #7: P2P Node and Channel Discovery
7. BOLT #8: Encrypted and Authenticated Transport
8. BOLT #9: Assigned Feature Flags
9. BOLT #10: DNS Bootstrap and Assisted Node Location
10. BOLT #11: Invoice Protocol for Lightning Payments
11. BOLT #12: Negotiation Protocol for Lightning Payments

The Spark: A Short Introduction to Lightning

Lightning is a protocol for making fast payments with Bitcoin using a network of channels.

Channels

Lightning works by establishing *channels*: two participants create a Lightning payment channel that contains some amount of bitcoin (e.g., 0.1 bitcoin) that they've locked up on the Bitcoin network. It is spendable only with both their signatures.

Initially they each hold a bitcoin transaction that sends all the bitcoin (e.g. 0.1 bitcoin) back to one party. They can later sign a new bitcoin transaction that splits these funds differently, e.g. 0.09 bitcoin to one party, 0.01 bitcoin to the other, and invalidate the previous bitcoin transaction so it won't be spent.

See BOLT #2: Channel Establishment for more on channel establishment and BOLT #3: Funding Transaction Output for the format of the bitcoin transaction that creates the channel. See BOLT #5: Recommendations for On-chain Transaction Handling for the requirements when participants disagree or fail, and the cross-signed bitcoin transaction must be spent.

Conditional Payments

A Lightning channel only allows payment between two participants, but channels can be connected together to form a network that allows payments between all members of the network. This requires the technology of a conditional payment, which can be added to a channel, e.g. "you get 0.01 bitcoin if you reveal the secret within 6 hours". Once the recipient presents the secret, that bitcoin transaction is replaced with one lacking the conditional payment and adding the funds to that recipient's output.

See BOLT #2: Adding an HTLC for the commands a participant uses to add a conditional payment, and BOLT #3: Commitment Transaction for the complete format of the bitcoin transaction.

Forwarding

Such a conditional payment can be safely forwarded to another participant with a lower time limit, e.g. "you get 0.01 bitcoin if you reveal the secret within 5 hours". This allows channels to be chained into a network without trusting the intermediaries.

See BOLT #2: Forwarding HTLCs for details on forwarding payments, BOLT #4: Packet Structure for how payment instructions are transported.

Network Topology

To make a payment, a participant needs to know what channels it can send through. Participants tell each other about channel and node creation, and updates.

See BOLT #7: P2P Node and Channel Discovery for details on the communication protocol, and BOLT #10: DNS Bootstrap and Assisted Node Location for initial network bootstrap.

Payment Invoicing

A participant receives invoices that tell them what payments to make.

See BOLT #11: Invoice Protocol for Lightning Payments for the protocol describing the destination and purpose of a payment such that the payer can later prove successful payment.

Glossary and Terminology Guide

- ***Announcement***:
    - A gossip message sent between *peers* intended to aid the discovery of a *channel* or a *node*.
- **`chain_hash`**:
    - The uniquely identifying hash of the target blockchain (usually the genesis hash). This allows *nodes* to create and reference *channels* on several blockchains. Nodes are to ignore any messages that reference a `chain_hash` that are unknown to them. Unlike `bitcoin-cli`, the hash is not reversed but is used directly.

      For the main chain Bitcoin blockchain, the `chain_hash` value MUST be (encoded in hex): `6fe28c0ab6f1b372c1a6a246ae63f74f931e8365e15a089c68d6190000000000`.
- ***Channel***:
    - A fast, off-chain method of mutual exchange between two *peers*. To transact funds, peers exchange signatures to create an updated *commitment transaction*.
    - *See closure methods: mutual close, revoked transaction close, unilateral close*
    - *See related: route*
- ***Closing transaction***:
    - A transaction generated as part of a *mutual close*. A closing transaction is similar to a *commitment transaction*, but with no pending payments.
    - *See related: commitment transaction, funding transaction, penalty transaction*
- ***Commitment number***:
    - A 48-bit incrementing counter for each *commitment transaction*; counters are independent for each *peer* in the *channel* and start at 0.
    - *See container: commitment transaction*

    - *See related: closing transaction, funding transaction, penalty transaction*

- ***Commitment revocation private key*:**

    - Every *commitment transaction* has a unique commitment revocation private-key value that allows the other *peer* to spend all outputs immediately: revealing this key is how old commitment transactions are revoked. To support revocation, each output of the commitment transaction refers to the commitment revocation public key.
    - *See container: commitment transaction*
    - *See originator: per-commitment secret*

- ***Commitment transaction*:**

    - A transaction that spends the *funding transaction*. Each *peer* holds the other peer's signature for this transaction, so that each always has a commitment transaction that it can spend. After a new commitment transaction is negotiated, the old one is *revoked*.
    - *See parts: commitment number, commitment revocation private key, HTLC, per-commitment secret, outpoint*
    - *See related: closing transaction, funding transaction, penalty transaction*
    - *See types: revoked commitment transaction*

- ***Fail the channel*:**

    - This is a forced close of the channel. Very early on (before opening), this may not require any action but forgetting the existence of the channel. Usually it requires signing and broadcasting the latest commitment transaction, although during mutual close it can also be performed by signing and broadcasting a mutual close transaction. See BOLT #5.

- ***Close the connection*:**

    - This means closing communication with the peer (such as closing the TCP socket). It does not imply closing any channels with the peer, but does cause the discarding of uncommitted state for connections with channels: see BOLT #2.

- ***Final node*:**

    - The final recipient of a packet that is routing a payment from an *origin node* through some number of *hops*. It is also the final *receiving peer* in a chain.
    - *See category: node*
    - *See related: origin node, processing node*

- ***Funding transaction*:**

    - An irreversible on-chain transaction that pays to both *peers* on a *channel*. It can only be spent by mutual consent.
    - *See related: closing transaction, commitment transaction, penalty transaction*

- ***Hop*:**
    - A *node*. Generally, an intermediate node lying between an *origin node* and a *final node*.
    - *See category: node*
- ***HTLC*: Hashed Time Locked Contract.**
    - A conditional payment between two *peers*: the recipient can spend the payment by presenting its signature and a *payment preimage*, otherwise the payer can cancel the contract by spending it after a given time. These are implemented as outputs from the *commitment transaction*.
    - *See container: commitment transaction*
    - *See parts: Payment hash, Payment preimage*
- ***Invoice*: A request for funds on the Lightning Network, possibly**

  including payment type, payment amount, expiry, and other information. This is how payments are made on the Lightning Network, rather than using Bitcoin-style addresses.
- ***It's ok to be odd*:**
    - A rule applied to some numeric fields that indicates either optional or compulsory support for features. Even numbers indicate that both endpoints MUST support the feature in question, while odd numbers indicate that the feature MAY be disregarded by the other endpoint.
- ***MSAT*:**
    - A millisatoshi, often used as a field name.
- ***Mutual close*:**
    - A cooperative close of a *channel*, accomplished by broadcasting an unconditional spend of the *funding transaction* with an output to each *peer* (unless one output is too small, and thus is not included).
    - *See related: revoked transaction close, unilateral close*
- ***Node*:**
    - A computer or other device that is part of the Lightning network.
    - *See related: peers*
    - *See types: final node, hop, origin node, processing node, receiving node, sending node*
- ***Origin node*:**
    - The *node* that originates a packet that will route a payment through some number of hops to a *final node*. It is also the first sending peer in a chain.
    - *See category: node*
    - *See related: final node, processing node*

- ***Outpoint***:
    - A transaction hash and output index that uniquely identify an unspent transaction output. Needed to compose a new transaction, as an input.
    - *See related: funding transaction, commitment transaction*

- ***Payment hash***:
    - The *HTLC* contains the payment hash, which is the hash of the *payment preimage*.
    - *See container: HTLC*
    - *See originator: Payment preimage*

- ***Payment preimage***:
    - Proof that payment has been received, held by the final recipient, who is the only person who knows this secret. The final recipient releases the preimage in order to release funds. The payment preimage is hashed as the *payment hash* in the *HTLC*.
    - *See container: HTLC*
    - *See derivation: payment hash*

- ***Peers***:
    - Two *nodes* that are in communication with each other.
        - Two peers may gossip with each other prior to setting up a channel.
        - Two peers may establish a *channel* through which they transact.
    - *See related: node*

- ***Penalty transaction***:
    - A transaction that spends all outputs of a *revoked commitment transaction*, using the *commitment revocation private key*. A *peer* uses this if the other peer tries to "cheat" by broadcasting a *revoked commitment transaction*.
    - *See related: closing transaction, commitment transaction, funding transaction*

- ***Per-commitment secret***:
    - Every *commitment transaction* derives its keys from a per-commitment secret, which is generated such that the series of per-commitment secrets for all previous commitments can be stored compactly.
    - *See container: commitment transaction*
    - *See derivation: commitment revocation private key*

- ***Processing node***:
    - A *node* that is processing a packet that originated with an *origin node* and that is being sent toward a *final node* in order to route a payment. It acts as a *receiving peer* to receive the message, then a sending peer to send on the packet.
    - *See category: node*

    - *See related: final node, origin node*

- ***Receiving node***:

    - A *node* that is receiving a message.
    - *See category: node*
    - *See related: sending node*

- ***Receiving peer***:

    - A *node* that is receiving a message from a directly connected *peer*.
    - *See category: peer*
    - *See related: sending peer*

- ***Revoked commitment transaction***:

    - An old *commitment transaction* that has been revoked because a new commitment transaction has been negotiated.
    - *See category: commitment transaction*

- ***Revoked transaction close***:

    - An invalid close of a *channel*, accomplished by broadcasting a *revoked commitment transaction*. Since the other *peer* knows the *commitment revocation secret key*, it can create a *penalty transaction*.
    - *See related: mutual close, unilateral close*

- ***Route***:

    - A path across the Lightning Network that enables a payment from an *origin node* to a *final node* across one or more *hops*.
    - *See related: channel*

- ***Sending node***:

    - A *node* that is sending a message.
    - *See category: node*
    - *See related: receiving node*

- ***Sending peer***:

    - A *node* that is sending a message to a directly connected *peer*.
    - *See category: peer*
    - *See related: receiving peer*.

- ***Unilateral close***:

    - An uncooperative close of a *channel*, accomplished by broadcasting a *commitment transaction*. This transaction is larger (i.e. less efficient) than a *closing transaction*, and the *peer* whose commitment is broadcast cannot access its own outputs for some previously-negotiated duration.

    - *See related: mutual close, revoked transaction close*

Theme Song

```
Why this network could be democratic...
Numismatic...
Cryptographic!
Why it could be released Lightning!
(Release Lightning!)


We'll have some timelocked contracts with hashed pubkeys, oh yeah.
(Keep talking, whoa keep talkin')
We'll segregate the witness for trustless starts, oh yeah.
(I'll get the money, I've got to get the money)
With dynamic onion routes, they'll be shakin' in their boots;
You know that's just the truth, we'll be scaling through the roof.
Release Lightning!
(Go, go, go, go; go, go, go, go, go, go)


[Chorus:]
Oh released Lightning, it's better than a debit card..
(Release Lightning, go release Lightning!)
With released Lightning, micropayments just ain't hard...
(Release Lightning, go release Lightning!)
Then kaboom: we'll hit the moon -- release Lightning!
(Go, go, go, go; go, go, go, go, go, go)


We'll have QR codes, and smartphone apps, oh yeah.
(Ooo ooo ooo ooo ooo ooo ooo)
P2P messaging, and passive incomes, oh yeah.
(Ooo ooo ooo ooo ooo ooo ooo)
Outsourced closure watch, gives me feelings in my crotch.
You'll know it's not a brag when the repo gets a tag:
Released Lightning.


[Chorus]
[Instrumental, ~1m10s]
[Chorus]
(Lightning! Lightning! Lightning! Lightning!
 Lightning! Lightning! Lightning! Lightning!)


C'mon guys, let's get to work!
```

– Anthony Towns aj@erisian.com.au

Authors

[FIXME: Insert Author List]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

Connecting Nodes

Before two Lightning nodes can open a channel, they have to talk. That means a TCP connection, an encrypted handshake, and a shared idea of which messages exist.

BOLT 08 is the transport. Lightning uses a Noise protocol handshake (`Noise_XK`) so that by the time the first application message is sent, both sides have authenticated keys and an encrypted session. If you've ever wondered why Lightning node IDs are public keys, this is why: the transport identity *is* the node identity.

BOLT 01 is the messaging layer that rides on that session. It defines the binary framing, the type system, and the messages every peer needs — `init`, `error`, `ping` / `pong`. Everything later in the book is a BOLT 01 message with a more interesting type.

BOLT 09 is the feature bits. Lightning is still changing, so peers advertise what they support during `init` and inside invoices, node announcements, and other gossip. If a message in a later BOLT is optional, BOLT 09 is where that option got a number.

Read these three in order. Transport, then messages, then the flags that say which messages you're allowed to send.

In this chapter

- BOLT 08 — Encrypted and authenticated transport
- BOLT 01 — Base messaging
- BOLT 09 — Feature flags

BOLT #8: Encrypted and Authenticated Transport

All communications between Lightning nodes is encrypted in order to provide confidentiality for all transcripts between nodes and is authenticated in order to avoid malicious interference. Each node has a known long-term identifier that is a public key on Bitcoin's `secp256k1` curve. This long-term public key is used within the protocol to establish an encrypted and authenticated connection with peers, and also to authenticate any information advertised on behalf of a node.

Table of Contents

- Cryptographic Messaging Overview
  - Authenticated Key Agreement Handshake
  - Handshake Versioning
  - Noise Protocol Instantiation
- Authenticated Key Exchange Handshake Specification
  - Handshake State
  - Handshake State Initialization
  - Handshake Exchange

- Lightning Message Specification
    - Encrypting and Sending Messages
    - Receiving and Decrypting Messages
- Lightning Message Key Rotation
- Security Considerations
- Appendix A: Transport Test Vectors
    - Initiator Tests
    - Responder Tests
    - Message Encryption Tests
- Acknowledgments
- References
- Authors

Cryptographic Messaging Overview

Prior to sending any Lightning messages, nodes MUST first initiate the cryptographic session state that is used to encrypt and authenticate all messages sent between nodes. The initialization of this cryptographic session state is completely distinct from any inner protocol message header or conventions.

The transcript between two nodes is separated into two distinct segments:

1. Before any actual data transfer, both nodes participate in an authenticated key agreement handshake, which is based on the Noise Protocol Framework[2].
2. If the initial handshake is successful, then nodes enter the Lightning message exchange phase. In the Lightning message exchange phase, all messages are Authenticated Encryption with Associated Data (AEAD) ciphertexts.

Authenticated Key Agreement Handshake

The handshake chosen for the authenticated key exchange is `Noise_XK`. As a pre-message, the initiator must know the identity public key of the responder. This provides a degree of identity hiding for the responder, as its static public key is *never* transmitted during the handshake. Instead, authentication is achieved implicitly via a series of Elliptic-Curve Diffie-Hellman (ECDH) operations followed by a MAC check.

The authenticated key agreement (`Noise_XK`) is performed in three distinct steps (acts). During each act of the handshake the following occurs: some (possibly encrypted) keying material is sent to the other party; an ECDH is performed, based on exactly which act is being executed, with the result mixed into the current set of encryption keys (`ck` the chaining key and `k` the encryption key); and an AEAD payload with a zero-length cipher text is sent. As this payload has no length, only a MAC is sent across. The mixing of ECDH outputs into a hash digest forms an incremental TripleDH handshake.

Using the language of the Noise Protocol, `e` and `s` (both public keys with `e` being the ephemeral key and `s` being the static key which in our case is usually the `nodeid`) indicate possibly encrypted keying material, and `es`, `ee`, and `se` each indicate an ECDH operation between two keys. The handshake is laid out as follows:

```
Noise_XK(s, rs):
   <- s
   ...
   -> e, es
```

```
<- e, ee
-> s, se
```

All of the handshake data sent across the wire, including the keying material, is incrementally hashed into a session-wide "handshake digest", `h`. Note that the handshake state `h` is never transmitted during the handshake; instead, digest is used as the Associated Data within the zero-length AEAD messages.

Authenticating each message sent ensures that a man-in-the-middle (MITM) hasn't modified or replaced any of the data sent as part of a handshake, as the MAC check would fail on the other side if so.

A successful check of the MAC by the receiver indicates implicitly that all authentication has been successful up to that point. If a MAC check ever fails during the handshake process, then the connection is to be immediately terminated.

Handshake Versioning

Each message sent during the initial handshake starts with a single leading byte, which indicates the version used for the current handshake. A version of 0 indicates that no change is necessary, while a non-zero version indicate that the client has deviated from the protocol originally specified within this document.

Clients MUST reject handshake attempts initiated with an unknown version.

Noise Protocol Instantiation

Concrete instantiations of the Noise Protocol require the definition of three abstract cryptographic objects: the hash function, the elliptic curve, and the AEAD cipher scheme. For Lightning, `SHA-256` is chosen as the hash function, `secp256k1` as the elliptic curve, and `ChaChaPoly-1305` as the AEAD construction.

The composition of `ChaCha20` and `Poly1305` that are used MUST conform to `RFC 8439`[1].

The official protocol name for the Lightning variant of Noise is `Noise_XK_secp256k1_ChaChaPoly_SHA256`. The ASCII string representation of this value is hashed into a digest used to initialize the starting handshake state. If the protocol names of two endpoints differ, then the handshake process fails immediately.

Authenticated Key Exchange Handshake Specification

The handshake proceeds in three acts, taking 1.5 round trips. Each handshake is a *fixed* sized payload without any header or additional meta-data attached. The exact size of each act is as follows:

- **Act One**: 50 bytes
- **Act Two**: 50 bytes
- **Act Three**: 66 bytes

Handshake State

Throughout the handshake process, each side maintains these variables:

- `ck`: the **chaining key**. This value is the accumulated hash of all previous ECDH outputs. At the end of the handshake, `ck` is used to derive the encryption keys for Lightning messages.

- `h`: the **handshake hash**. This value is the accumulated hash of *all* handshake data that has been sent and received so far during the handshake process.
- `temp_k1`, `temp_k2`, `temp_k3`: the **intermediate keys**. These are used to encrypt and decrypt the zero-length AEAD payloads at the end of each handshake message.
- `e`: a party's **ephemeral keypair**. For each session, a node MUST generate a new ephemeral key with strong cryptographic randomness.
- `s`: a party's **static keypair** (`ls` for local, `rs` for remote)

The following functions will also be referenced:

- `ECDH(k, rk)`: performs an Elliptic-Curve Diffie-Hellman operation using `k`, which is a valid `secp256k1` private key, and `rk`, which is a valid public key
    - The returned value is the SHA256 of the compressed format of the generated point.
- `HKDF(salt,ikm)`: a function defined in `RFC 5869`[3], evaluated with a zero-length `info` field
    - All invocations of `HKDF` implicitly return 64 bytes of cryptographic randomness using the extract-and-expand component of the `HKDF`.
- `encryptWithAD(k, n, ad, plaintext)`: outputs `encrypt(k, n, ad, plaintext)`
    - Where `encrypt` is an evaluation of `ChaCha20-Poly1305` (IETF variant) with the passed arguments, with nonce `n` encoded as 32 zero bits, followed by a *little-endian* 64-bit value. Note: this follows the Noise Protocol convention, rather than our normal endian.
- `decryptWithAD(k, n, ad, ciphertext)`: outputs `decrypt(k, n, ad, ciphertext)`
    - Where `decrypt` is an evaluation of `ChaCha20-Poly1305` (IETF variant) with the passed arguments, with nonce `n` encoded as 32 zero bits, followed by a *little-endian* 64-bit value.
- `generateKey()`: generates and returns a fresh `secp256k1` keypair
    - Where the object returned by `generateKey` has two attributes:
        - `.pub`, which returns an abstract object representing the public key
        - `.priv`, which represents the private key used to generate the public key
    - Where the object also has a single method:
        - `.serializeCompressed()`
- `a || b` denotes the concatenation of two byte strings `a` and `b`

Handshake State Initialization

Before the start of Act One, both sides initialize their per-sessions state as follows:

1. `h = SHA-256(protocolName)`
    - where `protocolName = "Noise_XK_secp256k1_ChaChaPoly_SHA256"` encoded as an ASCII string
2. `ck = h`
3. `h = SHA-256(h || prologue)`
    - where `prologue` is the ASCII string: `lightning`

As a concluding step, both sides mix the responder's public key into the handshake digest:

- The initiating node mixes in the responding node's static public key serialized in Bitcoin's compressed format:
    - `h = SHA-256(h || rs.pub.serializeCompressed())`
- The responding node mixes in their local static public key serialized in Bitcoin's compressed format:
    - `h = SHA-256(h || ls.pub.serializeCompressed())`

Handshake Exchange

Act One

```
-> e, es
```

Act One is sent from initiator to responder. During Act One, the initiator attempts to satisfy an implicit challenge by the responder. To complete this challenge, the initiator must know the static public key of the responder.

The handshake message is *exactly* 50 bytes: 1 byte for the handshake version, 33 bytes for the compressed ephemeral public key of the initiator, and 16 bytes for the `poly1305` tag.

Sender Actions:

1. `e = generateKey()`
2. `h = SHA-256(h || e.pub.serializeCompressed())`
    - The newly generated ephemeral key is accumulated into the running handshake digest.
3. `es = ECDH(e.priv, rs)`
    - The initiator performs an ECDH between its newly generated ephemeral key and the remote node's static public key.
4. `ck, temp_k1 = HKDF(ck, es)`
    - A new temporary encryption key is generated, which is used to generate the authenticating MAC.
5. `c = encryptWithAD(temp_k1, 0, h, zero)`
    - where `zero` is a zero-length plaintext
6. `h = SHA-256(h || c)`
    - Finally, the generated ciphertext is accumulated into the authenticating handshake digest.
7. Send `m = 0 || e.pub.serializeCompressed() || c` to the responder over the network buffer.

Receiver Actions:

1. Read *exactly* 50 bytes from the network buffer.
2. Parse the read message (`m`) into `v`, `re`, and `c`:
    - where `v` is the *first* byte of `m`, `re` is the next 33 bytes of `m`, and `c` is the last 16 bytes of `m`
    - The raw bytes of the remote party's ephemeral public key (`re`) are to be deserialized into a point on the curve using affine coordinates as encoded by the key's serialized composed format.
3. If `v` is an unrecognized handshake version, then the responder MUST abort the connection attempt.
4. `h = SHA-256(h || re.serializeCompressed())`
    - The responder accumulates the initiator's ephemeral key into the authenticating handshake digest.

5. `es = ECDH(s.priv, re)`
    - The responder performs an ECDH between its static private key and the initiator's ephemeral public key.
6. `ck, temp_k1 = HKDF(ck, es)`
    - A new temporary encryption key is generated, which will shortly be used to check the authenticating MAC.
7. `p = decryptWithAD(temp_k1, 0, h, c)`
    - If the MAC check in this operation fails, then the initiator does *not* know the responder's static public key. If this is the case, then the responder MUST terminate the connection without any further messages.
8. `h = SHA-256(h || c)`
    - The received ciphertext is mixed into the handshake digest. This step serves to ensure the payload wasn't modified by a MITM.

Act Two

```
<- e, ee
```

Act Two is sent from the responder to the initiator. Act Two will *only* take place if Act One was successful. Act One was successful if the responder was able to properly decrypt and check the MAC of the tag sent at the end of Act One.

The handshake is *exactly* 50 bytes: 1 byte for the handshake version, 33 bytes for the compressed ephemeral public key of the responder, and 16 bytes for the `poly1305` tag.

Sender Actions:

1. `e = generateKey()`
2. `h = SHA-256(h || e.pub.serializeCompressed())`
    - The newly generated ephemeral key is accumulated into the running handshake digest.
3. `ee = ECDH(e.priv, re)`
    - where `re` is the ephemeral key of the initiator, which was received during Act One
4. `ck, temp_k2 = HKDF(ck, ee)`
    - A new temporary encryption key is generated, which is used to generate the authenticating MAC.
5. `c = encryptWithAD(temp_k2, 0, h, zero)`
    - where `zero` is a zero-length plaintext
6. `h = SHA-256(h || c)`
    - Finally, the generated ciphertext is accumulated into the authenticating handshake digest.
7. Send `m = 0 || e.pub.serializeCompressed() || c` to the initiator over the network buffer.

Receiver Actions:

1. Read *exactly* 50 bytes from the network buffer.
2. Parse the read message (`m`) into `v`, `re`, and `c`:
    - where `v` is the *first* byte of `m`, `re` is the next 33 bytes of `m`, and `c` is the last 16 bytes of `m`.
3. If `v` is an unrecognized handshake version, then the responder MUST abort the connection attempt.
4. `h = SHA-256(h || re.serializeCompressed())`

5. `ee = ECDH(e.priv, re)`
    - where `re` is the responder's ephemeral public key
    - The raw bytes of the remote party's ephemeral public key (`re`) are to be deserialized into a point on the curve using affine coordinates as encoded by the key's serialized composed format.
6. `ck, temp_k2 = HKDF(ck, ee)`
    - A new temporary encryption key is generated, which is used to generate the authenticating MAC.
7. `p = decryptWithAD(temp_k2, 0, h, c)`
    - If the MAC check in this operation fails, then the initiator MUST terminate the connection without any further messages.
8. `h = SHA-256(h || c)`
    - The received ciphertext is mixed into the handshake digest. This step serves to ensure the payload wasn't modified by a MITM.

Act Three

```
-> s, se
```

Act Three is the final phase in the authenticated key agreement described in this section. This act is sent from the initiator to the responder as a concluding step. Act Three is executed *if and only if* Act Two was successful. During Act Three, the initiator transports its static public key to the responder encrypted with *strong* forward secrecy, using the accumulated `HKDF` derived secret key at this point of the handshake.

The handshake is *exactly* 66 bytes: 1 byte for the handshake version, 33 bytes for the static public key encrypted with the `ChaCha20` stream cipher, 16 bytes for the encrypted public key's tag generated via the AEAD construction, and 16 bytes for a final authenticating tag.

Sender Actions:

1. `c = encryptWithAD(temp_k2, 1, h, s.pub.serializeCompressed())`
    - where `s` is the static public key of the initiator
2. `h = SHA-256(h || c)`
3. `se = ECDH(s.priv, re)`
    - where `re` is the ephemeral public key of the responder
4. `ck, temp_k3 = HKDF(ck, se)`
    - The final intermediate shared secret is mixed into the running chaining key.
5. `t = encryptWithAD(temp_k3, 0, h, zero)`
    - where `zero` is a zero-length plaintext
6. `sk, rk = HKDF(ck, zero)`
    - where `zero` is a zero-length plaintext, `sk` is the key to be used by the initiator to encrypt messages to the responder, and `rk` is the key to be used by the initiator to decrypt messages sent by the responder
    - The final encryption keys, to be used for sending and receiving messages for the duration of the session, are generated.
7. `rn = 0, sn = 0`
    - The sending and receiving nonces are initialized to 0.
8. `rck = sck = ck`
    - The sending and receiving chaining keys are initialized the same.

9. Send `m = 0 || c || t` over the network buffer.

Receiver Actions:

1. Read *exactly* 66 bytes from the network buffer.
2. Parse the read message (`m`) into `v`, `c`, and `t`:
    - where `v` is the *first* byte of `m`, `c` is the next 49 bytes of `m`, and `t` is the last 16 bytes of `m`
3. If `v` is an unrecognized handshake version, then the responder MUST abort the connection attempt.
4. `rs = decryptWithAD(temp_k2, 1, h, c)`
    - At this point, the responder has recovered the static public key of the initiator.
    - If the MAC check in this operation fails, then the responder MUST terminate the connection without any further messages.
5. `h = SHA-256(h || c)`
6. `se = ECDH(e.priv, rs)`
    - where `e` is the responder's original ephemeral key
7. `ck, temp_k3 = HKDF(ck, se)`
8. `p = decryptWithAD(temp_k3, 0, h, t)`
    - If the MAC check in this operation fails, then the responder MUST terminate the connection without any further messages.
9. `rk, sk = HKDF(ck, zero)`
    - where `zero` is a zero-length plaintext, `rk` is the key to be used by the responder to decrypt the messages sent by the initiator, and `sk` is the key to be used by the responder to encrypt messages to the initiator
    - The final encryption keys, to be used for sending and receiving messages for the duration of the session, are generated.
10. `rn = 0, sn = 0`
    - The sending and receiving nonces are initialized to 0.
11. `rck = sck = ck`
    - The sending and receiving chaining keys are initialized the same.

Lightning Message Specification

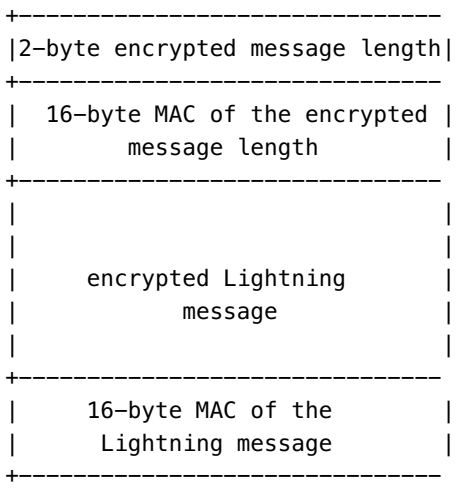
At the conclusion of Act Three, both sides have derived the encryption keys, which will be used to encrypt and decrypt messages for the remainder of the session.

The actual Lightning protocol messages are encapsulated within AEAD ciphertexts. Each message is prefixed with another AEAD ciphertext, which encodes the total length of the following Lightning message (not including its MAC).

The *maximum* size of *any* Lightning message MUST NOT exceed `65535` bytes. A maximum size of `65535` simplifies testing, makes memory management easier, and helps mitigate memory-exhaustion attacks.

In order to make traffic analysis more difficult, the length prefix for all encrypted Lightning messages is also encrypted. Additionally a 16-byte `Poly-1305` tag is added to the encrypted length prefix in order to ensure that the packet length hasn't been modified when in-flight and also to avoid creating a decryption oracle.

The structure of packets on the wire resembles the following:



The prefixed message length is encoded as a 2-byte big-endian integer, for a total maximum packet length of `2 + 16 + 65535 + 16` = `65569` bytes.

Encrypting and Sending Messages

In order to encrypt and send a Lightning message (`m`) to the network stream, given a sending key (`sk`) and a nonce (`sn`), the following steps are completed:

1. Let `l = len(m)`.
    - where `len` obtains the length in bytes of the Lightning message
2. Serialize `l` into 2 bytes encoded as a big-endian integer.
3. Encrypt `l` (using `ChaChaPoly-1305`, `sn`, and `sk`), to obtain `lc` (18 bytes)
    - The nonce `sn` is encoded as a 96-bit little-endian number. As the decoded nonce is 64 bits, the 96-bit nonce is encoded as: 32 bits of leading 0s followed by a 64-bit value.
        - The nonce `sn` MUST be incremented after this step.
    - A zero-length byte slice is to be passed as the AD (associated data).
4. Finally, encrypt the message itself (`m`) using the same procedure used to encrypt the length prefix. Let encrypted ciphertext be known as `c`.
    - The nonce `sn` MUST be incremented after this step.
5. Send `lc || c` over the network buffer.

Receiving and Decrypting Messages

In order to decrypt the *next* message in the network stream, the following steps are completed:

1. Read *exactly* 18 bytes from the network buffer.
2. Let the encrypted length prefix be known as `lc`.
3. Decrypt `lc` (using `ChaCha20-Poly1305`, `rn`, and `rk`), to obtain the size of the encrypted packet `l`.
    - A zero-length byte slice is to be passed as the AD (associated data).
    - The nonce `rn` MUST be incremented after this step.
4. Read *exactly* `l+16` bytes from the network buffer, and let the bytes be known as `c`.
5. Decrypt `c` (using `ChaCha20-Poly1305`, `rn`, and `rk`), to obtain decrypted plaintext packet `p`.
    - The nonce `rn` MUST be incremented after this step.

Lightning Message Key Rotation

Changing keys regularly and forgetting previous keys is useful to prevent the decryption of old messages, in the case of later key leakage (i.e. backwards secrecy).

Key rotation is performed for *each* key (`sk` and `rk`) *individually*, using `sck` and `rck` respectively. A key is to be rotated after a party encrypts or decrypts 1000 times with it (i.e. every 500 messages). This can be properly accounted for by rotating the key once the nonce dedicated to it reaches 1000.

Key rotation for a key `k` is performed according to the following steps:

1. Let `ck` be the chaining key (i.e. `rck` for `rk` or `sck` for `sk`)
2. `ck', k' = HKDF(ck, k)`
3. Reset the nonce for the key to `n = 0`.
4. `k = k'`
5. `ck = ck'`

Security Considerations

It is strongly recommended that existing, commonly-used, validated libraries be used for encryption and decryption, to avoid the many possible implementation pitfalls.

Appendix A: Transport Test Vectors

To make a repeatable test handshake, the following specifies what `generateKey()` will return (i.e. the value for `e.priv`) for each side. Note that this is a violation of the spec, which requires randomness.

Initiator Tests

The initiator SHOULD produce the given output when fed this input. The comments reflect internal states, for debugging purposes.

```
    name: transport-initiator successful handshake
    rs.pub: 0x028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    ls.priv: 0x1111111111111111111111111111111111111111111111111111111111111111
    ls.pub: 0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    e.priv: 0x1212121212121212121212121212121212121212121212121212121212121212
    e.pub: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f7
    # Act One
    # h=0x9e0e7de8bb75554f21db034633de04be41a2b8a18da7a319a03c803bf02b396c
    # ss=0x1e2fb3c8fe8fb9f262f649f64d26ecf0f2c0a805a767cf02dc2d77a6ef1fdcc3
    #
HKDF(0x2640f52eebcd9e882958951c794250eedb28002c05d7dc2ea0f195406042caf1,0x1e2fb3c8fe8fb9f262f649
f64d26ecf0f2c0a805a767cf02dc2d77a6ef1fdcc3)
    #
ck,temp_k1=0xb61ec1191326fa240decc9564369dbb3ae2b34341d1e11ad64ed89f89180582f,0xe68f69b7f096d791
7245f5e5cf8ae1595febe4d4644333c99f9c4a1282031c9f
    # encryptWithAD(0xe68f69b7f096d7917245f5e5cf8ae1595febe4d4644333c99f9c4a1282031c9f,
0x000000000000000000000000, 0x9e0e7de8bb75554f21db034633de04be41a2b8a18da7a319a03c803bf02b396c,
<empty>)
    # c=0df6086551151f58b8afe6c195782c6a
    # h=0x9d1ffbb639e7e20021d9259491dc7b160aab270fb1339ef135053f6f2cebe9ce
```

```
    output:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    input:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # re=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # h=0x38122f669819f906000621a14071802f93f2ef97df100097bcac3ae76c6dc0bf
    # ss=0xc06363d6cc549bcb7913dbb9ac1c33fc1158680c89e972000ecd06b36c472e47
    #
HKDF(0xb61ec1191326fa240decc9564369dbb3ae2b34341d1e11ad64ed89f89180582f,0xc06363d6cc549bcb7913db
b9ac1c33fc1158680c89e972000ecd06b36c472e47)
    #
ck,temp_k2=0xe89d31033a1b6bf68c07d22e08ea4d7884646c4b60a9528598ccb4ee2c8f56ba,0x908b166535c01a93
5cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc
    # decryptWithAD(0x908b166535c01a935cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc,
0x000000000000000000000000, 0x38122f669819f906000621a14071802f93f2ef97df100097bcac3ae76c6dc0bf,
0x6e2470b93aac583c9ef6eafca3f730ae)
    # h=0x90578e247e98674e661013da3c5c1ca6a8c8f48c90b485c0dfa1494e23d56d72
    # Act Three
    # encryptWithAD(0x908b166535c01a935cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc,
0x000000000100000000000000, 0x90578e247e98674e661013da3c5c1ca6a8c8f48c90b485c0dfa1494e23d56d72,
0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa)
    #
c=0xb9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c3822
    # h=0x5dcb5ea9b4ccc755e0e3456af3990641276e1d5dc9afd82f974d90a47c918660
    # ss=0xb36b6d195982c5be874d6d542dc268234379e1ae4ff1709402135b7de5cf0766
    #
HKDF(0xe89d31033a1b6bf68c07d22e08ea4d7884646c4b60a9528598ccb4ee2c8f56ba,0xb36b6d195982c5be874d6d
542dc268234379e1ae4ff1709402135b7de5cf0766)
    #
ck,temp_k3=0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01,0x981a46c820fb7a24
1bc8184ba4bb1f01bcdfafb00dde80098cb8c38db9141520
    # encryptWithAD(0x981a46c820fb7a241bc8184ba4bb1f01bcdfafb00dde80098cb8c38db9141520,
0x000000000000000000000000, 0x5dcb5ea9b4ccc755e0e3456af3990641276e1d5dc9afd82f974d90a47c918660,
<empty>)
    # t=0x8dc68b1c466263b47fdf31e560e139ba
    output:
0x00b9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c38228dc68b1c466263b47fdf31e560e139ba
    # HKDF(0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01,zero)
    output:
sk,rk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9,0xbb9020b8965f4df047e07
f955f3c4b88418984aadc5cdb35096b9ea8fa5c3442

    name: transport-initiator act2 short read test
    rs.pub: 0x028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    ls.priv: 0x1111111111111111111111111111111111111111111111111111111111111111
    ls.pub: 0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    e.priv: 0x1212121212121212121212121212121212121212121212121212121212121212
    e.pub: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f7
    output:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
```

```
782c6a
    # Act Two
    input:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730
    output: ERROR (ACT2_READ_FAILED)

    name: transport-initiator act2 bad version test
    rs.pub: 0x028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    ls.priv: 0x1111111111111111111111111111111111111111111111111111111111111111
    ls.pub: 0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    e.priv: 0x1212121212121212121212121212121212121212121212121212121212121212
    e.pub: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f7
    output:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    input:
0x0102466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    output: ERROR (ACT2_BAD_VERSION 1)

    name: transport-initiator act2 bad key serialization test
    rs.pub: 0x028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    ls.priv: 0x1111111111111111111111111111111111111111111111111111111111111111
    ls.pub: 0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    e.priv: 0x1212121212121212121212121212121212121212121212121212121212121212
    e.pub: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f7
    output:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    input:
0x0004466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    output: ERROR (ACT2_BAD_PUBKEY)

    name: transport-initiator act2 bad MAC test
    rs.pub: 0x028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    ls.priv: 0x1111111111111111111111111111111111111111111111111111111111111111
    ls.pub: 0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    e.priv: 0x1212121212121212121212121212121212121212121212121212121212121212
    e.pub: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f7
    output:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    input:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730af
    output: ERROR (ACT2_BAD_TAG)
```

Responder Tests

The responder SHOULD produce the given output when fed this input.

```
    name: transport-responder successful handshake
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # re=0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f7
    # h=0x9e0e7de8bb75554f21db034633de04be41a2b8a18da7a319a03c803bf02b396c
    # ss=0x1e2fb3c8fe8fb9f262f649f64d26ecf0f2c0a805a767cf02dc2d77a6ef1fdcc3
    #
HKDF(0x2640f52eebcd9e882958951c794250eedb28002c05d7dc2ea0f195406042caf1,0x1e2fb3c8fe8fb9f262f649
f64d26ecf0f2c0a805a767cf02dc2d77a6ef1fdcc3)
    #
ck,temp_k1=0xb61ec1191326fa240decc9564369dbb3ae2b34341d1e11ad64ed89f89180582f,0xe68f69b7f096d791
7245f5e5cf8ae1595febe4d4644333c99f9c4a1282031c9f
    # decryptWithAD(0xe68f69b7f096d7917245f5e5cf8ae1595febe4d4644333c99f9c4a1282031c9f,
0x000000000000000000000000, 0x9e0e7de8bb75554f21db034633de04be41a2b8a18da7a319a03c803bf02b396c,
0x0df6086551151f58b8afe6c195782c6a)
    # h=0x9d1ffbb639e7e20021d9259491dc7b160aab270fb1339ef135053f6f2cebe9ce
    # Act Two
    # e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    # h=0x38122f669819f906000621a14071802f93f2ef97df100097bcac3ae76c6dc0bf
    # ss=0xc06363d6cc549bcb7913dbb9ac1c33fc1158680c89e972000ecd06b36c472e47
    #
HKDF(0xb61ec1191326fa240decc9564369dbb3ae2b34341d1e11ad64ed89f89180582f,0xc06363d6cc549bcb7913db
b9ac1c33fc1158680c89e972000ecd06b36c472e47)
    #
ck,temp_k2=0xe89d31033a1b6bf68c07d22e08ea4d7884646c4b60a9528598ccb4ee2c8f56ba,0x908b166535c01a93
5cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc
    # encryptWithAD(0x908b166535c01a935cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc,
0x000000000000000000000000, 0x38122f669819f906000621a14071802f93f2ef97df100097bcac3ae76c6dc0bf,
<empty>)
    # c=0x6e2470b93aac583c9ef6eafca3f730ae
    # h=0x90578e247e98674e661013da3c5c1ca6a8c8f48c90b485c0dfa1494e23d56d72
    output:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # Act Three
    input:
0x00b9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c38228dc68b1c466263b47fdf31e560e139ba
    # decryptWithAD(0x908b166535c01a935cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc,
0x000000000100000000000000, 0x90578e247e98674e661013da3c5c1ca6a8c8f48c90b485c0dfa1494e23d56d72,
0xb9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad0c
3822)
    # rs=0x034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    # h=0x5dcb5ea9b4ccc755e0e3456af3990641276e1d5dc9afd82f974d90a47c918660
    # ss=0xb36b6d195982c5be874d6d542dc268234379e1ae4ff1709402135b7de5cf0766
    #
HKDF(0xe89d31033a1b6bf68c07d22e08ea4d7884646c4b60a9528598ccb4ee2c8f56ba,0xb36b6d195982c5be874d6d
542dc268234379e1ae4ff1709402135b7de5cf0766)
```

```
    #
ck,temp_k3=0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01,0x981a46c820fb7a24
1bc8184ba4bb1f01bcdfafb00dde80098cb8c38db9141520
    # decryptWithAD(0x981a46c820fb7a241bc8184ba4bb1f01bcdfafb00dde80098cb8c38db9141520,
0x000000000000000000000000, 0x5dcb5ea9b4ccc755e0e3456af3990641276e1d5dc9afd82f974d90a47c918660,
0x8dc68b1c466263b47fdf31e560e139ba)
    # HKDF(0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01,zero)
    output:
rk,sk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9,0xbb9020b8965f4df047e07
f955f3c4b88418984aadc5cdb35096b9ea8fa5c3442

    name: transport-responder act1 short read test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c
    output: ERROR (ACT1_READ_FAILED)

    name: transport-responder act1 bad version test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x01036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    output: ERROR (ACT1_BAD_VERSION)

    name: transport-responder act1 bad key serialization test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00046360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    output: ERROR (ACT1_BAD_PUBKEY)

    name: transport-responder act1 bad MAC test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6b
    output: ERROR (ACT1_BAD_TAG)
```

```
    name: transport-responder act3 bad version test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    output:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # Act Three
    input:
0x01b9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c38228dc68b1c466263b47fdf31e560e139ba
    output: ERROR (ACT3_BAD_VERSION 1)

    name: transport-responder act3 short read test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    output:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # Act Three
    input:
0x00b9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c38228dc68b1c466263b47fdf31e560e139
    output: ERROR (ACT3_READ_FAILED)

    name: transport-responder act3 bad MAC for ciphertext test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    output:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # Act Three
    input:
0x00c9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c38228dc68b1c466263b47fdf31e560e139ba
```

```
    output: ERROR (ACT3_BAD_CIPHERTEXT)

    name: transport-responder act3 bad rs test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    output:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # Act Three
    input:
0x00bfe3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa2235536ad09a8ee351870c2bb7f78b754
a26c6cef79a98d25139c856d7efd252c2ae73c
    # decryptWithAD(0x908b166535c01a935cf1e130a5fe895ab4e6f3ef8855d87e9b7581c4ab663ddc,
0x000000000000000000000001, 0x90578e247e98674e661013da3c5c1ca6a8c8f48c90b485c0dfa1494e23d56d72,
0xd7fedc211450dd9602b41081c9bd05328b8bf8c0238880f7b7cb8a34bb6d8354081e8d4b81887fae47a74fe8aab300
8653)
    # rs=0x044f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
    output: ERROR (ACT3_BAD_PUBKEY)

    name: transport-responder act3 bad MAC test
    ls.priv=2121212121212121212121212121212121212121212121212121212121212121
    ls.pub=028d7500dd4c12685d1f568b4c2b5048e8534b873319f3a8daa612b469132ec7f7
    e.priv=0x2222222222222222222222222222222222222222222222222222222222222222
    e.pub=0x02466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f27
    # Act One
    input:
0x00036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddbd21325eff73f70df6086551151f58b8afe6c195
782c6a
    # Act Two
    output:
0x0002466d7fcae563e5cb09a0d1870bb580344804617879a14949cf22285f1bae3f276e2470b93aac583c9ef6eafca3
f730ae
    # Act Three
    input:
0x00b9e3a702e93e3a9948c2ed6e5fd7590a6e1c3a0344cfc9d5b57357049aa22355361aa02e55a8fc28fef5bd6d71ad
0c38228dc68b1c466263b47fdf31e560e139bb
    output: ERROR (ACT3_BAD_TAG)
```

Message Encryption Tests

In this test, the initiator sends length 5 messages containing "hello" 1001 times. Only six example outputs are shown, for brevity and to test two key rotations:

```
name: transport-message test
ck=0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01
sk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9
rk=0xbb9020b8965f4df047e07f955f3c4b88418984aadc5cdb35096b9ea8fa5c3442
# encrypt l: cleartext=0x0005, AD=NULL, sn=0x000000000000000000000000,
```

```
sk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9 =>
0xcf2b30ddf0cf3f80e7c35a6e6730b59fe802
# encrypt m: cleartext=0x68656c6c6f, AD=NULL, sn=0x000000000100000000000000,
sk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9 =>
0x473180f396d88a8fb0db8cbcf25d2f214cf9ea1d95
output 0: 0xcf2b30ddf0cf3f80e7c35a6e6730b59fe802473180f396d88a8fb0db8cbcf25d2f214cf9ea1d95
# encrypt l: cleartext=0x0005, AD=NULL, sn=0x000000000200000000000000,
sk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9 =>
0x72887022101f0b6753e0c7de21657d35a4cb
# encrypt m: cleartext=0x68656c6c6f, AD=NULL, sn=0x000000000300000000000000,
sk=0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9 =>
0x2a1f5cde2650528bbc8f837d0f0d7ad833b1a256a1
output 1: 0x72887022101f0b6753e0c7de21657d35a4cb2a1f5cde2650528bbc8f837d0f0d7ad833b1a256a1
# 0xcc2c6e467efc8067720c2d09c139d1f77731893aad1defa14f9bf3c48d3f1d31,
0x3fbdc101abd1132ca3a0ae34a669d8d9ba69a587e0bb4ddd59524541cf4813d8 =
HKDF(0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01,
0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9)
# 0xcc2c6e467efc8067720c2d09c139d1f77731893aad1defa14f9bf3c48d3f1d31,
0x3fbdc101abd1132ca3a0ae34a669d8d9ba69a587e0bb4ddd59524541cf4813d8 =
HKDF(0x919219dbb2920afa8db80f9a51787a840bcf111ed8d588caf9ab4be716e42b01,
0x969ab31b4d288cedf6218839b27a3e2140827047f2c0f01bf5c04435d43511a9)
output 500: 0x178cb9d7387190fa34db9c2d50027d21793c9bc2d40b1e14dcf30ebeeeb220f48364f7a4c68bf8
output 501: 0x1b186c57d44eb6de4c057c49940d79bb838a145cb528d6e8fd26dbe50a60ca2c104b56b60e45bd
# 0x728366ed68565dc17cf6dd97330a859a6a56e87e2beef3bd828a4c4a54d8df06,
0x9e0477f9850dca41e42db0e4d154e3a098e5a000d995e421849fcd5df27882bd =
HKDF(0xcc2c6e467efc8067720c2d09c139d1f77731893aad1defa14f9bf3c48d3f1d31,
0x3fbdc101abd1132ca3a0ae34a669d8d9ba69a587e0bb4ddd59524541cf4813d8)
# 0x728366ed68565dc17cf6dd97330a859a6a56e87e2beef3bd828a4c4a54d8df06,
0x9e0477f9850dca41e42db0e4d154e3a098e5a000d995e421849fcd5df27882bd =
HKDF(0xcc2c6e467efc8067720c2d09c139d1f77731893aad1defa14f9bf3c48d3f1d31,
0x3fbdc101abd1132ca3a0ae34a669d8d9ba69a587e0bb4ddd59524541cf4813d8)
output 1000: 0x4a2f3cc3b5e78ddb83dcb426d9863d9d9a723b0337c89dd0b005d89f8d3c05c52b76b29b740f09
output 1001: 0x2ecd8c8a5629d0d02ab457a0fdd0f7b90a192cd46be5ecb6ca570bfc5e268338b1a16cf4ef2d36
```

Acknowledgments

TODO(roasbeef); fin

References

1. https://tools.ietf.org/html/rfc8439
2. http://noiseprotocol.org/noise.html
3. https://tools.ietf.org/html/rfc5869

Authors

FIXME

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

BOLT #1: Base Protocol

Overview

This protocol assumes an underlying authenticated and ordered transport mechanism that takes care of framing individual messages. BOLT #8 specifies the canonical transport layer used in Lightning, though it can be replaced by any transport that fulfills the above guarantees.

The default TCP port depends on the network used. The most common networks are:

- Bitcoin mainnet with port number 9735 or the corresponding hexadecimal `0x2607`;
- Bitcoin testnet with port number 19735 (`0x4D17`);
- Bitcoin signet with port number 39735 (`0x9B37`).

The Unicode code point for LIGHTNING [1], and the port convention try to follow the Bitcoin Core convention.

All data fields are unsigned big-endian unless otherwise specified.

Table of Contents

- Connection Handling and Multiplexing
- Lightning Message Format
- Type-Length-Value Format
- Fundamental Types
- Setup Messages
    - The `init` Message
    - The `error` and `warning` Messages
- Control Messages
    - The `ping` and `pong` Messages
- Peer Storage
    - The `peer_storage` and `peer_storage_retrieval` Messages
- Appendix A: BigSize Test Vectors
- Appendix B: Type-Length-Value Test Vectors
- Appendix C: Message Extension
- Appendix D: Signed Integers Test Vectors
- Acknowledgments
- References
- Authors

Connection Handling and Multiplexing

Implementations MUST use a single connection per peer; channel messages (which include a channel ID) are multiplexed over this single connection.

Lightning Message Format

After decryption, all Lightning messages are of the form:

1. `type`: a 2-byte big-endian field indicating the type of message
2. `payload`: a variable-length payload that comprises the remainder of the message and that conforms to a format matching the `type`
3. `extension`: an optional TLV stream

The `type` field indicates how to interpret the `payload` field. The format for each individual type is defined by a specification in this repository. The type follows the *it's ok to be odd* rule, so nodes MAY send *odd*-numbered types without ascertaining that the recipient understands it.

The messages are grouped logically into five groups, ordered by the most significant bit that is set:

- Setup & Control (types `0-31`): messages related to connection setup, control, supported features, and error reporting (described below)
- Channel (types `32-127`): messages used to setup and tear down micropayment channels (described in BOLT #2)
- Commitment (types `128-255`): messages related to updating the current commitment transaction, which includes adding, revoking, and settling HTLCs as well as updating fees and exchanging signatures (described in BOLT #2)
- Routing (types `256-511`): messages containing node and channel announcements, as well as any active route exploration (described in BOLT #7)
- Custom (types `32768-65535`): experimental and application-specific messages

The size of the message is required by the transport layer to fit into a 2-byte unsigned int; therefore, the maximum possible size is 65535 bytes.

A sending node: - MUST NOT send an evenly-typed message not listed here without prior negotiation. - MUST NOT send evenly-typed TLV records in the `extension` without prior negotiation. - that negotiates an option in this specification: - MUST include all the fields annotated with that option. - When defining custom messages: - SHOULD pick a random `type` to avoid collision with other custom types. - SHOULD pick a `type` that doesn't conflict with other experiments listed in this issue. - SHOULD pick an odd `type` identifiers when regular nodes should ignore the additional data. - SHOULD pick an even `type` identifiers when regular nodes should reject the message and close the connection.

A receiving node: - upon receiving a message of *odd*, unknown type: - MUST ignore the received message. - upon receiving a message of *even*, unknown type: - MUST close the connection. - MAY fail the channels. - upon receiving a known message with insufficient length for the contents: - MUST close the connection. - MAY fail the channels. - upon receiving a message with an `extension`: - MAY ignore the `extension`. - Otherwise, if the `extension` is invalid: - MUST close the connection. - MAY fail the channels.

Rationale

By default `SHA2` and Bitcoin public keys are both encoded as big endian, thus it would be unusual to use a different endian for other fields.

Length is limited to 65535 bytes by the cryptographic wrapping, and messages in the protocol are never more than that length anyway.

The *it's ok to be odd* rule allows for future optional extensions without negotiation or special coding in clients. The *extension* field similarly allows for future expansion by letting senders include additional TLV data. Note that an *extension* field can only be added when the message `payload` doesn't already fill the 65535 bytes maximum length.

Implementations may prefer to have message data aligned on an 8-byte boundary (the largest natural alignment requirement of any type here); however, adding a 6-byte padding after the type field was considered wasteful: alignment may be achieved by decrypting the message into a buffer with 6-bytes of pre-padding.

Type-Length-Value Format

Throughout the protocol, a TLV (Type-Length-Value) format is used to allow for the backwards-compatible addition of new fields to existing message types.

A `tlv_record` represents a single field, encoded in the form:

- [`bigsize`: `type`]
- [`bigsize`: `length`]
- [`length`: `value`]

A `tlv_stream` is a series of (possibly zero) `tlv_record`s, represented as the concatenation of the encoded `tlv_record`s. When used to extend existing messages, a `tlv_stream` is typically placed after all currently defined fields.

The `type` is encoded using the BigSize format. It functions as a message-specific, 64-bit identifier for the `tlv_record` determining how the contents of `value` should be decoded. `type` identifiers below 2^16 are reserved for use in this specification. `type` identifiers greater than or equal to 2^16 are available for custom records. Any record not defined in this specification is considered a custom record. This includes experimental and application-specific messages.

The `length` is encoded using the BigSize format signaling the size of `value` in bytes.

The `value` depends entirely on the `type`, and should be encoded or decoded according to the message-specific format determined by `type`.

Requirements

The sending node: - MUST order `tlv_record`s in a `tlv_stream` by strictly-increasing `type`, hence MUST not produce more than a single TLV record with the same `type` - MUST minimally encode `type` and `length`. - When defining custom record `type` identifiers: - SHOULD pick random `type` identifiers to avoid collision with other custom types. - SHOULD pick odd `type` identifiers when regular nodes should ignore the additional data. - SHOULD pick even `type` identifiers when regular nodes should reject the full tlv stream containing the custom record. - SHOULD NOT use redundant, variable-length encodings in a `tlv_record`.

The receiving node: - if zero bytes remain before parsing a `type`: - MUST stop parsing the `tlv_stream`. - if a `type` or `length` is not minimally encoded: - MUST fail to parse the `tlv_stream`. - if decoded `type`s are not strictly-increasing (including situations when two or more occurrences of the same `type` are met): - MUST fail

to parse the `tlv_stream`. - if `length` exceeds the number of bytes remaining in the message: - MUST fail to parse the `tlv_stream`. - if `type` is known: - MUST decode the next `length` bytes using the known encoding for `type`. - if `length` is not exactly equal to that required for the known encoding for `type`: - MUST fail to parse the `tlv_stream`. - if variable-length fields within the known encoding for `type` are not minimal: - MUST fail to parse the `tlv_stream`. - otherwise, if `type` is unknown: - if `type` is even: - MUST fail to parse the `tlv_stream`. - otherwise, if `type` is odd: - MUST discard the next `length` bytes.

Rationale

The primary advantage in using TLV is that a reader is able to ignore new fields that it does not understand, since each field carries the exact size of the encoded element. Without TLV, even if a node does not wish to use a particular field, the node is forced to add parsing logic for that field in order to determine the offset of any fields that follow.

The strict monotonicity constraint ensures that all `type`s are unique and can appear at most once. Fields that map to complex objects, e.g. vectors, maps, or structs, should do so by defining the encoding such that the object is serialized within a single `tlv_record`. The uniqueness constraint, among other things, enables the following optimizations: - canonical ordering is defined independent of the encoded `value`s. - canonical ordering can be known at compile-time, rather than being determined dynamically at the time of encoding. - verifying canonical ordering requires less state and is less-expensive. - variable-size fields can reserve their expected size up front, rather than appending elements sequentially and incurring double-and-copy overhead.

The use of a bigsize for `type` and `length` permits a space savings for small `type`s or short `value`s. This potentially leaves more space for application data over the wire or in an onion payload.

All `type`s must appear in increasing order to create a canonical encoding of the underlying `tlv_record`s. This is crucial when computing signatures over a `tlv_stream`, as it ensures verifiers will be able to recompute the same message digest as the signer. Note that the canonical ordering over the set of fields can be enforced even if the verifier does not understand what the fields contain.

Writers should avoid using redundant, variable-length encodings in a `tlv_record` since this results in encoding the length twice and complicates computing the outer length. As an example, when writing a variable length byte array, the `value` should contain only the raw bytes and forgo an additional internal length since the `tlv_record` already carries the number of bytes that follow. On the other hand, if a `tlv_record` contains multiple, variable-length elements then this would not be considered redundant, and is needed to allow the receiver to parse individual elements from `value`.

Fundamental Types

Various fundamental types are referred to in the message specifications:

- `byte`: an 8-bit byte
- `s8`: an 8-bit signed integer
- `u16`: a 2 byte unsigned integer
- `s16`: a 2 byte signed integer
- `u32`: a 4 byte unsigned integer
- `s32`: a 4 byte signed integer
- `u64`: an 8 byte unsigned integer
- `s64`: an 8 byte signed integer

Signed integers use standard big-endian two's complement representation (see test vectors below).

For the final value in TLV records, truncated integers may be used. Leading zeros in truncated integers MUST be omitted:

- `tu16`: a 0 to 2 byte truncated unsigned integer
- `tu32`: a 0 to 4 byte truncated unsigned integer
- `tu64`: a 0 to 8 byte truncated unsigned integer

When used to encode amounts, the previous fields MUST comply with the upper bound of 21 million BTC:

- satoshi amounts MUST be at most `0x000775f05a074000`
- milli-satoshi amounts MUST be at most `0x1d24b2dfac520000`

The following convenience types are also defined:

- `chain_hash`: a 32-byte chain identifier (see BOLT #0)
- `channel_id`: a 32-byte channel_id (see BOLT #2)
- `sha256`: a 32-byte SHA2-256 hash
- `signature`: a 64-byte bitcoin Elliptic Curve signature
- `bip340sig`: a 64-byte bitcoin Elliptic Curve Schnorr signature as per BIP-340
- `point`: a 33-byte Elliptic Curve point (compressed encoding as per SEC 1 standard)
- `short_channel_id`: an 8 byte value identifying a channel (see BOLT #7)
- `sciddir_or_pubkey`: either 9 or 33 bytes referencing or identifying a node, respectively
    - if the first byte is 0 or 1, then an 8-byte `short_channel_id` follows for a total of 9 bytes
        - 0 for the first byte indicates this refers to `node_id_1` in the `channel_announcement` for `short_channel_id`
        - 1 for the first byte indicates this refers to `node_id_2` in the `channel_announcement` for `short_channel_id` (see BOLT #7
    - if the first byte is 2 or 3, then the value is a 33-byte `point`
- `bigsize`: a variable-length, unsigned integer similar to Bitcoin's CompactSize encoding, but big-endian. Described in BigSize.
- `utf8`: a byte as part of a UTF-8 string. A writer MUST ensure an array of these is a valid UTF-8 string, a reader MAY reject any messages containing an array of these which is not a valid UTF-8 string.

Setup Messages

The `init` Message

Once authentication is complete, the first message reveals the features supported or required by this node, even if this is a reconnection.

BOLT #9 specifies lists of features. Each feature is generally represented by 2 bits. The least-significant bit is numbered 0, which is *even*, and the next most significant bit is numbered 1, which is *odd*. For historical reasons, features are divided into global and local feature bitmasks.

A feature is *offered* if a peer set it in the `init` message for the current connection (as either even or odd). A feature is *negotiated* if either both peers offered it, or the local node offered it as even: it can assume the peer supports it, as it did not disconnect as it would be required to do.

The `features` field MUST be padded to bytes with 0s.

1. type: 16 (`init`)
2. data:
    - [`u16:gflen`]
    - [`gflen*byte:globalfeatures`]
    - [`u16:flen`]
    - [`flen*byte:features`]
    - [`init_tlvs:tlvs`]
3. `tlv_stream`: `init_tlvs`
4. types:
    1. type: 1 (`networks`)
    2. data:
        - [`...*chain_hash:chains`]
    3. type: 3 (`remote_addr`)
    4. data:
        - [`...*byte:data`]

The optional `networks` indicates the chains the node is interested in. The optional `remote_addr` can be used to circumvent NAT issues.

Requirements

The sending node: - MUST send `init` as the first Lightning message for any connection. - MUST set feature bits as defined in BOLT #9. - MUST set any undefined feature bits to 0. - SHOULD NOT set features greater than 13 in `globalfeatures`. - SHOULD use the minimum length required to represent the `features` field. - SHOULD set `networks` to all chains it will gossip or open channels for. - SHOULD set `remote_addr` to reflect the remote IP address (and port) of an incoming connection, if the node is the receiver and the connection was done via IP. - if it sets `remote_addr`: - MUST set it to a valid `address descriptor` (1 byte type and data) as described in BOLT 7. - SHOULD NOT set private addresses as `remote_addr`.

The receiving node: - MUST wait to receive `init` before sending any other messages. - MUST combine (bitwise OR) the two feature bitmaps into one logical `features` map. - MUST respond to known feature bits as specified in BOLT #9. - upon receiving unknown *odd* feature bits that are non-zero: - MUST ignore the bit. - upon receiving unknown *even* feature bits that are non-zero: - MUST close the connection. - upon receiving `networks` containing no common chains - MAY close the connection. - if the feature vector does not set all known, transitive dependencies: - MUST close the connection. - MAY use the `remote_addr` to update its `node_announcement`

Rationale

There used to be two feature bitfields here, but for backwards compatibility they're now combined into one.

This semantic allows both future incompatible changes and future backward compatible changes. Bits should generally be assigned in pairs, in order that optional features may later become compulsory.

Nodes wait for receipt of the other's features to simplify error diagnosis when features are incompatible.

Since all networks share the same port, but most implementations only support a single network, the `networks` fields avoids nodes erroneously believing they will receive updates about their preferred network, or that they can open channels.

The `error` and `warning` Messages

For simplicity of diagnosis, it's often useful to tell a peer that something is incorrect.

1. type: 17 (`error`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u16`:`len`]
    - [`len*byte`:`data`]
3. type: 1 (`warning`)
4. data:
    - [`channel_id`:`channel_id`]
    - [`u16`:`len`]
    - [`len*byte`:`data`]

Requirements

The channel is referred to by `channel_id`, unless `channel_id` is 0 (i.e. all bytes are 0), in which case it refers to all channels.

The funding node using channel establishment v1 (`open_channel`): - for all error messages sent before (and including) the `funding_created` message: - MUST use `temporary_channel_id` in lieu of `channel_id`.

The fundee node using channel establishment v1 (`accept_channel`): - for all error messages sent before (and not including) the `funding_signed` message: - MUST use `temporary_channel_id` in lieu of `channel_id`.

The opener node using channel establishment v2 (`open_channel2`): - for all error messages sent before the `accept_channel2` message is received: - MUST use `temporary_channel_id` in lieu of `channel_id`.

The accepter node using channel establishment v2 (`open_channel2`): - for all error messages sent before (and including) the `accept_channel2` message: - MUST use `temporary_channel_id` in lieu of `channel_id`.

A sending node: - SHOULD send `error` for protocol violations or internal errors that make channels unusable or that make further communication unusable. - SHOULD send `error` with the unknown `channel_id` in reply to messages of type `32-255` related to unknown channels. - when sending `error`: - MUST fail the channel(s) referred to by the error message. - MAY set `channel_id` to all zero to indicate all channels. - when sending `warning`: - MAY set `channel_id` to all zero if the warning is not related to a specific channel. - MAY send an empty `data` field. - when failure was caused by an invalid signature check: - SHOULD include the raw, hex-encoded transaction in reply to a `funding_created`, `funding_signed`, `closing_signed`, or `commitment_signed` message.

The receiving node: - upon receiving `error`: - if `channel_id` is all zero: - MUST fail all channels with the sending node. - otherwise: - MUST fail the channel referred to by `channel_id`, if that channel is with the sending node. - upon receiving `warning`: - SHOULD log the message for later diagnosis. - MAY attempt `shutdown` if permitted at this point. - if no existing channel is referred to by `channel_id`: - MUST ignore the

message. - if `data` is not composed solely of printable ASCII characters (For reference: the printable character set includes byte values 32 through 126, inclusive): - SHOULD NOT print out `data` verbatim.

Rationale

There are unrecoverable errors that require an abort of conversations; if the connection is simply dropped, then the peer may retry the connection. It's also useful to describe protocol violations for diagnosis, as this indicates that one peer has a bug.

On the other hand, overuse of error messages has lead to implementations ignoring them (to avoid an otherwise expensive channel break), so the "warning" message was added to allow some degree of retry or recovery for spurious errors.

It may be wise not to distinguish errors in production settings, lest it leak information — hence, the optional `data` field.

Control Messages

The `ping` and `pong` Messages

In order to allow for the existence of long-lived TCP connections, at times it may be required that both ends keep alive the TCP connection at the application level. Such messages also allow obfuscation of traffic patterns.

1. type: 18 (`ping`)
2. data:
    - [`u16:num_pong_bytes`]
    - [`u16:byteslen`]
    - [`byteslen*byte:ignored`]

The `pong` message is to be sent whenever a `ping` message is received. It serves as a reply and also serves to keep the connection alive, while explicitly notifying the other end that the receiver is still active. Within the received `ping` message, the sender will specify the number of bytes to be included within the data payload of the `pong` message.

1. type: 19 (`pong`)
2. data:
    - [`u16:byteslen`]
    - [`byteslen*byte:ignored`]

Requirements

A node sending a `ping` message: - SHOULD set `ignored` to 0s. - MUST NOT set `ignored` to sensitive data such as secrets or portions of initialized memory. - if it doesn't receive a corresponding `pong`: - MAY close the network connection, - and MUST NOT fail the channels in this case.

A node sending a `pong` message: - SHOULD set `ignored` to 0s. - MUST NOT set `ignored` to sensitive data such as secrets or portions of initialized memory.

A node receiving a `ping` message: - if `num_pong_bytes` is less than 65532: - MUST respond by sending a `pong` message, with `byteslen` equal to `num_pong_bytes`. - otherwise (`num_pong_bytes` is **not** less than 65532): - MUST ignore the `ping`.

A node receiving a `pong` message: - if `byteslen` does not correspond to any `ping`'s `num_pong_bytes` value it has sent: - MAY close the connection.

Rationale

The largest possible message is 65535 bytes; thus, the maximum sensible `byteslen` is 65531 — in order to account for the type field (`pong`) and the `byteslen` itself. This allows a convenient cutoff for `num_pong_bytes` to indicate that no reply should be sent.

Connections between nodes within the network may be long lived, as payment channels have an indefinite lifetime. However, it's likely that no new data will be exchanged for a significant portion of a connection's lifetime. Also, on several platforms it's possible that Lightning clients will be put to sleep without prior warning. Hence, a distinct `ping` message is used, in order to probe for the liveness of the connection on the other side, as well as to keep the established connection active.

Additionally, the ability for a sender to request that the receiver send a response with a particular number of bytes enables nodes on the network to create *synthetic* traffic. Such traffic can be used to partially defend against packet and timing analysis — as nodes can fake the traffic patterns of typical exchanges without applying any true updates to their respective channels.

When combined with the onion routing protocol defined in BOLT #4, careful statistically driven synthetic traffic can serve to further bolster the privacy of participants within the network.

Limited precautions are recommended against `ping` flooding, however some latitude is given because of network delays. Note that there are other methods of incoming traffic flooding (e.g. sending *odd* unknown message types, or padding every message maximally).

Finally, the usage of periodic `ping` messages serves to promote frequent key rotations as specified within BOLT #8.

Peer storage

The `peer_storage` and `peer_storage_retrieval` Messages

Nodes that advertise the `option_provide_storage` feature offer storing arbitrary data for their peers. The data stored must not exceed 65531 bytes, which lets it fit in lightning messages.

Nodes can verify that their `option_provide_storage` peers correctly store their data at each reconnection, by comparing the contents of the retrieved data with the last one they sent. However, nodes should not expect their peers to always have their latest data available.

Nodes ask their peers to store data using the `peer_storage` message and expect peers to return the latest data to them using the `peer_storage_retrieval` message:

1. type: 7 (`peer_storage`)

2. data:
    - [u16:length]
    - [length*byte:blob]
3. type: 9 (peer_storage_retrieval)
4. data:
    - [u16:length]
    - [length*byte:blob]

Requirements:

The sender of peer_storage: - MAY send peer_storage whenever necessary. - MUST limit its blob to 65531 bytes. - MUST encrypt the data in a manner that ensures its integrity upon receipt. - SHOULD pad the blob to ensure its length is always exactly 65531 bytes.

The receiver of peer_storage: - If it offered option_provide_storage: - if it has an open channel with the sender: - MUST store the message. - MAY store the message anyway.

- If it does store the message:
    - MAY delay storage to ratelimit peer to no more than one update per minute.
    - MUST replace the old blob with the latest received.
    - MUST send peer_storage_retrieval again after reconnection, after exchanging init messages.

The sender of peer_storage_retrieval: - MUST include the last blob it stored for that peer. - when all channels with that peer are closed: - SHOULD wait at least 2016 blocks before deleting the blob.

The receiver of peer_storage_retrieval: - when it receives peer_storage_retrieval with an outdated or irrelevant data: - MAY send a warning.

Rationale:

The peer_storage and peer_storage_retrieval messages enable nodes to securely store and share data with other nodes in the network, serving as a backup mechanism for important information. By utilizing them, nodes can safeguard crucial data, enhancing the network's resilience and reliability. Additionally, even if we don't have an open channel, some nodes might provide this service in exchange for some sats so they may store peer_storage.

Nodes should pad the blob to obscure its actual size, enhancing privacy by making size-based analysis more difficult for the receiver.

peer_storage_retrieval should not be sent after channel_reestablish because then the user wouldn't have an option to recover the node and update its state in case they lost data.

Nodes should send a peer_storage message whenever they wish to update the blob stored with their peers. This blob can be used to distribute encrypted data, which could be helpful in restoring the node.

Appendix A: BigSize Test Vectors

The following test vectors can be used to assert the correctness of a BigSize implementation used in the TLV format. The format is identical to the CompactSize encoding used in bitcoin, but replaces the little-endian encoding of multi-byte values with big-endian.

Values encoded with BigSize will produce an encoding of either 1, 3, 5, or 9 bytes depending on the size of the integer. The encoding is a piece-wise function that takes a `uint64` value `x` and produces:

```
uint8(x)                if x < 0xfd
0xfd + be16(uint16(x))  if x < 0x10000
0xfe + be32(uint32(x))  if x < 0x100000000
0xff + be64(x)          otherwise.
```

Here `+` denotes concatenation and `be16`, `be32`, and `be64` produce a big-endian encoding of the input for 16, 32, and 64-bit integers, respectively.

A value is said to be *minimally encoded* if it could not be encoded using fewer bytes. For example, a BigSize encoding that occupies 5 bytes but whose value is less than 0x10000 is not minimally encoded. All values decoded with BigSize should be checked to ensure they are minimally encoded.

BigSize Decoding Tests

The following is an example of how to execute the BigSize decoding tests.

```
func testReadBigSize(t *testing.T, test bigSizeTest) {
        var buf [8]byte
        r := bytes.NewReader(test.Bytes)
        val, err := tlv.ReadBigSize(r, &buf)
        if err != nil && err.Error() != test.ExpErr {
                t.Fatalf("expected decoding error: %v, got: %v",
                        test.ExpErr, err)
        }

        // If we expected a decoding error, there's no point checking the value.
        if test.ExpErr != "" {
                return
        }

        if val != test.Value {
                t.Fatalf("expected value: %d, got %d", test.Value, val)
        }
}
```

A correct implementation should pass against these test vectors:


```
[
    {
        "name": "zero",
        "value": 0,
        "bytes": "00"
    },
    {
        "name": "one byte high",
        "value": 252,
        "bytes": "fc"
    },
    {
        "name": "two byte low",
        "value": 253,
```

```
    "bytes": "fd00fd"
},
{
    "name": "two byte high",
    "value": 65535,
    "bytes": "fdffff"
},
{
    "name": "four byte low",
    "value": 65536,
    "bytes": "fe00010000"
},
{
    "name": "four byte high",
    "value": 4294967295,
    "bytes": "feffffffff"
},
{
    "name": "eight byte low",
    "value": 4294967296,
    "bytes": "ff0000000100000000"
},
{
    "name": "eight byte high",
    "value": 18446744073709551615,
    "bytes": "ffffffffffffffffff"
},
{
    "name": "two byte not canonical",
    "value": 0,
    "bytes": "fd00fc",
    "exp_error": "decoded bigsize is not canonical"
},
{
    "name": "four byte not canonical",
    "value": 0,
    "bytes": "fe0000ffff",
    "exp_error": "decoded bigsize is not canonical"
},
{
    "name": "eight byte not canonical",
    "value": 0,
    "bytes": "ff00000000ffffffff",
    "exp_error": "decoded bigsize is not canonical"
},
{
    "name": "two byte short read",
    "value": 0,
    "bytes": "fd00",
    "exp_error": "unexpected EOF"
},
{
    "name": "four byte short read",
    "value": 0,
    "bytes": "feffff",
```

```
        "exp_error": "unexpected EOF"
    },
    {
        "name": "eight byte short read",
        "value": 0,
        "bytes": "fffffffffff",
        "exp_error": "unexpected EOF"
    },
    {
        "name": "one byte no read",
        "value": 0,
        "bytes": "",
        "exp_error": "EOF"
    },
    {
        "name": "two byte no read",
        "value": 0,
        "bytes": "fd",
        "exp_error": "unexpected EOF"
    },
    {
        "name": "four byte no read",
        "value": 0,
        "bytes": "fe",
        "exp_error": "unexpected EOF"
    },
    {
        "name": "eight byte no read",
        "value": 0,
        "bytes": "ff",
        "exp_error": "unexpected EOF"
    }
]
```

BigSize Encoding Tests

The following is an example of how to execute the BigSize encoding tests.

```
func testWriteBigSize(t *testing.T, test bigSizeTest) {
        var (
                w   bytes.Buffer
                buf [8]byte
        )
        err := tlv.WriteBigSize(&w, test.Value, &buf)
        if err != nil {
                t.Fatalf("unable to encode %d as bigsize: %v",
                        test.Value, err)
        }

        if bytes.Compare(w.Bytes(), test.Bytes) != 0 {
                t.Fatalf("expected bytes: %v, got %v",
                        test.Bytes, w.Bytes())
        }
}
```

A correct implementation should pass against the following test vectors:


```
[
    {
        "name": "zero",
        "value": 0,
        "bytes": "00"
    },
    {
        "name": "one byte high",
        "value": 252,
        "bytes": "fc"
    },
    {
        "name": "two byte low",
        "value": 253,
        "bytes": "fd00fd"
    },
    {
        "name": "two byte high",
        "value": 65535,
        "bytes": "fdffff"
    },
    {
        "name": "four byte low",
        "value": 65536,
        "bytes": "fe00010000"
    },
    {
        "name": "four byte high",
        "value": 4294967295,
        "bytes": "feffffffff"
    },
    {
        "name": "eight byte low",
        "value": 4294967296,
        "bytes": "ff0000000100000000"
    },
    {
        "name": "eight byte high",
        "value": 18446744073709551615,
        "bytes": "ffffffffffffffffff"
    }
]
```


Appendix B: Type-Length-Value Test Vectors

The following tests assume that two separate TLV namespaces exist: n1 and n2.

The n1 namespace supports the following TLV types:

1. `tlv_stream`: `n1`
2. types:
    1. type: 1 (`tlv1`)

2. data:
   - [`tu64:amount_msat`]
1. type: 2 (`tlv2`)
2. data:
   - [`short_channel_id:scid`]
1. type: 3 (`tlv3`)
2. data:
   - [`point:node_id`]
   - [`u64:amount_msat_1`]
   - [`u64:amount_msat_2`]
1. type: 254 (`tlv4`)
2. data:
   - [`u16:cltv_delta`]

The n2 namespace supports the following TLV types:

1. `tlv_stream`: `n2`
2. types:
    1. type: 0 (`tlv1`)
    2. data:
       - [`tu64:amount_msat`]
    1. type: 11 (`tlv2`)
    2. data:
       - [`tu32:cltv_expiry`]

TLV Decoding Failures

The following TLV streams in any namespace should trigger a decoding failure:

1. Invalid stream: 0xfd
2. Reason: type truncated
3. Invalid stream: 0xfd01
4. Reason: type truncated
5. Invalid stream: 0xfd0001 00
6. Reason: not minimally encoded type
7. Invalid stream: 0xfd0101
8. Reason: missing length
9. Invalid stream: 0x0f fd
10. Reason: (length truncated)
11. Invalid stream: 0x0f fd26

12. Reason: (length truncated)
13. Invalid stream: 0x0f fd2602
14. Reason: missing value
15. Invalid stream: 0x0f fd0001 00
16. Reason: not minimally encoded length
17. Invalid stream: 0x0f fd0201
    0000000000

11. Invalid stream: 0x01 05 0001000000
12. Reason: encoding for `n1s` `tlv1s` `amount_msat` is not minimal
13. Invalid stream: 0x01 06 000100000000
14. Reason: encoding for `n1s` `tlv1s` `amount_msat` is not minimal
15. Invalid stream: 0x01 07 00010000000000
16. Reason: encoding for `n1s` `tlv1s` `amount_msat` is not minimal
17. Invalid stream: 0x01 08 0001000000000000
18. Reason: encoding for `n1s` `tlv1s` `amount_msat` is not minimal
19. Invalid stream: 0x02 07 01010101010101
20. Reason: less than encoding length for `n1s` `tlv2`.
21. Invalid stream: 0x02 09 010101010101010101
22. Reason: greater than encoding length for `n1s` `tlv2`.
23. Invalid stream: 0x03 21 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb
24. Reason: less than encoding length for `n1s` `tlv3`.
25. Invalid stream: 0x03 29 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb0000000000000001
26. Reason: less than encoding length for `n1s` `tlv3`.
27. Invalid stream: 0x03 30 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb000000000000000100000000000001
28. Reason: less than encoding length for `n1s` `tlv3`.
29. Invalid stream: 0x03 31 043da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb0000000000000001000000000000000002
30. Reason: `n1s` `node_id` is not a valid point.
31. Invalid stream: 0x03 32 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb00000000000000010000000000000000001
32. Reason: greater than encoding length for `n1s` `tlv3`.
33. Invalid stream: 0xfd00fe 00
34. Reason: less than encoding length for `n1s` `tlv4`.
35. Invalid stream: 0xfd00fe 01 01

36. Reason: less than encoding length for `n1s` `tlv4`.
37. Invalid stream: 0xfd00fe 03 010101
38. Reason: greater than encoding length for `n1s` `tlv4`.
39. Invalid stream: 0x00 00
40. Reason: unknown even field for `n1s` namespace.

TLV Decoding Successes

The following TLV streams in either namespace should correctly decode, and be ignored:

1. Valid stream: 0x
2. Explanation: empty message
3. Valid stream: 0x21 00
4. Explanation: Unknown odd type.
5. Valid stream: 0xfd0201 00
6. Explanation: Unknown odd type.
7. Valid stream: 0xfd00fd 00
8. Explanation: Unknown odd type.
9. Valid stream: 0xfd00ff 00
10. Explanation: Unknown odd type.
11. Valid stream: 0xfe02000001 00
12. Explanation: Unknown odd type.
13. Valid stream: 0xff0200000000000001 00
14. Explanation: Unknown odd type.

The following TLV streams in `n1` namespace should correctly decode, with the values given here:

1. Valid stream: 0x01 00
2. Values: `tlv1` `amount_msat`=0
3. Valid stream: 0x01 01 01
4. Values: `tlv1` `amount_msat`=1
5. Valid stream: 0x01 02 0100
6. Values: `tlv1` `amount_msat`=256

7. Valid stream: 0x01 03 010000
8. Values: `tlv1` `amount_msat`=65536
9. Valid stream: 0x01 04 01000000
10. Values: `tlv1` `amount_msat`=16777216
11. Valid stream: 0x01 05 0100000000
12. Values: `tlv1` `amount_msat`=4294967296
13. Valid stream: 0x01 06 010000000000
14. Values: `tlv1` `amount_msat`=1099511627776
15. Valid stream: 0x01 07 01000000000000
16. Values: `tlv1` `amount_msat`=281474976710656
17. Valid stream: 0x01 08 0100000000000000
18. Values: `tlv1` `amount_msat`=72057594037927936
19. Valid stream: 0x02 08 0000000000000226
20. Values: `tlv2` `scid`=0x0x550
21. Valid stream: 0x03 31 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb0000000000000000100000000000000002
22. Values: `tlv3` `node_id`=023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb `amount_msat_1`=1 `amount_msat_2`=2
23. Valid stream: 0xfd00fe 02 0226
24. Values: `tlv4` `cltv_delta`=550

TLV Stream Decoding Failure

Any appending of an invalid stream to a valid stream should trigger a decoding failure.

Any appending of a higher-numbered valid stream to a lower-numbered valid stream should not trigger a decoding failure.

In addition, the following TLV streams in namespace `n1` should trigger a decoding failure:

1. Invalid stream: 0x02 08 0000000000000226 01 01 2a
2. Reason: valid TLV records but invalid ordering
3. Invalid stream: 0x02 08 0000000000000231 02 08 0000000000000451
4. Reason: duplicate TLV type

5. Invalid stream: 0x1f 00 0f 01 2a
6. Reason: valid (ignored) TLV records but invalid ordering
7. Invalid stream: 0x1f 00 1f 01 2a
8. Reason: duplicate TLV type (ignored)

The following TLV stream in namespace `n2` should trigger a decoding failure:

1. Invalid stream: 0xffffffffffffffffff 00 00 00
2. Reason: valid TLV records but invalid ordering

Appendix C: Message Extension

This section contains examples of valid and invalid extensions on the `init` message. The base `init` message (without extensions) for these examples is `0x001000000000` (all features turned off).

The following `init` messages are valid:

- `0x001000000000`: no extension provided
- `0x001000000000c9012acb0104`: the extension contains two unknown *odd* TLV records (with types `0xc9` and `0xcb`)

The following `init` messages are invalid:

- `0x00100000000001`: the extension is present but truncated
- `0x001000000000ca012a`: the extension contains unknown *even* TLV records (assuming that TLV type `0xca` is unknown)
- `0x001000000000c90101c90102`: the extension TLV stream is invalid (duplicate TLV record type `0xc9`)

Note that when messages are signed, the *extension* is part of the signed bytes. Nodes should store the *extension* bytes even if they don't understand them to be able to correctly verify signatures.

Appendix D: Signed Integers Test Vectors

The following test vector show how signed integers (`s8`, `s16`, `s32` and `s64`) are encoded using big-endian two's complement.


```
[
    {
        "value": 0,
        "bytes": "00"
    },
    {
        "value": 42,
        "bytes": "2a"
    },
    {
        "value": -42,
        "bytes": "d6"
    },
    {
```

```
    "value": 127,
    "bytes": "7f"
},
{
    "value": -128,
    "bytes": "80"
},
{
    "value": 128,
    "bytes": "0080"
},
{
    "value": -129,
    "bytes": "ff7f"
},
{
    "value": 15000,
    "bytes": "3a98"
},
{
    "value": -15000,
    "bytes": "c568"
},
{
    "value": 32767,
    "bytes": "7fff"
},
{
    "value": -32768,
    "bytes": "8000"
},
{
    "value": 32768,
    "bytes": "00008000"
},
{
    "value": -32769,
    "bytes": "ffff7fff"
},
{
    "value": 21000000,
    "bytes": "01406f40"
},
{
    "value": -21000000,
    "bytes": "febf90c0"
},
{
    "value": 2147483647,
    "bytes": "7fffffff"
},
{
    "value": -2147483648,
    "bytes": "80000000"
},
```

```
    {
        "value": 2147483648,
        "bytes": "0000000080000000"
    },
    {
        "value": -2147483649,
        "bytes": "ffffffff7fffffff"
    },
    {
        "value": 500000000000,
        "bytes": "000000746a528800"
    },
    {
        "value": -500000000000,
        "bytes": "ffffff8b95ad7800"
    },
    {
        "value": 9223372036854775807,
        "bytes": "7fffffffffffffff"
    },
    {
        "value": -9223372036854775808,
        "bytes": "8000000000000000"
    }
]
```


Acknowledgments

[TODO: (roasbeef); fin]

References

1. http://www.unicode.org/charts/PDF/U2600.pdf

Authors

[FIXME: Insert Author List]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

BOLT #9: Assigned Feature Flags

This document tracks the assignment of `features` flags in the `init` message (BOLT #1), as well as `features` fields in the `channel_announcement` and `node_announcement` messages (BOLT #7). The flags are tracked separately, since new flags will likely be added over time.

Some features were introduced and became so widespread they are `ASSUMED` to be present by all nodes, and can be safely ignored (and the semantics are only defined in prior revisions of this spec).

Flags are numbered from the least-significant bit, at bit 0 (i.e. 0x1, an *even* bit). They are generally assigned in pairs so that features can be introduced as optional (*odd* bits) and later upgraded to be compulsory (*even* bits), which will be refused by outdated nodes: see BOLT #1: The `init` Message.

Some features don't make sense on a per-channels or per-node basis, so each feature defines how it is presented in those contexts. Some features may be required for opening a channel, but not a requirement for use of the channel, so the presentation of those features depends on the feature itself.

The Context column decodes as follows:

- `I`: presented in the `init` message.
- `N`: presented in the `node_announcement` messages
- `C`: presented in the `channel_announcement` message.
- `C-`: presented in the `channel_announcement` message, but always odd (optional).
- `C+`: presented in the `channel_announcement` message, but always even (required).
- `9`: presented in BOLT 11 invoices.
- `B`: presented in the `allowed_features` field of a blinded path.
- `T`: used in the `channel_type` field when opening channels.

Bits	Name	Description	Context	Dependencies	Link
0/1	`option_data_loss_protect`	ASSUMED			
4/5	`option_upfront_shutdown_script`	Commits to a shutdown scriptpubkey when opening channel	IN		BOLT #2
6/7	`gossip_queries`	Peer has useful gossip to share			
8/9	`var_onion_optin`	ASSUMED			
10/11	`gossip_queries_ex`	Gossip queries can include additional information	IN		BOLT #7
12/13	`option_static_remotekey`	ASSUMED			
14/15	`payment_secret`	ASSUMED			[Routing Onion Specifica [bolt04]
16/17	`basic_mpp`	Node can receive basic multi-part payments	IN9	`payment_secret`	BOLT #4
18/19	`option_support_large_channel`	Can create large channels	IN		BOLT #2
22/23	`option_anchors`	Anchor commitment type with zero fee HTLC transactions	INT		BOLT #3 lightning dev

Bits	Name	Description	Context	Dependencies	Link
24/25	`option_route_blinding`	Node supports blinded paths	IN9		BOLT #4
26/27	`option_shutdown_anysegwit`	Future segwit versions allowed in `shutdown`	IN		BOLT #2
28/29	`option_dual_fund`	Use v2 of channel open, enables dual funding	IN		BOLT #2
34/35	`option_quiesce`	Support for `stfu` message	IN		BOLT #2
36/37	`option_attribution_data`	Can generate/relay attribution data and `fulfillment_payload`	IN9		BOLT #4 successfu payment
38/39	`option_onion_messages`	Can forward onion messages	IN		BOLT #4
40/41	`zero_fee_commitments`	Zero-fee commitment and HTLC transactions	IN	`option_channel_type`	BOLT #3
42/43	`option_provide_storage`	Can store other nodes' encrypted backup data	IN		BOLT #1
44/45	`option_channel_type`	ASSUMED			
46/47	`option_scid_alias`	Supply channel aliases for routing	INT		BOLT #2
48/49	`option_payment_metadata`	Payment metadata in tlv record	9		BOLT #1
50/51	`option_zeroconf`	Understands zeroconf channel types	INT	`option_scid_alias`	BOLT #2
60/61	`option_simple_close`	Simplified closing negotiation	IN	`option_shutdown_anysegwit`	BOLT #2
62/63	`option_splice`	Allows replacing the funding transaction with a new one	IN		BOLT #2
66/67	`option_onion_messages_only_channels`	Only accepts onion messages from peers with a channel	IN	`option_onion_messages`	BOLT #4

Requirements

The origin node: * If it supports a feature above, SHOULD set the corresponding odd bit in all feature fields indicated by the Context column unless indicated that it must set the even feature bit instead. * If it requires a feature above, MUST set the corresponding even feature bit in all feature fields indicated by the Context column, unless indicated that it must set the odd feature bit instead. * MUST NOT set feature bits it does not support. * MUST NOT set feature bits in fields not specified by the table above. * MUST NOT set both the optional and mandatory bits. * MUST set all transitive feature dependencies. * MUST support: * `var_onion_optin`

The receiving node: * if both the optional and the mandatory feature bits in a pair are set, the feature should be treated as mandatory.

The requirements for receiving specific bits are defined in the linked sections in the table above. The requirements for feature bits that are not defined above can be found in BOLT #1: The `init` Message.

Rationale

Note that for feature flags which are available in both the `node_announcement` and BOLT 11 invoice contexts, the features as set in the BOLT 11 invoice should override those set in the `node_announcement`. This keeps things consistent with the unknown features behavior as specified in BOLT 7.

The origin must set all transitive feature dependencies in order to create a well-formed feature vector. By validating all known dependencies up front, this simplifies logic gated on a single feature bit; the feature's dependencies are known to be set, and do not need to be validated at every feature gate.

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

Opening and Running Channels

A Lightning channel is a Bitcoin output that two people share, plus a set of rules for updating who owns how much without going back to the chain every time.

BOLT 02 is the peer protocol for that. It covers the whole channel lifecycle: opening (`open_channel` / `accept_channel` / `funding_created` / `funding_signed`), going live (`channel_ready`), sending payments with HTLCs (`update_add_htlc` and friends), committing a new state (`commitment_signed` / `revoke_and_ack`), and cooperative close. If you're implementing a node, you'll live in this document.

BOLT 03 is the Bitcoin underneath. It specifies the funding transaction, the commitment transactions, HTLC-timeout and HTLC-success transactions, and the scripts and signature formats that make the penalty model work. BOLT 02 is the conversation; BOLT 03 is the money.

You can read BOLT 02 first for the flow, then BOLT 03 when you need to know exactly what gets signed. Or keep them side by side. They were meant to be a pair.

In this chapter

- BOLT 02 — Peer protocol for channel management

- BOLT 03 — Bitcoin transactions and scripts

BOLT #2: Peer Protocol for Channel Management

The peer channel protocol has three phases: establishment, normal operation, and closing.

Table of Contents

- Channel
  - Definition of `channel_id`
  - Interactive Transaction Construction
    - Set-Up and Vocabulary
    - Fee Responsibility
    - Overview
    - The `tx_add_input` Message
    - The `tx_add_output` Message
    - The `tx_remove_input` and `tx_remove_output` Messages
    - The `tx_complete` Message
    - The `tx_signatures` Message
    - The `tx_init_rbf` Message
    - The `tx_ack_rbf` Message
    - The `tx_abort` Message
  - Channel Establishment v1
    - The `open_channel` Message
    - The `accept_channel` Message
    - The `funding_created` Message
    - The `funding_signed` Message
    - The `channel_ready` Message
  - Channel Establishment v2
    - The `open_channel2` Message
    - The `accept_channel2` Message
    - Funding Composition
    - The `commitment_signed` Message
    - Sharing funding signatures: `tx_signatures`
    - Fee bumping: `tx_init_rbf` and `tx_ack_rbf`
  - Channel Quiescence
  - Channel Splicing
    - The `splice_init` Message
    - The `splice_ack` Message
    - Splice Transaction Construction
    - Splice Completion
  - Channel Close
    - Closing Initiation: `shutdown`
    - Closing Negotiation: `closing_complete` and `closing_sig`
    - Legacy Closing Negotiation: `closing_signed`
  - Normal Operation
    - Forwarding HTLCs

        - cltv_expiry_delta Selection
        - Adding an HTLC: update_add_htlc
        - Removing an HTLC: update_fulfill_htlc, update_fail_htlc, and update_fail_malformed_htlc
        - Batching channel messages
        - Committing Updates So Far: commitment_signed
        - Completing the Transition to the Updated State: revoke_and_ack
        - Updating Fees: update_fee
    - Message Retransmission: channel_reestablish message
- Authors

Channel

Definition of `channel_id`

Some messages use a `channel_id` to identify the channel. It's derived from the funding transaction by combining the `funding_txid` and the `funding_output_index`, using big-endian exclusive-OR (i.e. `funding_output_index` alters the last 2 bytes).

Prior to channel establishment, a `temporary_channel_id` is used, which is a random nonce.

Note that as duplicate `temporary_channel_id`s may exist from different peers, APIs which reference channels by their channel id before the funding transaction is created are inherently unsafe. The only protocol-provided identifier for a channel before funding_created has been exchanged is the (source_node_id, destination_node_id, temporary_channel_id) tuple. Note that any such APIs which reference channels by their channel id before the funding transaction is confirmed are also not persistent - until you know the script pubkey corresponding to the funding output nothing prevents duplicative channel ids.

`channel_id`, v2

For channels established using the v2 protocol, the `channel_id` is the `SHA256(lesser-revocation-basepoint || greater-revocation-basepoint)`, where the lesser and greater is based off the order of the basepoint.

When sending `open_channel2`, the peer's revocation basepoint is unknown. A `temporary_channel_id` must be computed by using a zeroed out basepoint for the non-initiator.

When sending `accept_channel2`, the `temporary_channel_id` from `open_channel2` must be used, to allow the initiator to match the response to its request.

Rationale

The revocation basepoints must be remembered by both peers for correct operation anyway. They're known after the first exchange of messages, obviating the need for a `temporary_channel_id` in subsequent messages. By mixing information from both sides, they avoid `channel_id` collisions, and they remove the dependency on the funding txid.

Interactive Transaction Construction

Interactive transaction construction allows two peers to collaboratively build a transaction for broadcast. This protocol is the foundation for dual-funded channels establishment (v2).

Set-Up and Vocabulary

There are two parties to a transaction construction: an *initiator* and a *non-initiator*. The *initiator* is the peer which initiates the protocol, e.g. for channel establishment v2 the *initiator* would be the peer which sends `open_channel2`.

The protocol makes the following assumptions:

- The `feerate` for the transaction is known.
- The `dust_limit` for the transaction is known.
- The `nLocktime` for the transaction is known.
- The `nVersion` for the transaction is known.

Fee Responsibility

The *initiator* is responsible for paying the fees for the following fields, to be referred to as the `common fields`.

- version
- segwit marker + flag
- input count
- output count
- locktime

The rest of the transaction bytes' fees are the responsibility of the peer who contributed that input or output via `tx_add_input` or `tx_add_output`, at the agreed upon `feerate`.

Overview

The *initiator* initiates the interactive transaction construction protocol with `tx_add_input`. The *non-initiator* responds with any of `tx_add_input`, `tx_add_output`, `tx_remove_input`, `tx_remove_output`, or `tx_complete`. The protocol continues with the synchronous exchange of interactive transaction protocol messages until both nodes have sent and received a consecutive `tx_complete`. This is a turn-based protocol.

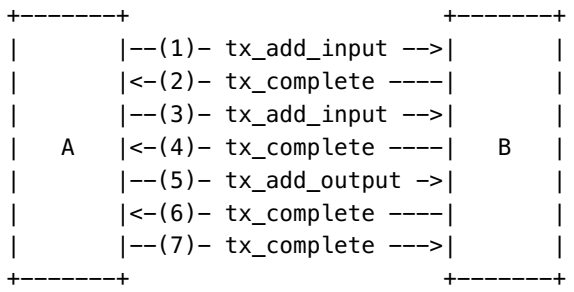
Once peers have exchanged consecutive `tx_completes`, the interactive transaction construction protocol is considered concluded. Both peers should construct the transaction and fail the negotiation if an error is discovered.

This protocol is expressly designed to allow for parallel, multi-party sessions to collectively construct a single transaction. This preserves the ability to open multiple channels in a single transaction. While `serial_ids` are generally chosen randomly, to maintain consistent transaction ordering across all peer sessions, it is simplest to reuse received `serial_ids` when forwarding them to other peers, inverting the bottom bit as necessary to satisfy the parity requirement.

Here are a few example exchanges.

***initiator* only**

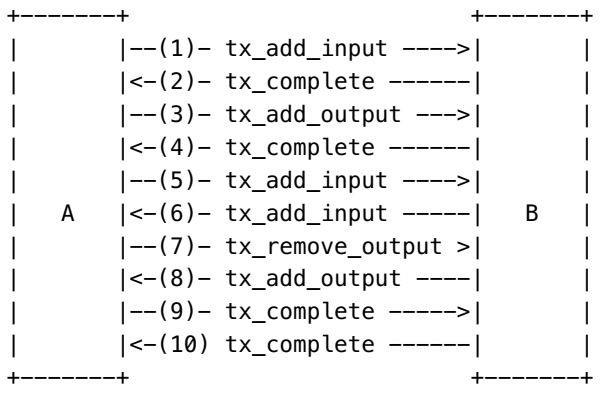
A, the *initiator*, has two inputs and an output (the funding output). B, the *non-initiator* has nothing to contribute.



initiator* and *non-initiator

A, the *initiator*, contributes 2 inputs and an output that they then remove. B, the *non-initiator*, contributes 1 input and an output, but waits until A adds a second input before contributing.

Note that if A does not send a second input, the negotiation will end without B's contributions.



The `tx_add_input` Message

This message contains a transaction input.

1. type: 66 (`tx_add_input`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u64`:`serial_id`]
    - [`u16`:`prevtx_len`]
    - [`prevtx_len*byte`:`prevtx`]
    - [`u32`:`prevtx_vout`]
    - [`u32`:`sequence`]
    - [`tx_add_input_tlvs`:`tlvs`]
3. `tlv_stream`: `tx_add_input_tlvs`
4. types:
    1. type: 0 (`shared_input_txid`)

2. data:
   - [sha256:funding_txid]

Requirements

The sending node: - MUST add all sent inputs to the transaction - MUST use a unique `serial_id` for each input currently added to the transaction - MUST set `sequence` to be less than or equal to 4294967293 (`0xFFFFFFFD`) - MUST NOT re-transmit inputs it has received from the peer - if is the *initiator*: - MUST send even `serial_id`s - if is the *non-initiator*: - MUST send odd `serial_id`s

The receiving node: - MUST add all received inputs to the transaction - MUST fail the negotiation if: - `sequence` is set to `0xFFFFFFFE` or `0xFFFFFFFF` - if `prevtx_len` is `0`: - `shared_input_txid` is not set - `shared_input_txid` and `prevtx_vout` don't match the previous funding output - a previously added (and not removed) input already exists with `shared_input_txid` set - if `prevtx_len` is not `0`: - `prevtx` and `prevtx_vout` are identical to a previously added (and not removed) input - `prevtx` is not a valid transaction - `prevtx_vout` is greater or equal to the number of outputs on `prevtx` - the `scriptPubKey` of the `prevtx_vout` output of `prevtx` is not exactly a 1-byte push opcode (for the numeric values `0` to `16`) followed by a data push between 2 and 40 bytes - the `serial_id` is already included in the transaction - the `serial_id` has the wrong parity - if has received 4096 `tx_add_input` messages during this negotiation

Rationale

Each node must know the set of the transaction inputs. The *non-initiator* MAY omit this message.

`serial_id` is a randomly chosen number which uniquely identifies this input. Inputs in the constructed transaction MUST be sorted by `serial_id`.

`prevtx` is the serialized transaction that contains the output this input spends, used to verify that the input is non-malleable. It can be ommitted (`prevtx_len` set to `0`) when both peers already know that the input is non-malleable (e.g. when it is the previous funding output).

`prevtx_vout` is the index of the output being spent.

`sequence` is the sequence number of this input: it must signal replaceability, and the same value should be used across implementations to avoid on-chain fingerprinting.

Liquidity griefing

When sending `tx_add_input`, senders have no guarantee that the remote node will complete the protocol in a timely manner. Malicious remote nodes could delay messages or stop responding, which can result in a partially created transaction that cannot be broadcast by the honest node. If the honest node is locking the corresponding UTXO exclusively for this remote node, this can be exploited to lock up the honest node's liquidity.

It is thus recommended that implementations keep UTXOs unlocked and actively reuse them in concurrent sessions, which guarantees that transactions created with honest nodes double-spend pending transactions with malicious nodes at no additional cost for the honest node.

Unfortunately, this will also create conflicts between concurrent sessions with honest nodes. This is a reasonable trade-off though because:

- on-chain funding attempts are relatively infrequent operations
- honest nodes should complete the protocol quickly, reducing the risk of conflicts
- failed attempts can simply be retried at no cost

The `tx_add_output` Message

This message adds a transaction output.

1. type: 67 (`tx_add_output`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u64`:`serial_id`]
    - [`u64`:`sats`]
    - [`u16`:`scriptlen`]
    - [`scriptlen*byte`:`script`]

Requirements

Either node: - MAY omit this message

The sending node: - MUST add all sent outputs to the transaction - if is the *initiator*: - MUST send even `serial_id`s - if is the *non-initiator*: - MUST send odd `serial_id`s

The receiving node: - MUST add all received outputs to the transaction - MUST accept P2WSH, P2WPKH, P2TR `script`s - MAY fail the negotiation if `script` is non-standard - MUST fail the negotiation if: - the `serial_id` is already included in the transaction - the `serial_id` has the wrong parity - it has received 4096 `tx_add_output` messages during this negotiation - the `sats` amount is less than the `dust_limit` - the `sats` amount is greater than 2,100,000,000,000,000 (`MAX_MONEY`)

Rationale

Each node must know the set of the transaction outputs.

`serial_id` is a randomly chosen number which uniquely identifies this output. Outputs in the constructed transaction MUST be sorted by `serial_id`.

`sats` is the satoshi value of the output.

`script` is the scriptPubKey for the output (with its length omitted). `script`s are not required to follow standardness rules: non-standard scripts such as `OP_RETURN` may be accepted, but the corresponding transaction may fail to relay across the network.

The `tx_remove_input` and `tx_remove_output` Messages

This message removes an input from the transaction.

1. type: 68 (`tx_remove_input`)

2. data:
    - [channel_id:channel_id]
    - [u64:serial_id]

This message removes an output from the transaction.

1. type: 69 (tx_remove_output)
2. data:
    - [channel_id:channel_id]
    - [u64:serial_id]

Requirements

The sending node: - MUST NOT send a tx_remove with a serial_id it did not add to the transaction or has already been removed

The receiving node: - MUST remove the indicated input or output from the transaction - MUST fail the negotiation if: - the input or output identified by the serial_id was not added by the sender - the serial_id does not correspond to a currently added input (or output)

The tx_complete Message

This message signals the conclusion of a peer's transaction contributions.

1. type: 70 (tx_complete)
2. data:
    - [channel_id:channel_id]

Requirements

The nodes: - MUST send this message in succession to conclude this protocol

The receiving node: - MUST use the negotiated inputs and outputs to construct a transaction - MUST fail the negotiation if: - the peer's total input satoshis is less than their outputs. One MUST account for the peer's portion of the funding output when verifying compliance with this requirement. - the peer's paid feerate does not meet or exceed the agreed feerate (based on the minimum fee). - if is the *non-initiator*: - the *initiator*'s fees do not cover the common fields - there are more than 252 inputs - there are more than 252 outputs - the estimated weight of the tx is greater than 400,000 (MAX_STANDARD_TX_WEIGHT)

Rationale

To signal the conclusion of exchange of transaction inputs and outputs.

Upon successful exchange of tx_complete messages, both nodes should construct the transaction and proceed to the next portion of the protocol. For channel establishment v2, exchanging commitment transactions.

For the minimum fee calculation see BOLT #3.

The maximum inputs and outputs are capped at 252. This effectively fixes the byte size of the input and output counts on the transaction to one (1).

The `tx_signatures` Message

1. type: 71 (`tx_signatures`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`sha256`:`txid`]
    - [`u16`:`num_witnesses`]
    - [`num_witnesses*witness`:`witnesses`]
    - [`tx_signatures_tlvs`:`tlvs`]
3. subtype: `witness`
4. data:
    - [`u16`:`len`]
    - [`len*byte`:`witness_data`]
5. `tlv_stream`: `tx_signatures_tlvs`
6. types:
    1. type: 0 (`shared_input_signature`)
    2. data:
    - [`signature`:`signature`]

Requirements

The sending node: - if it has the lowest total satoshis contributed, as defined by total `tx_add_input` values, or both peers have contributed equal amounts but it has the lowest `node_id` (sorted lexicographically): - MUST transmit their `tx_signatures` first - MUST order the `witnesses` by the `serial_id` of the input they correspond to - `num_witnessess` MUST equal the number of inputs they added - MUST use the `SIGHASH_ALL` (0x01) flag on each signature

The receiving node: - MUST fail the negotiation if: - the message contains an empty `witness` - the number of `witnesses` does not equal the number of inputs added by the sending node - the `txid` does not match the txid of the transaction - the `witnesses` are non-standard - a signature uses a flag that is not `SIGHASH_ALL` (0x01) - SHOULD apply the `witnesses` to the transaction and broadcast it - MUST reply with their `tx_signatures` if not already transmitted

Rationale

A strict ordering is used to decide which peer sends `tx_signatures` first. This prevents deadlocks where each peer is waiting for the other peer to send `tx_signatures`, and enables multiparty tx collaboration.

The `witness_data` is encoded as per bitcoin's wire protocol (a CompactSize number of elements, with each element a CompactSize length and that many bytes following).

While the `minimum fee` is calculated and verified at `tx_complete` conclusion, it is possible for the fee for the exchanged witness data to be underpaid. It is the responsibility of the sending peer to correctly account for the required fee.

The `tx_init_rbf` Message

This message initiates a replacement of the transaction after it's been completed.

1. type: 72 (`tx_init_rbf`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u32`:`locktime`]
    - [`u32`:`feerate`]
    - [`tx_init_rbf_tlvs`:`tlvs`]
3. `tlv_stream`: `tx_init_rbf_tlvs`
4. types:
    1. type: 0 (`funding_output_contribution`)
    2. data:
        - [`s64`:`satoshis`]
    3. type: 2 (`require_confirmed_inputs`)

Requirements

The sender: - MUST set `feerate` greater than or equal to the maximum of: - 25/24 times the `feerate` of the previously constructed transaction, rounded down. - 25 sat per kw greater than the `feerate` of the previously constructed transaction. - If it contributes to the transaction's funding output: - MUST set `funding_output_contribution` - If it requires the receiving node to only use confirmed inputs: - MUST set `require_confirmed_inputs` - If it contributed to previous transactions: - MUST ensure that the new transaction double-spends all other attempts, by sending `tx_add_input` with at least one input from each previous transaction construction attempt.

The recipient: - MUST respond either with `tx_abort` or with `tx_ack_rbf` - MUST respond with `tx_abort` if: - the `feerate` is not greater than or equal to the maximum of: - 25/24 times the `feerate` of the last successfully constructed transaction, rounded down. - 25 sat per kw greater than the `feerate` of the last successfully constructed transaction - MAY send `tx_abort` for any reason - MUST fail the negotiation if: - `require_confirmed_inputs` is set but it cannot provide confirmed inputs

Rationale

`feerate` is the feerate this transaction will pay. It must be at least the maximum of 25/24 times the previous `feerate` (rounded down) and 25 sat per kw above the previous `feerate`. The multiplicative rule ensures progress at higher feerates, while the additive floor of 25 sat per kw (0.1 sat/vB) ensures that the replacement transaction meets BIP125's minimum relay fee requirement (default `incrementalRelayFee` of 0.1 sat/vB since Bitcoin Core v30.0), which the multiplicative increase alone could fail to satisfy at low feerates.

E.g. if the last `feerate` was 520, the next sent `feerate` must be at least 545 (max(520 * 25 / 24, 520 + 25) = max(541, 545) = 545).

If the previous transaction confirms in the middle of an RBF attempt, the attempt MUST be abandoned.

`funding_output_contribution` is the amount of satoshis that this peer will contribute to the funding output of the transaction, when there is such an output. Note that it may be different from the contribution made in the previously completed transaction. If omitted, the sender is not contributing to the funding output.

The `tx_ack_rbf` Message

1. type: 73 (`tx_ack_rbf`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`tx_ack_rbf_tlvs`:`tlvs`]
3. `tlv_stream`: `tx_ack_rbf_tlvs`
4. types:
    1. type: 0 (`funding_output_contribution`)
    2. data:
        - [`s64`:`satoshis`]
    3. type: 2 (`require_confirmed_inputs`)

Requirements

The sender: - If it contributes to the transaction's funding output: - MUST set `funding_output_contribution` - If it requires the receiving node to only use confirmed inputs: - MUST set `require_confirmed_inputs` - If it contributed to previous transactions: - MUST ensure that the new transaction double-spends all other attempts, by sending `tx_add_input` with at least one input from each previous transaction construction attempt.

The recipient: - MUST respond with `tx_abort` or with a `tx_add_input` message, restarting the interactive tx collaboration protocol. - MUST fail the negotiation if: - `require_confirmed_inputs` is set but it cannot provide confirmed inputs

Rationale

`funding_output_contribution` is the amount of satoshis that this peer will contribute to the funding output of the transaction, when there is such an output. Note that it may be different from the contribution made in the previously completed transaction. If omitted, the sender is not contributing to the funding output.

It's recommended that a peer, rather than fail the RBF negotiation due to a large feerate change, instead stop contributing to the funding output, and decline to participate further in the transaction (by not contributing, they may obtain incoming liquidity at no cost).

The `tx_abort` Message

1. type: 74 (`tx_abort`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u16`:`len`]
    - [`len*byte`:`data`]

Requirements

A sending node: - MUST NOT have already transmitted `tx_signatures` - SHOULD forget the current negotiation and reset their state. - MAY send an empty `data` field. - when failure was caused by an invalid signature check: - SHOULD include the raw, hex-encoded transaction in reply to a `tx_signatures` or `commitment_signed` message.

A receiving node: - if they have already sent `tx_signatures` to the peer: - MUST NOT forget the channel until any inputs to the negotiated tx have been spent. - if they have not sent `tx_signatures`: - SHOULD forget the current negotiation and reset their state. - if they have not sent `tx_abort`: - MUST echo back `tx_abort` - if `data` is not composed solely of printable ASCII characters (For reference: the printable character set includes byte values 32 through 126, inclusive): - SHOULD NOT print out `data` verbatim.

Rationale

A receiving node, if they've already sent their `tx_signatures` has no guarantee that the transaction won't be signed and published by their peer. They must remember the transaction and channel (if appropriate) until the transaction is no longer eligible to be spent (i.e. any input has been spent in a different transaction).

The `tx_abort` message allows for the cancellation of an in progress negotiation, and a return to the initial starting state. It is distinct from the `error` message, which triggers a channel close.

Echoing back `tx_abort` allows the peer to ack that they've seen the abort message, permitting the originating peer to terminate the in-flight process without worrying about stale messages.

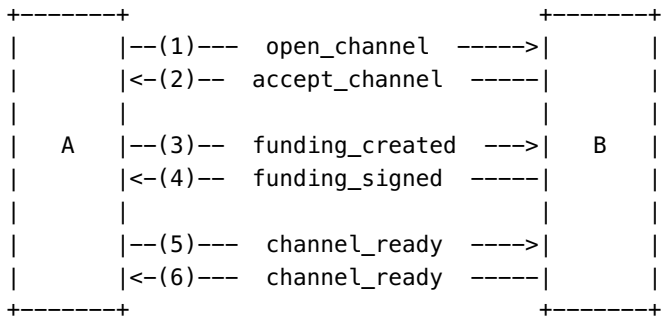
Channel Establishment v1

After authenticating and initializing a connection (BOLT #8 and BOLT #1, respectively), channel establishment may begin.

There are two pathways for establishing a channel, a legacy version presented here, and a second version (below). Which channel establishment protocols are available for use is negotiated in the `init` message.

This consists of the funding node (funder) sending an `open_channel` message, followed by the responding node (fundee) sending `accept_channel`. With the channel parameters locked in, the funder is able to create the funding transaction and both versions of the commitment transaction, as described in BOLT #3. The funder then sends the outpoint of the funding output with the `funding_created` message, along with the signature for the fundee's version of the commitment transaction. Once the fundee learns the funding outpoint, it's able to generate the signature for the funder's version of the commitment transaction and send it over using the `funding_signed` message.

Once the channel funder receives the `funding_signed` message, it must broadcast the funding transaction to the Bitcoin network. After the `funding_signed` message is sent/received, both sides should wait for the funding transaction to enter the blockchain and reach the specified depth (number of confirmations). After both sides have sent the `channel_ready` message, the channel is established and can begin normal operation. The `channel_ready` message includes information that will be used to construct channel authentication proofs.



- where node A is 'funder' and node B is 'fundee'

If this fails at any stage, or if one node decides the channel terms offered by the other node are not suitable, the channel establishment fails.

Note that multiple channels can operate in parallel, as all channel messages are identified by either a `temporary_channel_id` (before the funding transaction is created) or a `channel_id` (derived from the funding transaction).

The `open_channel` Message

This message contains information about a node and indicates its desire to set up a new channel. This is the first step toward creating the funding transaction and both versions of the commitment transaction.

1. type: 32 (`open_channel`)
2. data:
    - [`chain_hash`:`chain_hash`]
    - [`32*byte`:`temporary_channel_id`]
    - [`u64`:`funding_satoshis`]
    - [`u64`:`push_msat`]
    - [`u64`:`dust_limit_satoshis`]
    - [`u64`:`max_htlc_value_in_flight_msat`]
    - [`u64`:`channel_reserve_satoshis`]
    - [`u64`:`htlc_minimum_msat`]
    - [`u32`:`feerate_per_kw`]
    - [`u16`:`to_self_delay`]
    - [`u16`:`max_accepted_htlcs`]
    - [`point`:`funding_pubkey`]
    - [`point`:`revocation_basepoint`]
    - [`point`:`payment_basepoint`]
    - [`point`:`delayed_payment_basepoint`]
    - [`point`:`htlc_basepoint`]
    - [`point`:`first_per_commitment_point`]
    - [`byte`:`channel_flags`]
    - [`open_channel_tlvs`:`tlvs`]
3. `tlv_stream`: `open_channel_tlvs`
4. types:
    1. type: 0 (`upfront_shutdown_script`)
    2. data:
        - [`...*byte`:`shutdown_scriptpubkey`]
    3. type: 1 (`channel_type`)
    4. data:
        - [`...*byte`:`type`]

The `chain_hash` value denotes the exact blockchain that the opened channel will reside within. This is usually the genesis hash of the respective blockchain. The existence of the `chain_hash` allows nodes to open channels

across many distinct blockchains as well as have channels within multiple blockchains opened to the same peer (if it supports the target chains).

The `temporary_channel_id` is used to identify this channel on a per-peer basis until the funding transaction is established, at which point it is replaced by the `channel_id`, which is derived from the funding transaction.

`funding_satoshis` is the amount the sender is putting into the channel. `push_msat` is an amount of initial funds that the sender is unconditionally giving to the receiver. `dust_limit_satoshis` is the threshold below which outputs should not be generated for this node's commitment or HTLC transactions (i.e. HTLCs below this amount plus HTLC transaction fees are not enforceable on-chain). This reflects the reality that tiny outputs are not considered standard transactions and will not propagate through the Bitcoin network. `channel_reserve_satoshis` is the minimum amount that the other node is to keep as a direct payment. `htlc_minimum_msat` indicates the smallest value HTLC this node will accept.

`max_htlc_value_in_flight_msat` is a cap on total value of outstanding HTLCs offered by the remote node, which allows the local node to limit its exposure to HTLCs; similarly, `max_accepted_htlcs` limits the number of outstanding HTLCs the remote node can offer.

`feerate_per_kw` indicates the initial fee rate in satoshi per 1000-weight (i.e. 1/4 the more normally-used 'satoshi per 1000 vbytes') that this side will pay for commitment and HTLC transactions, as described in BOLT #3 (this can be adjusted later with an `update_fee` message).

`to_self_delay` is the number of blocks that the other node's to-self outputs must be delayed, using `OP_CHECKSEQUENCEVERIFY` delays; this is how long it will have to wait in case of breakdown before redeeming its own funds.

`funding_pubkey` is the public key in the 2-of-2 multisig script of the funding transaction output.

The various `_basepoint` fields are used to derive unique keys as described in BOLT #3 for each commitment transaction. Varying these keys ensures that the transaction ID of each commitment transaction is unpredictable to an external observer, even if one commitment transaction is seen; this property is very useful for preserving privacy when outsourcing penalty transactions to third parties.

`first_per_commitment_point` is the per-commitment point to be used for the first commitment transaction,

Only the least-significant bit of `channel_flags` is currently defined: `announce_channel`. This indicates whether the initiator of the funding flow wishes to advertise this channel publicly to the network, as detailed within BOLT #7.

The `shutdown_scriptpubkey` allows the sending node to commit to where funds will go on mutual close, which the remote node should enforce even if a node is compromised later.

The `option_support_large_channel` is a feature used to let everyone know this node will accept `funding_satoshis` greater than or equal to 2^24. Since it's broadcast in the `node_announcement` message other nodes can use it to identify peers willing to accept large channel even before exchanging the `init` message with them.

Defined Channel Types

Channel types are an explicit enumeration: for convenience of future definitions they reuse even feature bits, but they are not an arbitrary combination (they represent the persistent features which affect the channel operation).

The currently defined basic types are: - `option_static_remotekey` (bit 12) - `option_anchors` and `option_static_remotekey` (bits 22 and 12) - `zero_fee_commitments` (bit 40)

Each basic type has the following variations allowed: - `option_scid_alias` (bit 46) - `option_zeroconf` (bit 50)

Requirements

The sending node: - MUST ensure the `chain_hash` value identifies the chain it wishes to open the channel within. - MUST ensure `temporary_channel_id` is unique from any other channel ID with the same peer. - if both nodes advertised `option_support_large_channel`: - MAY set `funding_satoshis` greater than or equal to 2^24 satoshi. - otherwise: - MUST set `funding_satoshis` to less than 2^24 satoshi. - MUST set `push_msat` to equal or less than 1000 * `funding_satoshis`. - MUST set `funding_pubkey`, `revocation_basepoint`, `htlc_basepoint`, `payment_basepoint`, and `delayed_payment_basepoint` to valid secp256k1 pubkeys in compressed format. - MUST set `first_per_commitment_point` to the per-commitment point to be used for the initial commitment transaction, derived as specified in BOLT #3. - MUST set `channel_reserve_satoshis` greater than or equal to `dust_limit_satoshis`. - MUST set undefined bits in `channel_flags` to 0. - if both nodes advertised the `option_upfront_shutdown_script` feature: - MUST include `upfront_shutdown_script` with either a valid `shutdown_scriptpubkey` as required by `shutdown` `scriptpubkey`, or a zero-length `shutdown_scriptpubkey` (ie. `0x0000`). - otherwise: - MAY include `upfront_shutdown_script`. - if it includes `open_channel_tlvs`: - MUST include `upfront_shutdown_script`. - MUST set `channel_type`: - MUST set it to a defined type representing the type it wants. - MUST use the smallest bitmap possible to represent the channel type. - SHOULD NOT set it to a type containing a feature which was not negotiated. - if `channel_type` includes `zero_fee_commitments`: - MUST set `feerate_per_kw` to `0`. - if `announce_channel` is `true` (not `0`): - MUST NOT send `channel_type` with the `option_scid_alias` bit set.

The sending node SHOULD: - set `to_self_delay` sufficient to ensure the sender can irreversibly spend a commitment transaction output, in case of misbehavior by the receiver. - set `dust_limit_satoshis` to a sufficient value to allow commitment transactions to propagate through the Bitcoin network. - set `htlc_minimum_msat` to the minimum value HTLC it's willing to accept from this peer. - when using `zero_fee_commitments` or `option_anchors`: - set `dust_limit_satoshis` to a sufficient value to ensure that it is economical to spend, taking the cost of HTLC transactions into account. - otherwise (not using `zero_fee_commitments` or `option_anchors`): - set `feerate_per_kw` to at least the rate it estimates would cause the transaction to be immediately included in a block.

The receiving node MUST: - ignore undefined bits in `channel_flags`. - if the message doesn't include a `channel_type`: - fail the channel. - if the connection has been re-established after receiving a previous `open_channel`, BUT before receiving a `funding_created` message: - accept a new `open_channel` message. - discard the previous `open_channel` message. - if `option_dual_fund` has been negotiated: - fail the channel.

The receiving node MAY fail the channel if: - `announce_channel` is `false` (`0`), yet it wishes to publicly announce the channel. - `funding_satoshis` is too small. - it considers `htlc_minimum_msat` too large. - it considers `max_htlc_value_in_flight_msat` too small. - it considers `channel_reserve_satoshis` too large. - it considers `max_accepted_htlcs` too small. - it considers `dust_limit_satoshis` too large (see BOLT 3).

The receiving node MUST fail the channel if: - the `chain_hash` value is set to a hash of a chain that is unknown to the receiver. - `push_msat` is greater than `funding_satoshis` * 1000. - `to_self_delay` is unreasonably large. - `channel_type` includes `zero_fee_commitments` and: - `max_accepted_htlcs` is greater than 114. - `feerate_per_kw` is not `0`. - `channel_type` does not include `zero_fee_commitments` and: - `max_accepted_htlcs` is greater than 483. - it considers `feerate_per_kw` too small for timely processing or unreasonably large. - `funding_pubkey`, `revocation_basepoint`, `htlc_basepoint`, `payment_basepoint`, or `delayed_payment_basepoint` are not valid secp256k1 pubkeys in compressed format. - `dust_limit_satoshis` is greater than `channel_reserve_satoshis`. - `dust_limit_satoshis` is smaller than `354 satoshis` (see BOLT 3). - the funder's amount for the initial commitment transaction is not sufficient for full fee payment. - both `to_local` and `to_remote` amounts for the initial commitment transaction are less than or equal to `channel_reserve_satoshis` (see BOLT 3). - `funding_satoshis` is greater than or equal to 2^24 and the receiver does not support `option_support_large_channel`. - the `channel_type` is not suitable. - the `channel_type` includes `option_zeroconf` and it does not trust the sender to open an unconfirmed channel.

The receiving node MUST NOT: - consider funds received, using `push_msat`, to be received until the funding transaction has reached sufficient depth.

Rationale

The requirement for `funding_satoshis` to be less than 2^24 satoshi was a temporary self-imposed limit while implementations were not yet considered stable, it can be lifted by advertising `option_support_large_channel`.

The *channel reserve* is specified by the peer's `channel_reserve_satoshis`: 1% of the channel total is suggested. Each side of a channel maintains this reserve so it always has something to lose if it were to try to broadcast an old, revoked commitment transaction. Initially, this reserve may not be met, as only one side has funds; but the protocol ensures that there is always progress toward meeting this reserve, and once met, it is maintained.

The sender can unconditionally give initial funds to the receiver using a non-zero `push_msat`, but even in this case we ensure that the funder has sufficient remaining funds to pay fees and that one side has some amount it can spend (which also implies there is at least one non-dust output). Note that, like any other on-chain transaction, this payment is not certain until the funding transaction has been confirmed sufficiently (with a danger of double-spend until this occurs) and may require a separate method to prove payment via on-chain confirmation.

The `feerate_per_kw` is generally only of concern to the sender (who pays the fees), but there is also the fee rate paid by HTLC transactions; thus, unreasonably large fee rates can also penalize the recipient.

Separating the `htlc_basepoint` from the `payment_basepoint` improves security: a node needs the secret associated with the `htlc_basepoint` to produce HTLC signatures for the protocol, but the secret for the `payment_basepoint` can be in cold storage.

The requirement that `channel_reserve_satoshis` is not considered dust according to `dust_limit_satoshis` eliminates cases where all outputs would be eliminated as dust. The similar requirements in `accept_channel` ensure that both sides' `channel_reserve_satoshis` are above both `dust_limit_satoshis`.

The receiver should not accept large `dust_limit_satoshis`, as this could be used in griefing attacks, where the peer publishes its commitment with a lot of dust htlcs, which effectively become miner fees. But it must allow

values higher than the standard Bitcoin Core dust limits, since HTLC outputs need to be spent by a second-stage transaction at a feerate matching the current on-chain feerate.

Details for how to handle a channel failure can be found in BOLT 5:Failing a Channel.

The `accept_channel` Message

This message contains information about a node and indicates its acceptance of the new channel. This is the second step toward creating the funding transaction and both versions of the commitment transaction.

1. type: 33 (`accept_channel`)
2. data:
    - [`32*byte:temporary_channel_id`]
    - [`u64:dust_limit_satoshis`]
    - [`u64:max_htlc_value_in_flight_msat`]
    - [`u64:channel_reserve_satoshis`]
    - [`u64:htlc_minimum_msat`]
    - [`u32:minimum_depth`]
    - [`u16:to_self_delay`]
    - [`u16:max_accepted_htlcs`]
    - [`point:funding_pubkey`]
    - [`point:revocation_basepoint`]
    - [`point:payment_basepoint`]
    - [`point:delayed_payment_basepoint`]
    - [`point:htlc_basepoint`]
    - [`point:first_per_commitment_point`]
    - [`accept_channel_tlvs:tlvs`]
3. `tlv_stream`: `accept_channel_tlvs`
4. types:
    1. type: 0 (`upfront_shutdown_script`)
    2. data:
        - [`...*byte:shutdown_scriptpubkey`]
    3. type: 1 (`channel_type`)
    4. data:
        - [`...*byte:type`]

Requirements

The `temporary_channel_id` MUST be the same as the `temporary_channel_id` in the `open_channel` message.

The sender: - if `channel_type` includes `option_zeroconf`: - MUST set `minimum_depth` to zero. - otherwise: - SHOULD set `minimum_depth` to a number of blocks it considers reasonable to avoid double-spending of the funding transaction. - MUST set `channel_reserve_satoshis` greater than or equal to `dust_limit_satoshis` from the `open_channel` message. - MUST set `dust_limit_satoshis` less than or equal to `channel_reserve_satoshis` from the `open_channel` message. - MUST set `channel_type` to the `channel_type` from `open_channel`.

The receiver: - if `minimum_depth` is unreasonably large: - MAY fail the channel. - if `channel_reserve_satoshis` is less than `dust_limit_satoshis` within the `open_channel` message: - MUST fail the channel. - if `channel_reserve_satoshis` from the `open_channel` message is less than `dust_limit_satoshis`: - MUST fail the channel. - if the message doesn't include a `channel_type`: - MUST fail the channel. - if `channel_type` does not match the `channel_type` from `open_channel`: - MUST fail the channel.

Other fields have the same requirements as their counterparts in `open_channel`.

The `funding_created` Message

This message describes the outpoint which the funder has created for the initial commitment transactions. After receiving the peer's signature, via `funding_signed`, it will broadcast the funding transaction.

1. type: 34 (`funding_created`)
2. data:
    - [`32*byte:temporary_channel_id`]
    - [`sha256:funding_txid`]
    - [`u16:funding_output_index`]
    - [`signature:signature`]

Requirements

The sender MUST set: - `temporary_channel_id` the same as the `temporary_channel_id` in the `open_channel` message. - `funding_txid` to the transaction ID of a non-malleable transaction, - and MUST NOT broadcast this transaction. - `funding_output_index` to the output number of that transaction that corresponds the funding transaction output, as defined in BOLT #3. - `signature` to the valid signature using its `funding_pubkey` for the initial commitment transaction, as defined in BOLT #3.

The sender: - when creating the funding transaction: - SHOULD use only BIP141 (Segregated Witness) inputs. - SHOULD ensure the funding transaction confirms in the next 2016 blocks.

The recipient: - if `signature` is incorrect OR non-compliant with LOW-S-standard rule[LOWS]: - MUST send a `warning` and close the connection, or send an `error` and fail the channel.

Rationale

The `funding_output_index` can only be 2 bytes, since that's how it's packed into the `channel_id` and used throughout the gossip protocol. The limit of 65535 outputs should not be overly burdensome.

A transaction with all Segregated Witness inputs is not malleable, hence the funding transaction recommendation.

The funder may use CPFP on a change output to ensure that the funding transaction confirms before 2016 blocks, otherwise the fundee may forget that channel.

The `funding_signed` Message

This message gives the funder the signature it needs for the first commitment transaction, so it can broadcast the transaction knowing that funds can be redeemed, if need be.

This message introduces the `channel_id` to identify the channel. It's derived from the funding transaction by combining the `funding_txid` and the `funding_output_index`, using big-endian exclusive-OR (i.e. `funding_output_index` alters the last 2 bytes).

1. type: 35 (`funding_signed`)
2. data:
    - [`channel_id:channel_id`]
    - [`signature:signature`]

Requirements

Both peers: - MUST use the negotiated `channel_type` for all commitment transactions.

The sender MUST set: - `channel_id` by exclusive-OR of the `funding_txid` and the `funding_output_index` from the `funding_created` message. - `signature` to the valid signature, using its `funding_pubkey` for the initial commitment transaction, as defined in BOLT #3.

The recipient: - if `signature` is incorrect OR non-compliant with LOW-S-standard rule[LOWS]: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - MUST NOT broadcast the funding transaction before receipt of a valid `funding_signed`. - on receipt of a valid `funding_signed`: - SHOULD broadcast the funding transaction.

Rationale

We generate the commitment transaction at this point, using the `channel_type` that was communicated in the `open_channel` and `accept_channel` messages. This `channel_type` determines the channel commitment format for the total lifetime of the channel.

The `channel_ready` Message

This message (which used to be called `funding_locked`) indicates that the funding transaction has sufficient confirms for channel use. Once both nodes have sent this, the channel enters normal operating mode.

Note that the opener is free to send this message at any time (since it presumably trusts itself), but the accepter would usually wait until the funding has reached the `minimum_depth` asked for in `accept_channel`.

1. type: 36 (`channel_ready`)
2. data:
    - [`channel_id:channel_id`]
    - [`point:second_per_commitment_point`]
    - [`channel_ready_tlvs:tlvs`]
3. `tlv_stream`: `channel_ready_tlvs`
4. types:
    1. type: 1 (`short_channel_id`)
    2. data:
        - [`short_channel_id:alias`]

Requirements

The sender: - MUST NOT send `channel_ready` unless outpoint of given by `funding_txid` and `funding_output_index` in the `funding_created` message pays exactly `funding_satoshis` to the scriptpubkey specified in BOLT #3. - if it is not the node opening the channel: - SHOULD wait until the funding transaction has reached `minimum_depth` before sending this message. - MUST wait for at least 100 blocks if the funding transaction is the coinbase transaction. - MUST set `second_per_commitment_point` to the per-commitment point to be used for commitment transaction #1, derived as specified in BOLT #3. - if `option_scid_alias` was negotiated: - MUST set `short_channel_id` `alias`. - otherwise: - MAY set `short_channel_id` `alias`. - if it sets `alias`: - if the `announce_channel` bit was set in `open_channel`: - SHOULD initially set `alias` to value not related to the real `short_channel_id`. - otherwise: - MUST set `alias` to a value not related to the real `short_channel_id`. - MUST NOT send the same `alias` for multiple peers or use an alias which collides with a `short_channel_id` of a channel on the same node. - MUST always recognize the `alias` as a `short_channel_id` for incoming HTLCs to this channel. - if `channel_type` has `option_scid_alias` set: - MUST NOT allow incoming HTLCs to this channel using the real `short_channel_id` - MAY send multiple `channel_ready` messages to the same peer with different `alias` values. - otherwise: - MUST wait until the funding transaction has reached `minimum_depth` before sending this message.

The sender:

A non-funding node (fundee): - SHOULD forget the channel if it does not see the correct funding transaction after a timeout of 2016 blocks.

The receiver: - MAY use any of the `alias` it received, in BOLT 11 `r` fields. - if `channel_type` has `option_scid_alias` set: - MUST NOT use the real `short_channel_id` in BOLT 11 `r` fields.

From the point of waiting for `channel_ready` onward, either node MAY send an `error` and fail the channel if it does not receive a required response from the other node after a reasonable timeout.

Rationale

The non-funder can simply forget the channel ever existed, since no funds are at risk. If the fundee were to remember the channel forever, this would create a Denial of Service risk; therefore, forgetting it is recommended (even if the promise of `push_msat` is significant).

If the fundee forgets the channel before it was confirmed, the funder will need to broadcast the commitment transaction to get his funds back and open a new channel. To avoid this, the funder should ensure the funding transaction confirms in the next 2016 blocks.

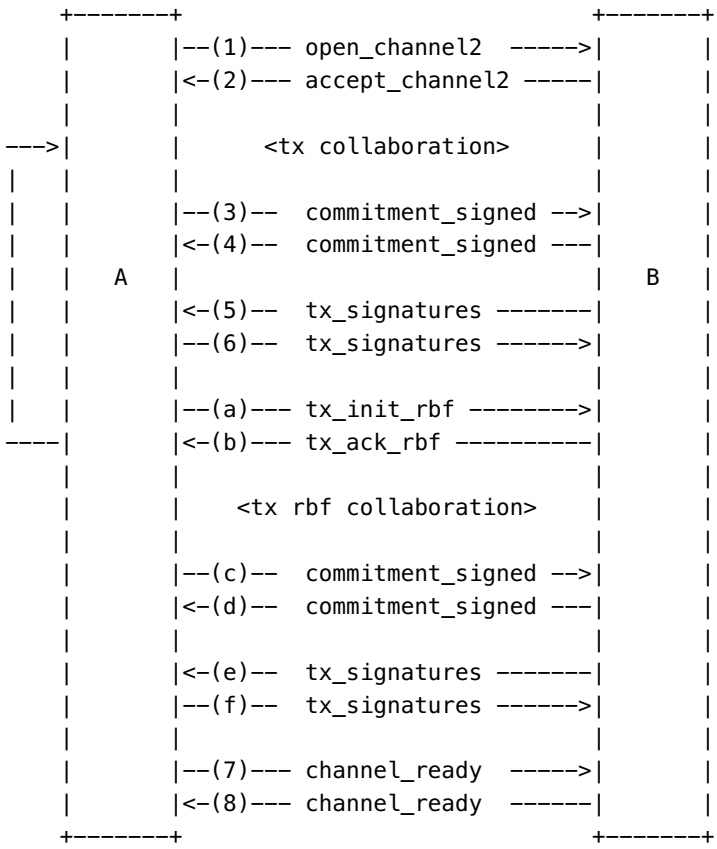
The `alias` here is required for two distinct use cases. The first one is for routing payments through channels that are not confirmed yet (since the real `short_channel_id` is unknown until confirmation). The second one is to provide one or more aliases to use for private channels (even once a real `short_channel_id` is available).

While a node can send multiple `alias`, it must remember all of the ones it has sent so it can use them should they be requested by incoming HTLCs. The recipient only need remember one, for use in `r` route hints in BOLT 11 invoices.

If an RBF negotiation is in progress when a `channel_ready` message is exchanged, the negotiation must be abandoned.

Channel Establishment v2

This is a revision of the channel establishment protocol. It changes the previous protocol to allow the `accept_channel2` peer (the *accepter*/*non-initiator*) to contribute inputs to the funding transaction, via the interactive transaction construction protocol.



- where node A is *opener*/*initiator* and node B is *accepter*/*non-initiator*

The `open_channel2` Message

This message initiates the v2 channel establishment workflow.

1. type: 64 (`open_channel2`)
2. data:
    - [`chain_hash`:`chain_hash`]
    - [`channel_id`:`temporary_channel_id`]
    - [`u32`:`funding_feerate_perkw`]
    - [`u32`:`commitment_feerate_perkw`]
    - [`u64`:`funding_satoshis`]
    - [`u64`:`dust_limit_satoshis`]
    - [`u64`:`max_htlc_value_in_flight_msat`]
    - [`u64`:`htlc_minimum_msat`]
    - [`u16`:`to_self_delay`]
    - [`u16`:`max_accepted_htlcs`]

        - [u32:locktime]
        - [point:funding_pubkey]
        - [point:revocation_basepoint]
        - [point:payment_basepoint]
        - [point:delayed_payment_basepoint]
        - [point:htlc_basepoint]
        - [point:first_per_commitment_point]
        - [point:second_per_commitment_point]
        - [byte:channel_flags]
        - [opening_tlvs:tlvs]
3. tlv_stream: opening_tlvs
4. types:
    1. type: 0 (upfront_shutdown_script)
    2. data:
        - [...*byte:shutdown_scriptpubkey]
    3. type: 1 (channel_type)
    4. data:
        - [...*byte:type]
    5. type: 2 (require_confirmed_inputs)

Rationale and Requirements are the same as for open_channel, with the following additions.

Requirements

If nodes have negotiated option_dual_fund: - the opening node: - MUST NOT send open_channel

The sending node: - MUST set channel_type - MUST set funding_feerate_perkw to the feerate for this transaction - if channel_type includes zero_fee_commitments: - MUST set commitment_feerate_perkw to 0. - If it requires the receiving node to only use confirmed inputs: - MUST set require_confirmed_inputs

The receiving node: - MAY fail the negotiation if: - the locktime is unacceptable - the funding_feerate_perkw is unacceptable - MUST fail the negotiation if: - channel_type includes zero_fee_commitments and commitment_feerate_perkw is not 0. - require_confirmed_inputs is set but it cannot provide confirmed inputs - channel_type is not set

Rationale

temporary_channel_id MUST be derived using a zeroed out basepoint for the peer's revocation basepoint. This allows the peer to return channel-assignable errors before the *accepter*'s revocation basepoint is known.

funding_feerate_perkw indicates the fee rate that the opening node will pay for the funding transaction in satoshi per 1000-weight, as described in BOLT-3, Appendix F.

locktime is the locktime for the funding transaction.

The receiving node, if the locktime or funding_feerate_perkw is considered out of an acceptable range, may fail the negotiation. However, it is recommended that the *accepter* permits the channel open to proceed without their participation in the channel's funding.

Note that `open_channel`'s `channel_reserve_satoshi` has been omitted. Instead, the channel reserve is fixed at 1% of the total channel balance (`open_channel2.funding_satoshis` + `accept_channel2.funding_satoshis`) rounded down to the nearest whole satoshi or the `dust_limit_satoshis`, whichever is greater.

Note that `push_msat` has been omitted.

`second_per_commitment_point` is now sent here (as well as in `channel_ready`) as a convenience for implementations.

The sending node may require the other participant to only use confirmed inputs. This ensures that the sending node doesn't end up paying the fees of a low feerate unconfirmed ancestor of one of the other participant's inputs.

The `accept_channel2` Message

This message contains information about a node and indicates its acceptance of the new channel.

1. type: 65 (`accept_channel2`)
2. data:
    - [`channel_id:temporary_channel_id`]
    - [`u64:funding_satoshis`]
    - [`u64:dust_limit_satoshis`]
    - [`u64:max_htlc_value_in_flight_msat`]
    - [`u64:htlc_minimum_msat`]
    - [`u32:minimum_depth`]
    - [`u16:to_self_delay`]
    - [`u16:max_accepted_htlcs`]
    - [`point:funding_pubkey`]
    - [`point:revocation_basepoint`]
    - [`point:payment_basepoint`]
    - [`point:delayed_payment_basepoint`]
    - [`point:htlc_basepoint`]
    - [`point:first_per_commitment_point`]
    - [`point:second_per_commitment_point`]
    - [`accept_tlvs:tlvs`]
3. `tlv_stream`: `accept_tlvs`
4. types:
    1. type: 0 (`upfront_shutdown_script`)
    2. data:
        - [`...*byte:shutdown_scriptpubkey`]
    3. type: 1 (`channel_type`)
    4. data:
        - [`...*byte:type`]
    5. type: 2 (`require_confirmed_inputs`)

Rationale and Requirements are the same as listed above, for `accept_channel` with the following additions.

Requirements

The accepting node: - MUST use the `temporary_channel_id` of the `open_channel2` message. - MUST set `channel_type` to the `channel_type` from `open_channel2`. - MAY respond with a `funding_satoshis` value of zero. - If it requires the opening node to only use confirmed inputs: - MUST set `require_confirmed_inputs`.

The receiving node: - MUST fail the negotiation if: - `require_confirmed_inputs` is set but it cannot provide confirmed inputs. - `channel_type` is not set.

Rationale

The `funding_satoshis` is the amount of bitcoin in satoshis the *accepter* will be contributing to the channel's funding transaction.

Note that `accept_channel`'s `channel_reserve_satoshi` has been omitted. Instead, the channel reserve is fixed at 1% of the total channel balance (`open_channel2.funding_satoshis` + `accept_channel2.funding_satoshis`) rounded down to the nearest whole satoshi or the `dust_limit_satoshis`, whichever is greater.

Funding Composition

Funding composition for channel establishment v2 makes use of the Interactive Transaction Construction protocol, with the following additional caveats.

The `tx_add_input` Message

Requirements

The sending node: - if the receiver set `require_confirmed_inputs` in `open_channel2`, `accept_channel2`, `tx_init_rbf` or `tx_ack_rbf`: - MUST NOT send a `tx_add_input` that contains an unconfirmed input

The `tx_add_output` Message

Requirements

The sending node: - if is the *opener*: - MUST send at least one `tx_add_output`, which contains the channel's funding output

Rationale

The channel funding output must be added by the *opener*, who pays its fees.

The `tx_complete` Message

Upon receipt of consecutive `tx_completes`, the receiving node: - if is the *accepter*: - MUST fail the negotiation if: - no funding output was received - the value of the funding output is not equal to the sum of `open_channel2.funding_satoshis` and `accept_channel2`. `funding_satoshis` - the value of the funding output is less than the `dust_limit` - if this is an RBF attempt: - MUST fail the negotiation if: - the transaction's total fees is less than the last successfully negotiated transaction's fees - the transaction does not share at least one input with each previous funding transaction - if it has sent `require_confirmed_inputs` in `open_channel2`,

`accept_channel2`, `tx_init_rbf` or `tx_ack_rbf`: - MUST fail the negotiation if: - one of the inputs added by the other peer is unconfirmed

The `commitment_signed` Message

This message is exchanged by both peers. It contains the signatures for the first commitment transaction, which uses a format determined by the `channel_type` sent in `open_channel2` and `accept_channel2`.

Rationale and Requirements are the same as listed below, for `commitment_signed` with the following additions.

Requirements

The sending node: - MUST send zero HTLCs. - MUST remember the details of this funding transaction.

The receiving node: - if the message has one or more HTLCs: - MUST fail the negotiation - if it has not already transmitted its `commitment_signed`: - MUST send `commitment_signed` - Otherwise: - MUST send `tx_signatures` if it should sign first, as specified in the `tx_signatures` requirements

Rationale

The first commitment transaction has no HTLCs.

Once peers are ready to exchange commitment signatures, they must remember the details of the funding transaction to allow resuming the signatures exchange if a disconnection happens.

Sharing funding signatures: `tx_signatures`

After a valid `commitment_signed` has been received from the peer and a `commitment_signed` has been sent, a peer: - MUST transmit `tx_signatures` with their signatures for the funding transaction, following the order specified in the `tx_signatures` requirements

Requirements

The sending node: - MUST verify it has received a valid commitment signature from its peer - MUST remember the details of this funding transaction - if it has NOT received a valid `commitment_signed` message: - MUST NOT send a `tx_signatures` message

The receiving node: - if has already sent or received a `channel_ready` message for this channel: - MUST ignore this message - if the `witness` weight lowers the effective `feerate` below the *opener*'s feerate for the funding transaction and the effective `feerate` is determined by the receiving node to be insufficient for getting the transaction confirmed in a timely manner: - SHOULD broadcast their commitment transaction, closing the channel - SHOULD double-spend their channel inputs when there is a productive opportunity to do so; effectively canceling this channel open - SHOULD apply `witnesses` to the funding transaction and broadcast it

Rationale

A peer sends their `tx_signatures` after receiving a valid `commitment_signed` message, following the order specified in the `tx_signatures` section.

In the case where a peer provides valid witness data that causes their paid feerate to fall beneath the `open_channel2.funding_feerate_perkw`, the channel should be considered failed and the channel should be double-spent when there is a productive opportunity to do so. This should disincentivize peers from underpaying fees.

Fee bumping: `tx_init_rbf` and `tx_ack_rbf`

After the funding transaction has been broadcast, it can be replaced by a transaction paying more fees to make the channel confirm faster.

Requirements

The sender of `tx_init_rbf`: - MAY be either the *initiator* or the *accepter* - If the sender is the accepter, it becomes the initiator of the `interactive-tx` session and thus: - MUST send `tx_add_output` for the channel output - MUST pay the fees for the shared transaction fields - MUST NOT have sent or received a `channel_ready` message.

The recipient: - MUST fail the negotiation if they have already sent or received `channel_ready` - MAY fail the negotiation for any reason

Rationale

If a valid `channel_ready` message is received in the middle of an RBF attempt, the attempt MUST be abandoned.

Peers can use different values in `tx_init_rbf.funding_output_contribution` and `tx_ack_rbf.funding_output_contribution` from the amounts transmitted in `open_channel2` and `accept_channel2`: they are allowed to change how much they wish to commit to the funding output.

It's recommended that a peer, rather than fail the RBF negotiation due to a large feerate change, instead sets their `sats` to zero, and decline to participate further in the channel funding: by not contributing, they may obtain incoming liquidity at no cost.

We allow both nodes to initiate RBF, because any one of them may want to take this opportunity to contribute additional funds to the channel without waiting for the initial funding transaction to confirm.

Channel Quiescence

Various fundamental changes, in particular protocol upgrades, are easiest on channels where both commitment transactions match, and no pending updates are in flight. We define a protocol to quiesce the channel by indicating that "SomeThing Fundamental is Underway".

`stfu`

1. type: 2 (`stfu`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u8`:`initiator`]

Requirements

The sender of `stfu`: - MUST NOT send `stfu` unless `option_quiesce` is negotiated. - MUST NOT send `stfu` if any of the sender's htlc additions, htlc removals or fee updates are pending for either peer. - MUST NOT send `stfu` twice. - if it is replying to an `stfu`: - MUST set `initiator` to 0 - otherwise: - MUST set `initiator` to 1 - MUST set `channel_id` to the id of the channel to quiesce. - MUST now consider the channel to be quiescing. - MUST NOT send an update message after `stfu`.

The receiver of `stfu`: - if it has sent `stfu` then: - MUST now consider the channel to be quiescent - otherwise: - SHOULD NOT send any more update messages. - MUST reply with `stfu` once it can do so.

Both nodes: - MUST disconnect after 60 seconds of quiescence if the HTLCs are pending.

Upon disconnection: - the channel is no longer considered quiescent.

Dependent Protocols: - MUST specify all states that terminate quiescence. - NOTE: this prevents batching executions of protocols that depend on quiescence.

Rationale

The normal use would be to cease sending updates, then wait for all the current updates to be acknowledged by both peers, then start quiescence. For some protocols, choosing the initiator matters, so this flag is sent.

If both sides send `stfu` simultaneously, they will both set `initiator` to `1`, in which case the "initiator" is arbitrarily considered to be the channel funder (the sender of `open_channel`). The quiescence effect is exactly the same as if one had replied to the other.

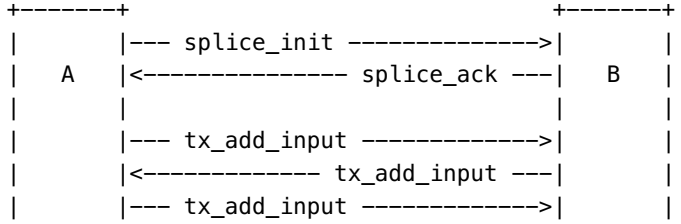
Dependent protocols have to specify termination conditions to prevent the need for disconnection to resume channel traffic. An explicit resume message was considered but rejected since it introduces a number of edge cases that make bilateral consensus of channel state significantly more complex to maintain. This introduces the derivative property that it is impossible to batch multiple downstream protocols in the same quiescence session.

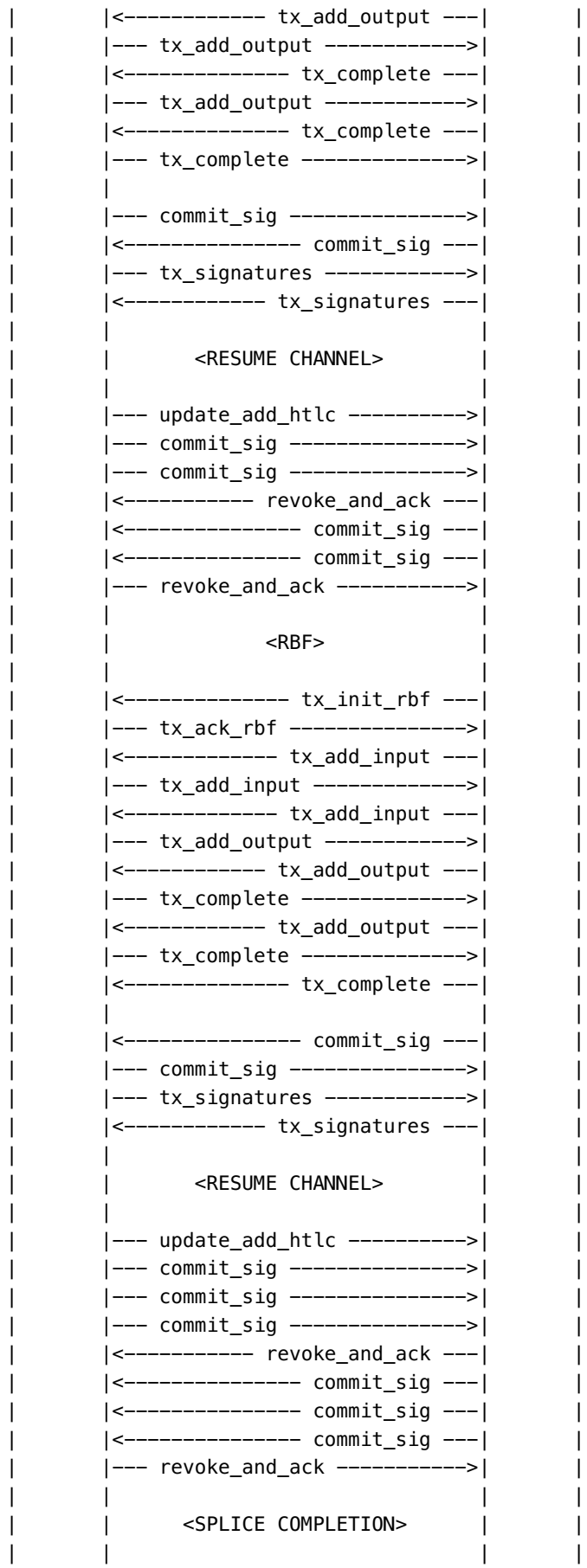
Channel Splicing

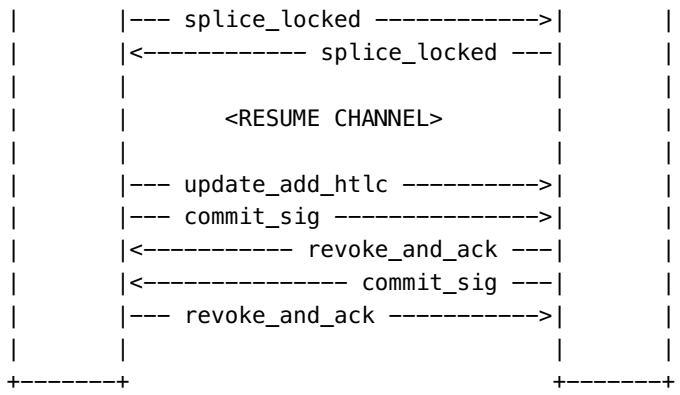
Splicing is the term given for replacing the funding transaction with a new one. For simplicity, splicing takes place once a channel is quiescent.

Operation returns to normal once the splice transaction has been signed (while waiting for one of the splice transactions to confirm), at which point the channel isn't quiescent anymore.

The splice is finally terminated when both sides send `splice_locked` to indicate that one of the splice transactions reached acceptable depth.




tx_add_output
tx_add_output
tx_complete
tx_add_output
tx_complete
tx_complete
commit_sig
commit_sig
tx_signatures
tx_signatures
<RESUME CHANNEL>
update_add_htlc
commit_sig
commit_sig
revoke_and_ack
commit_sig
commit_sig
revoke_and_ack
<RBF>
tx_init_rbf
tx_ack_rbf
tx_add_input
tx_add_input
tx_add_output
tx_add_output
tx_complete
tx_add_output
tx_complete
tx_complete
commit_sig
commit_sig
tx_signatures
tx_signatures
<RESUME CHANNEL>
update_add_htlc
commit_sig
commit_sig
commit_sig
revoke_and_ack
commit_sig
commit_sig
commit_sig
revoke_and_ack
<SPLICE COMPLETION>



The `splice_init` Message

1. type: 80 (`splice_init`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`s64`:`funding_contribution_satoshis`]
    - [`u32`:`funding_feerate_perkw`]
    - [`u32`:`locktime`]
    - [`point`:`funding_pubkey`]
    - [`splice_init_tlvs`:`tlvs`]
3. `tlv_stream`: `splice_init_tlvs`
4. types:
    1. type: 2 (`require_confirmed_inputs`)

`funding_contribution_satoshis` is the amount the sender is adding to their channel balance (splice-in) or removing from their channel balance (splice-out).

Requirements

The sending node: - MUST NOT send `splice_init` if the channel is not quiescent. - MUST NOT send `splice_init` if it is not the quiescence initiator. - MUST NOT send `splice_init` before sending and receiving `channel_ready`. - MUST NOT send `splice_init` while another splice is being negotiated. - MUST NOT send `splice_init` if another splice has been negotiated but `splice_locked` has not been sent and received. - MUST NOT send `splice_init` if it has previously sent `shutdown`. - MUST set `funding_feerate_perkw` to the feerate for the splice transaction. - If it is splicing funds out of the channel: - MUST set `funding_contribution_satoshis` to a negative value matching the amount that will be subtracted from its current channel balance. - If it is splicing funds into the channel: - MUST set `funding_contribution_satoshis` to a positive value matching the amount that will be added to its current channel balance. - If it requires the receiving node to only use confirmed inputs: - MUST set `require_confirmed_inputs`. - SHOULD use a different `funding_pubkey` than the one used for the previous funding transaction.

The receiving node: - If the channel is not quiescent: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If the sending node is not the quiescence initiator: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If another splice is already being negotiated: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If another splice has been negotiated but isn't locked yet: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If it has received `shutdown`: - MUST send a `warning` and close the connection or send an `error`

and fail the channel. - If the `funding_feerate_perkw` is unacceptable: - MUST respond with `tx_abort`. - If `funding_contribution_satoshis` is negative and its absolute value is greater than the sending node's current channel balance: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If it accepts the splice attempt: - MUST respond with `splice_ack`. - Otherwise (it rejects the splice): - MUST respond with `tx_abort`.

The `splice_ack` Message

1. type: 81 (`splice_ack`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`s64`:`funding_contribution_satoshis`]
    - [`point`:`funding_pubkey`]
    - [`splice_ack_tlvs`:`tlvs`]
3. `tlv_stream`: `splice_ack_tlvs`
4. types:
    1. type: 2 (`require_confirmed_inputs`)

Requirements

The sending node: - SHOULD use a different `funding_pubkey` than the one used for the previous funding transaction. - MAY set `funding_contribution_satoshis` to `0` if they don't want to contribute to the splice. - If it requires the receiving node to only use confirmed inputs: - MUST set `require_confirmed_inputs`.

The receiving node: - If it has sent `splice_init`: - If `funding_contribution_satoshis` is negative and its absolute value is greater than the sending node's current channel balance: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If it accepts the splice attempt: - MUST start an `interactive-tx` session to create the splice transaction. - Otherwise: - MUST reject the splice attempt by sending `tx_abort`. - Otherwise (it has not sent `splice_init`): - MUST send a `warning` and close the connection or send an `error` and fail the channel.

Splice Transaction Construction

The splice transaction is created using the Interactive Transaction Construction protocol, with the following additional requirements.

The `tx_add_input` Message

Requirements

The sending node: - If it is the splice initiator: - MUST add the current channel input to the splice transaction by sending `tx_add_input` with `shared_input_txid` containing the `txid` of the previous funding transaction. - MUST NOT include `prevtx` for that shared input. - MUST set `prevtx_vout` to the previous funding output index. - If the receiver set `require_confirmed_inputs` in `splice_init`, `splice_ack`, `tx_init_rbf` or `tx_ack_rbf`: - MUST NOT send a `tx_add_input` that contains an unconfirmed input.

The receiving node: - If `shared_input_txid` is set: - If it doesn't match the `txid` of the previous funding transaction: - MUST fail the negotiation by sending `tx_abort`. - If `prevtx_vout` doesn't match the previous funding output index: - MUST fail the negotiation by sending `tx_abort`.

Rationale

The splice transaction must spend the current channel funding output. The splice initiator is responsible for adding that input to the transaction, and pay the fees for its weight. It would be wasteful to transmit the previous funding transaction in the `prevtx` field, and wouldn't even be possible for funding transactions that exceed 65kB, so we only transmit its `txid` using the `shared_input_txid` field.

The `tx_add_output` Message

Requirements

The sending node: - If it is the splice initiator: - MUST send at least one `tx_add_output`, which contains the new channel's funding output based on the `funding_pubkeys` from `splice_init` and `splice_ack`. - MUST set the amount of that `tx_add_output` to the previous channel capacity with the `funding_contribution_satoshiss` from `splice_init` and `splice_ack` applied.

Rationale

The splice initiator is responsible for adding the new channel funding output to the transaction and paying the fees for its weight.

The `tx_complete` Message

Requirements

The receiving node: - MUST compute the channel balance for each side by adding their respective `funding_contribution_satoshis` to their previous channel balance. - MUST fail the negotiation by sending `tx_abort` if: - There is not exactly one input spending the current funding transaction. - There is not exactly one channel funding output using the funding public keys and funding contributions from `splice_init` and `splice_ack`. - This is an RBF attempt and the transaction's total fees is less than the last successfully negotiated splice transaction's fees. - Either side has added an output other than the channel funding output and the balance for that side is less than the channel reserve that matches the new channel capacity.

Rationale

If a side does not meet the reserve requirements, that's OK: but if they take funds out of the channel, they must ensure that they do meet them. If your peer adds a massive amount to the channel, then you only have to add more reserve if you want to contribute to the splice (and you can use `tx_remove_output` and/or `tx_remove_input` part-way through if this happens).

The `commitment_signed` Message

After exchanging `tx_complete`, both peers send `commitment_signed` to commit to the splice transaction by creating a commitment transaction spending the new channel funding output.

The usual `commitment_signed` requirements apply with the following additions.

Requirements

The sending node: - MUST create a commitment transaction that spends the splice funding output and: - Adds `funding_contribution_satoshis` from `splice_init` and `splice_ack` to the main balance of their respective sender. - Uses the same feerate as the existing commitment transaction. - Uses the same `commitment_number` as the existing commitment transaction. - MUST send signatures for pending HTLCs. - MUST remember the details of this splice transaction.

The receiving node: - MUST NOT respond with `revoke_and_ack`. - If it has not already transmitted its `commitment_signed`: - MUST send `commitment_signed`. - If it should sign first, as specified in the `tx_signatures` requirements: - MUST send `tx_signatures`. - Note that since the initiator sends `tx_add_input` for the shared input (corresponding to the previous channel output), 100% of the previous channel capacity is attributed to the initiator when computing who must send `tx_signatures` first (instead of using each node's previous balance).

On reconnection: - If `next_funding` matches the splice transaction: - MUST retransmit `commitment_signed`.

Rationale

Once peers are ready to exchange commitment signatures, they must remember the details of the splice transaction to allow resuming the signatures exchange if a disconnection happens.

The `tx_signatures` Message

Requirements

The sending node: - MUST set `shared_input_signature` to a valid ECDSA signature for the `tx_add_input` spending the previous channel funding output using the `funding_pubkey` that matches this input.

The receiving node: - If `shared_input_signature` is not set: - MUST send an `error` and fail the channel. - If `shared_input_signature` is not valid or non-compliant with the LOW-S-standard rule[LOWS]: - MUST send an `error` and fail the channel. - MUST consider the channel no longer quiescent.

On reconnection: - If `next_funding` matches the splice transaction: - MUST retransmit `tx_signatures`.

Rationale

Spending the channel funding output requires a signature from both peers. Each peer transmits its own signature, which allows creating a valid witness for the shared input without adding an additional message.

Once `tx_signatures` have been exchanged, the splice transaction can be broadcast. The channel is no longer quiescent: normal operation can resume while waiting for the transaction to confirm and `splice_locked` messages to be exchanged.

The `tx_init_rbf` Message

Requirements

The sending node: - MUST NOT send `tx_init_rbf` if the channel is not quiescent. - MUST NOT send `tx_init_rbf` if it is not the quiescence initiator. - MAY send `tx_init_rbf` even if it is not the splice initiator. - If there are more than 10 pending RBF attempts: - MUST set a high enough `feerate` to ensure quick confirmation. - MUST NOT send `tx_init_rbf` if it has previously sent `splice_locked`. - MUST NOT send `tx_init_rbf` if `option_zeroconf` has been negotiated. - MAY set `funding_output_contribution` to a different value than the `funding_contribution_satoshis` used in `splice_init` or `splice_ack`, or in previous RBF attempts.

The receiving node: - If the channel is not quiescent: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If the sending node is not the quiescence initiator: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If another RBF attempt has been created recently: - SHOULD send `tx_abort` to reject this RBF attempt and wait for the previous RBF attempt to confirm. - If there are more than 10 pending RBF attempts and the `feerate` is not high enough to ensure quick confirmation: - SHOULD send `tx_abort` to reject the RBF attempt. - If the sender previously sent `splice_locked`: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If `option_zeroconf` has been negotiated: - MUST send a `warning` and close the connection or send an `error` and fail the channel. - If `funding_output_contribution` is negative and its absolute value is greater than the sending node's current channel balance: - MUST send a `warning` and close the connection or send an `error` and fail the channel.

Rationale

Splice transactions can be RBF-ed to react to changes in the mempool feerate. We allow both nodes to initiate RBF, because any one of them may want to take this opportunity to splice additional funds into or out of the channel without waiting for the initial splice transaction to confirm.

We limit the number of pending RBF attempts, otherwise we may reach the `batch_size` limit defined in `start_batch`. We require using a large enough feerate when we've already created many RBF attempts, and we wait between RBF attempts to allow a previous attempt to confirm.

Since splice transactions always spend the current channel funding output, the RBF attempts automatically double-spend each other. We thus disallow RBF when `option_zeroconf` has been negotiated, because that creates a risk of losing funds.

The `tx_ack_rbf` Message

Requirements

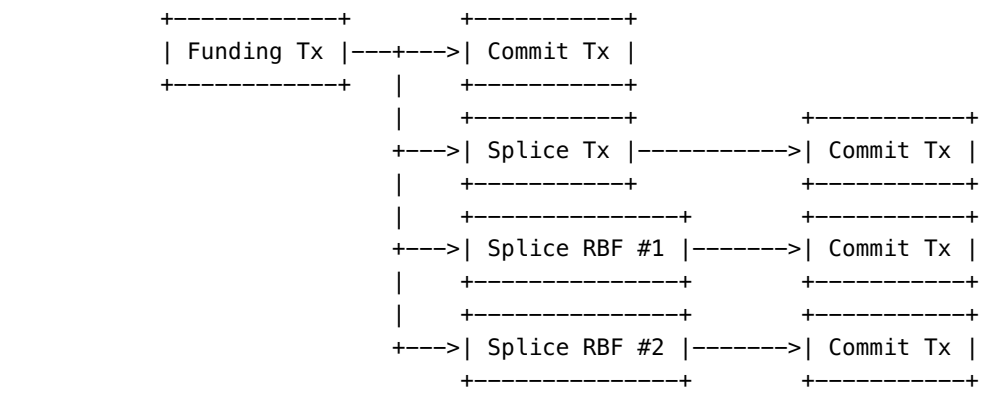
The sending node: - MAY set `funding_output_contribution` to a different value than the `funding_contribution_satoshis` used in `splice_init` or `splice_ack`, or in previous RBF attempts.

The receiving node: - If `funding_output_contribution` is negative and its absolute value is greater than the sending node's current channel balance: - MUST send a `warning` and close the connection or send an `error` and fail the channel.

Splice Completion

Once splice transactions have been signed but haven't reached acceptable depth, channel operations go back to normal and HTLCs can be exchanged, with the caveat that payments must be valid for all splice transactions.

Nodes keep track of multiple commitment transactions (one for the current funding transaction and one for each splice transaction) and exchange signatures for each of these commitment transactions.



The splice completes by exchanging `splice_locked` messages, at which point the locked transaction replaces the previous funding transaction.

The `splice_locked` Message

1. type: 77 (`splice_locked`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`sha256`:`splice_txid`]

Requirements

Each node: - If any splice transaction reaches acceptable depth: - MUST send `splice_locked` with the `txid` of that transaction. - If `option_zeroconf` has been negotiated: - SHOULD send `splice_locked` immediately after exchanging `tx_signatures`.

The receiving node: - If `splice_txid` doesn't match any of its pending splice transactions: - MUST send a `warning` and close the connection, or send an `error` and fail the channel.

Once a node has sent and received `splice_locked`: - If the `splice_txid`s match: - MUST stop sending `commitment_signed` for RBF attempts and ancestors of this splice transaction. - MAY discard RBF attempts and ancestor transactions. - If `announce_channel` is set for this channel: - MUST send `announcement_signatures` with `short_channel_id` matching this splice transaction. - If the `splice_txid`s are for different RBF candidates: - SHOULD ignore the message. - MAY send an `error` and fail the channel.

Rationale

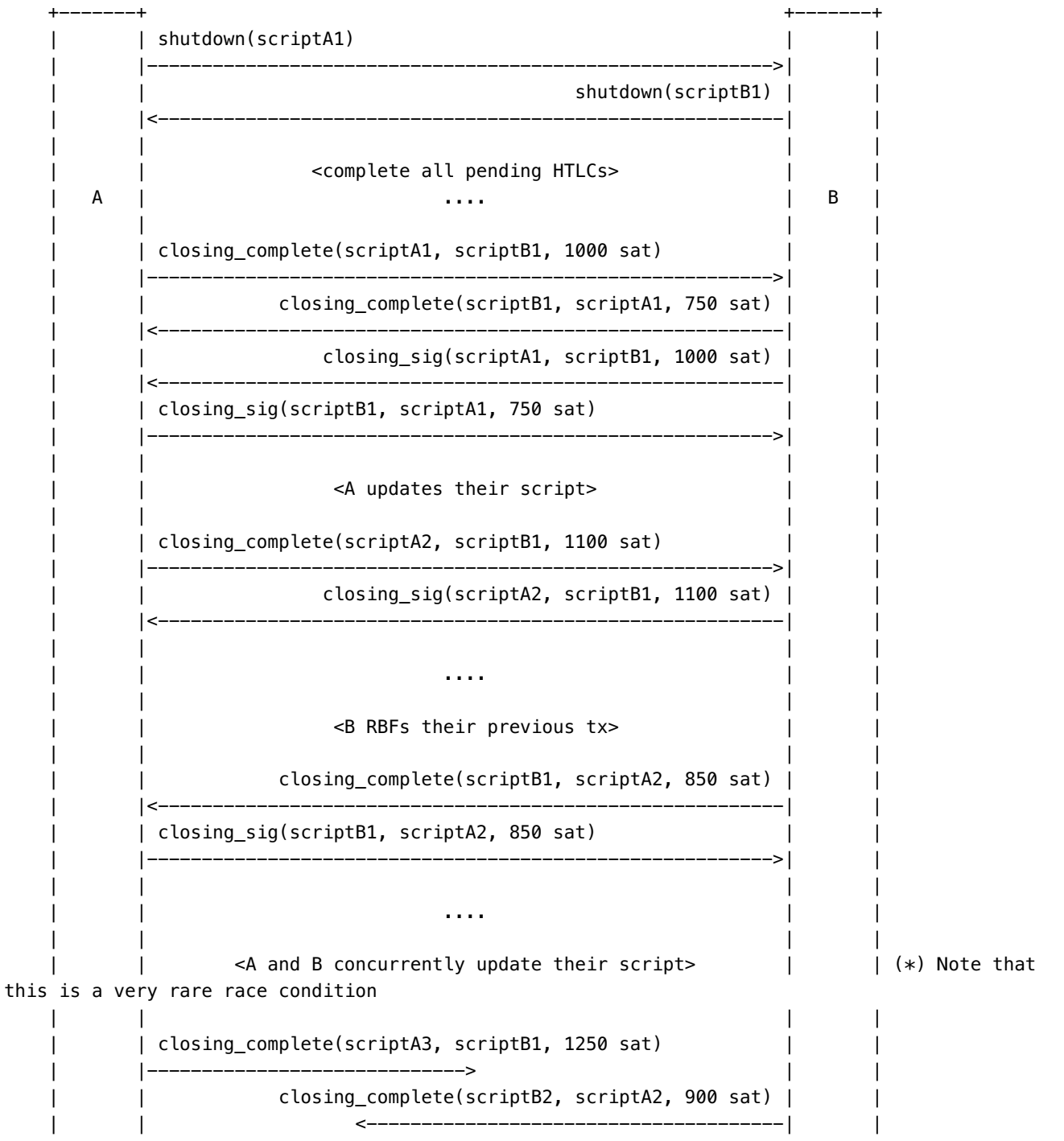
If nodes are on a different fork of the blockchain, they may disagree on which RBF attempt has been confirmed: in that case nodes can either close the channel or simply ignore `splice_locked` and wait for one of the forks to

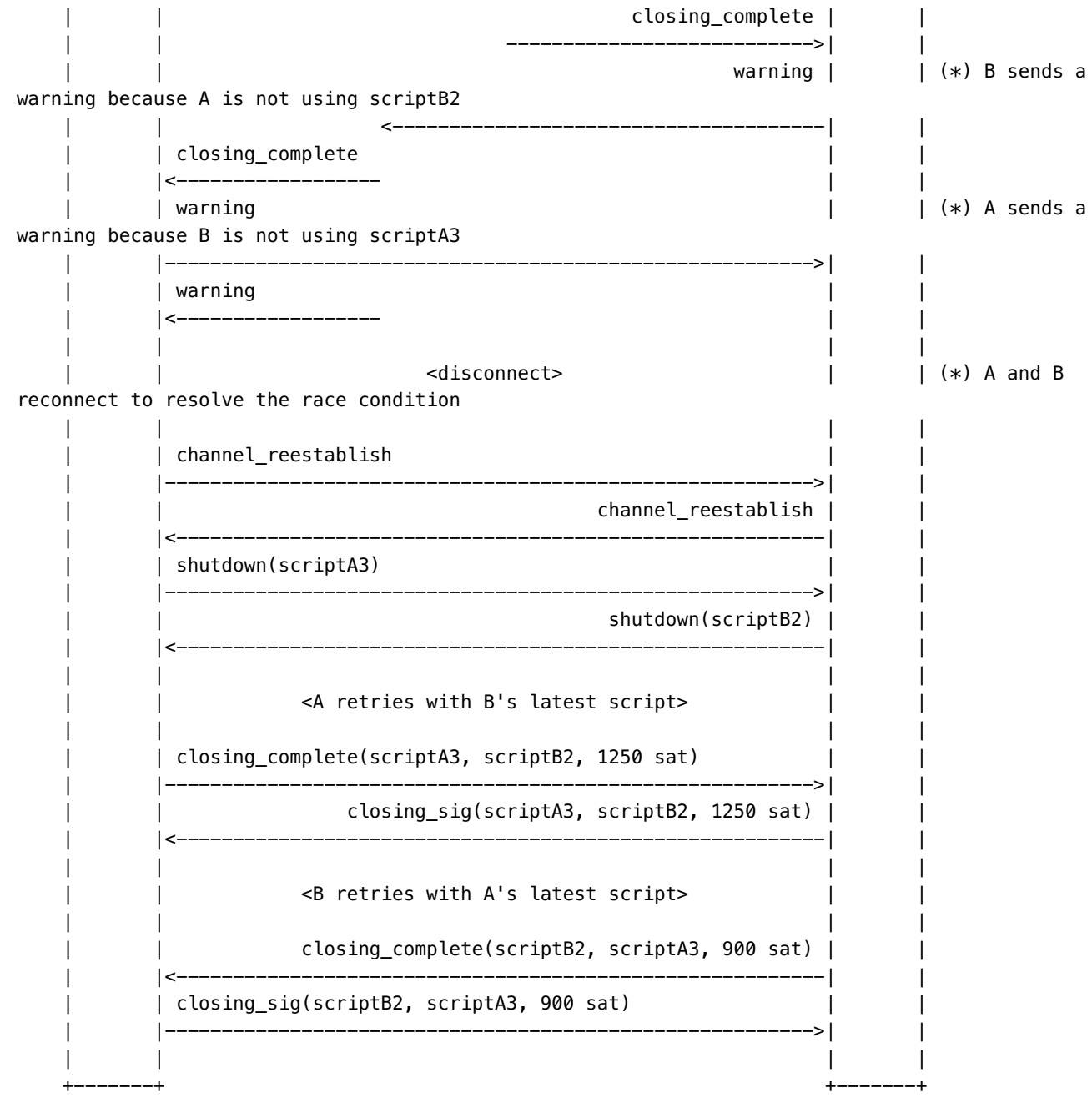
eventually replace the other, at which point both nodes should agree on which RBF attempt confirmed and exchange `splice_locked` for the same `splice_txid` to complete the splice.

Channel Close

Nodes can negotiate a mutual close of the connection, which unlike a unilateral close, allows them to access their funds immediately and can be negotiated with lower fees.

Closing happens in two stages: 1. one side indicates it wants to clear the channel (and thus will accept no new HTLCs) 2. once all HTLCs are resolved, the final channel close negotiation begins.





Closing Initiation: `shutdown`

Either node (or both) can send a `shutdown` message to initiate closing, along with the `scriptpubkey` it wants to be paid to.

1. type: 38 (`shutdown`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u16`:`len`]
    - [`len*byte`:`scriptpubkey`]

Requirements

A sending node: - if it hasn't sent a `funding_created` (if it is a funder) or a `funding_signed` (if it is a fundee): - MUST NOT send a `shutdown` - MAY send a `shutdown` before a `channel_ready`, i.e. before the funding transaction has reached `minimum_depth`. - if there are updates pending on the receiving node's commitment transaction: - MUST NOT send a `shutdown`. - MUST NOT send multiple `shutdown` messages. - MUST NOT send `shutdown` if there is a splice transaction that isn't locked yet. - MUST NOT send an `update_add_htlc` after a `shutdown`. - if no HTLCs remain in either commitment transaction (including dust HTLCs) and neither side has a pending `revoke_and_ack` to send: - MUST NOT send any `update` message after that point. - SHOULD fail to route any HTLC added after it has sent `shutdown`. - if it sent a non-zero-length `shutdown_scriptpubkey` in `open_channel` or `accept_channel`: - MUST send the same value in `scriptpubkey`. - MUST set `scriptpubkey` in one of the following forms:

```
1. `OP_0` `20` 20-bytes (version 0 pay to witness pubkey hash), OR
2. `OP_0` `32` 32-bytes (version 0 pay to witness script hash), OR
3. if (and only if) `option_shutdown_anysegwit` is negotiated:
  * `OP_1` through `OP_16` inclusive, followed by a single push of 2 to 40 bytes
    (witness program versions 1 through 16)
4. if (and only if) `option_simple_close` is negotiated:
  * `OP_RETURN` followed by one of:
    * `6` to `75` inclusive followed by exactly that many bytes
    * `76` followed by `76` to `80` followed by exactly that many bytes
```

A receiving node: - if it hasn't received a `funding_signed` (if it is a funder) or a `funding_created` (if it is a fundee): - SHOULD send an `error` and fail the channel. - if the `scriptpubkey` is not in one of the above forms: - SHOULD send a `warning`. - if it hasn't sent a `channel_ready` yet: - MAY reply to a `shutdown` message with a `shutdown` - once there are no outstanding updates on the peer, UNLESS it has already sent a `shutdown`: - MUST reply to a `shutdown` message with a `shutdown` - if both nodes advertised the `option_upfront_shutdown_script` feature, and the receiving node received a non-zero-length `shutdown_scriptpubkey` in `open_channel` or `accept_channel`, and that `shutdown_scriptpubkey` is not equal to `scriptpubkey`: - MAY send a `warning`. - MUST fail the connection.

Rationale

If channel state is always "clean" (no pending changes) when a shutdown starts, the question of how to behave if it wasn't is avoided: the sender always sends a `commitment_signed` first.

As shutdown implies a desire to terminate, it implies that no new HTLCs will be added or accepted. Once any HTLCs are cleared, there are no commitments for which a revocation is owed, and all updates are included on both commitment transactions, the peer may immediately begin closing negotiation, so we ban further updates to the commitment transaction (in particular, `update_fee` would be possible otherwise). However, while there are HTLCs on the commitment transaction, the initiator may find it desirable to increase the feerate as there may be pending HTLCs on the commitment which could timeout.

The `scriptpubkey` forms include only standard segwit forms accepted by the Bitcoin network, which ensures the resulting transaction will propagate to miners. However old nodes may send non-segwit scripts, which may be accepted for backwards-compatibility (with a caveat to force-close if this output doesn't meet dust relay requirements).

The `option_upfront_shutdown_script` feature means that the node wanted to pre-commit to `shutdown_scriptpubkey` in case it was compromised somehow. This is a weak commitment (a malevolent implementation tends to ignore specifications like this one!), but it provides an incremental improvement in security by requiring the cooperation of the receiving node to change the `scriptpubkey`.

The `shutdown` response requirement implies that the node sends `commitment_signed` to commit any outstanding changes before replying; however, it could theoretically reconnect instead, which would simply erase all outstanding uncommitted changes.

`OP_RETURN` is only standard if followed by PUSH opcodes, and the total script is 83 bytes or less. We are slightly stricter, to only allow a single PUSH, but there are two forms in script: one which pushes up to 75 bytes, and a longer one (`OP_PUSHDATA1`) which is needed for 76-80 bytes.

Closing Negotiation: `closing_complete` and `closing_sig`

Once shutdown is complete, the channel is empty of HTLCs, there are no commitments for which a revocation is owed, and all updates are included on both commitments, the final current commitment transactions will have no HTLCs.

If `option_simple_close` is not negotiated, see Legacy Closing Negotiation below.

Each peer creates their own closing transaction where they pay the fee, and sends `closing_complete` to the other peer with the transaction details. The other peer simply signs that transaction and sends back `closing_sig`. Each peer will thus independently send `closing_complete` and receive `closing_sig`, resulting in two independent (but conflicting) closing transactions being created.

The lesser-paid peer (if either is) can opt to omit their own output from the closing transaction.

This process can be repeated multiple times by sending `closing_complete` again, which allows increasing the fees and changing the output script.

1. type: 40 (`closing_complete`)
2. data:
    - [`channel_id:channel_id`]
    - [`u16:closer_scriptpubkey_len`]
    - [`closer_scriptpubkey_len*byte:closer_scriptpubkey`]
    - [`u16:closee_scriptpubkey_len`]
    - [`closee_scriptpubkey_len*byte:closee_scriptpubkey`]
    - [`u64:fee_satoshis`]
    - [`u32:locktime`]
    - [`closing_tlvs:tlvs`]
3. type: 41 (`closing_sig`)
4. data:
    - [`channel_id:channel_id`]
    - [`u16:closer_scriptpubkey_len`]
    - [`closer_scriptpubkey_len*byte:closer_scriptpubkey`]
    - [`u16:closee_scriptpubkey_len`]
    - [`closee_scriptpubkey_len*byte:closee_scriptpubkey`]
    - [`u64:fee_satoshis`]

        - [u32:locktime]
        - [closing_tlvs:tlvs]
    5. tlv_stream: closing_tlvs
    6. types:
        1. type: 1 (closer_output_only)
        2. data:
            - [signature:sig]
        3. type: 2 (closee_output_only)
        4. data:
            - [signature:sig]
        5. type: 3 (closer_and_closee_outputs)
        6. data:
            - [signature:sig]

Requirements

Note: the details and requirements for the transaction being signed are in BOLT 3.

An output is *dust* if the amount is less than the Bitcoin Core Dust Thresholds.

Note: These requirements only apply if `option_simple_close` is negotiated, otherwise the requirements are in Legacy Closing Negotiation.

Both nodes: - After a `shutdown` has been sent and received, AND no HTLCs remain in either commitment transaction: - SHOULD send a `closing_complete` message.

The sender of `closing_complete` (aka. "the closer"): - MUST set `fee_satoshis` to a fee less than or equal to its outstanding balance, rounded down to whole satoshis. - MUST set `fee_satoshis` so that at least one output is not dust. - MUST set `closer_scriptpubkey` to its desired output script. - MUST set `closee_scriptpubkey` to the last script it received from its peer (from `closing_complete` or from the initial `shutdown`). - MUST set `locktime` to the desired `nLockTime` of the closing transaction. - If the local outstanding balance (in millisatoshi) is less than the remote outstanding balance: - MUST NOT set `closer_output_only`. - MUST set `closee_output_only` if the local output amount is dust. - MAY set `closee_output_only` if it considers the local output amount uneconomical AND its `closer_scriptpubkey` is not `OP_RETURN`. - Otherwise (not lesser amount, cannot remove its own output): - MUST NOT set `closee_output_only`. - If it considers the local output amount uneconomical: - MAY send a `closer_scriptpubkey` that is a valid `OP_RETURN` script. - If it does, the output value MUST be set to zero so that all funds go to fees, as specified in BOLT #3. - If the closee's output amount is dust: - MUST set `closer_output_only`. - MUST NOT set `closer_and_closee_outputs`. - Otherwise: - MUST set both `closer_output_only` and `closer_and_closee_outputs`. - MUST generate its closing transaction as specified in BOLT #3. - MUST set `signature` fields as valid signature using its `funding_pubkey` of: - `closer_output_only`: closing transaction with only the local ("closer") output. - `closee_output_only`: closing transaction with only the remote ("closee") output. - `closer_and_closee_outputs`: closing transaction with both the closer and closee outputs. - If it wants to send another `closing_complete` (e.g. with a different `fee_satoshis` or `closer_scriptpubkey`): - MUST wait until it has received `closing_sig` first. - SHOULD close the connection if it doesn't receive `closing_sig`.

The receiver of `closing_complete` (aka. "the closee"): - If `fee_satoshis` is greater than the closer's outstanding balance: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If

`closee_scriptpubkey` does not match the last script it sent (from `closing_complete` or from the initial `shutdown`): - SHOULD ignore `closing_complete`. - SHOULD send a `warning`. - SHOULD close the connection. - If `closer_scriptpubkey` is invalid (as detailed in the `shutdown` requirements): - SHOULD ignore `closing_complete`. - SHOULD send a `warning`. - SHOULD close the connection. - If `closer_scriptpubkey` is a valid `OP_RETURN` script: - MUST set the closer's output amount to zero so that all funds go to fees, as specified in BOLT #3. - MUST generate the remote closing transaction as specified in BOLT #3. - Select a signature for validation: - If the local output amount is dust: - MUST use `closer_output_only`. - Otherwise, if it considers the local output amount uneconomical AND its `closee_scriptpubkey` is not `OP_RETURN`: - MUST use `closer_output_only`. - Otherwise, if `closer_and_closee_outputs` is present: - MUST use `closer_and_closee_outputs`. - Otherwise: - MUST use `closee_output_only`. - If the selected signature field does not exist: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If the signature field is not valid for the corresponding closing transaction specified in BOLT #3: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If the signature field is non-compliant with LOW-S-standard rule[LOWS]: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - MUST sign and broadcast the corresponding closing transaction. - MUST send `closing_sig` with a single valid signature in the same TLV field as the `closing_complete`. - MUST use `closer_scriptpubkey` for its own future `closing_complete` messages.

The receiver of `closing_sig`: - If `closer_scriptpubkey`, `closee_scriptpubkey`, `fee_satoshis` or `locktime` don't match what was sent in `closing_complete`: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If `tlvs` does not contain exactly one signature: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If `tlvs` does not contain one of the TLV fields sent in `closing_complete`: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If the signature field is not valid for the corresponding closing transaction specified in BOLT #3: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - If the signature field is non-compliant with LOW-S-standard rule[LOWS]: - MUST either send a `warning` and close the connection, or send an `error` and fail the channel. - otherwise: - MUST broadcast the corresponding closing transaction. - MAY send another `closing_complete` (e.g. with a different `fee_satoshis` or `closer_scriptpubkey`).

Rationale

The close protocol is designed to avoid any failure scenarios caused by fee disagreement, since each side offers to pay its own desired fee.

If one side has less funds than the other, it may choose to omit its own output, and in this case dust MUST be omitted, to ensure that the resulting transaction can be broadcast.

The corner case where fees are so high that both outputs are dust is addressed in two ways: paying a low fee to avoid the problem, or using an `OP_RETURN` (which is never "dust"). If one side chooses to use an `OP_RETURN` output, its amount must be 0 to ensure that the resulting transaction can be broadcast.

Note that there is usually no reason to pay a high fee for rapid processing, since an urgent child could pay the fee on the closing transactions' behalf. If rapid processing is desired and CPFP is not an option, the closer can RBF its previous closing transactions by sending `closing_complete` again.

Sending a new `closing_complete` message overrides previous ones, so you can negotiate again (even changing the output address if `upfront_shutdown_script` was not negotiated). This creates a rare race condition if both

nodes send `closing_complete` to change their `closer_scriptpubkey` at the same time: when that happens, the `closing_complete` they receive will be using their previous output script, so they shouldn't sign the corresponding transaction. When that happens, we simply reconnect, which provides the opportunity for both nodes to send their latest output script in `shutdown` and restart the signing flow. We include the closer and closee scripts in `closing_sig` to allow our peer to detect a script mismatch and correctly ignore the signatures, which also helps debugging race conditions.

If the closer proposes a transaction which will not relay (an output is dust, or the fee rate it proposes is too low), it doesn't harm the closee to sign the transaction.

Similarly, if the closer proposes a high fee, it doesn't harm the closee to sign the transaction, as the closer is paying.

There's a slight game where each side would prefer the other side pay the fee, and proposes a minimal fee. If neither side proposes a fee which will relay, the negotiation can occur again, or the final commitment transaction can be spent. In practice, the opener has an incentive to offer a reasonable closing fee, as they would pay the fee for the commitment transaction, which also costs more to spend.

Legacy Closing Negotiation: `closing_signed`

Once shutdown is complete, the channel is empty of HTLCs, there are no commitments for which a revocation is owed, and all updates are included on both commitments, the final current commitment transactions will have no HTLCs, and closing fee negotiation begins: if `option_simple_close` is negotiated, the section above applies, otherwise this legacy section applies.

The funder chooses a fee it thinks is fair, and signs the closing transaction with the `scriptpubkey` fields from the `shutdown` messages (along with its chosen fee) and sends the signature; the other node then replies similarly, using a fee it thinks is fair. This exchange continues until both agree on the same fee or when one side fails the channel.

In the modern method, the funder sends its permissible fee range, and the non-funder has to pick a fee in this range. If the non-funder chooses the same value, negotiation is complete after two messages, otherwise the funder will reply with the same value (completing after three messages).

1. type: 39 (`closing_signed`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u64`:`fee_satoshis`]
    - [`signature`:`signature`]
    - [`closing_signed_tlvs`:`tlvs`]
3. `tlv_stream`: `closing_signed_tlvs`
4. types:
    1. type: 1 (`fee_range`)
    2. data:
        - [`u64`:`min_fee_satoshis`]
        - [`u64`:`max_fee_satoshis`]

Requirements

Note: These requirements only apply if `option_simple_close` is NOT negotiated, otherwise the requirements Closing Negotiation: `closing_complete` and `closing_sig` apply.

The funding node: - after `shutdown` has been received, AND no HTLCs remain in either commitment transaction: - SHOULD send a `closing_signed` message.

The sending node: - SHOULD set the initial `fee_satoshis` according to its estimate of cost of inclusion in a block. - SHOULD set `fee_range` according to the minimum and maximum fees it is prepared to pay for a close transaction. - if it doesn't receive a `closing_signed` response after a reasonable amount of time: - MUST fail the channel - if it is not the funder: - SHOULD set `max_fee_satoshis` to at least the `max_fee_satoshis` received - SHOULD set `min_fee_satoshis` to a fairly low value - MUST set `signature` to the Bitcoin signature of the close transaction, as specified in BOLT #3.

The receiving node: - if the `signature` is not valid for either variant of closing transaction specified in BOLT #3 OR non-compliant with LOW-S-standard rule[LOWS]: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - if `fee_satoshis` is equal to its previously sent `fee_satoshis`: - SHOULD sign and broadcast the final closing transaction. - MAY close the connection. - if `fee_satoshis` matches its previously sent `fee_range`: - SHOULD use `fee_satoshis` to sign and broadcast the final closing transaction - SHOULD reply with a `closing_signed` with the same `fee_satoshis` value if it is different from its previously sent `fee_satoshis` - MAY close the connection. - if the message contains a `fee_range`: - if there is no overlap between that and its own `fee_range`: - SHOULD send a warning - MUST fail the channel if it doesn't receive a satisfying `fee_range` after a reasonable amount of time - otherwise: - if it is the funder: - if `fee_satoshis` is not in the overlap between the sent and received `fee_range`: - MUST fail the channel - otherwise: - MUST reply with the same `fee_satoshis`. - otherwise (it is not the funder): - if it has already sent a `closing_signed`: - if `fee_satoshis` is not the same as the value it sent: - MUST fail the channel - otherwise: - MUST propose a `fee_satoshis` in the overlap between received and (about-to-be) sent `fee_range`. - otherwise, if `fee_satoshis` is not strictly between its last-sent `fee_satoshis` and its previously-received `fee_satoshis`, UNLESS it has since reconnected: - SHOULD send a `warning` and close the connection, or send an `error` and fail the channel. - otherwise, if the receiver agrees with the fee: - SHOULD reply with a `closing_signed` with the same `fee_satoshis` value. - otherwise: - MUST propose a value "strictly between" the received `fee_satoshis` and its previously-sent `fee_satoshis`.

The receiving node: - if one of the outputs in the closing transaction is below the dust limit for its `scriptpubkey` (see BOLT 3): - MUST fail the channel

Rationale

When `fee_range` is not provided, the "strictly between" requirement ensures that forward progress is made, even if only by a single satoshi at a time. To avoid keeping state and to handle the corner case, where fees have shifted between disconnection and reconnection, negotiation restarts on reconnection.

Note there is limited risk if the closing transaction is delayed, but it will be broadcast very soon; so there is usually no reason to pay a premium for rapid processing.

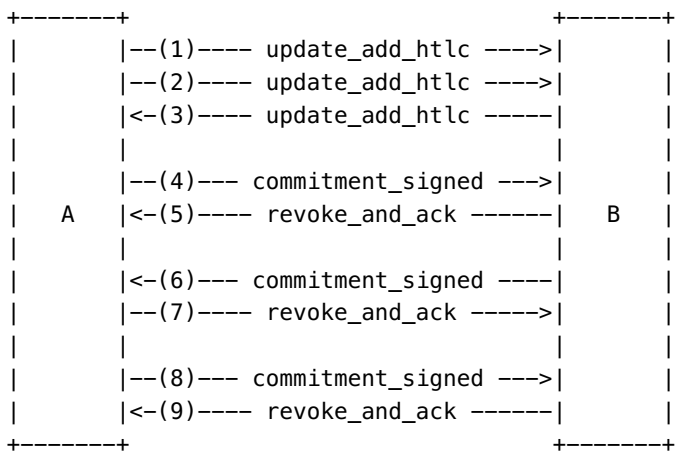
Note that the non-funder is not paying the fee, so there is no reason for it to have a maximum feerate. It may want a minimum feerate, however, to ensure that the transaction propagates. It can always use CPFP later to speed up confirmation if necessary, so that minimum should be low.

It may happen that the closing transaction doesn't meet bitcoin's default relay policies (e.g. when using a non-segwit shutdown script for an output below 546 satoshis, which is possible if `dust_limit_satoshis` is below 546 satoshis). No funds are at risk when that happens, but the channel must be force-closed as the closing transaction will likely never reach miners.

Normal Operation

Once both nodes have exchanged `channel_ready` (and optionally `announcement_signatures`), the channel can be used to make payments via Hashed Time Locked Contracts.

Changes are sent in batches: one or more `update_` messages are sent before a `commitment_signed` message, as in the following diagram:



Counter-intuitively, these updates apply to the *other node's* commitment transaction; the node only adds those updates to its own commitment transaction when the remote node acknowledges it has applied them via `revoke_and_ack`.

Thus each update traverses through the following states:

1. pending on the receiver
2. in the receiver's latest commitment transaction
3. … and the receiver's previous commitment transaction has been revoked, and the update is pending on the sender
4. … and in the sender's latest commitment transaction
5. … and the sender's previous commitment transaction has been revoked

As the two nodes' updates are independent, the two commitment transactions may be out of sync indefinitely. This is not concerning: what matters is whether both sides have irrevocably committed to a particular update or not (the final state, above).

Forwarding HTLCs

In general, a node offers HTLCs for two reasons: to initiate a payment of its own, or to forward another node's payment. In the forwarding case, care must be taken to ensure the *outgoing* HTLC cannot be redeemed unless the *incoming* HTLC can be redeemed. The following requirements ensure this is always true.

The respective **addition/removal** of an HTLC is considered *irrevocably committed* when:

1. The commitment transaction **with/without** it is committed to by both nodes, and any previous commitment transaction **without/with** it has been revoked, OR
2. The commitment transaction **with/without** it has been irreversibly committed to the blockchain.

Requirements

A node: - until an incoming HTLC has been irrevocably committed: - MUST NOT offer the corresponding outgoing HTLC (`update_add_htlc`) in response to that incoming HTLC. - until the removal of an outgoing HTLC is irrevocably committed, OR until the outgoing on-chain HTLC output has been spent via the HTLC-timeout transaction (with sufficient depth): - MUST NOT fail the incoming HTLC (`update_fail_htlc`) that corresponds to that outgoing HTLC. - once the `cltv_expiry` of an incoming HTLC has been reached, OR if `cltv_expiry` minus `current_height` is less than `cltv_expiry_delta` for the corresponding outgoing HTLC: - MUST fail that incoming HTLC (`update_fail_htlc`). - if an incoming HTLC's `cltv_expiry` is unreasonably far in the future: - SHOULD fail that incoming HTLC (`update_fail_htlc`). - upon receiving an `update_fulfill_htlc` for an outgoing HTLC, OR upon discovering the `payment_preimage` from an on-chain HTLC spend: - MUST fulfill the incoming HTLC that corresponds to that outgoing HTLC.

Rationale

In general, one side of the exchange needs to be dealt with before the other. Fulfilling an HTLC is different: knowledge of the preimage is, by definition, irrevocable and the incoming HTLC should be fulfilled as soon as possible to reduce latency.

An HTLC with an unreasonably long expiry is a denial-of-service vector and therefore is not allowed. Note that the exact value of "unreasonable" is currently unclear and may depend on network topology.

`cltv_expiry_delta` Selection

Once an HTLC has timed out, it can either be fulfilled or timed-out; care must be taken around this transition, both for offered and received HTLCs.

Consider the following scenario, where A sends an HTLC to B, who forwards to C, who delivers the goods as soon as the payment is received.

1. C needs to be sure that the HTLC from B cannot time out, even if B becomes unresponsive; i.e. C can fulfill the incoming HTLC on-chain before B can time it out on-chain.

2. B needs to be sure that if C fulfills the HTLC from B, it can fulfill the incoming HTLC from A; i.e. B can get the preimage from C and fulfill the incoming HTLC on-chain before A can time it out on-chain.

The critical settings here are the `cltv_expiry_delta` in BOLT #7 and the related `min_final_cltv_expiry_delta` in BOLT #11. `cltv_expiry_delta` is the minimum difference in HTLC CLTV timeouts, in the forwarding case (B). `min_final_cltv_expiry_delta` is the minimum difference between HTLC CLTV timeout and the current block height, for the terminal case (C).

Note that a node is at risk if it accepts an HTLC in one channel and offers an HTLC in another channel with too small of a difference between the CLTV timeouts. For this reason, the `cltv_expiry_delta` for the *outgoing* channel is used as the delta across a node.

The worst-case number of blocks between outgoing and incoming HTLC resolution can be derived, given a few assumptions:

- a worst-case reorganization depth `R` blocks
- a grace-period `G` blocks after HTLC timeout before giving up on an unresponsive peer and dropping to chain
- a number of blocks `S` between transaction broadcast and the transaction being included in a block

The worst case is for a forwarding node (B) that takes the longest possible time to spot the outgoing HTLC fulfillment and also takes the longest possible time to redeem it on-chain:

1. The B->C HTLC times out at block `N`, and B waits `G` blocks until it gives up waiting for C. B or C commits to the blockchain, and B spends HTLC, which takes `S` blocks to be included.
2. Bad case: C wins the race (just) and fulfills the HTLC, B only sees that transaction when it sees block `N+G+S+1`.
3. Worst case: There's reorganization `R` deep in which C wins and fulfills. B only sees transaction at `N+G+S+R`.
4. B now needs to fulfill the incoming A->B HTLC, but A is unresponsive: B waits `G` more blocks before giving up waiting for A. A or B commits to the blockchain.
5. Bad case: B sees A's commitment transaction in block `N+G+S+R+G+1` and has to spend the HTLC output, which takes `S` blocks to be mined.
6. Worst case: there's another reorganization `R` deep which A uses to spend the commitment transaction, so B sees A's commitment transaction in block `N+G+S+R+G+R` and has to spend the HTLC output, which takes `S` blocks to be mined.
7. B's HTLC spend needs to be at least `R` deep before it times out, otherwise another reorganization could allow A to timeout the transaction.

Thus, the worst case is `3R+2G+2S`, assuming `R` is at least 1. Note that the chances of three reorganizations in which the other node wins all of them is low for `R` of 2 or more. Since high fees are used (and HTLC spends can use almost arbitrary fees), `S` should be small during normal operation; although, given that block times are irregular, empty blocks still occur, fees may vary greatly, and the fees cannot be bumped on HTLC transactions, `S=12` should be considered a minimum. `S` is also the parameter that may vary the most under attack, so a higher value may be desirable when non-negligible amounts are at risk. The grace period `G` can be low (1 or 2), as nodes are required to timeout or fulfill as soon as possible; but if `G` is too low it increases the risk of unnecessary channel closure due to networking delays.

There are four values that need be derived:

1. the `cltv_expiry_delta` for channels, `3R+2G+2S`: if in doubt, a `cltv_expiry_delta` of at least 34 is reasonable (R=2, G=2, S=12).

2. the deadline for offered HTLCs: the deadline after which the channel has to be failed and timed out on-chain. This is `G` blocks after the HTLC's `cltv_expiry`: 1 or 2 blocks is reasonable.

3. the deadline for received HTLCs this node has fulfilled: the deadline after which the channel has to be failed and the HTLC fulfilled on-chain before its `cltv_expiry`. See steps 4-7 above, which imply a deadline of `2R+G+S` blocks before `cltv_expiry`: 18 blocks is reasonable.

4. the minimum `cltv_expiry` accepted for terminal payments: the worst case for the terminal node C is `2R+G+S` blocks (as, again, steps 1-3 above don't apply). The default in BOLT #11 is 18, which matches this calculation.

Requirements

An offering node: - MUST estimate a timeout deadline for each HTLC it offers. - MUST NOT offer an HTLC with a timeout deadline before its `cltv_expiry`. - if an HTLC which it offered is in either node's current commitment transaction, AND is past this timeout deadline: - SHOULD send an `error` to the receiving peer (if connected). - MUST fail the channel.

A fulfilling node: - for each HTLC it is attempting to fulfill: - MUST estimate a fulfillment deadline. - MUST fail (and not forward) an HTLC whose fulfillment deadline is already past. - if an HTLC it has fulfilled is in either node's current commitment transaction, AND is past this fulfillment deadline: - SHOULD send an `error` to the offering peer (if connected). - MUST fail the channel.

Bounding exposure to trimmed in-flight HTLCs: `max_dust_htlc_exposure_msat`

When an HTLC in a channel is below the "trimmed" threshold in BOLT3 #3, the HTLC cannot be claimed on-chain, instead being turned into additional miner fees if either party unilaterally closes the channel. Because the threshold is per-HTLC, the total exposure to such HTLCs may be substantial if there are many dust HTLCs committed when the channel is force-closed.

This can be exploited in griefing attacks or even in miner-extractable-value attacks, if the malicious entity wins mining capabilities.

The total exposure is given by the following back-of-the-envelope computation:

```
remote `max_accepted_htlcs` * (`HTLC-success-kiloweight` * `feerate_per_kw` + remote
`dust_limit_satoshis`)
    + local `max_accepted_htlcs` * (`HTLC-timeout-kiloweight` * `feerate_per_kw` + remote
`dust_limit_satoshis`)
```

To mitigate this scenario, a `max_dust_htlc_exposure_msat` threshold can be applied when sending, forwarding and receiving HTLCs.

A node: - when receiving an HTLC: - if the HTLC's `amount_msat` is smaller than the remote `dust_limit_satoshis` plus the HTLC-timeout fee at `feerate_per_kw`: - if the `amount_msat` plus the dust balance of the remote transaction is greater than `max_dust_htlc_exposure_msat`: - SHOULD fail this HTLC once it's committed - SHOULD NOT reveal a preimage for this HTLC - if the HTLC's `amount_msat` is smaller than the local `dust_limit_satoshis` plus the HTLC-success fee at `feerate_per_kw`: - if the `amount_msat` plus the dust balance of the local transaction is greater than `max_dust_htlc_exposure_msat`: - SHOULD fail this HTLC once it's committed - SHOULD NOT reveal a preimage for this HTLC - when offering an HTLC: - if the HTLC's `amount_msat` is smaller than the remote `dust_limit_satoshis` plus the HTLC-success fee at `feerate_per_kw`: - if the `amount_msat` plus the dust balance of the remote transaction is greater than `max_dust_htlc_exposure_msat`: - SHOULD NOT send this HTLC - SHOULD fail the corresponding incoming HTLC (if any) - if the HTLC's `amount_msat` is inferior to the holder's `dust_limit_satoshis` plus the HTLC-timeout fee at the `feerate_per_kw`: - if the `amount_msat` plus the dust balance of the local transaction is greater

than `max_dust_htlc_exposure_msat`: - SHOULD NOT send this HTLC - SHOULD fail the corresponding incoming HTLC (if any)

The `max_dust_htlc_exposure_msat` is an upper bound on the trimmed balance from dust exposure. The exact value used is a matter of node policy.

For channels that don't use `zero_fee_commitments` or `option_anchors`, an increase of the `feerate_per_kw` may trim multiple htlcs from commitment transactions, which could create a large increase in dust exposure.

Adding an HTLC: `update_add_htlc`

Either node can send `update_add_htlc` to offer an HTLC to the other, which is redeemable in return for a payment preimage. Amounts are in millisatoshi, though on-chain enforcement is only possible for whole satoshi amounts greater than the dust limit (in commitment transactions these are rounded down as specified in BOLT #3).

The format of the `onion_routing_packet` portion, which indicates where the payment is destined, is described in BOLT #4.

1. type: 128 (`update_add_htlc`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u64`:`id`]
    - [`u64`:`amount_msat`]
    - [`sha256`:`payment_hash`]
    - [`u32`:`cltv_expiry`]
    - [`1366*byte`:`onion_routing_packet`]
3. `tlv_stream`: `update_add_htlc_tlvs`
4. types:
    1. type: 0 (`blinded_path`)
    2. data:
        - [`point`:`path_key`]

Requirements

A sending node: - if `zero_fee_commitments` has not been negotiated: - if it is *responsible* for paying the Bitcoin fee: - MUST NOT offer `amount_msat` if, after adding that HTLC to its commitment transaction, it cannot pay the fee for either the local or remote commitment transaction at the current `feerate_per_kw` while maintaining its channel reserve (see Updating Fees). - if `option_anchors` applies to this commitment transaction and the sending node is the funder: - MUST be able to additionally pay for `to_local_anchor` and `to_remote_anchor` above its reserve. - SHOULD NOT offer `amount_msat` if, after adding that HTLC to its commitment transaction, its remaining balance doesn't allow it to pay the commitment transaction fee when receiving or sending a future additional non-dust HTLC while maintaining its channel reserve. It is recommended that this "fee spike buffer" can handle twice the current `feerate_per_kw` to ensure predictability between implementations. - if it is *not responsible* for paying the Bitcoin fee: - SHOULD NOT offer `amount_msat` if, once the remote node adds that HTLC to its commitment transaction, it cannot pay the fee for the updated local or remote transaction at the current `feerate_per_kw` while maintaining its channel reserve. - MUST offer `amount_msat` greater than 0. - MUST NOT offer `amount_msat` below the receiving node's `htlc_minimum_msat` - MUST set `cltv_expiry` less

than 500000000. - if result would be offering more than the remote's `max_accepted_htlcs` HTLCs, in the remote commitment transaction: - MUST NOT add an HTLC. - if the total value of offered HTLCs would exceed the remote's `max_htlc_value_in_flight_msat`: - MUST NOT add an HTLC. - for the first HTLC it offers: - MUST set `id` to 0. - MUST increase the value of `id` by 1 for each successive offer. - if it is relaying a payment inside a blinded route: - MUST set `path_key` (see Route Blinding) - if a splice is pending: - MUST ensure that requirements are met for all commmitment transactions.

`id` MUST NOT be reset to 0 after the update is complete (i.e. after `revoke_and_ack` has been received). It MUST continue incrementing instead.

A receiving node: - receiving an `amount_msat` equal to 0, OR less than its own `htlc_minimum_msat`: - SHOULD send a `warning` and close the connection, or send an `error` and fail the channel. - receiving an `amount_msat` that the sending node cannot afford at the current `feerate_per_kw` (while maintaining its channel reserve and any `to_local_anchor` and `to_remote_anchor` costs): - SHOULD send a `warning` and close the connection, or send an `error` and fail the channel. - if a sending node adds more than receiver `max_accepted_htlcs` HTLCs to its local commitment transaction, OR adds more than receiver `max_htlc_value_in_flight_msat` worth of offered HTLCs to its local commitment transaction: - SHOULD send a `warning` and close the connection, or send an `error` and fail the channel. - if sending node sets `cltv_expiry` to greater or equal to 500000000: - SHOULD send a `warning` and close the connection, or send an `error` and fail the channel. - MUST allow multiple HTLCs with the same `payment_hash`. - if the sender did not previously acknowledge the commitment of that HTLC: - MUST ignore a repeated `id` value after a reconnection. - if other `id` violations occur: - MAY send a `warning` and close the connection, or send an `error` and fail the channel. - MUST decrypt `onion_routing_packet` as described in Onion Decryption to extract a `payload`. - MUST use `path_key` (if specified). - MUST use `payment_hash` as `associated_data`. - If decryption fails, the result is not a valid `payload` TLV, or it contains unknown even types: - MUST respond with an error as detailed in Failure Messages - Otherwise: - MUST follow the requirements for the reader of `payload` in Payload Format - If a splice is pending: - MUST ensure that requirements are met for all commmitment transactions.

The `onion_routing_packet` contains an obfuscated list of hops and instructions for each hop along the path. It commits to the HTLC by setting the `payment_hash` as associated data, i.e. includes the `payment_hash` in the computation of HMACs. This prevents replay attacks that would reuse a previous `onion_routing_packet` with a different `payment_hash`.

Rationale

Invalid amounts are a clear protocol violation and indicate a breakdown.

If a node did not accept multiple HTLCs with the same payment hash, an attacker could probe to see if a node had an existing HTLC. This requirement, to deal with duplicates, leads to the use of a separate identifier; it's assumed a 64-bit counter never wraps.

Retransmissions of unacknowledged updates are explicitly allowed for reconnection purposes; allowing them at other times simplifies the recipient code (though strict checking may help debugging).

`max_accepted_htlcs` is limited to 483 to ensure that, even if both sides send the maximum number of HTLCs, the `commitment_signed` message will still be under the maximum message size. It also ensures that a single penalty transaction can spend the entire commitment transaction, as calculated in BOLT #5. When using `zero_fee_commitments`, we further limit `max_accepted_htlcs` to 114 because v3 transactions are limited to 10kvB, which decreases the number of outputs the commitment transaction can have.

`cltv_expiry` values equal to or greater than 500000000 would indicate a time in seconds, and the protocol only supports an expiry in blocks.

When `zero_fee_commitments` is not used, the node *responsible* for paying the Bitcoin fee should maintain a "fee spike buffer" on top of its reserve to accommodate a future fee increase. Without this buffer, the node *responsible* for paying the Bitcoin fee may reach a state where it is unable to send or receive any non-dust HTLC while maintaining its channel reserve (because of the increased weight of the commitment transaction), resulting in a degraded channel. See #728 for more details.

If splicing is supported, there can be more than one commitment transaction at a time: proposed changes must be valid for all of them.

Removing an HTLC: `update_fulfill_htlc`, `update_fail_htlc`, and `update_fail_malformed_htlc`

For simplicity, a node can only remove HTLCs added by the other node. There are four reasons for removing an HTLC: the payment preimage is supplied, it has timed out, it has failed to route, or it is malformed.

To supply the preimage:

1. type: 130 (`update_fulfill_htlc`)
2. data:
    - [`channel_id:channel_id`]
    - [`u64:id`]
    - [`32*byte:payment_preimage`]
3. `tlv_stream`: `update_fulfill_htlc_tlvs`
4. types:
    1. type: 1 (`attribution_data`)
    2. data:
        - [`20*u32:htlc_hold_times`]
        - [`210*sha256[..4]:truncated_hmacs`]
    3. type: 3 (`fulfillment_payload`)
    4. data:
        - [`...*byte:fulfillment_payload`]

The `fulfillment_payload` field is an opaque encrypted blob for the benefit of the original HTLC initiator, as defined in BOLT #4. It mirrors the `reason` field of `update_fail_htlc`: the final node originates it, and intermediate nodes obfuscate and relay it.

For a timed out or route-failed HTLC:

1. type: 131 (`update_fail_htlc`)
2. data:
    - [`channel_id:channel_id`]
    - [`u64:id`]
    - [`u16:len`]
    - [`len*byte:reason`]
3. `tlv_stream`: `update_fail_htlc_tlvs`
4. types:
    1. type: 1 (`attribution_data`)

2. data:
    - [20*u32:htlc_hold_times]
    - [210*sha256[..4]:truncated_hmacs]

The reason field is an opaque encrypted blob for the benefit of the original HTLC initiator, as defined in BOLT #4; however, there's a special malformed failure variant for the case where the peer couldn't parse it: in this case the current node instead takes action, encrypting it into a update_fail_htlc for relaying.

For an unparsable HTLC:

1. type: 135 (update_fail_malformed_htlc)
2. data:
    - [channel_id:channel_id]
    - [u64:id]
    - [sha256:sha256_of_onion]
    - [u16:failure_code]

Requirements

A node: - SHOULD remove an HTLC as soon as it can. - SHOULD fail an HTLC which has timed out. - until the corresponding HTLC is irrevocably committed in both sides' commitment transactions: - MUST NOT send an update_fulfill_htlc, update_fail_htlc, or update_fail_malformed_htlc. - When failing an incoming HTLC: - If current_path_key is set in the onion payload and it is not the final node: - MUST send an update_fail_htlc error using the invalid_onion_blinding failure code for any local or downstream errors. - SHOULD use the sha256_of_onion of the onion it received. - MAY use an all zero sha256_of_onion. - SHOULD add a random delay before sending update_fail_htlc. - If path_key is set in the incoming update_add_htlc: - MUST send an update_fail_malformed_htlc error using the invalid_onion_blinding failure code for any local or downstream errors. - SHOULD use the sha256_of_onion of the onion it received. - MAY use an all zero sha256_of_onion. - When supporting option_attribution_data: - if path_key is not set in the incoming update_add_htlc: - MUST initialize attribution_data and include it in update_fail_htlc and update_fulfill_htlc. See BOLT04. - if a fulfillment_payload is present in the update_fulfill_htlc received from the downstream node: - MUST obfuscate and relay it in the outgoing update_fulfill_htlc, including in a blinded path. See BOLT04.

A receiving node: - if the id does not correspond to an HTLC in its current commitment transaction: - MUST send a warning and close the connection, or send an error and fail the channel. - if the payment_preimage value in update_fulfill_htlc doesn't SHA256 hash to the corresponding HTLC payment_hash: - MUST send a warning and close the connection, or send an error and fail the channel. - if the fulfillment_payload in update_fulfill_htlc is longer than 32768 bytes (32 KiB): - MUST send an error and fail the channel. - if the BADONION bit in failure_code is not set for update_fail_malformed_htlc: - MUST send a warning and close the connection, or send an error and fail the channel. - if the sha256_of_onion in update_fail_malformed_htlc doesn't match the onion it sent and is not all zero: - MAY retry or choose an alternate error response. - otherwise, a receiving node which has an outgoing HTLC canceled by update_fail_malformed_htlc: - MUST return an error in the update_fail_htlc sent to the link which originally sent the HTLC, using the failure_code given and setting the data to sha256_of_onion.

Rationale

A node that doesn't time out HTLCs risks channel failure (see `cltv_expiry_delta` Selection).

A node that sends `update_fulfill_htlc`, before the sender, is also committed to the HTLC and risks losing funds.

If the onion is malformed, the upstream node won't be able to extract the shared key to generate a response — hence the special failure message, which makes this node do it.

The node can check that the SHA256 that the upstream is complaining about does match the onion it sent, which may allow it to detect random bit errors. However, without re-checking the actual encrypted packet sent, it won't know whether the error was its own or the remote's; so such detection is left as an option.

Nodes inside a blinded route must use `invalid_onion_blinding` to avoid leaking information to senders trying to probe the blinded route.

Batching channel messages

Multiple channel messages can be grouped together as a single logical message using the `start_batch` message.

1. type: 127 (`start_batch`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u16`:`batch_size`]
3. `tlv_stream`: `start_batch_tlvs`
4. types:
    1. type: 1 (`message_type`)
    2. data:
    - [`u16`:`message_type`]

Requirements

A sending node: - MUST set `batch_size` to a value strictly greater than 1. - MUST set `batch_size` to a value lower than or equal to 20. - MUST set `message_type` to `132` (`commitment_signed` message type as defined in Bolt 1). - After sending `start_batch`: - MUST send `batch_size` `commitment_signed` messages for the same `channel_id`, without any other unrelated messages in-between.

A receiving node: - If `batch_size` is not strictly greater than 1: - MUST ignore the `start_batch` message. - SHOULD send a `warning`. - If `batch_size` is strictly greater than 20: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - MUST group the next `batch_size` messages and process them together. - If one of those messages is not for the specified `channel_id`: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - If `message_type` is missing or not set to the type for `commitment_signed`: - MUST ignore the `start_batch` message and process the following messages sequentially.

Rationale

The `start_batch` message is only used when sending multiple `commitment_signed` messages during splicing. We thus narrow the requirements for this specific scenario: they can be relaxed in the future if we use `start_batch` for other features.

We limit the `batch_size` to 20 elements to protect against excessive queuing that could be abused to DoS receiving nodes. The `start_batch` message is only used for splice RBF attempts so far, which shouldn't need that many attempts to get transactions confirmed.

Committing Updates So Far: `commitment_signed`

When a node has changes for the remote commitment, it can apply them, sign the resulting transaction (as defined in BOLT #3), and send a `commitment_signed` message.

1. type: 132 (`commitment_signed`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`signature`:`signature`]
    - [`u16`:`num_htlcs`]
    - [`num_htlcs*signature`:`htlc_signature`]
    - [`commitment_signed_tlvs`:`tlvs`]
3. `tlv_stream`: `commitment_signed_tlvs`
4. types:
    1. type: 1 (`funding_txid`)
    2. data:
    - [`sha256`:`funding_txid`]

Requirements

A sending node: - MUST NOT send a `commitment_signed` message that does not include any updates. - MAY send a `commitment_signed` message that only alters the fee. - MAY send a `commitment_signed` message that doesn't change the commitment transaction aside from the new revocation number (due to dust, identical HTLC replacement, or insignificant or multiple fee changes). - MUST include one `htlc_signature` for every HTLC transaction corresponding to the ordering of the commitment transaction (see BOLT #3). - MUST set `funding_txid` to the funding transaction spent by this commitment transaction. - if it has not recently received a message from the remote node: - SHOULD use `ping` and await the reply `pong` before sending `commitment_signed`. - If there are `N` pending splice transactions (with `N` greater than `0`): - MUST send `start_batch` first with `batch_size` set to `N + 1` and `message_type` set to `132` (`commitment_signed`). - MUST send `commitment_signed` for the current channel funding output. - MUST send `commitment_signed` for each of the splice transactions. - MUST set `funding_txid` in each `commitment_signed` message to match the funding transaction spent by that commitment transaction. - MUST NOT send any other message before it has sent the batch of `commitment_signed` messages.

A receiving node: - once all pending updates are applied: - if `signature` is not valid for its local commitment transaction OR non-compliant with LOW-S-standard rule LOWS: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - if `num_htlcs` is not equal to the number of HTLC outputs in the local commitment transaction: - MUST send a `warning` and close the connection, or send an `error` and fail

the channel. - if any `htlc_signature` is not valid for the corresponding HTLC transaction OR non-compliant with LOW-S-standard rule LOWS: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - If there are pending splice transactions and the sending node did not send `start_batch` followed by a batch of `commitment_signed` messages: - MUST send an `error` and fail the channel. - If the sending node sent `start_batch` and we are processing a batch of `commitment_signed` messages: - If `funding_txid` is missing in one of the `commitment_signed` messages: - MUST send an `error` and fail the channel. - If there are pending splice transactions: - MUST validate each `commitment_signed` based on `funding_txid`. - If `commitment_signed` is missing for a funding transaction: - MUST send an `error` and fail the channel. - Otherwise: - MUST respond with a `revoke_and_ack` message. - Otherwise (no pending splice transactions): - MUST ignore `commitment_signed` where `funding_txid` does not match the current funding transaction. - If `commitment_signed` is missing for the current funding transaction: - MUST send an `error` and fail the channel. - Otherwise: - MUST respond with a `revoke_and_ack` message. - Otherwise: - MUST respond with a `revoke_and_ack` message.

Rationale

There's little point offering spam updates: it implies a bug.

The `num_htlcs` field is redundant, but makes the packet length check fully self-contained.

The recommendation to require recent messages recognizes the reality that networks are unreliable: nodes might not realize their peers are offline until after sending `commitment_signed`. Once `commitment_signed` is sent, the sender considers itself bound to those HTLCs, and cannot fail the related incoming HTLCs until the output HTLCs are fully resolved.

Note that the `htlc_signature` implicitly enforces the time-lock mechanism in the case of offered HTLCs being timed out or received HTLCs being spent. This is done to reduce fees by creating smaller scripts compared to explicitly stating time-locks on HTLC outputs.

Splicing requires us to send and receive additional signatures, as we don't know which (if any) of the splice transactions will end up being the new channel funding transaction. We use `start_batch` to send a batch of `commitment_signed` messages containing signatures for each of the pending splice transactions and for the current funding transaction. After sending `splice_locked`, we may receive a batch containing obsolete `commitment_signed` messages from our peer that they started sending before receiving our `splice_locked`: we can safely ignore them by filtering on `funding_txid`.

Completing the Transition to the Updated State: `revoke_and_ack`

Once the recipient of `commitment_signed` checks the signature and knows it has a valid new commitment transaction, it replies with the commitment preimage for the previous commitment transaction in a `revoke_and_ack` message.

This message also implicitly serves as an acknowledgment of receipt of the `commitment_signed`, so this is a logical time for the `commitment_signed` sender to apply (to its own commitment) any pending updates it sent before that `commitment_signed`.

The description of key derivation is in BOLT #3.

1. type: 133 (`revoke_and_ack`)

2. data:
    - [channel_id:channel_id]
    - [32*byte:per_commitment_secret]
    - [point:next_per_commitment_point]

Requirements

A sending node: - MUST set `per_commitment_secret` to the secret used to generate keys for the previous commitment transaction. - MUST set `next_per_commitment_point` to the values for its next commitment transaction. - MUST send a single `revoke_and_ack` message, even if it is responding to a batch of `commitment_signed` messages.

A receiving node: - if `per_commitment_secret` is not a valid secret key or does not generate the previous `per_commitment_point`: - MUST send an `error` and fail the channel. - if the `per_commitment_secret` was not generated by the protocol in BOLT #3: - MAY send a `warning` and close the connection, or send an `error` and fail the channel.

A node: - MUST NOT broadcast old (revoked) commitment transactions, - Note: doing so will allow the other node to seize all channel funds. - SHOULD NOT sign commitment transactions, unless it's about to broadcast them (due to a failed connection), - Note: this is to reduce the above risk.

Updating Fees: `update_fee`

For channels that don't use `zero_fee_commitments`, an `update_fee` message is sent by the node which is paying the Bitcoin fee. Like any update, it's first committed to the receiver's commitment transaction and then (once acknowledged) committed to the sender's. Unlike an HTLC, `update_fee` is never closed but simply replaced.

There is a possibility of a race, as the recipient can add new HTLCs before it receives the `update_fee`. Under this circumstance, the sender may not be able to afford the fee on its own commitment transaction, once the `update_fee` is finally acknowledged by the recipient. In this case, the fee will be less than the fee rate, as described in BOLT #3.

The exact calculation used for deriving the fee from the fee rate is given in BOLT #3.

1. type: 134 (`update_fee`)
2. data:
    - [channel_id:channel_id]
    - [u32:feerate_per_kw]

Requirements

When using `zero_fee_commitments`, nodes: - MUST NOT send `update_fee`.

The node *responsible* for paying the Bitcoin fee: - SHOULD send `update_fee` to ensure the current fee rate is sufficient (by a significant margin) for timely processing of the commitment transaction.

The node *not responsible* for paying the Bitcoin fee: - MUST NOT send `update_fee`.

A sending node: - if `option_anchors` was not negotiated: - if the `update_fee` increases `feerate_per_kw`: - if the dust balance of the remote transaction at the updated `feerate_per_kw` is greater than

`max_dust_htlc_exposure_msat`: - MAY NOT send `update_fee` - MAY fail the channel - if the dust balance of the local transaction at the updated `feerate_per_kw` is greater than `max_dust_htlc_exposure_msat`: - MAY NOT send `update_fee` - MAY fail the channel - if a splice is pending: - MUST ensure that requirements are met for all commmitment transactions.

A receiving node: - if `zero_fee_commitments` is used: - MUST send an `error` and fail the channel. - if the `update_fee` is too low for timely processing, OR is unreasonably large: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - if the sender is not responsible for paying the Bitcoin fee: - MUST send a `warning` and close the connection, or send an `error` and fail the channel. - if the sender cannot afford the new fee rate on the receiving node's current commitment transaction: - SHOULD send a `warning` and close the connection, or send an `error` and fail the channel. - but MAY delay this check until the `update_fee` is committed. - if `option_anchors` was not negotiated: - if the `update_fee` increases `feerate_per_kw`: - if the dust balance of the remote transaction at the updated `feerate_per_kw` is greater then `max_dust_htlc_exposure_msat`: - MAY fail the channel - if the dust balance of the local transaction at the updated `feerate_per_kw` is greater than `max_dust_htlc_exposure_msat`: - MAY fail the channel - if a splice is pending: - MUST ensure that requirements are met for all commmitment transactions.

Rationale

Bitcoin fees are required for unilateral closes to be effective. With `option_anchors`, `feerate_per_kw` is not as critical anymore to guarantee confirmation as it was in the legacy commitment format, but it still needs to be enough to be able to enter the mempool (satisfy min relay fee and mempool min fee). With `zero_fee_commitments`, transactions can enter the mempool as a package with a child transaction paying the fees: channel transactions thus don't need to directly include a fee, so this message isn't used.

For the legacy commitment format, there is no general method for the broadcasting node to use child-pays-for-parent to increase its effective fee.

Given the variance in fees, and the fact that the transaction may be spent in the future, it's a good idea for the fee payer to keep a good margin (say 5x the expected fee requirement) for legacy commitment txes; but, due to differing methods of fee estimation, an exact value is not specified.

Since the fees are currently one-sided (the party which requested the channel creation always pays the fees for the commitment transaction), it's simplest to only allow it to set fee levels; however, as the same fee rate applies to HTLC transactions, the receiving node must also care about the reasonableness of the fee.

If on-chain fees increase while commitments contain many HTLCs that will be trimmed at the updated feerate, this could overflow the configured `max_dust_htlc_exposure_msat`. Whether to close the channel preemptively or not is left as a matter of node policy.

If splicing is supported, there can be more than one commitment transaction at a time: proposed changes must be valid for all of them.

Message Retransmission

Because communication transports are unreliable, and may need to be re-established from time to time, the design of the transport has been explicitly separated from the protocol.

Nonetheless, it's assumed our transport is ordered and reliable. Reconnection introduces doubt as to what has been received, so there are explicit acknowledgments at that point.

This is fairly straightforward in the case of channel establishment and close, where messages have an explicit order, but during normal operation, acknowledgments of updates are delayed until the `commitment_signed` / `revoke_and_ack` exchange; so it cannot be assumed that the updates have been received. This also means that the receiving node only needs to store updates upon receipt of `commitment_signed`.

Note that messages described in BOLT #7 are independent of particular channels; their transmission requirements are covered there, and besides being transmitted after `init` (as all messages are), they are independent of requirements here.

1. type: 136 (`channel_reestablish`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`u64`:`next_commitment_number`]
    - [`u64`:`next_revocation_number`]
    - [`32*byte`:`your_last_per_commitment_secret`]
    - [`point`:`my_current_per_commitment_point`]
3. `tlv_stream`: `channel_reestablish_tlvs`
4. types:
    1. type: 1 (`next_funding`)
    2. data:
        - [`sha256`:`next_funding_txid`]
        - [`byte`:`retransmit_flags`]
    3. type: 5 (`my_current_funding_locked`)
    4. data:
        - [`sha256`:`my_current_funding_locked_txid`]
        - [`byte`:`retransmit_flags`]

`next_commitment_number`: A commitment number is a 48-bit incrementing counter for each commitment transaction; counters are independent for each peer in the channel and start at 0. They're only explicitly relayed to the other node in the case of re-establishment, otherwise they are implicit.

The `next_funding.retransmit_flags` bitfield is used to let the receiving peer know which messages they must retransmit for the corresponding `next_funding_txid` after the reconnection:

Bit Position	Name
0	`commitment_signed`

The `my_current_funding_locked.retransmit_flags` bitfield is used to let our peer know which messages we expect them to retransmit after the reconnection:

Bit Position	Name
0	`announcement_signatures`

Requirements

A funding node: - upon disconnection: - if it has broadcast the funding transaction: - MUST remember the channel for reconnection. - otherwise: - SHOULD NOT remember the channel for reconnection.

A non-funding node: - upon disconnection: - if it has sent the `funding_signed` message: - MUST remember the channel for reconnection. - otherwise: - SHOULD NOT remember the channel for reconnection.

A node: - MUST handle continuation of a previous channel on a new encrypted transport. - upon disconnection: - MUST reverse any uncommitted updates sent by the other side (i.e. all messages beginning with `update_` for which no `commitment_signed` has been received). - Note: a node MAY have already used the `payment_preimage` value from the `update_fulfill_htlc`, so the effects of `update_fulfill_htlc` are not completely reversed. - upon reconnection: - if a channel is in an error state: - SHOULD retransmit the error packet and ignore any other packets for that channel. - otherwise: - MUST transmit `channel_reestablish` for each channel. - MUST wait to receive the other node's `channel_reestablish` message before sending any other messages for that channel.

The sending node: - MUST set `next_commitment_number` to the commitment number of the next `commitment_signed` it expects to receive. - MUST set `next_revocation_number` to the commitment number of the next `revoke_and_ack` message it expects to receive. - MUST set `my_current_per_commitment_point` to a valid point. - if `next_revocation_number` equals 0: - MUST set `your_last_per_commitment_secret` to all zeroes - otherwise: - MUST set `your_last_per_commitment_secret` to the last `per_commitment_secret` it received - if it has sent `commitment_signed` for an interactive transaction construction but it has not received `tx_signatures`: - MUST include the `next_funding` TLV. - MUST set `next_funding_txid` to the txid of that interactive transaction. - if it has not received `commitment_signed` for this `next_funding_txid`: - MUST set the `commitment_signed` bit in `retransmit_flags`. - otherwise: - MUST NOT include the `next_funding` TLV. - if `option_splice` was negotiated: - if a splice transaction reached acceptable depth while disconnected: - MUST include `my_current_funding_locked` with the txid of the latest such transaction. - otherwise, if it has already sent `splice_locked` for any transaction: - MUST include `my_current_funding_locked` with the txid of the last `splice_locked` it sent. - otherwise, if it has already sent `channel_ready`: - MUST include `my_current_funding_locked` with the txid of the channel funding transaction. - otherwise (it has never sent `channel_ready` or `splice_locked`): - MUST NOT include `my_current_funding_locked`. - if `my_current_funding_locked` is included: - if `announce_channel` is set for this channel: - if it has not received `announcement_signatures` for that transaction: - MUST set the `announcement_signatures` bit to `1` in `retransmit_flags`. - otherwise: - MUST set the `announcement_signatures` bit to `0` in `retransmit_flags`.

A node: - if `next_commitment_number` is zero: - MUST immediately fail the channel and broadcast any relevant latest commitment transaction. - if `next_commitment_number` is 1 in both the `channel_reestablish` it sent and received, and none of those `channel_reestablish` messages contain `my_current_funding_locked` or `next_funding` for a splice transaction: - MUST retransmit `channel_ready`. - otherwise: - MUST NOT retransmit `channel_ready`, but MAY send `channel_ready` with a different `short_channel_id` `alias` field. - upon reconnection: - MUST ignore any redundant `channel_ready` it receives. - if `next_commitment_number` is equal to the commitment number of the last `commitment_signed` message the receiving node has sent: - MUST reuse the same commitment number for its next `commitment_signed`. - otherwise: - if `next_commitment_number` is not equal to the commitment number of the next `commitment_signed` that the receiving node would send: - SHOULD send an `error` and fail the channel. - if `next_revocation_number` is equal to the commitment number of the last `revoke_and_ack` the receiving node sent, AND the receiving node hasn't already received a `closing_signed`: - MUST re-send the `revoke_and_ack`. - if it has previously sent a `commitment_signed` that

needs to be retransmitted: - MUST retransmit `revoke_and_ack` and `commitment_signed` in the same relative order as initially transmitted. - otherwise: - if `next_revocation_number` is not equal to 1 greater than the commitment number of the last `revoke_and_ack` the receiving node has sent: - SHOULD send an `error` and fail the channel. - if it has not sent `revoke_and_ack`, AND `next_revocation_number` is not equal to 0: - SHOULD send an `error` and fail the channel.

A receiving node: - MUST ignore `my_current_per_commitment_point`, but MAY require it to be a valid point. - if `next_revocation_number` is greater than expected above, AND `your_last_per_commitment_secret` is correct for that `next_revocation_number` minus 1: - MUST NOT broadcast its commitment transaction. - SHOULD send an `error` to request the peer to fail the channel. - otherwise: - if `your_last_per_commitment_secret` does not match the expected values: - SHOULD send an `error` and fail the channel.

A receiving node: - if the `next_funding` TLV is set: - if `next_funding_txid` matches the latest interactive funding transaction: - if it has not received `tx_signatures` for that funding transaction: - if the `commitment_signed` bit is set in `retransmit_flags`: - MUST retransmit its `commitment_signed` for that funding transaction. - if it has already received `commitment_signed` and it should sign first, as specified in the `tx_signatures` requirements: - MUST send its `tx_signatures` for that funding transaction. - if it has already received `tx_signatures` for that funding transaction: - MUST send its `tx_signatures` for that funding transaction. - if it also sets `next_funding` in its own `channel_reestablish`, but the values don't match: - MUST send an `error` and fail the channel. - otherwise: - MUST send `tx_abort` to let the sending node know that they can forget this funding transaction.

A receiving node: - if splice transactions are pending and `my_current_funding_locked` matches one of those splice transactions, for which it hasn't received `splice_locked` yet: - MUST process `my_current_funding_locked` as if it was receiving `splice_locked` for this `txid`. - if `my_current_funding_locked` is included with the `announcement_signatures` bit set in the `retransmit_flags`: - if `announce_channel` is set for this channel and the receiving node is ready to send `announcement_signatures` for the corresponding splice transaction: - MUST retransmit `announcement_signatures`.

A node: - MUST NOT assume that previously-transmitted messages were lost, - if it has sent a previous `commitment_signed` message: - MUST handle the case where the corresponding commitment transaction is broadcast at any time by the other side, - Note: this is particularly important if the node does not simply retransmit the exact `update_` messages as previously sent. - upon reconnection: - if it has sent a previous `shutdown`: - MUST retransmit `shutdown`.

Rationale

The requirements above ensure that the opening phase is nearly atomic: if it doesn't complete, it starts again. The only exception is if the `funding_signed` message is sent but not received. In this case, the funder will forget the channel, and presumably open a new one upon reconnection; meanwhile, the other node will eventually forget the original channel, due to never receiving `channel_ready` or seeing the funding transaction on-chain.

There's no acknowledgment for `error`, so if a reconnect occurs it's polite to retransmit before disconnecting again; however, it's not a MUST, because there are also occasions where a node can simply forget the channel altogether.

`closing_signed` also has no acknowledgment so must be retransmitted upon reconnection (though negotiation restarts on reconnection, so it needs not be an exact retransmission). The only acknowledgment for `shutdown` is `closing_signed`, so one or the other needs to be retransmitted.

The handling of updates is similarly atomic: if the commit is not acknowledged (or wasn't sent) the updates are re-sent. However, it's not insisted they be identical: they could be in a different order, involve different fees, or even be missing HTLCs which are now too old to be added. Requiring they be identical would effectively mean a write to disk by the sender upon each transmission, whereas the scheme here encourages a single persistent write to disk for each `commitment_signed` sent or received. But if you need to retransmit both a `commitment_signed` and a `revoke_and_ack`, the relative order of these two must be preserved, otherwise it will lead to a channel closure.

A re-transmittal of `revoke_and_ack` should never be asked for after a `closing_signed` has been received, since that would imply a shutdown has been completed — which can only occur after the `revoke_and_ack` has been received by the remote node.

Note that the `next_commitment_number` starts at 1, since commitment number 0 is created during opening. `next_revocation_number` will be 0 until the `commitment_signed` for commitment number 1 is send and then the revocation for commitment number 0 is received.

`channel_ready` is implicitly acknowledged by the start of normal operation, which is known to have begun after a `commitment_signed` has been received (hence, the test for a `next_commitment_number` greater than 1) or after a splice transaction has been created.

A node, which has somehow fallen behind (e.g. has been restored from old backup), can detect that it has fallen behind. A fallen-behind node must know it cannot broadcast its current commitment transaction — which would lead to total loss of funds — as the remote node can prove it knows the revocation preimage. The `error` returned by the fallen-behind node should make the other node drop its current commitment transaction to the chain. The other node should wait for that `error` to give the fallen-behind node an opportunity to fix its state first (e.g by restarting with a different backup).

The `next_funding` TLV allows peers to finalize the signing steps of an interactive transaction construction, or safely abort that transaction if it was not signed by one of the peers, who has thus already removed it from its state.

`my_current_funding_locked` is equivalent to sending `splice_locked`, but is handled atomically during `channel_reestablish` (instead of requiring a retransmission of the `splice_locked` message). This is useful to avoid race conditions with channel updates (for more details about this race condition, see this example). It handles the case where the `splice_locked` message was lost during the disconnection, or when a splice transaction reaches acceptable depth while peers are disconnected. It also allows requesting a retransmission of the `announcement_signatures` message for the latest splice transaction, in case it wasn't received before disconnecting.

Authors

[FIXME: Insert Author List]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

BOLT #3: Bitcoin Transaction and Script Formats

This details the exact format of on-chain transactions, which both sides need to agree on to ensure signatures are valid. This consists of the funding transaction output script, the commitment transactions, and the HTLC transactions.

Table of Contents

- Transactions
  - Transaction Output Ordering
  - Use of Segwit
  - Funding Transaction Output
  - Commitment Transaction
    - Commitment Transaction Outputs
      - `to_local` Output
      - `to_remote` Output
      - `to_local_anchor` and `to_remote_anchor`
      - `shared_anchor`
      - Offered HTLC Outputs
      - Received HTLC Outputs
    - Trimmed Outputs
  - HTLC-timeout and HTLC-success Transactions
  - Legacy Closing Transaction
  - Closing Transaction
  - Fees
    - Fee Calculation
    - Fee Payment
    - Calculating Fees for Collaborative Transactions
  - Dust Limits
  - Commitment Transaction Construction
- Keys
  - Key Derivation
    - `localpubkey`, `local_htlcpubkey`, `remote_htlcpubkey`, `local_delayedpubkey`, and `remote_delayedpubkey` Derivation
    - `remotepubkey` Derivation
    - `revocationpubkey` Derivation
    - Per-commitment Secret Requirements
  - Efficient Per-commitment Secret Storage
- Appendix A: Expected Weights
  - Expected Weight of the Funding Transaction (v2 Channel Establishment)
  - Expected Weight of the Commitment Transaction
  - Expected Weight of HTLC-timeout and HTLC-success Transactions
- Appendix B: Funding Transaction Test Vectors
- Appendix C: Commitment and HTLC Transaction Test Vectors
- Appendix D: Per-commitment Secret Generation Test Vectors
  - Generation Tests
  - Storage Tests

- Appendix E: Key Derivation Test Vectors
- Appendix F: Commitment and HTLC Transaction Test Vectors (anchors)
- Appendix G: Dual Funded Transaction Test Vectors
- Appendix H: Commitment and HTLC Transaction Test Vectors (zero-fee-commitments)
- References
- Authors

Transactions

Transaction Output Ordering

Outputs in transactions are always sorted according to: * first according to their value, smallest first (in whole satoshis, note that for HTLC outputs, the millisatoshi part must be ignored) * followed by `scriptpubkey`, comparing the common-length prefix lexicographically as if by `memcmp`, then selecting the shorter script (if they differ in length), * finally, for HTLC outputs, in increasing `cltv_expiry` order.

Rationale

Two offered HTLCs which have the same `amount` (rounded from `amount_msat`) and `payment_hash` will have identical outputs, even if their `cltv_expiry` differs. This only matters because the same ordering is used to send `htlc_signatures` and the HTLC transactions themselves are different, thus the two peers must agree on the canonical ordering for this case.

Use of Segwit

Most transaction outputs used here are pay-to-witness-script-hash[BIP141] (P2WSH) outputs: the Segwit version of P2SH. To spend such outputs, the last item on the witness stack must be the actual script that was used to generate the P2WSH output that is being spent. This last item has been omitted for brevity in the rest of this document.

A <> designates an empty vector as required for compliance with MINIMALIF-standard rule.[MINIMALIF]

Funding Transaction Output

- The funding output script is a P2WSH to:

```
2 <pubkey1> <pubkey2> 2 OP_CHECKMULTISIG
```

- Where `pubkey1` is the lexicographically lesser of the two `funding_pubkey` in compressed format, and where `pubkey2` is the lexicographically greater of the two.

Commitment Transaction

- version: 3 when using `zero_fee_commitments`, otherwise 2
- locktime: upper 8 bits are 0x20, lower 24 bits are the lower 24 bits of the obscured commitment number
- txin count: 1
    - `txin[0]` outpoint: `txid` and `output_index` from `funding_created` message

    - `txin[0]` sequence: upper 8 bits are 0x80, lower 24 bits are upper 24 bits of the obscured commitment number
    - `txin[0]` script bytes: 0
    - `txin[0]` witness: `0 <signature_for_pubkey1> <signature_for_pubkey2>`

The 48-bit commitment number is obscured by `XOR` with the lower 48 bits of:

```
SHA256(payment_basepoint from open_channel || payment_basepoint from accept_channel)
```

This obscures the number of commitments made on the channel in the case of unilateral close, yet still provides a useful index for both nodes (who know the `payment_basepoints`) to quickly find a revoked commitment transaction.

Commitment Transaction Outputs

To allow an opportunity for penalty transactions, in case of a revoked commitment transaction, all outputs that return funds to the owner of the commitment transaction (a.k.a. the "local node") must be delayed for `to_self_delay` blocks. For HTLCs this delay is done in a second-stage HTLC transaction (HTLC-success for HTLCs accepted by the local node, HTLC-timeout for HTLCs offered by the local node).

The reason for the separate transaction stage for HTLC outputs is so that HTLCs can timeout or be fulfilled even though they are within the `to_self_delay` delay. Otherwise, the required minimum timeout on HTLCs is lengthened by this delay, causing longer timeouts for HTLCs traversing the network.

The amounts for each output MUST be rounded down to whole satoshis. If this amount, minus the fees for the HTLC transaction, is less than the `dust_limit_satoshis` set by the owner of the commitment transaction, the output MUST NOT be produced (thus the funds add to fees). If `zero_fee_commitments` is used, the amount may be partially added to the `shared_anchor` as detailed here.

`to_local` Output

This output sends funds back to the owner of this commitment transaction and thus must be timelocked using `OP_CHECKSEQUENCEVERIFY`. It can be claimed, without delay, by the other party if they know the revocation private key. The output is a version-0 P2WSH, with a witness script:

```
OP_IF
    # Penalty transaction
    <revocationpubkey>
OP_ELSE
    `to_self_delay`
    OP_CHECKSEQUENCEVERIFY
    OP_DROP
    <local_delayedpubkey>
OP_ENDIF
OP_CHECKSIG
```

The output is spent by an input with `nSequence` field set to `to_self_delay` (which can only be valid after that duration has passed) and witness:

```
<local_delayedsig> <>
```

If a revoked commitment transaction is published, the other party can spend this output immediately with the following witness:

```
<revocation_sig> 1
```

`to_remote` Output

If `option_anchors` applies to the commitment transaction, the `to_remote` output is encumbered by a one block csv lock.

```
<remotepubkey> OP_CHECKSIGVERIFY 1 OP_CHECKSEQUENCEVERIFY
```

The output is spent by an input with `nSequence` field set to `1` and witness:

```
<remote_sig>
```

Otherwise, this output is a simple P2WPKH to `remotepubkey`.

`to_local_anchor` and `to_remote_anchor` Output (option_anchors)

This output can be spent by the local and remote nodes respectively to provide incentive to mine the transaction, using child-pays-for-parent. Both anchor outputs are always added, except for the case where there are no htlcs and one of the parties has a commitment output that is below the dust limit. In that case only an anchor is added for the commitment output that does materialize. This typically happens if the initiator closes right after opening (no `to_remote` output).

```
<local_funding_pubkey/remote_funding_pubkey> OP_CHECKSIG OP_IFDUP
OP_NOTIF
    OP_16 OP_CHECKSEQUENCEVERIFY
OP_ENDIF
```

Each party has its own anchor output that locks to their funding key. This is to prevent a malicious peer from attaching child transactions with a low fee density to an anchor and thereby blocking the victim from getting the commit tx confirmed in time. This defense is supported by a change in Bitcoin core 0.19: https://github.com/bitcoin/bitcoin/pull/15681. This is also the reason that every non-anchor output on the commit tx is CSV locked.

To prevent utxo set pollution, any anchor that remains unspent can be spent by anyone after the commitment tx confirms. This is also the reason to lock the anchor outputs to the funding key. Third parties can observe this key and reconstruct the spend script, even if none of the commitment outputs would be spent. This does assume that at some point the fee market goes down to a level where sweeping the anchors is economical.

The amount of the output is fixed at 330 sats, the default dust limit for P2WSH.

Spending of the output requires the following witness:

```
<local_sig/remote_sig>
```

After 16 blocks, anyone can sweep the anchor with witness:

```
<>
```

`shared_anchor` Output (zero_fee_commitments)

This output can be spent by anyone to provide incentive to mine the transaction, using child-pays-for-parent. This output is only added when the commitment transaction is using version 3 (`zero_fee_commitments`) and prevents pinning attacks on the commitment transaction.

It is using the following standard P2A (pay-to-anchor) script:

```
OP_1 <0x4e73>
```

The standard dust limit for this type of script is `240 sat`. However, if the parent transaction doesn't pay any on-chain fees, using values below this dust limit is allowed under the "ephemeral dust" rule.

If the difference between the commitment transaction input and the sum of all other commitment transaction outputs is smaller than `240 sat`, we set the anchor amount to this value. This ensures that the commitment transaction doesn't pay any on-chain fees whenever the anchor amount is smaller than `240 sat`.

Otherwise, we set the anchor amount to `240 sat`, and the remaining difference between the commitment transaction input and its outputs directly contributes to the transaction's mining fees (which is standard since the `shared_anchor` has reached its dust limit).

Spending of the output requires the following (empty) witness:

```
<>
```

Offered HTLC Outputs

This output sends funds to either an HTLC-timeout transaction after the HTLC-timeout or to the remote node using the payment preimage or the revocation key. The output is a P2WSH, with a witness script (no `option_anchors`):

```
# To remote node with revocation key
OP_DUP OP_HASH160 <RIPEMD160(SHA256(revocationpubkey))> OP_EQUAL
OP_IF
    OP_CHECKSIG
OP_ELSE
    <remote_htlcpubkey> OP_SWAP OP_SIZE 32 OP_EQUAL
    OP_NOTIF
        # To local node via HTLC-timeout transaction (timelocked).
        OP_DROP 2 OP_SWAP <local_htlcpubkey> 2 OP_CHECKMULTISIG
    OP_ELSE
        # To remote node with preimage.
        OP_HASH160 <RIPEMD160(payment_hash)> OP_EQUALVERIFY
        OP_CHECKSIG
    OP_ENDIF
OP_ENDIF
```

Or, with `option_anchors`:

```
# To remote node with revocation key
OP_DUP OP_HASH160 <RIPEMD160(SHA256(revocationpubkey))> OP_EQUAL
OP_IF
    OP_CHECKSIG
```

```
OP_ELSE
    <remote_htlcpubkey> OP_SWAP OP_SIZE 32 OP_EQUAL
    OP_NOTIF
        # To local node via HTLC-timeout transaction (timelocked).
        OP_DROP 2 OP_SWAP <local_htlcpubkey> 2 OP_CHECKMULTISIG
    OP_ELSE
        # To remote node with preimage.
        OP_HASH160 <RIPEMD160(payment_hash)> OP_EQUALVERIFY
        OP_CHECKSIG
    OP_ENDIF
    1 OP_CHECKSEQUENCEVERIFY OP_DROP
OP_ENDIF
```

The remote node can redeem the HTLC with the witness:

```
<remotehtlcsig> <payment_preimage>
```

Note that if `option_anchors` applies, the nSequence field of the spending input must be `1`.

If a revoked commitment transaction is published, the remote node can spend this output immediately with the following witness:

```
<revocation_sig> <revocationpubkey>
```

The sending node can use the HTLC-timeout transaction to timeout the HTLC once the HTLC is expired, as shown below. This is the only way that the local node can timeout the HTLC, and this branch requires `<remotehtlcsig>`, which ensures that the local node cannot prematurely timeout the HTLC since the HTLC-timeout transaction has `cltv_expiry` as its specified `locktime`. The local node must also wait `to_self_delay` before accessing these funds, allowing for the remote node to claim these funds if the transaction has been revoked.

Received HTLC Outputs

This output sends funds to either the remote node after the HTLC-timeout or using the revocation key, or to an HTLC-success transaction with a successful payment preimage. The output is a P2WSH, with a witness script (no `option_anchors`):

```
# To remote node with revocation key
OP_DUP OP_HASH160 <RIPEMD160(SHA256(revocationpubkey))> OP_EQUAL
OP_IF
    OP_CHECKSIG
OP_ELSE
    <remote_htlcpubkey> OP_SWAP OP_SIZE 32 OP_EQUAL
    OP_IF
        # To local node via HTLC-success transaction.
        OP_HASH160 <RIPEMD160(payment_hash)> OP_EQUALVERIFY
        2 OP_SWAP <local_htlcpubkey> 2 OP_CHECKMULTISIG
    OP_ELSE
        # To remote node after timeout.
        OP_DROP <cltv_expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP
        OP_CHECKSIG
    OP_ENDIF
OP_ENDIF
```

Or, with `option_anchors`:

```
# To remote node with revocation key
OP_DUP OP_HASH160 <RIPEMD160(SHA256(revocationpubkey))> OP_EQUAL
OP_IF
    OP_CHECKSIG
OP_ELSE
    <remote_htlcpubkey> OP_SWAP OP_SIZE 32 OP_EQUAL
    OP_IF
        # To local node via HTLC-success transaction.
        OP_HASH160 <RIPEMD160(payment_hash)> OP_EQUALVERIFY
        2 OP_SWAP <local_htlcpubkey> 2 OP_CHECKMULTISIG
    OP_ELSE
        # To remote node after timeout.
        OP_DROP <cltv_expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP
        OP_CHECKSIG
    OP_ENDIF
    1 OP_CHECKSEQUENCEVERIFY OP_DROP
OP_ENDIF
```

To timeout the HTLC, the remote node spends it with the witness:

```
<remotehtlcsig> <>
```

Note that if `option_anchors` applies, the nSequence field of the spending input must be `1`.

If a revoked commitment transaction is published, the remote node can spend this output immediately with the following witness:

```
<revocation_sig> <revocationpubkey>
```

To redeem the HTLC, the HTLC-success transaction is used as detailed below. This is the only way that the local node can spend the HTLC, since this branch requires `<remotehtlcsig>`, which ensures that the local node must wait `to_self_delay` before accessing these funds allowing for the remote node to claim these funds if the transaction has been revoked.

Trimmed Outputs

Each peer specifies a `dust_limit_satoshis` below which outputs should not be produced; these outputs that are not produced are termed “trimmed”. A trimmed output is considered too small to be worth creating: it is instead either added to the commitment transaction fee or, if `zero_fee_commitments` applies, to the shared anchor output.

For HTLCs, it needs to be taken into account that the second-stage HTLC transaction may also be below the limit. Note that when using `option_anchors` or `zero_fee_commitments`, HTLC transactions don’t include a fee and thus don’t contribute to trimming: setting a higher `dust_limit_satoshis` makes sense for those channels to ensure that outputs are economical to spend.

Requirements

Before the commitment transaction outputs are determined: - the base fee MUST be subtracted from the `to_local` or `to_remote` output, as specified in Fee Calculation. - if `option_anchors` applies: - the

`to_local_anchor` output value MUST be subtracted from the funder's output. - the `to_remote_anchor` output value MUST be subtracted from the funder's output.

The commitment transaction: - if the amount of the commitment transaction `to_local` output would be less than `dust_limit_satoshis` set by the transaction owner: - MUST NOT contain that output. - otherwise: - MUST be generated as specified in `to_local` Output. - if the amount of the commitment transaction `to_remote` output would be less than `dust_limit_satoshis` set by the transaction owner: - MUST NOT contain that output. - otherwise: - MUST be generated as specified in `to_remote` Output. - for every offered HTLC: - if the HTLC amount minus the HTLC-timeout fee would be less than `dust_limit_satoshis` set by the transaction owner: - MUST NOT contain that output. - otherwise: - MUST be generated as specified in Offered HTLC Outputs. - for every received HTLC: - if the HTLC amount minus the HTLC-success fee would be less than `dust_limit_satoshis` set by the transaction owner: - MUST NOT contain that output. - otherwise: - MUST be generated as specified in Received HTLC Outputs. - if `zero_fee_commitments` applies: - MUST add a `shared_anchor` output with an amount computed as detailed in the `shared_anchor` section.

HTLC-Timeout and HTLC-Success Transactions

These HTLC transactions are almost identical, except the HTLC-timeout transaction is timelocked. Both HTLC-timeout/HTLC-success transactions can be spent by a valid penalty transaction.

- version: 3 when using `zero_fee_commitments`, otherwise 2
- locktime: `0` for HTLC-success, `cltv_expiry` for HTLC-timeout
- txin count: 1
    - `txin[0]` outpoint: `txid` of the commitment transaction and `output_index` of the matching HTLC output for the HTLC transaction
    - `txin[0]` sequence: `1` for `option_anchors`, `0` otherwise
    - `txin[0]` script bytes: `0`
    - `txin[0]` witness stack: `0 <remotehtlcsig> <localhtlcsig> <payment_preimage>` for HTLC-success, `0 <remotehtlcsig> <localhtlcsig> <>` for HTLC-timeout
- txout count: 1
    - `txout[0]` amount: the HTLC `amount_msat` divided by 1000 (rounding down) minus fees in satoshis (see Fee Calculation)
    - `txout[0]` script: version-0 P2WSH with witness script as shown below
- if `option_anchors` or `zero_fee_commitments` applies to this commitment transaction, `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` is used as described in BOLT #5.

The witness script for the output is:

```
OP_IF
    # Penalty transaction
    <revocationpubkey>
OP_ELSE
    `to_self_delay`
    OP_CHECKSEQUENCEVERIFY
    OP_DROP
    <local_delayedpubkey>
OP_ENDIF
OP_CHECKSIG
```

To spend this via penalty, the remote node uses a witness stack `<revocationsig> 1`, and to collect the output, the local node uses an input with nSequence `to_self_delay` and a witness stack `<local_delayedsig> 0`.

Legacy Closing Transaction

This variant is used for `closing_signed` messages (i.e. where `option_simple_close` is not negotiated).

Note that there are two possible variants for each node.

- version: 2
- locktime: 0
- txin count: 1
    - `txin[0]` outpoint: `txid` and `output_index` from `funding_created` message
    - `txin[0]` sequence: 0xFFFFFFFF
    - `txin[0]` script bytes: 0
    - `txin[0]` witness: `0 <signature_for_pubkey1> <signature_for_pubkey2>`
- txout count: 1 or 2
    - `txout` amount: final balance to be paid to one node (minus `fee_satoshis` from `closing_signed`, if this peer funded the channel)
    - `txout` script: as specified in that node's `scriptpubkey` in its `shutdown` message

Requirements

Each node offering a signature: - MUST round each output down to whole satoshis. - MUST subtract the fee given by `fee_satoshis` from the output to the funder. - MUST remove any output below its own `dust_limit_satoshis`. - MAY eliminate its own output.

Rationale

There is a possibility of irreparable differences on closing if one node considers the other's output too small to allow propagation on the Bitcoin network (a.k.a. "dust"), and that other node instead considers that output too valuable to discard. This is why each side uses its own `dust_limit_satoshis`, and the result can be a signature validation failure, if they disagree on what the closing transaction should look like.

However, if one side chooses to eliminate its own output, there's no reason for the other side to fail the closing protocol; so this is explicitly allowed. The signature indicates which variant has been used.

There will be at least one output, if the funding amount is greater than twice `dust_limit_satoshis`.

Closing Transaction

This variant is used for `closing_complete` and `closing_sig` messages (i.e. where `option_simple_close` is negotiated).

In this case, the node sending `closing_complete` ("the closer") pays the fees. The outputs are ordered as detailed in Transaction Output Ordering.

The side with lesser funds can opt to omit their own output.

- version: 2

- locktime: `locktime` from the `closing_complete` message
- txin count: 1
    - `txin[0]` outpoint: `txid` and `output_index` of the channel output
    - `txin[0]` sequence: 0xFFFFFFFD
    - `txin[0]` script bytes: 0
    - `txin[0]` witness: `0 <signature_for_pubkey1> <signature_for_pubkey2>`
- txout count: 1 or 2
    - The closer output:
        - `txout` amount:
            - 0 if the `scriptpubkey` starts with `OP_RETURN`
            - otherwise the final balance for the closer, minus `closing_complete.fee_satoshis`, rounded down to whole satoshis
        - `txout` script: as specified in `closer_scriptpubkey` from the `closing_complete` message
    - The closee output:
        - `txout` amount:
            - 0 if the `scriptpubkey` starts with `OP_RETURN`
            - otherwise the final balance for the closee, rounded down to whole satoshis
        - `txout` script: as specified in `closee_scriptpubkey` from the `closing_complete` message

Requirements

Each node offering a signature: - MUST round each output down to whole satoshis. - MUST subtract the fee given by `fee_satoshis` from the closer output. - MUST set the output amount to 0 if the `scriptpubkey` is `OP_RETURN`.

Fees

Fee Calculation

The fee calculation for both commitment transactions and HTLC transactions is based on the current `feerate_per_kw` and the *expected weight* of the transaction. Note that `feerate_per_kw` is set to `0` when using `zero_fee_commitments`: in that case the commitment transactions and HTLC transactions don't pay any fee, which must be paid using child-pays-for-parent (for the commitment transactions) or by adding inputs (for the HTLC transactions).

The actual and expected weights vary for several reasons:

- Bitcoin uses DER-encoded signatures, which vary in size.
- Bitcoin also uses variable-length integers, so a large number of outputs will take 3 bytes to encode rather than 1.
- The `to_remote` output may be below the dust limit.
- The `to_local` output may be below the dust limit once fees are extracted.

Thus, a simplified formula for *expected weight* is used, which assumes:

- Signatures are 73 bytes long (the maximum length).
- There are a small number of outputs (thus 1 byte to count them).
- There are always both a `to_local` output and a `to_remote` output.

- (if `option_anchors`) there are always both a `to_local_anchor` and `to_remote_anchor` output.

This yields the following *expected weights* (details of the computation in Appendix A):

```
Commitment weight (no option_anchors):   724 + 172 * num-untrimmed-htlc-outputs
Commitment weight (option_anchors):     1124 + 172 * num-untrimmed-htlc-outputs
HTLC-timeout weight (no option_anchors): 663
HTLC-timeout weight (option_anchors): 666
HTLC-success weight (no option_anchors): 703
HTLC-success weight (option_anchors): 706
```

Note the reference to the "base fee" for a commitment transaction in the requirements below, which is what the funder pays. The actual fee may be higher than the amount calculated here, due to rounding and trimmed outputs.

Requirements

The fee for an HTLC-timeout transaction: - If `option_anchors` or `zero_fee_commitments` applies: 1. MUST be 0. - Otherwise, MUST be calculated to match: 1. Multiply `feerate_per_kw` by 663 and divide by 1000 (rounding down).

The fee for an HTLC-success transaction: - If `option_anchors` or `zero_fee_commitments` applies: 1. MUST be 0. - Otherwise, MUST be calculated to match: 1. Multiply `feerate_per_kw` by 703 and divide by 1000 (rounding down).

The base fee for a commitment transaction: - If `zero_fee_commitments` applies: 1. MUST be 0. - Otherwise, MUST be calculated to match: 1. Start with `weight` = 724 (1124 if `option_anchors` applies). 2. For each committed HTLC, if that output is not trimmed as specified in Trimmed Outputs, add 172 to `weight`. 3. Multiply `feerate_per_kw` by `weight`, divide by 1000 (rounding down).

Example

For example, suppose there is a `feerate_per_kw` of 5000, a `dust_limit_satoshis` of 546 satoshis, and a commitment transaction with: * two offered HTLCs of 5000000 and 1000000 millisatoshis (5000 and 1000 satoshis) * two received HTLCs of 7000000 and 800000 millisatoshis (7000 and 800 satoshis)

The HTLC-timeout transaction `weight` is 663, and thus the fee is 3315 satoshis. The HTLC-success transaction `weight` is 703, and thus the fee is 3515 satoshis

The commitment transaction `weight` is calculated as follows:

- `weight` starts at 724.
- The offered HTLC of 5000 satoshis is above 546 + 3315 and results in:
    - an output of 5000 satoshi in the commitment transaction
    - an HTLC-timeout transaction of 5000 - 3315 satoshis that spends this output
    - `weight` increases to 896
- The offered HTLC of 1000 satoshis is below 546 + 3315 so it is trimmed.

- The received HTLC of 7000 satoshis is above 546 + 3515 and results in:
    - an output of 7000 satoshi in the commitment transaction
    - an HTLC-success transaction of 7000 - 3515 satoshis that spends this output
    - `weight` increases to 1068
- The received HTLC of 800 satoshis is below 546 + 3515 so it is trimmed.

The base commitment transaction fee is 5340 satoshi; the actual fee (which adds the 1000 and 800 satoshi HTLCs that would make dust outputs) is 7140 satoshi. The final fee may be even higher if the `to_local` or `to_remote` outputs fall below `dust_limit_satoshis`.

Fee Payment

Base commitment transaction fees and amounts for `to_local_anchor` and `to_remote_anchor` outputs are extracted from the funder's amount.

When using `zero_fee_commitments`, the `shared_anchor` output amount is taken from trimmed outputs, as detailed in the `shared_anchor` section.

Note that after the fee amount is subtracted from the to-funder output, that output may be below `dust_limit_satoshis`, and thus will also contribute to fees (or, if `zero_fee_commitments` applies, to the `shared_anchor`).

A node: - if `zero_fee_commitments` is not used and the resulting fee rate is too low: - MAY send a `warning` and close the connection, or send an `error` and fail the channel.

Calculating Fees for Collaborative Transactions

For transactions constructed using the interactive protocol, fees are paid by each party to the transaction, at a `feerate` determined during the initiation, with the initiator covering the fees for the common transaction fields.

Dust Limits

The `dust_limit_satoshis` parameter is used to configure the threshold below which nodes will not produce on-chain transaction outputs.

There is no consensus rule in Bitcoin that makes outputs below dust thresholds invalid or unspendable, but policy rules in popular implementations will prevent relaying transactions that contain such outputs.

Bitcoin Core defines the following dust thresholds:

- pay to pubkey hash (p2pkh): 546 satoshis
- pay to script hash (p2sh): 540 satoshis
- pay to witness pubkey hash (p2wpkh): 294 satoshis
- pay to witness script hash (p2wsh): 330 satoshis
- pay to anchor (p2a): 240 satoshis
- unknown segwit versions: 354 satoshis
- `OP_RETURN` outputs: these are never dust

The rationale of this calculation (implemented here) is explained in the following sections.

In all these sections, the calculations are done with a feerate of 3000 sat/kB as per Bitcoin Core's implementation.

Note that since the introduction of `option_anchors`, the second-stage HTLC transaction's weight is not taken into account when deciding whether outputs should be included in the commitment transaction or not. It thus makes sense to use a `dust_limit_satoshis` that takes into account the cost of those second-stage HTLC transactions, to ensure that outputs added to the commitment transaction can actually be claimed on-chain, otherwise they may pollute the utxo set indefinitely. At a minimum, nodes should allow their peer to use a `dust_limit_satoshis` that is higher than the values defined by Bitcoin Core. We cannot predict future feerates, so this will not always work and can still result in HTLC outputs that are unspendable if the on-chain fees are too high. We cannot use very large `dust_limit_satoshis` values either since it would create too much dust exposure in the commitment transaction (more details here).

Pay to pubkey hash (p2pkh)

A p2pkh output is 34 bytes:

- 8 bytes for the output amount
- 1 byte for the script length
- 25 bytes for the script (`OP_DUP` `OP_HASH160` `20` 20-bytes `OP_EQUALVERIFY` `OP_CHECKSIG`)

A p2pkh input is at least 148 bytes:

- 36 bytes for the previous output (32 bytes hash + 4 bytes index)
- 4 bytes for the sequence
- 1 byte for the script sig length
- 107 bytes for the script sig:
    - 1 byte for the items count
    - 1 byte for the signature length
    - 71 bytes for the signature
    - 1 byte for the public key length
    - 33 bytes for the public key

The p2pkh dust threshold is then `(34 + 148) * 3000 / 1000 = 546 satoshis`

Pay to script hash (p2sh)

A p2sh output is 32 bytes:

- 8 bytes for the output amount
- 1 byte for the script length
- 23 bytes for the script (`OP_HASH160` `20` 20-bytes `OP_EQUAL`)

A p2sh input doesn't have a fixed size, since it depends on the underlying script, so we use 148 bytes as a lower bound.

The p2sh dust threshold is then `(32 + 148) * 3000 / 1000 = 540 satoshis`

Pay to witness pubkey hash (p2wpkh)

A p2wpkh output is 31 bytes:

- 8 bytes for the output amount
- 1 byte for the script length
- 22 bytes for the script (`OP_0` `20` 20-bytes)

A p2wpkh input is at least 67 bytes (depending on the signature length):

- 36 bytes for the previous output (32 bytes hash + 4 bytes index)
- 4 bytes for the sequence
- 1 byte for the script sig length
- 26 bytes for the witness (rounded down from 26.75, with the 75% segwit discount applied):
    - 1 byte for the items count
    - 1 byte for the signature length
    - 71 bytes for the signature
    - 1 byte for the public key length
    - 33 bytes for the public key

The p2wpkh dust threshold is then `(31 + 67) * 3000 / 1000 = 294 satoshis`

Pay to witness script hash (p2wsh)

A p2wsh output is 43 bytes:

- 8 bytes for the output amount
- 1 byte for the script length
- 34 bytes for the script (`OP_0` `32` 32-bytes)

A p2wsh input doesn't have a fixed size, since it depends on the underlying script, so we use 67 bytes as a lower bound.

The p2wsh dust threshold is then `(43 + 67) * 3000 / 1000 = 330 satoshis`

Unknown segwit versions

Unknown segwit outputs are at most 51 bytes:

- 8 bytes for the output amount
- 1 byte for the script length
- 42 bytes for the script (`OP_1` through `OP_16` inclusive, followed by a single push of 2 to 40 bytes)

The input doesn't have a fixed size, since it depends on the underlying script, so we use 67 bytes as a lower bound.

The unknown segwit version dust threshold is then `(51 + 67) * 3000 / 1000 = 354 satoshis`

Commitment Transaction Construction

This section ties the previous sections together to detail the algorithm for constructing the commitment transaction for one peer: given that peer's `dust_limit_satoshis`, the current `feerate_per_kw`, the amounts due to each peer (`to_local` and `to_remote`), and all committed HTLCs:

- Initialize the commitment transaction input and locktime, as specified in Commitment Transaction.
- Calculate which committed HTLCs need to be trimmed (see Trimmed Outputs).
- Calculate the base commitment transaction fee.
- Subtract this base fee from the funder (either `to_local` or `to_remote`). If `option_anchors` applies to the commitment transaction, also subtract two times the fixed anchor size of 330 sats from the funder (either `to_local` or `to_remote`).
- For every offered HTLC, if it is not trimmed, add an offered HTLC output.
- For every received HTLC, if it is not trimmed, add a received HTLC output.
- If the `to_local` amount is greater or equal to `dust_limit_satoshis`, add a `to_local` output.
- If the `to_remote` amount is greater or equal to `dust_limit_satoshis`, add a `to_remote` output.
- If `option_anchors` applies to the commitment transaction:
    - if `to_local` exists or there are untrimmed HTLCs, add a `to_local_anchor` output.
    - if `to_remote` exists or there are untrimmed HTLCs, add a `to_remote_anchor` output.
- If `zero_fee_commitments` applies to the commitment transaction:
    - add a `shared_anchor` output.
- Sort the outputs into BIP 69+CLTV order.

Keys

Key Derivation

Each commitment transaction uses a unique `localpubkey`. The HTLC-success and HTLC-timeout transactions use `local_delayedpubkey` and `revocationpubkey`. These are changed for every transaction based on the `per_commitment_point`.

The reason for key change is so that trustless watching for revoked transactions can be outsourced. Such a *watcher* should not be able to determine the contents of a commitment transaction — even if the *watcher* knows which transaction ID to watch for and can make a reasonable guess as to which HTLCs and balances may be included. Nonetheless, to avoid storage of every commitment transaction, a *watcher* can be given the `per_commitment_secret` values (which can be stored compactly) and the `revocation_basepoint` and `delayed_payment_basepoint` used to regenerate the scripts required for the penalty transaction; thus, a *watcher* need only be given (and store) the signatures for each penalty input.

Changing the `localpubkey` every time ensures that commitment transaction ID cannot be guessed except in the trivial case where there is no `to_local` output, as every commitment transaction uses an ID in its output script. Splitting the `local_delayedpubkey`, which is required for the penalty transaction, allows it to be shared with the *watcher* without revealing `localpubkey`; even if both peers use the same *watcher*, nothing is revealed.

Finally, even in the case of normal unilateral close, the HTLC-success and/or HTLC-timeout transactions do not reveal anything to the *watcher*, as it does not know the corresponding `per_commitment_secret` and cannot relate the `local_delayedpubkey` or `revocationpubkey` with their bases.

For efficiency, keys are generated from a series of per-commitment secrets that are generated from a single seed, which allows the receiver to compactly store them (see below).

`localpubkey`, `local_htlcpubkey`, `remote_htlcpubkey`, `local_delayedpubkey`, and `remote_delayedpubkey` Derivation

These pubkeys are simply generated by addition from their base points:

```
pubkey = basepoint + SHA256(per_commitment_point || basepoint) * G
```

- The `localpubkey` uses the local node's `payment_basepoint`;
- The `local_htlcpubkey` uses the local node's `htlc_basepoint`;
- The `remote_htlcpubkey` uses the remote node's `htlc_basepoint`;
- The `local_delayedpubkey` uses the local node's `delayed_payment_basepoint`;
- The `remote_delayedpubkey` uses the remote node's `delayed_payment_basepoint`.

The corresponding private keys can be similarly derived, if the basepoint secrets are known (i.e. the private keys corresponding to `localpubkey`, `local_htlcpubkey`, and `local_delayedpubkey` only):

```
privkey = basepoint_secret + SHA256(per_commitment_point || basepoint)
```

`remotepubkey` Derivation

The `remotepubkey` is simply the remote node's `payment_basepoint`.

This means that a node can spend a commitment transaction even if it has lost data and doesn't know the corresponding `per_commitment_point`. A watchtower could correlate transactions given to it which only have a `to_remote` output if it sees one of them onchain, but such transactions do not need any enforcement and should not be handed to a watchtower.

`revocationpubkey` Derivation

The `revocationpubkey` is a blinded key: when the local node wishes to create a new commitment for the remote node, it uses its own `revocation_basepoint` and the remote node's `per_commitment_point` to derive a new `revocationpubkey` for the commitment. After the remote node reveals the `per_commitment_secret` used (thereby revoking that commitment), the local node can then derive the `revocationprivkey`, as it now knows the two secrets necessary to derive the key (`revocation_basepoint_secret` and `per_commitment_secret`).

The `per_commitment_point` is generated using elliptic-curve multiplication:

```
per_commitment_point = per_commitment_secret * G
```

And this is used to derive the revocation pubkey from the remote node's `revocation_basepoint`:

```
revocationpubkey = revocation_basepoint * SHA256(revocation_basepoint || per_commitment_point) +
per_commitment_point * SHA256(per_commitment_point || revocation_basepoint)
```

This construction ensures that neither the node providing the basepoint nor the node providing the `per_commitment_point` can know the private key without the other node's secret.

The corresponding private key can be derived once the `per_commitment_secret` is known:

```
revocationprivkey = revocation_basepoint_secret * SHA256(revocation_basepoint ||
per_commitment_point) + per_commitment_secret * SHA256(per_commitment_point ||
revocation_basepoint)
```

Per-commitment Secret Requirements

A node: - MUST select an unguessable 256-bit seed for each connection, - MUST NOT reveal the seed.

Up to (2^48 - 1) per-commitment secrets can be generated.

The first secret used: - MUST be index 281474976710655, - and from there, the index is decremented.

The I'th secret P: - MUST match the output of this algorithm:

```
generate_from_seed(seed, I):
    P = seed
    for B in 47 down to 0:
        if B set in I:
            flip(B) in P
            P = SHA256(P)
    return P
```

Where "flip(B)" alternates the (B mod 8) bit of the (B div 8) byte of the value. So, "flip(0) in e3b0…" is "e2b0…", and "flip(10) in"e3b0…" is "e3b4…".

The receiving node: - MAY store all previous per-commitment secrets. - MAY calculate them from a compact representation, as described below.

Efficient Per-commitment Secret Storage

The receiver of a series of secrets can store them compactly in an array of 49 (value,index) pairs. Because, for a given secret on a 2^X boundary, all secrets up to the next 2^X boundary can be derived; and secrets are always received in descending order starting at `0xFFFFFFFFFFFF`.

In binary, it's helpful to think of any index in terms of a *prefix*, followed by some trailing 0s. You can derive the secret for any index that matches this *prefix*.

For example, secret `0xFFFFFFFFFFF0` allows the secrets to be derived for `0xFFFFFFFFFFF1` through `0xFFFFFFFFFFFF`, inclusive; and secret `0xFFFFFFFFF08` allows the secrets to be derived for `0xFFFFFFFFF09` through `0xFFFFFFFFF0F`, inclusive.

This is done using a slight generalization of `generate_from_seed` above:

```
# Return I'th secret given base secret whose index has bits..47 the same.
derive_secret(base, bits, I):
    P = base
    for B in bits - 1 down to 0:
        if B set in I:
            flip(B) in P
            P = SHA256(P)
    return P
```

Only one secret for each unique prefix need be saved; in effect, the number of trailing 0s is counted, and this determines where in the storage array the secret is stored:

```
# a.k.a. count trailing 0s
where_to_put_secret(I):
    for B in 0 to 47:
        if testbit(I) in B == 1:
            return B
    # I = 0, this is the seed.
    return 48
```

A double-check, that all previous secrets derive correctly, is needed; if this check fails, the secrets were not generated from the same seed:

```
insert_secret(secret, I):
    B = where_to_put_secret(I)

    # This tracks the index of the secret in each bucket across the traversal.
    for b in 0 to B:
        if derive_secret(secret, B, known[b].index) != known[b].secret:
            error The secret for I is incorrect
            return

    # Assuming this automatically extends known[] as required.
    known[B].index = I
    known[B].secret = secret
```

Finally, if an unknown secret at index `I` needs be derived, it must be discovered which known secret can be used to derive it. The simplest method is iterating over all the known secrets, and testing if each can be used to derive the unknown secret:

```
derive_old_secret(I):
    for b in 0 to len(secrets):
        # Mask off the non-zero prefix of the index.
        MASK = ~((1 << b) - 1)
        if (I & MASK) == secrets[b].index:
            return derive_secret(known, i, I)
    error Index 'I' hasn't been received yet.
```

This looks complicated, but remember that the index in entry `b` has `b` trailing 0s; the mask and compare simply checks if the index at each bucket is a prefix of the desired index.

Appendix A: Expected Weights

Expected Weight of the Funding Transaction (v2 Channel Establishment)

The *expected weight* of a funding transaction is calculated as follows:

```
  inputs: 41 bytes
    - previous_out_point: 36 bytes
      - hash: 32 bytes
      - index: 4 bytes
    - var_int: 1 byte
    - script_sig: 0 bytes
```

```
    - witness <---- Cost for "witness" data calculated separately.
    - sequence: 4 bytes

  non_funding_outputs: 9 bytes + `scriptlen`
    - value: 8 bytes
    - var_int: 1 byte <---- assuming a standard output script
    - script: `scriptlen`

  funding_output: 43 bytes
    - value: 8 bytes
    - var_int: 1 byte
    - script: 34 bytes
      - OP_0: 1 byte
      - PUSHDATA(32-byte-hash): 33 bytes
```

Multiplying non-witness data by 4 results in a weight of:

```
  // transaction_fields = 10 (version, input count, output count, locktime)
  // segwit_fields = 2 (marker + flag)
  // funding_transaction = 43 + num_inputs * 41 + num_outputs * 9 + sum(scriptlen)
  funding_transaction_weight = 4 * (funding_transaction + transaction_fields) + segwit_fields

  witness_weight = sum(witness_len)

  overall_weight = funding_transaction_weight + witness_weight
```

Expected Weight of the Commitment Transaction

The *expected weight* of a commitment transaction is calculated as follows:

```
p2wsh: 34 bytes
    - OP_0: 1 byte
    - OP_DATA: 1 byte (witness_script_SHA256 length)
    - witness_script_SHA256: 32 bytes

p2wpkh: 22 bytes
    - OP_0: 1 byte
    - OP_DATA: 1 byte (public_key_HASH160 length)
    - public_key_HASH160: 20 bytes

multi_sig: 71 bytes
    - OP_2: 1 byte
    - OP_DATA: 1 byte (pub_key_alice length)
    - pub_key_alice: 33 bytes
    - OP_DATA: 1 byte (pub_key_bob length)
    - pub_key_bob: 33 bytes
    - OP_2: 1 byte
    - OP_CHECKMULTISIG: 1 byte

witness: 222 bytes
    - number_of_witness_elements: 1 byte
    - nil_length: 1 byte
    - sig_alice_length: 1 byte
    - sig_alice: 73 bytes
    - sig_bob_length: 1 byte
```

```
    - sig_bob: 73 bytes
    - witness_script_length: 1 byte
    - witness_script (multi_sig)

funding_input: 41 bytes
    - previous_out_point: 36 bytes
        - hash: 32 bytes
        - index: 4 bytes
    - var_int: 1 byte (script_sig length)
    - script_sig: 0 bytes
    - witness <---- "witness" is used instead of "script_sig" for
            transaction validation; however, "witness" is stored
            separately, and the cost for its size is smaller. So,
            the calculation of ordinary data is separated
            from the witness data.
    - sequence: 4 bytes

output_paying_to_local: 43 bytes
    - value: 8 bytes
    - var_int: 1 byte (pk_script length)
    - pk_script (p2wsh): 34 bytes

output_paying_to_remote (no option_anchors): 31 bytes
    - value: 8 bytes
    - var_int: 1 byte (pk_script length)
    - pk_script (p2wpkh): 22 bytes

output_paying_to_remote (option_anchors): 43 bytes
    - value: 8 bytes
    - var_int: 1 byte (pk_script length)
    - pk_script (p2wsh): 34 bytes

output_anchor (option_anchors): 43 bytes
    - value: 8 bytes
    - var_int: 1 byte (pk_script length)
    - pk_script (p2wsh): 34 bytes

htlc_output: 43 bytes
    - value: 8 bytes
    - var_int: 1 byte (pk_script length)
    - pk_script (p2wsh): 34 bytes

 witness_header: 2 bytes
    - flag: 1 byte
    - marker: 1 byte

 commitment_transaction (no option_anchors): 125 + 43 * num-htlc-outputs bytes
    - version: 4 bytes
    - witness_header <---- part of the witness data
    - count_tx_in: 1 byte
    - tx_in: 41 bytes
        funding_input
    - count_tx_out: 1 byte
    - tx_out: 74 + 43 * num-htlc-outputs bytes
        output_paying_to_remote,
```

```
        output_paying_to_local,
        ....htlc_output's...
    - lock_time: 4 bytes

 commitment_transaction (option_anchors): 225 + 43 * num-htlc-outputs bytes
    - version: 4 bytes
    - witness_header <---- part of the witness data
    - count_tx_in: 1 byte
    - tx_in: 41 bytes
        funding_input
    - count_tx_out: 3 byte
    - tx_out: 172 + 43 * num-htlc-outputs bytes
        output_paying_to_remote,
        output_paying_to_local,
        output_anchor,
        output_anchor,
        ....htlc_output's...
    - lock_time: 4 bytes
```

Multiplying non-witness data by 4 results in a weight of:

```
// 500 + 172 * num-htlc-outputs weight (no option_anchors)
// 900 + 172 * num-htlc-outputs weight (option_anchors)
commitment_transaction_weight = 4 * commitment_transaction

// 224 weight
witness_weight = witness_header + witness

overall_weight (no option_anchors) = 500 + 172 * num-htlc-outputs + 224 weight
overall_weight (option_anchors) = 900 + 172 * num-htlc-outputs + 224 weight
```

Expected Weight of HTLC-timeout and HTLC-success Transactions

The *expected weight* of an HTLC transaction is calculated as follows:

```
accepted_htlc_script: 140 bytes (143 bytes with option_anchors)
    - OP_DUP: 1 byte
    - OP_HASH160: 1 byte
    - OP_DATA: 1 byte (RIPEMD160(SHA256(revocationpubkey)) length)
    - RIPEMD160(SHA256(revocationpubkey)): 20 bytes
    - OP_EQUAL: 1 byte
    - OP_IF: 1 byte
    - OP_CHECKSIG: 1 byte
    - OP_ELSE: 1 byte
    - OP_DATA: 1 byte (remote_htlcpubkey length)
    - remote_htlcpubkey: 33 bytes
    - OP_SWAP: 1 byte
    - OP_SIZE: 1 byte
    - OP_DATA: 1 byte (32 length)
    - 32: 1 byte
    - OP_EQUAL: 1 byte
    - OP_IF: 1 byte
    - OP_HASH160: 1 byte
    - OP_DATA: 1 byte (RIPEMD160(payment_hash) length)
    - RIPEMD160(payment_hash): 20 bytes
```

    - OP_EQUALVERIFY: 1 byte
    - 2: 1 byte
    - OP_SWAP: 1 byte
    - OP_DATA: 1 byte (local_htlcpubkey length)
    - local_htlcpubkey: 33 bytes
    - 2: 1 byte
    - OP_CHECKMULTISIG: 1 byte
    - OP_ELSE: 1 byte
    - OP_DROP: 1 byte
    - OP_DATA: 1 byte (cltv_expiry length)
    - cltv_expiry: 4 bytes
    - OP_CHECKLOCKTIMEVERIFY: 1 byte
    - OP_DROP: 1 byte
    - OP_CHECKSIG: 1 byte
    - OP_ENDIF: 1 byte
    - OP_1: 1 byte (option_anchors)
    - OP_CHECKSEQUENCEVERIFY: 1 byte (option_anchors)
    - OP_DROP: 1 byte (option_anchors)
    - OP_ENDIF: 1 byte

offered_htlc_script: 133 bytes (136 bytes with option_anchors)
    - OP_DUP: 1 byte
    - OP_HASH160: 1 byte
    - OP_DATA: 1 byte (RIPEMD160(SHA256(revocationpubkey)) length)
    - RIPEMD160(SHA256(revocationpubkey)): 20 bytes
    - OP_EQUAL: 1 byte
    - OP_IF: 1 byte
    - OP_CHECKSIG: 1 byte
    - OP_ELSE: 1 byte
    - OP_DATA: 1 byte (remote_htlcpubkey length)
    - remote_htlcpubkey: 33 bytes
    - OP_SWAP: 1 byte
    - OP_SIZE: 1 byte
    - OP_DATA: 1 byte (32 length)
    - 32: 1 byte
    - OP_EQUAL: 1 byte
    - OP_NOTIF: 1 byte
    - OP_DROP: 1 byte
    - 2: 1 byte
    - OP_SWAP: 1 byte
    - OP_DATA: 1 byte (local_htlcpubkey length)
    - local_htlcpubkey: 33 bytes
    - 2: 1 byte
    - OP_CHECKMULTISIG: 1 byte
    - OP_ELSE: 1 byte
    - OP_HASH160: 1 byte
    - OP_DATA: 1 byte (RIPEMD160(payment_hash) length)
    - RIPEMD160(payment_hash): 20 bytes
    - OP_EQUALVERIFY: 1 byte
    - OP_CHECKSIG: 1 byte
    - OP_ENDIF: 1 byte
    - OP_1: 1 byte (option_anchors)
    - OP_CHECKSEQUENCEVERIFY: 1 byte (option_anchors)
    - OP_DROP: 1 byte (option_anchors)
    - OP_ENDIF: 1 byte

```
timeout_witness: 285 bytes (288 bytes with option_anchors)
    - number_of_witness_elements: 1 byte
    - nil_length: 1 byte
    - sig_alice_length: 1 byte
    - sig_alice: 73 bytes
    - sig_bob_length: 1 byte
    - sig_bob: 73 bytes
    - nil_length: 1 byte
    - witness_script_length: 1 byte
    - witness_script (offered_htlc_script)

success_witness: 324 bytes (327 bytes with option_anchors)
    - number_of_witness_elements: 1 byte
    - nil_length: 1 byte
    - sig_alice_length: 1 byte
    - sig_alice: 73 bytes
    - sig_bob_length: 1 byte
    - sig_bob: 73 bytes
    - preimage_length: 1 byte
    - preimage: 32 bytes
    - witness_script_length: 1 byte
    - witness_script (accepted_htlc_script)

commitment_input: 41 bytes
    - previous_out_point: 36 bytes
        - hash: 32 bytes
        - index: 4 bytes
    - var_int: 1 byte (script_sig length)
    - script_sig: 0 bytes
    - witness (success_witness or timeout_witness)
    - sequence: 4 bytes

htlc_output: 43 bytes
    - value: 8 bytes
    - var_int: 1 byte (pk_script length)
    - pk_script (p2wsh): 34 bytes

htlc_transaction:
    - version: 4 bytes
    - witness_header <---- part of the witness data
    - count_tx_in: 1 byte
    - tx_in: 41 bytes
        commitment_input
    - count_tx_out: 1 byte
    - tx_out: 43
        htlc_output
    - lock_time: 4 bytes
```

Multiplying non-witness data by 4 results in a weight of 376. Adding the witness data for each case (285 or 288 + 2 for HTLC-timeout, 324 or 327 + 2 for HTLC-success) results in weights of:

```
663 (HTLC-timeout) (666 with option_anchors)
703 (HTLC-success) (706 with option_anchors)
    - (really 702 and 705, but we use these numbers for historical reasons)
```

Appendix B: Funding Transaction Test Vectors

In the following: - It's assumed that *local* is the funder. - Private keys are displayed as 32 bytes plus a trailing 1 (Bitcoin's convention for "compressed" private keys, i.e. keys for which the public key is compressed). - Transaction signatures are all deterministic, using RFC6979 (using HMAC-SHA256). A valid signature MUST sign all inputs and outputs of the relevant transaction (i.e. MUST be created with a `SIGHASH_ALL` signature hash), unless explicitly stated otherwise. Note that clients MUST send the signature in compact encoding and not in Bitcoin-script format, thus the signature hash byte is not transmitted.

The input for the funding transaction was created using a test chain with the following first two blocks; the second block contains a spendable coinbase (note that such a P2PKH input is inadvisable, as detailed in BOLT #2, but provides the simplest example):

```
Block 0 (genesis):
0100000000000000000000000000000000000000000000000000000000000000000000003ba3edfd7a7b12b27ac72c3e
67768f617fc81bc3888a51323a9fb8aa4b1e5e4adae5494dffff7f2002000000010100000001000000000000000000000
0000000000000000000000000000000000000000000ffffffff4d04ffff001d0104455468652054696d65732030332f
4a616e2f32303039204368616e63656c6c6f72206f6e206272696e6b206f66207365636f6e64206261696c6f75742066
6f722062616e6b73ffffffff0100f2052a01000000434104678afdb0fe5548271967f1a67130b7105cd6a828e03909a6
7962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec112de5c384df7ba0b8d578a4c702b6bf11d5fac00000000
Block 1:
0000002006226e46111a0b59caaf126043eb5bbf28c34f3a5e332a1fc7b2b73cf188910fadbb20ea41a8423ea937e76e
8151636bf6093b70eaff942930d20576600521fdc30f9858ffff7f2000000000010100000001000000000000000000000
0000000000000000000000000000000000000000000ffffffff03510101ffffffff0100f2052a010000001976a9143c
a33c2e4446f4a305f23c80df8ad1afdcf652f988ac00000000
Block 1 coinbase transaction:
0100000001000000000000000000000000000000000000000000000000000000000000000000ffffffff03510101ffffff
ff0100f2052a010000001976a9143ca33c2e4446f4a305f23c80df8ad1afdcf652f988ac00000000
Block 1 coinbase privkey: 6bd078650fcee8444e4e09825227b801a1ca928debb750eb36e6d56124bb20e801
# privkey in base58: cRCH7YNcarfvaiY1GWUKQrRGmoezvfAiqHtdRvxe16shzbd7LDMz
# pubkey in base68: mm3aPLSv9fBrbS68JzurAMp4xGoddJ6pSf
```

The funding transaction is paid to the following pubkeys:

```
local_funding_pubkey: 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb
remote_funding_pubkey: 030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c1
# funding witness script =
5221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d
66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae
```

The funding transaction has a single input and a change output (order determined by BIP69 in this case):

```
input txid: fd2105607605d2302994ffea703b09f66b6351816ee737a93e42a841ea20bbad
input index: 0
input satoshis: 5000000000
funding satoshis: 10000000
# funding witness script =
5221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d
66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae
# feerate_per_kw: 15000
change satoshis: 4989986080
funding output: 0
```

The resulting funding transaction is:

```
funding tx:
0200000001adbb20ea41a8423ea937e76e8151636bf6093b70eaff942930d20576600521fd000000006b483045022100
90587b6201e166ad6af0227d3036a9454223d49a1f11839c1a362184340ef0240220577f7cd5cca78719405cbf1de741
4ac027f0239ef6e214c90fcaab0454d84b3b012103535b32d5eb0a6ed0982a0479bbadc9868d9836f6ba94dd5a63be16
d875069184ffffffff028096980000000000220020c015c4a6be010e21657068fc2e6a9d02b27ebe4d490a25846f7237
f104d1a3cd20256d29010000001600143ca33c2e4446f4a305f23c80df8ad1afdcf652f900000000
# txid: 8984484a580b825b9972d7adb15050b3ab624ccd731946b3eeddb92f4e7ef6be
```

Appendix C: Commitment and HTLC Transaction Test Vectors

In the following: - *local* transactions are considered, which implies that all payments to *local* are delayed. - It's assumed that *local* is the opener. - Private keys are displayed as 32 bytes plus a trailing 1 (Bitcoin's convention for "compressed" private keys, i.e. keys for which the public key is compressed). - Transaction signatures are all deterministic, using RFC6979 (using HMAC-SHA256).

To start, common basic parameters for each test vector are defined: the HTLCs are not used for the first "simple commitment tx with no HTLCs" test, and HTLCs 5 and 6 are only used in the "same amount and preimage" test.

```
funding_tx_id: 8984484a580b825b9972d7adb15050b3ab624ccd731946b3eeddb92f4e7ef6be
funding_output_index: 0
funding_amount_satoshi: 10000000
commitment_number: 42
local_delay: 144
local_dust_limit_satoshi: 546
htlc 0 direction: remote->local
htlc 0 amount_msat: 1000000
htlc 0 expiry: 500
htlc 0 payment_preimage: 0000000000000000000000000000000000000000000000000000000000000000
htlc 1 direction: remote->local
htlc 1 amount_msat: 2000000
htlc 1 expiry: 501
htlc 1 payment_preimage: 0101010101010101010101010101010101010101010101010101010101010101
htlc 2 direction: local->remote
htlc 2 amount_msat: 2000000
htlc 2 expiry: 502
htlc 2 payment_preimage: 0202020202020202020202020202020202020202020202020202020202020202
htlc 3 direction: local->remote
htlc 3 amount_msat: 3000000
htlc 3 expiry: 503
htlc 3 payment_preimage: 0303030303030303030303030303030303030303030303030303030303030303
htlc 4 direction: remote->local
htlc 4 amount_msat: 4000000
htlc 4 expiry: 504
htlc 4 payment_preimage: 0404040404040404040404040404040404040404040404040404040404040404
htlc 5 direction: local->remote
htlc 5 amount_msat: 5000000
htlc 5 expiry: 506
htlc 5 payment_preimage: 0505050505050505050505050505050505050505050505050505050505050505
htlc 6 direction: local->remote
htlc 6 amount_msat: 5000001
htlc 6 expiry: 505
htlc 6 payment_preimage: 0505050505050505050505050505050505050505050505050505050505050505
```

Here are the points used to derive the obscuring factor for the commitment number:

```
local_payment_basepoint: 034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
remote_payment_basepoint: 032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991
# obscured commitment number = 0x2bb038521914 ^ 42
```

We use the same values for the HTLC basepoints:

```
local_htlc_basepoint: 034f355bdcb7cc0af728ef3cceb9615d90684bb5b2ca5f859ab0f0b704075871aa
remote_htlc_basepoint: 032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991
```

And, here are the keys needed to create the transactions:

```
local_funding_privkey: 30ff4956bbdd3222d44cc5e8a1261dab1e07957bdac5ae88fe3261ef321f374901
local_funding_pubkey: 023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb
remote_funding_pubkey: 030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c1
local_privkey: bb13b121cdc357cd2e608b0aea294afca36e2b34cf958e2e6451a2f27469449101
local_htlcpubkey: 030d417a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e7
remote_htlcpubkey: 0394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b
local_delayedpubkey: 03fd5960528dc152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c
local_revocation_pubkey: 0212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b19
# funding wscript =
5221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d
66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae
```

And, here are the test vectors themselves:

```
name: simple commitment tx with no HTLCs
to_local_msat: 7000000000
to_remote_msat: 3000000000
local_feerate_per_kw: 15000
# base commitment transaction fee = 10860
# actual commitment transaction fee = 10860
# to_local amount 6989140 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
3045022100c3127b33dcc741dd6b05b1e63cbd1a9a7d816f37af9b6756fa2376b056f032370220408b96279808fe57eb
7e463710804cdf4f108388bc5cf722d8c848d2c7f9f3b0
# local_signature =
30440220616210b2cc4d3afb601013c373bbd8aac54febd9f15400379a8cb65ce7deca60022034236c010991beb7ff77
0510561ae8dc885b8d38d1947248c38f2ae055647142
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
02c0c62d0000000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e48454a56a0000000002200204adb4e2f00
643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e04004730440220616210b2cc4d3afb601013c373bb
d8aac54febd9f15400379a8cb65ce7deca60022034236c010991beb7ff770510561ae8dc885b8d38d1947248c38f2ae0
5564714201483045022100c3127b33dcc741dd6b05b1e63cbd1a9a7d816f37af9b6756fa2376b056f032370220408b96
279808fe57eb7e463710804cdf4f108388bc5cf722d8c848d2c7f9f3b001475221023da092f6980e58d2c037173180e9
a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c3
9fa711c152ae3e195220
num_htlcs: 0

name: commitment tx with all five HTLCs untrimmed (minimum feerate)
```

```
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 0
# base commitment transaction fee = 0
# actual commitment transaction fee = 0
# HTLC #2 offered amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868
# HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
# HTLC #0 received amount 1000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f401b175ac6868
# HTLC #1 received amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f501b175ac6868
# HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
# to_local amount 6988000 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
3044022009b048187705a8cbc9ad73adbe5af148c3d012e1f067961486c822c7af08158c022006d66f3704cfab3eb2dc
49dae24e4aa22a6910fc9b424007583204e3621af2e5
# local_signature =
304402206fc2d1f10ea59951eefac0b4b7c396a3c3d87b71ff0b019796ef4535beaf36f902201765b0181e514d04f4c8
ad75659d7037be26cdb3f8bb6f78fe61decef484c3ea
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
07e8030000000000022002052bfef0479d7b293c27e0f1eb294bea154c63a3294ef092c19af51409bce0e2ad0070000
00000000220020403d394747cae42e98ff01734ad5c08f82ba123d3d9a620abda88989651e2ab5d00700000000000022
0020748eba944fedc8827f6b06bc44678f93c0f9e6078b35c6331ed31e75f8ce0c2db80b0000000000000220020c20b5d
1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f0000000000002200208c48d15160397c97
31df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d0000000000160014cc1b07838e387deacd0e5232e1
e8b49f4c29e484e0a06a0000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7d
d80e040047304402206fc2d1f10ea59951eefac0b4b7c396a3c3d87b71ff0b019796ef4535beaf36f902201765b0181e
514d04f4c8ad75659d7037be26cdb3f8bb6f78fe61decef484c3ea01473044022009b048187705a8cbc9ad73adbe5af1
48c3d012e1f067961486c822c7af08158c022006d66f3704cfab3eb2dc49dae24e4aa22a6910fc9b424007583204e362
1af2e501475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2c
cc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 5
# signature for output #0 (htlc-success for htlc #0)
remote_htlc_signature =
3045022100d9e29616b8f3959f1d3d7f7ce893ffedcdc407717d0de8e37d808c91d3a7c50d022078c3033f6d00095c87
20a4bc943c1b45727818c082e4e3ddbc6d3116435b624b
# local_htlc_signature =
```

30440220636de5682ef0c5b61f124ec74e8aa2461a69777521d6998295dcea36bc333811022016528559 4b23c50b28b8
2df200234566628a27bcd17f7f14404bd865354eb3ce
htlc_success_tx (htlc #0):
02000000000101ab84ff284f162cfbfef241f853b47d4368d171f9e2a1445160cd591c4c7d882b00000000000000000001
01e8030000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100d9e29616b8f3959f1d3d7f7ce893ffedcdc407717d0de8e37d808c91d3a7c50d022078c3033f6d00095c8720
a4bc943c1b45727818c082e4e3ddbc6d3116435b624b014730440220636de5682ef0c5b61f124ec74e8aa2461a697775
21d6998295dcea36bc333811022016528559 4b23c50b28b82df200234566628a27bcd17f7f14404bd865354eb3ce0120
0000000000000000000000000000000000000000000000000000000000000008a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f401b175ac686800000000
signature for output #1 (htlc-timeout for htlc #2)
remote_htlc_signature =
30440220649fe8b20e67e46cbb0d09b4acea87dbec001b39b08dee7bdd0b1f03922a8640022037c462dff79df501cecf
db12ea7f4de91f99230bb544726f6e04527b1f896004
local_htlc_signature =
3045022100803159dee7935dba4a1d36a61055ce8fd62caa528573cc221ae288515405a252022029c59e7cffce374fe8
60100a4a63787e105c3cf5156d40b12dd53ff55ac8cf3f
htlc_timeout_tx (htlc #2):
02000000000101ab84ff284f162cfbfef241f853b47d4368d171f9e2a1445160cd591c4c7d882b01000000000000000001
01d0070000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
440220649fe8b20e67e46cbb0d09b4acea87dbec001b39b08dee7bdd0b1f03922a8640022037c462dff79df501cecfdb
12ea7f4de91f99230bb544726f6e04527b1f8960040148304502210080 3159dee7935dba4a1d36a61055ce8fd62caa52
8573cc221ae288515405a252022029c59e7cffce374fe860100a4a63787e105c3cf5156d40b12dd53ff55ac8cf3f0100
8576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a21
84d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca
813e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868f6010000
signature for output #2 (htlc-success for htlc #1)
remote_htlc_signature =
30440220770fc321e97a19f38985f2e7732dd9fe08d16a2efa4bcbc0429400a447faf49102204d40b417f3113e1b0944
ae0986f517564ab4acd3d190503faf97a6e420d43352
local_htlc_signature =
3045022100a437cc2ce77400ecde441b3398fea3c3ad8bdad8132be818227fe3c5b8345989022069d45e7fa0ae551ec3
7240845e2c561ceb2567eacf3076a6a43a502d05865faa
htlc_success_tx (htlc #1):
02000000000101ab84ff284f162cfbfef241f853b47d4368d171f9e2a1445160cd591c4c7d882b02000000000000000001
01d0070000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
440220770fc321e97a19f38985f2e7732dd9fe08d16a2efa4bcbc0429400a447faf49102204d40b417f3113e1b0944ae
0986f517564ab4acd3d190503faf97a6e420d4335201483045022100a437cc2ce77400ecde441b3398fea3c3ad8bdad8
132be818227fe3c5b8345989022069d45e7fa0ae551ec37240845e2c561ceb2567eacf3076a6a43a502d05865faa0120
0101010101010101010101010101010101010101010101010101010101010101018a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f501b175ac686800000000
signature for output #3 (htlc-timeout for htlc #3)
remote_htlc_signature =
304402207bcbf4f60a9829b05d2dbab84ed593e0291836be715dc7db6b72a64caf646af802201e489a5a84f7c5cc1303
98b841d138d031a5137ac8f4c49c770a4959dc3c1363
local_htlc_signature =
304402203121d9b9c055f354304b016a36662ee99e1110d9501cb271b087ddb6f382c2c80220549882f3f3b78d9c492d
e47543cb9a697cecc493174726146536c5954dac7487
htlc_timeout_tx (htlc #3):
02000000000101ab84ff284f162cfbfef241f853b47d4368d171f9e2a1445160cd591c4c7d882b03000000000000000001
01b80b0000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730

```
4402207bcbf4f60a9829b05d2dbab84ed593e0291836be715dc7db6b72a64caf646af802201e489a5a84f7c5cc130398
b841d138d031a5137ac8f4c49c770a4959dc3c1363014730440220312 1d9b9c055f354304b016a36662ee99e1110d950
1cb271b087ddb6f382c2c80220549882f3f3b78d9c492de47543cb9a697cecc493174726146536c5954dac7487010085
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
# signature for output #4 (htlc-success for htlc #4)
remote_htlc_signature =
3044022076dca5cb81ba7e466e349b7128cdba216d4d01659e29b96025b9524aaf0d1899022060de85697b88b21c7497
02b7d2cfa7dfeaa1f472c8f1d7d9c23f2bf968464b87
# local_htlc_signature =
3045022100d9080f103cc92bac15ec42464a95f070c7fb6925014e673ee2ea1374d36a7f7502200c65294d22eb20d485
64954d5afe04a385551919d8b2ddb4ae2459daaeee1d95
htlc_success_tx (htlc #4):
02000000000101ab84ff284f162cfbfef241f853b47d4368d171f9e2a1445160cd591c4c7d882b040000000000000000
01a00f000000000000 2200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
44022076dca5cb81ba7e466e349b7128cdba216d4d01659e29b96025b9524aaf0d1899022060de85697b88b21c749702
b7d2cfa7dfeaa1f472c8f1d7d9c23f2bf968464b8701483045022100d9080f103cc92bac15ec42464a95f070c7fb6925
014e673ee2ea1374d36a7f7502200c65294d22eb20d48564954d5afe04a385551919d8b2ddb4ae2459daaeee1d950120
0404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with seven outputs untrimmed (maximum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 647
# base commitment transaction fee = 1024
# actual commitment transaction fee = 1024
# HTLC #2 offered amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868
# HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
# HTLC #0 received amount 1000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f401b175ac6868
# HTLC #1 received amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f501b175ac6868
# HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
# to_local amount 6986976 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
```

```
remote_signature =
3045022100a135f9e8a5ed25f7277446c67956b00ce6f610ead2bdec2c2f686155b7814772022059f1f6e1a8b336a68e
fcc1af3fe4d422d4827332b5b067501b099c47b7b5b5ee
# local_signature =
30450221009ec15c687898bb4da8b3a833e5ab8bfc51ec6e9202aaa8e66611edfd4a85ed1102203d7183e45078b9735c
93450bc3415d3e5a8c576141a711ec6ddcb4a893926bb7
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a488489000000000038b02b80
07e80300000000000022002052bfef0479d7b293c27e0f1eb294bea154c63a3294ef092c19af51409bce0e2ad0070000
00000000220020403d394747cae42e98ff01734ad5c08f82ba123d3d9a620abda88989651e2ab5d00700000000000022
0020748eba944fedc8827f6b06bc44678f93c0f9e6078b35c6331ed31e75f8ce0c2db80b0000000000000220020c20b5d
1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f0000000000002200208c48d15160397c97
31df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d0000000000160014cc1b07838e387deacd0e5232e1
e8b49f4c29e484e09c6a0000000000022002004adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7d
d80e040048304502210091ec15c687898bb4da8b3a833e5ab8bfc51ec6e9202aaa8e66611edfd4a85ed1102203d7183e4
5078b9735c93450bc3415d3e5a8c576141a711ec6ddcb4a893926bb701483045022100a135f9e8a5ed25f7277446c679
56b00ce6f610ead2bdec2c2f686155b7814772022059f1f6e1a8b336a68efcc1af3fe4d422d4827332b5b067501b099c
47b7b5b5ee01475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b62
3d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 5
# signature for output #0 (htlc-success for htlc #0)
remote_htlc_signature =
304502210084376 27f9ad84ac67052e2a414a4367b8556fd1f94d8b02590f89f50525cd33502205b9c21ff6e7fc864f2
352746ad8ba59182510819acb644e25b8a12fc37bbf24f
# local_htlc_signature =
30440220344b0deb055230d01703e6c7acd45853c4af2328b49b5d8af4f88a060733406602202ea64f2a43d5751edfe7
5503cbc35a62e3141b5ed032fa03360faf4ca66f670b
htlc_success_tx (htlc #0):
02000000000101 2cfb3e4788c206881d38f2996b6cb2109b5935acb527d14bdaa7b908afa9b2fe00000000000000000000
0122020000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
450221008437627f9ad84ac67052e2a414a4367b8556fd1f94d8b02590f89f50525cd33502205b9c21ff6e7fc864f235
2746ad8ba59182510819acb644e25b8a12fc37bbf24f014730440220344b0deb055230d01703e6c7acd45853c4af2328
b49b5d8af4f88a060733406602202ea64f2a43d5751edfe75503cbc35a62e3141b5ed032fa03360faf4ca66f670b0120
0000000000000000000000000000000000000000000000000000000000000008a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f401b175ac686800000000
# signature for output #1 (htlc-timeout for htlc #2)
remote_htlc_signature =
304402205a67f92bf6845cf2892b48d874ac1daf88a36495cf8a06f93d83180d930a6f75022031da1621d95c3f335cc0
6a3056cf960199dae600b7cf89088f65fc53cdbef28c
# local_htlc_signature =
30450221009e5e3822b0185c6799a95288c597b671d6cc69ab80f43740f00c6c3d0752bdda02206da947a74bd98f3175
324dc56fdba86cc783703a120a6f0297537e60632f4c7f
htlc_timeout_tx (htlc #2):
02000000000101 2cfb3e4788c206881d38f2996b6cb2109b5935acb527d14bdaa7b908afa9b2fe01000000000000000000
0124060000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
4402205a67f92bf6845cf2892b48d874ac1daf88a36495cf8a06f93d83180d930a6f75022031da1621d95c3f335cc06a
3056cf960199dae600b7cf89088f65fc53cdbef28c014830450221009e5e3822b0185c6799a95288c597b671d6cc69ab
80f43740f00c6c3d0752bdda02206da947a74bd98f3175324dc56fdba86cc783703a120a6f0297537e60632f4c7f0100
8576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a21
84d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca
813e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868f6010000
# signature for output #2 (htlc-success for htlc #1)
remote_htlc_signature =
```

```
30440220437e21766054a3eef7f65690c5bcfa9920babbc5af92b819f772f6ea96df6c7402207173622024bd97328cfb
26c6665e25c2f5d67c319443ccdc60c903217005d8c8
# local_htlc_signature =
3045022100fcfc47e36b712624677626cef3dc1d67f6583bd46926a6398fe6b00b0c9a37760220525788257b187fc775
c6370d04eadf34d06f3650a63f8df851cee0ecb47a1673
htlc_success_tx (htlc #1):
020000000001012cfb3e4788c206881d38f2996b6cb2109b5935acb527d14bdaa7b908afa9b2fe020000000000000000
010a0600000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
440220437e21766054a3eef7f65690c5bcfa9920babbc5af92b819f772f6ea96df6c7402207173622024bd97328cfb26
c6665e25c2f5d67c319443ccdc60c903217005d8c801483045022100fcfc47e36b712624677626cef3dc1d67f6583bd4
6926a6398fe6b00b0c9a37760220525788257b187fc775c6370d04eadf34d06f3650a63f8df851cee0ecb47a16730120
0101010101010101010101010101010101010101010101010101010101010101018a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f501b175ac686800000000
# signature for output #3 (htlc-timeout for htlc #3)
remote_htlc_signature =
304402207436e10737e4df499fc051686d3e11a5bb2310e4d1f1e691d287cef66514791202207cb58e71a6b7a42dd001
b7e3ae672ea4f71ea3e1cd412b742e9124abb0739c64
# local_htlc_signature =
3045022100e78211b8409afb7255ffe37337da87f38646f1faebbdd61bc1920d69e3ead67a02201a626305adfcd16bfb
7e9340928d9b6305464eab4aa4c4a3af6646e9b9f69dee
htlc_timeout_tx (htlc #3):
020000000001012cfb3e4788c206881d38f2996b6cb2109b5935acb527d14bdaa7b908afa9b2fe030000000000000000
010c0a0000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
4402207436e10737e4df499fc051686d3e11a5bb2310e4d1f1e691d287cef66514791202207cb58e71a6b7a42dd001b7
e3ae672ea4f71ea3e1cd412b742e9124abb0739c6401483045022100e78211b8409afb7255ffe37337da87f38646f1fa
ebbdd61bc1920d69e3ead67a02201a626305adfcd16bfb7e9340928d9b6305464eab4aa4c4a3af6646e9b9f69dee0100
8576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a21
84d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca
813e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
# signature for output #4 (htlc-success for htlc #4)
remote_htlc_signature =
30450221009acd6a827a76bfee50806178dfe0495cd4e1d9c58279c194c7b01520fe68cb8d022024d439047c368883e5
70997a7d40f0b430cb5a742f507965e7d3063ae3feccca
# local_htlc_signature =
3044022048762cf546bbfe474f1536365ea7c416e3c0389d60558bc9412cb148fb6ab68202207215d7083b75c96ff9d2
b08c59c34e287b66820f530b486a9aa4cdd9c347d5b9
htlc_success_tx (htlc #4):
020000000001012cfb3e4788c206881d38f2996b6cb2109b5935acb527d14bdaa7b908afa9b2fe040000000000000000
01da0d0000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
450221009acd6a827a76bfee50806178dfe0495cd4e1d9c58279c194c7b01520fe68cb8d022024d439047c368883e570
997a7d40f0b430cb5a742f507965e7d3063ae3feccca01473044022048762cf546bbfe474f1536365ea7c416e3c0389d
60558bc9412cb148fb6ab68202207215d7083b75c96ff9d2b08c59c34e287b66820f530b486a9aa4cdd9c347d5b90120
0404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with six outputs untrimmed (minimum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 648
# base commitment transaction fee = 914
# actual commitment transaction fee = 1914
```

```
# HTLC #2 offered amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868
# HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
# HTLC #1 received amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f501b175ac6868
# HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
# to_local amount 6987086 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402203948f900a5506b8de36a4d8502f94f21dd84fd9c2314ab427d52feaa7a0a19f2022059b6a37a4adaa2c5419d
c8aea63c6e2a2ec4c4bde46207f6dc1fcd22152fc6e5
# local_signature =
3045022100b15f72908ba3382a34ca5b32519240a22300cc6015b6f9418635fb41f3d01d8802207adb331b9ed1575383
dca0f2355e86c173802feecf8298fbea53b9d4610583e9
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
06d007000000000000220020403d394747cae42e98ff01734ad5c08f82ba123d3d9a620abda88989651e2ab5d0070000
00000000220020748eba944fedc8827f6b06bc44678f93c0f9e6078b35c6331ed31e75f8ce0c2db80b00000000000022
0020c20b5d1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f0000000000002200208c48d1
5160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d0000000000160014cc1b07838e387dea
cd0e5232e1e8b49f4c29e4844e9d6a00000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857ac
cc862d6b7dd80e040048304502210 0b15f72908ba3382a34ca5b32519240a22300cc6015b6f9418635fb41f3d01d8802
207adb331b9ed1575383dca0f2355e86c173802feecf8298fbea53b9d4610583e90147304402203948f900a5506b8de3
6a4d8502f94f21dd84fd9c2314ab427d52feaa7a0a19f2022059b6a37a4adaa2c5419dc8aea63c6e2a2ec4c4bde46207
f6dc1fcd22152fc6e501475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb2103
0e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 4
# signature for output #0 (htlc-timeout for htlc #2)
remote_htlc_signature =
3045022100a031202f3be94678f0e998622ee95ebb6ada8da1e9a5110228b5e04a747351e4022010ca6a21e18314ed53
cfaae3b1f51998552a61a468e596368829a50ce40110e0
# local_htlc_signature =
304502210097e1873b57267730154595187a34949d3744f52933070c74757005e61ce2112e02204ecfba2aa42d4f14bd
f8bad4206bb97217b702e6c433e0e1b0ce6587e6d46ec6
htlc_timeout_tx (htlc #2):
020000000001010f44041fdfba175987cf4e6135ba2a154e3b7fb96483dc0ed5efc0678e5b6bf1000000000000000000
012306000000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100a031202f3be94678f0e998622ee95ebb6ada8da1e9a5110228b5e04a747351e4022010ca6a21e18314ed53cf
aae3b1f51998552a61a468e596368829a50ce40110e00148304502210097e1873b57267730154595187a34949d3744f5
2933070c74757005e61ce2112e02204ecfba2aa42d4f14bdf8bad4206bb97217b702e6c433e0e1b0ce6587e6d46ec601
008576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a
2184d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4da
```

```
ca813e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868f6010000
# signature for output #1 (htlc-success for htlc #1)
remote_htlc_signature =
304402202361012a634aee7835c5ecdd6413dcffa8f404b7e77364c792cff984e4ee71e90220715c5e90baa08daa45a7
439b1ee4fa4843ed77b19c058240b69406606d384124
# local_htlc_signature =
3044022019de73b00f1d818fb388e83b2c8c31f6bce35ac624e215bc12f88f9dc33edf48022006ff814bb9f700ee6abc
3294e146fac3efd4f13f0005236b41c0a946ee00c9ae
htlc_success_tx (htlc #1):
020000000001010f44041fdfba175987cf4e6135ba2a154e3b7fb96483dc0ed5efc0678e5b6bf1010000000000000000
0109060000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
4402202361012a634aee7835c5ecdd6413dcffa8f404b7e77364c792cff984e4ee71e90220715c5e90baa08daa45a743
9b1ee4fa4843ed77b19c058240b69406606d384124014730440220 19de73b00f1d818fb388e83b2c8c31f6bce35ac624
e215bc12f88f9dc33edf48022006ff814bb9f700ee6abc3294e146fac3efd4f13f0005236b41c0a946ee00c9ae012001
0101010101010101010101010101010101010101010101010101010101010101018a76a91414011f7254d96b819c76986c27
7d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c8201
208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d417a46946384f88d5f3337267c5e579765
875dc4daca813e21734b140639e752ae677502f501b175ac686800000000
# signature for output #2 (htlc-timeout for htlc #3)
remote_htlc_signature =
304402207e8e82cd71ed4febeb593732c260456836e97d81896153ecd2b3cf320ca6861702202dd4a30f68f98ced7cc5
6a36369ac1fdd978248c5ff4ed204fc00cc625532989
# local_htlc_signature =
3045022100bd0be6100c4fd8f102ec220e1b053e4c4e2ecca25615490150007b40d314dc3902201a1e0ea266965b4316
4d9e6576f58fa6726d42883dd1c3996d2925c2e2260796
htlc_timeout_tx (htlc #3):
020000000001010f44041fdfba175987cf4e6135ba2a154e3b7fb96483dc0ed5efc0678e5b6bf1020000000000000000
010b0a0000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
4402207e8e82cd71ed4febeb593732c260456836e97d81896153ecd2b3cf320ca6861702202dd4a30f68f98ced7cc56a
36369ac1fdd978248c5ff4ed204fc00cc625532989014830450221 00bd0be6100c4fd8f102ec220e1b053e4c4e2ecca2
5615490150007b40d314dc3902201a1e0ea266965b43164d9e6576f58fa6726d42883dd1c3996d2925c2e22607960100
8576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a21
84d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca
813e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
# signature for output #3 (htlc-success for htlc #4)
remote_htlc_signature =
3044022024cd52e4198c8ae0e414a86d86b5a65ea7450f2eb4e783096736d93395eca5ce022078f0094745b45be4d4b2
b04dd5978c9e66ba49109e5704403e84aaf5f387d6be
# local_htlc_signature =
3045022100bbfb9d0a946d420807c86e985d636cceb16e71c3694ed186316251a00cbd807202207773223f9a337e145f
64673825be9b30d07ef1542c82188b264bedcf7cda78c6
htlc_success_tx (htlc #4):
020000000001010f44041fdfba175987cf4e6135ba2a154e3b7fb96483dc0ed5efc0678e5b6bf1030000000000000000
01d90d0000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
44022024cd52e4198c8ae0e414a86d86b5a65ea7450f2eb4e783096736d93395eca5ce022078f0094745b45be4d4b2b0
4dd5978c9e66ba49109e5704403e84aaf5f387d6be01483045022100bbfb9d0a946d420807c86e985d636cceb16e71c3
694ed186316251a00cbd807202207773223f9a337e145f64673825be9b30d07ef1542c82188b264bedcf7cda78c60120
040404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with six outputs untrimmed (maximum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
```

```
local_feerate_per_kw: 2069
# base commitment transaction fee = 2921
# actual commitment transaction fee = 3921
# HTLC #2 offered amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868
# HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
# HTLC #1 received amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f501b175ac6868
# HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
# to_local amount 6985079 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304502210090b96a2498ce0c0f2fadbec2aab278fed54c1a7838df793ec4d2c78d96ec096202204fdd439c50f90d483b
aa7b68feeef4bd33bc277695405447bcd0bfb2ca34d7bc
# local_signature =
3045022100ad9a9bbbb75d506ca3b716b336ee3cf975dd7834fcf129d7dd188146eb58a8b4022061a759ee417339f7fe
2ea1e8deb83abb6a74db31a09b7648a932a639cda23e33
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
06d007000000000000220020403d394747cae42e98ff01734ad5c08f82ba123d3d9a620abda88989651e2ab5d0070000
0000000220020748eba944fedc8827f6b06bc44678f93c0f9e6078b35c6331ed31e75f8ce0c2db80b00000000000022
0020c20b5d1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f0000000000002200208c48d1
5160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d0000000000160014cc1b07838e387dea
cd0e5232e1e8b49f4c29e48477956a0000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857ac
cc862d6b7dd80e040048304502210090b96a2498ce0c0f2fadbec2aab278fed54c1a7838df793ec4d2c78d96ec096202204fdd439c50f90d483baa7b68feeef4bd33bc27769540
2061a759ee417339f7fe2ea1e8deb83abb6a74db31a09b7648a932a639cda23e3301483045022100ad9a9bbbb75d506ca3b716b336ee3cf975dd7834fcf129d7dd188146eb58a8b402
5447bcd0bfb2ca34d7bc01475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f54eb21
030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 4
# signature for output #0 (htlc-timeout for htlc #2)
remote_htlc_signature =
3045022100f33513ee38abf1c582876f921f8fddc06acff48e04515532a32d3938de938ffd02203aa308a2c1863b7d6f
df53159a1465bf2e115c13152546cc5d74483ceaa7f699
# local_htlc_signature =
3045022100a637902a5d4c9ba9e7c472a225337d5aac9e2e3f6744f76e237132e7619ba0400220035c60d784a031c0d9
f6df66b7eab8726a5c25397399ee4aa960842059eb3f9d
htlc_timeout_tx (htlc #2):
02000000000101adbe717a63fb658add30ada1e6e12ed257637581898abe475c11d7bbcd65bd4d000000000000000000
01750200000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100f33513ee38abf1c582876f921f8fddc06acff48e04515532a32d3938de938ffd02203aa308a2c1863b7d6fdf
53159a1465bf2e115c13152546cc5d74483ceaa7f6990148304502210 0a637902a5d4c9ba9e7c472a225337d5aac9e2e
```

3f6744f76e237132e7619ba0400220035c60d784a031c0d9f6df66b7eab8726a5c25397399ee4aa960842059eb3f9d01
008576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a
2184d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4da
ca813e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868f6010000
signature for output #1 (htlc-success for htlc #1)
remote_htlc_signature =
3045022100ce07682cf4b90093c22dc2d9ab2a77ad6803526b655ef857221cc96af5c9e0bf02200f501cee22e7a268af
40b555d15a8237c9f36ad67ef1841daf9f6a0267b1e6df
local_htlc_signature =
3045022100e57e46234f8782d3ff7aa593b4f7446fb5316c842e693dc63ee324fd49f6a1c302204a2f7b44c48bd26e15
54422afae13153eb94b29d3687b733d18930615fb2db61
htlc_success_tx (htlc #1):
02000000000101adbe717a63fb658add30ada1e6e12ed257637581898abe475c11d7bbcd65bd4d010000000000000000
0122020000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100ce07682cf4b90093c22dc2d9ab2a77ad6803526b655ef857221cc96af5c9e0bf02200f501cee22e7a268af40
b555d15a8237c9f36ad67ef1841daf9f6a0267b1e6df01483045022100e57e46234f8782d3ff7aa593b4f7446fb5316c
842e693dc63ee324fd49f6a1c302204a2f7b44c48bd26e1554422afae13153eb94b29d3687b733d18930615fb2db6101
20010101010101010101010101010101010101010101010101010101010101010101018a76a91414011f7254d96b819c7698
6c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c
8201208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d417a46946384f88d5f3337267c5e57
9765875dc4daca813e21734b140639e752ae677502f501b175ac686800000000
signature for output #2 (htlc-timeout for htlc #3)
remote_htlc_signature =
3045022100e3e35492e55f82ec0bc2f317ffd7a486d1f7024330fe9743c3559fc39f32ef0c02203d1d4db651fc388a91
d5ad8ecdd8e83673063bc8eefe27cfd8c189090e3a23e0
local_htlc_signature =
3044022068613fb1b98eb3aec7f44c5b115b12343c2f066c4277c82b5f873dfe68f37f50022028109b4650f3f528ca4b
fe9a467aff2e3e43893b61b5159157119d5d95cf1c18
htlc_timeout_tx (htlc #3):
02000000000101adbe717a63fb658add30ada1e6e12ed257637581898abe475c11d7bbcd65bd4d020000000000000000
015d0600000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100e3e35492e55f82ec0bc2f317ffd7a486d1f7024330fe9743c3559fc39f32ef0c02203d1d4db651fc388a91d5
ad8ecdd8e83673063bc8eefe27cfd8c189090e3a23e00147304402206861 3fb1b98eb3aec7f44c5b115b12343c2f066c
4277c82b5f873dfe68f37f50022028109b4650f3f528ca4bfe9a467aff2e3e43893b61b5159157119d5d95cf1c180100
8576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a21
84d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca
813e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
signature for output #3 (htlc-success for htlc #4)
remote_htlc_signature =
304402207475aeb0212ef9bf5130b60937817ad88c9a87976988ef1f323f026148cc4a850220739fea17ad3257dcad72
e509c73eebe86bee30b178467b9fdab213d631b109df
local_htlc_signature =
3045022100d315522e09e7d53d2a659a79cb67fef56d6c4bddf3f46df6772d0d20a7beb7c8022070bcc17e288607b6a7
2be0bd83368bb6d53488db266c1cdb4d72214e4f02ac33
htlc_success_tx (htlc #4):
02000000000101adbe717a63fb658add30ada1e6e12ed257637581898abe475c11d7bbcd65bd4d030000000000000000
01f2090000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
44022075475aeb0212ef9bf5130b60937817ad88c9a87976988ef1f323f026148cc4a850220739fea17ad3257dcad72e5
09c73eebe86bee30b178467b9fdab213d631b109df01483045022100d315522e09e7d53d2a659a79cb67fef56d6c4bdd
f3f46df6772d0d20a7beb7c8022070bcc17e288607b6a72be0bd83368bb6d53488db266c1cdb4d72214e4f02ac330120
040404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with five outputs untrimmed (minimum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 2070
base commitment transaction fee = 2566
actual commitment transaction fee = 5566
HTLC #2 offered amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868
HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
to_local amount 6985434 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402204ca1ba260dee913d318271d86e10ca0f5883026fb5653155cff600fb40895223022037b145204b7054a40e08
bb1fefbd826f827b40838d3e501423bcc57924bcb50c
local_signature =
3044022001014419b5ba00e083ac4e0a85f19afc848aacac2d483b4b525d15e2ae5adbfe022015ebddad6ee1e72b47cb
09f3e78459da5be01ccccd95dceca0e056a00cc773c1
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a488489000000000038b02b80
05d007000000000000220020403d394747cae42e98ff01734ad5c08f82ba123d3d9a620abda88989651e2ab5b80b0000
00000000220020c20b5d1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f00000000000022
00208c48d15160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d0000000000160014cc1b07
838e387deacd0e5232e1e8b49f4c29e484da966a0000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c1
1a40d857accc862d6b7dd80e0400473044022001014419b5ba00e083ac4e0a85f19afc848aacac2d483b4b525d15e2ae
5adbfe022015ebddad6ee1e72b47cb09f3e78459da5be01ccccd95dceca0e056a00cc773c10147304402204ca1ba260d
ee913d318271d86e10ca0f5883026fb5653155cff600fb40895223022037b145204b7054a40e08bb1fefbd826f827b40
838d3e501423bcc57924bcb50c01475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f
54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 3
signature for output #0 (htlc-timeout for htlc #2)
remote_htlc_signature =
304402205f6b6d12d8d2529fb24f4445630566cf4abbd0f9330ab6c2bdb94222d6a2a0c502202f556258ae6f05b19374
9e4c541dfcc13b525a5422f6291f073f15617ba8579b
local_htlc_signature =
30440220150b11069454da70caf2492ded9e0065c9a57f25ac2a4c52657b1d15b6c6ed85022068a38833b603c8892717
206383611bad210f1cbb4b1f87ea29c6c65b9e1cb3e5
htlc_timeout_tx (htlc #2):
02000000000101403ad7602b43293497a3a2235a12ecefda4f3a1f1d06e49b1786d945685de1ff000000000000000000
01740200000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
4402205f6b6d12d8d2529fb24f4445630566cf4abbd0f9330ab6c2bdb94222d6a2a0c502202f556258ae6f05b193749e
4c541dfcc13b525a5422f6291f073f15617ba8579b014730440220150b11069454da70caf2492ded9e0065c9a57f25ac
2a4c52657b1d15b6c6ed85022068a38833b603c8892717206383611bad210f1cbb4b1f87ea29c6c65b9e1cb3e5010085
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184

d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868f6010000
signature for output #1 (htlc-timeout for htlc #3)
remote_htlc_signature =
3045022100f960dfb1c9aee7ce1437efa65b523e399383e8149790e05d8fed27ff6e42fe0002202fe8613e062ffe0b0c
518cc4101fba1c6de70f64a5bcc7ae663f2efae43b8546
local_htlc_signature =
30450221009a6ed18e6873bc3644332a6ee21c152a5b102821865350df7a8c74451a51f9f2022050d801fb4895d7d7fb
f452824c0168347f5c0cbe821cf6a97a63af5b8b2563c6
htlc_timeout_tx (htlc #3):
02000000000101403ad7602b43293497a3a2235a12ecefda4f3a1f1d06e49b1786d945685de1ff010000000000000000
015c0600000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100f960dfb1c9aee7ce1437efa65b523e399383e8149790e05d8fed27ff6e42fe0002202fe8613e062ffe0b0c51
8cc4101fba1c6de70f64a5bcc7ae663f2efae43b8546014830450221009a6ed18e6873bc3644332a6ee21c152a5b1028
21865350df7a8c74451a51f9f2022050d801fb4895d7d7fbf452824c0168347f5c0cbe821cf6a97a63af5b8b2563c601
008576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a
2184d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4da
ca813e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
signature for output #2 (htlc-success for htlc #4)
remote_htlc_signature =
3045022100ae5fc7717ae684bc1fcf9020854e5dbe9842c9e7472879ac06ff95ac2bb10e4e022057728ada4c00083a3e
65493fb5d50a232165948a1a0f530ef63185c2c8c56504
local_htlc_signature =
30440220408ad3009827a8fccf774cb285587686bfb2ed041f89a89453c311ce9c8ee0f902203c7392d9f8306d3a4652
2a66bd2723a7eb2628cb2d9b34d4c104f1766bf37502
htlc_success_tx (htlc #4):
02000000000101403ad7602b43293497a3a2235a12ecefda4f3a1f1d06e49b1786d945685de1ff020000000000000000
01f109000000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100ae5fc7717ae684bc1fcf9020854e5dbe9842c9e7472879ac06ff95ac2bb10e4e022057728ada4c00083a3e65
493fb5d50a232165948a1a0f530ef63185c2c8c5650401473044022040 8ad3009827a8fccf774cb285587686bfb2ed04
1f89a89453c311ce9c8ee0f902203c7392d9f8306d3a46522a66bd2723a7eb2628cb2d9b34d4c104f1766bf375020120
0404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with five outputs untrimmed (maximum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 2194
base commitment transaction fee = 2720
actual commitment transaction fee = 5720
HTLC #2 offered amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868
HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
to_local amount 6985280 wscript

```
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402204bb3d6e279d71d9da414c82de42f1f954267c762b2e2eb8b76bc3be4ea07d4b0022014febc009c5edc8c3fc5
d94015de163200f780046f1c293bfed8568f08b70fb3
# local_signature =
3044022072c2e2b1c899b2242656a537dde2892fa3801be0d6df0a87836c550137acde8302201654aa1974d37a829083
c3ba15088689f30b56d6a4f6cb14c7bad0ee3116d398
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
05d007000000000000220020403d394747cae42e98ff01734ad5c08f82ba123d3d9a620abda88989651e2ab5b80b0000
00000000220020c20b5d1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f00000000000022
00208c48d15160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d0000000000160014cc1b07
838e387deacd0e5232e1e8b49f4c29e48440966a00000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c1
1a40d857accc862d6b7dd80e0400473044022072c2e2b1c899b2242656a537dde2892fa3801be0d6df0a87836c550137
acde8302201654aa1974d37a829083c3ba15088689f30b56d6a4f6cb14c7bad0ee3116d3980147304402204bb3d6e279
d71d9da414c82de42f1f954267c762b2e2eb8b76bc3be4ea07d4b0022014febc009c5edc8c3fc5d94015de163200f780
046f1c293bfed8568f08b70fb301475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f
54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 3
# signature for output #0 (htlc-timeout for htlc #2)
remote_htlc_signature =
304502210093972668035 1a7856c1bc386d4a1f422c7d29bd7b56afc139570f508474e6c40022023175a799ccf44c017
fbaadb924c40b2a12115a5b7d0dfd3228df803a2de8450
# local_htlc_signature =
304502210099c98c2edeeee6ec0fb5f3bea8b79bb016a2717afa9b5072370f34382de281d302206f5e2980a995e045cf
90a547f0752a7ee99d48547bc135258fe7bc07e0154301
htlc_timeout_tx (htlc #2):
02000000000101153cd825fdb3aa624bfe513e8031d5d08c5e582fb3d1d1fe8faf27d3eed410cd0000000000000000000
0122020000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
450221009393726680351a7856c1bc386d4a1f422c7d29bd7b56afc139570f508474e6c40022023175a799ccf44c017fb
aadb924c40b2a12115a5b7d0dfd3228df803a2de84500148304502210099c98c2edeeee6ec0fb5f3bea8b79bb016a271
7afa9b5072370f34382de281d302206f5e2980a995e045cf90a547f0752a7ee99d48547bc135258fe7bc07e015430101
008576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a
2184d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4da
ca813e21734b140639e752ae67a914b43e1b38138a41b37f7cd9a1d274bc63e3a9b5d188ac6868f6010000
# signature for output #1 (htlc-timeout for htlc #3)
remote_htlc_signature =
3044022021bb883bf324553d085ba2e821cad80c28ef8b303dbead8f98e548783c02d1600220638f9ef2a9bba25869af
c923f4b5dc38be3bb459f9efa5d869392d5f7779a4a0
# local_htlc_signature =
3045022100fd85bd7697b89c08ec12acc8ba89b23090637d83abd26ca37e01ae93e67c367302202b551fe69386116c47
f984aab9c8dfd25d864dcde5d3389cfbef2447a85c4b77
htlc_timeout_tx (htlc #3):
02000000000101153cd825fdb3aa624bfe513e8031d5d08c5e582fb3d1d1fe8faf27d3eed410cd0100000000000000000
010a0600000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
44022021bb883bf324553d085ba2e821cad80c28ef8b303dbead8f98e548783c02d1600220638f9ef2a9bba25869afc9
23f4b5dc38be3bb459f9efa5d869392d5f7779a4a0014830450221 00fd85bd7697b89c08ec12acc8ba89b23090637d83
abd26ca37e01ae93e67c367302202b551fe69386116c47f984aab9c8dfd25d864dcde5d3389cfbef2447a85c4b770100
8576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a21
84d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca
813e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
# signature for output #2 (htlc-success for htlc #4)
```

```
remote_htlc_signature =
3045022100c9e6f0454aa598b905a35e641a70cc9f67b5f38cc4b00843a041238c4a9f1c4a0220260a2822a62da97e44
583e837245995ca2e36781769c52f19e498efbdcca262b
# local_htlc_signature =
30450221008a9f2ea24cd455c2b64c1472a5fa83865b0a5f49a62b661801e884cf2849af8302204d44180e50bf6adfcf
1c1e581d75af91aba4e28681ce4a5ee5f3cbf65eca10f3
htlc_success_tx (htlc #4):
020000000000101153cd825fdb3aa624bfe513e8031d5d08c5e582fb3d1d1fe8faf27d3eed410cd020000000000000000
019a0900000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100c9e6f0454aa598b905a35e641a70cc9f67b5f38cc4b00843a041238c4a9f1c4a0220260a2822a62da97e4458
3e837245995ca2e36781769c52f19e498efbdcca262b014830450221008a9f2ea24cd455c2b64c1472a5fa83865b0a5f
49a62b661801e884cf2849af8302204d44180e50bf6adfcf1c1e581d75af91aba4e28681ce4a5ee5f3cbf65eca10f301
2004040404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c7698
6c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c
8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e57
9765875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with four outputs untrimmed (minimum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 2195
# base commitment transaction fee = 2344
# actual commitment transaction fee = 7344
# HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
# HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
# to_local amount 6985656 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402201a8c1b1f9671cd9e46c7323a104d7047cc48d3ee80d40d4512e0c72b8dc65666022066d7f9a2ce18c9eb22d2
739ffcce05721c767f9b607622a31b6ea5793ddce403
# local_signature =
3044022044d592025b610c0d678f65032e87035cdfe89d1598c522cc32524ae8172417c30220749fef9d5b2ae8cdd91e
ce442ba8809bc891efedae2291e578475f97715d1767
output commit_tx:
020000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a488489000000000038b02b80
04b80b0000000000000220020c20b5d1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f0000
000000002200208c48d15160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d000000000016
0014cc1b07838e387deacd0e5232e1e8b49f4c29e484b8976a000000000022002040adb4e2f00643db396dd120d4e7dc1
7625f5f2c11a40d857accc862d6b7dd80e0400473044022044d592025b610c0d678f65032e87035cdfe89d1598c522cc
32524ae8172417c30220749fef9d5b2ae8cdd91ece442ba8809bc891efedae2291e578475f97715d17670147304402201
1a8c1b1f9671cd9e46c7323a104d7047cc48d3ee80d40d4512e0c72b8dc65666022066d7f9a2ce18c9eb22d2739ffcce
05721c767f9b607622a31b6ea5793ddce40301475221023da092f6980e58d2c037173180e9a465476026ee50f9669596
3e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 2
# signature for output #0 (htlc-timeout for htlc #3)
remote_htlc_signature =
```

```
3045022100e57b845066a06ee7c2cbfc29eabffe52daa9bf6f6de760066d04df9f9b250e0002202ffb197f0e6e0a77a7
5a9aff27014bd3de83b7f748d7efef986abe655e1dd50e
# local_htlc_signature =
3045022100ecc8c6529d0b2316d046f0f0757c1e1c25a636db168ec4f3aa1b9278df685dc0022067ae6b65e936f13370
91f7b18a15935b608c5f2cdddb2f892ed0babfdd376d76
htlc_timeout_tx (htlc #3):
0200000000001018130a10f09b13677ba2885a8bca32860f3a952e5912b829a473639b5a2c07b900000000000000000000
0109060000000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100e57b845066a06ee7c2cbfc29eabffe52daa9bf6f6de760066d04df9f9b250e0002202ffb197f0e6e0a77a75a
9aff27014bd3de83b7f748d7efef986abe655e1dd50e01483045022100ecc8c6529d0b2316d046f0f0757c1e1c25a636
db168ec4f3aa1b9278df685dc0022067ae6b65e936f1337091f7b18a15935b608c5f2cdddb2f892ed0babfdd376d7601
008576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a
2184d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4da
ca813e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
# signature for output #1 (htlc-success for htlc #4)
remote_htlc_signature =
3045022100d193b7ecccad8057571620a0b1ffa6c48e9483311723b59cf536043b20bc51550220546d4bd37b3b101ecd
a14f6c907af46ec391abce1cd9c7ce22b1a62b534f2f2a
# local_htlc_signature =
3044022014d66f11f9cacf923807eba49542076c5fe5cccf252fb08fe98c78ef3ca6ab5402201b290dbe043cc512d9d7
8de074a5a129b8759bc6a6c546b190d120b690bd6e82
htlc_success_tx (htlc #4):
0200000000001018130a10f09b13677ba2885a8bca32860f3a952e5912b829a473639b5a2c07b900100000000000000000
0199090000000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100d193b7ecccad8057571620a0b1ffa6c48e9483311723b59cf536043b20bc51550220546d4bd37b3b101ecda1
4f6c907af46ec391abce1cd9c7ce22b1a62b534f2f2a01473044022014d66f11f9cacf923807eba49542076c5fe5cccf
252fb08fe98c78ef3ca6ab5402201b290dbe043cc512d9d78de074a5a129b8759bc6a6c546b190d120b690bd6e820120
0404040404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with four outputs untrimmed (maximum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 3702
# base commitment transaction fee = 3953
# actual commitment transaction fee = 8953
# HTLC #3 offered amount 3000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868
# HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
# to_local amount 6984047 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304502210092a587aeb777f869e7ff0d7898ea619ee26a3dacd1f3672b945eea600be431100220077ee9eae3528d1525
1f2a52b607b189820e57a6ccfac8d1af502b132ee40169
# local_signature =
```

```
3045022100e5efb73c32d32da2d79702299b6317de6fb24a60476e3855926d78484dd1b3c802203557cb66a42c944ef0
6e00bcc4da35a5bcb2f185aab0f8e403e519e1d66aaf75
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
04b80b000000000000220020c20b5d1f8584fd90443e7b7b720136174fa4b9333c261d04dbbd012635c0f419a00f0000
000000002200208c48d15160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d000000000016
0014cc1b07838e387deacd0e5232e1e8b49f4c29e4846f916a00000000002200204adb4e2f00643db396dd120d4e7dc1
7625f5f2c11a40d857accc862d6b7dd80e0400483045022100e5efb73c32d32da2d79702299b6317de6fb24a60476e38
55926d78484dd1b3c802203557cb66a42c944ef06e00bcc4da35a5bcb2f185aab0f8e403e519e1d66aaf750148304502
210092a587aeb777f869e7ff0d7898ea619ee26a3dacd1f3672b945eea600be431100220077ee9eae3528d15251f2a52
b607b189820e57a6ccfac8d1af502b132ee4016901475221023da092f6980e58d2c037173180e9a465476026ee50f966
95963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e1952
20
num_htlcs: 2
# signature for output #0 (htlc-timeout for htlc #3)
remote_htlc_signature =
304402206fa54c11f98c3bae1e93df43fc7affeb05b476bf8060c03e29c377c69bc08e8b0220672701cce50d5c379ff4
5a5d2cfe48ac44973adb066ac32608e21221d869bb89
# local_htlc_signature =
304402206e36c683ebf2cb16bcef3d5439cf8b53cd97280a365ed8acd7abb85a8ba5f21c02206e8621edfc2a5766cbc9
6eb67fd501127ff163eb6b85518a39f7d4974aef126f
htlc_timeout_tx (htlc #3):
020000000001018db483bff65c70ee71d8282aeec5a880e2e2b39e45772bda5460403095c62e3f0000000000000000000
0122020000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
4402206fa54c11f98c3bae1e93df43fc7affeb05b476bf8060c03e29c377c69bc08e8b0220672701cce50d5c379ff45a
5d2cfe48ac44973adb066ac32608e21221d869bb890147304402206e36c683ebf2cb16bcef3d5439cf8b53cd97280a36
5ed8acd7abb85a8ba5f21c02206e8621edfc2a5766cbc96eb67fd501127ff163eb6b85518a39f7d4974aef126f010085
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9148a486ff2e31d6158bf39e2608864d63fefd09d5b88ac6868f7010000
# signature for output #1 (htlc-success for htlc #4)
remote_htlc_signature =
3044022057649739b0eb74d541ead0dfdb3d4b2c15aa192720031044c3434c67812e5ca902201e5ede42d960ae551707
f4a6b34b09393cf4dee2418507daa022e3550dbb5817
# local_htlc_signature =
304402207faad26678c8850e01b4a0696d60841f7305e1832b786110ee9075cb92ed14a30220516ef8ee5dfa80824ea2
8cbcec0dd95f8b847146257c16960db98507db15ffdc
htlc_success_tx (htlc #4):
020000000001018db483bff65c70ee71d8282aeec5a880e2e2b39e45772bda5460403095c62e3f0100000000000000000
0176050000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
44022057649739b0eb74d541ead0dfdb3d4b2c15aa192720031044c3434c67812e5ca902201e5ede42d960ae551707f4
a6b34b09393cf4dee2418507daa022e3550dbb58170147304402207faad26678c8850e01b4a0696d60841f7305e1832b
786110ee9075cb92ed14a30220516ef8ee5dfa80824ea28cbcec0dd95f8b847146257c16960db98507db15ffdc012004
0404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c27
7d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c8201
208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e579765
875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with three outputs untrimmed (minimum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 3703
# base commitment transaction fee = 3317
# actual commitment transaction fee = 11317
# HTLC #4 received amount 4000 wscript
```

76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
to_local amount 6984683 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
3045022100b495d239772a237ff2cf354b1b11be152fd852704cb184e7356d13f2fb1e5e430220723db5cdb9cbd6ead7
bfd3deb419cf41053a932418cbb22a67b581f40bc1f13e
local_signature =
304402201b736d1773a124c745586217a75bed5f66c05716fbe8c7db4fdb3c3069741cdd02205083f39c321c1bcadfc8
d97e3c791a66273d936abac0c6a2fde2ed46019508e1
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
03a00f0000000000002200208c48d15160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d00
00000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e484eb936a0000000000220204adb4e2f00643db396dd
120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e040047304402201b736d1773a124c745586217a75bed5f66c057
16fbe8c7db4fdb3c3069741cdd02205083f39c321c1bcadfc8d97e3c791a66273d936abac0c6a2fde2ed46019508e101
483045022100b495d239772a237ff2cf354b1b11be152fd852704cb184e7356d13f2fb1e5e430220723db5cdb9cbd6ea
d7bfd3deb419cf41053a932418cbb22a67b581f40bc1f13e01475221023da092f6980e58d2c037173180e9a465476026
ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152
ae3e195220
num_htlcs: 1
signature for output #0 (htlc-success for htlc #4)
remote_htlc_signature =
3045022100c34c61735f93f2e324cc873c3b248111ccf8f6db15d5969583757010d4ad2b4602207867bb919b2ddd6387
873e425345c9b7fd18d1d66aba41f3607bc2896ef3c30a
local_htlc_signature =
3045022100988c143e2110067117d2321bdd4bd16ca1734c98b29290d129384af0962b634e02206c1b02478878c5f547
018b833986578f90c3e9be669fe5788ad0072a55acbb05
htlc_success_tx (htlc #4):
020000000010120060e4a29579d429f0f27c17ee5f1ee282f20d706d6f90b63d35946d8f3029a000000000000000000
01750500000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100c34c61735f93f2e324cc873c3b248111ccf8f6db15d5969583757010d4ad2b4602207867bb919b2ddd638787
3e425345c9b7fd18d1d66aba41f3607bc2896ef3c30a01483045022100988c143e2110067117d2321bdd4bd16ca1734c
98b29290d129384af0962b634e02206c1b02478878c5f547018b833986578f90c3e9be669fe5788ad0072a55acbb0501
2004040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c7698
6c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c
8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e57
9765875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with three outputs untrimmed (maximum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 4914
base commitment transaction fee = 4402
actual commitment transaction fee = 12402
HTLC #4 received amount 4000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f801b175ac6868
to_local amount 6983598 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d

```
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
3045022100b4b16d5f8cc9fc4c1aff48831e832a0d8990e133978a66e302c133550954a44d022073573ce127e2200d31
6f6b612803a5c0c97b8d20e1e44dbe2ac0dd2fb8c95244
# local_signature =
3045022100d72638bc6308b88bb6d45861aae83e5b9ff6e10986546e13bce769c70036e2620220320be7c6d66d22f30b
9fcd52af66531505b1310ca3b848c19285b38d8a1a8c19
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
03a00f0000000000002200208c48d15160397c9731df9bc3b236656efb6665fbfe92b4a6878e88a499f741c4c0c62d00
00000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e484ae8f6a0000000002200204adb4e2f00643db396dd
120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e04004830450221 00d72638bc6308b88bb6d45861aae83e5b9ff6
e10986546e13bce769c70036e2620220320be7c6d66d22f30b9fcd52af66531505b1310ca3b848c19285b38d8a1a8c19
01483045022100b4b16d5f8cc9fc4c1aff48831e832a0d8990e133978a66e302c133550954a44d022073573ce127e220
0d316f6b612803a5c0c97b8d20e1e44dbe2ac0dd2fb8c9524401475221023da092f6980e58d2c037173180e9a4654760
26ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c1
52ae3e195220
num_htlcs: 1
# signature for output #0 (htlc-success for htlc #4)
remote_htlc_signature =
3045022100f43591c156038ba217756006bb3c55f7d113a325cdd7d9303c82115372858d68022016355b5aadf222bc8d
12e426c75f4a03423917b2443a103eb2a498a3a2234374
# local_htlc_signature =
30440220585dee80fafa264beac535c3c0bb5838ac348b156fdc982f86adc08dfc9bfd250220130abb82f9f295cc9ef4
23dcfef772fde2acd85d9df48cc538981d26a10a9c10
htlc_success_tx (htlc #4):
02000000000101a9172908eace869cc35128c31fc2ab502f72e4dff31aab23e0244c4b04b11ab0000000000000000000
0122020000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100f43591c156038ba217756006bb3c55f7d113a325cdd7d9303c82115372858d68022016355b5aadf222bc8d12
e426c75f4a03423917b2443a103eb2a498a3a2234374014730440220585dee80fafa264beac535c3c0bb5838ac348b15
6fdc982f86adc08dfc9bfd250220130abb82f9f295cc9ef423dcfef772fde2acd85d9df48cc538981d26a10a9c100120
0404040404040404040404040404040404040404040404040404040404040404048a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a91418bc1a114ccf9c052d3d23e28d3b0a9d1227434288527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f801b175ac686800000000

name: commitment tx with two outputs untrimmed (minimum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 4915
# base commitment transaction fee = 3558
# actual commitment transaction fee = 15558
# to_local amount 6984442 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402203a286936e74870ca1459c700c71202af0381910a6bfab687ef494ef1bc3e02c902202506c362d0e3bee15e80
2aa729bf378e051644648253513f1c085b264cc2a720
# local_signature =
30450221008a953551f4d67cb4df3037207fc082ddaf6be84d417b0bd14c80aab66f1b01a402207508796dc75034b2de
e876fe01dc05a08b019f3e5d689ac8842ade2f1befccf5
```

```
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
02c0c62d0000000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e484fa926a00000000002200204adb4e2f00
643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e04004830450221008a953551f4d67cb4df3037207f
c082ddaf6be84d417b0bd14c80aab66f1b01a402207508796dc75034b2dee876fe01dc05a08b019f3e5d689ac8842ade
2f1befccf50147304402203a286936e74870ca1459c700c71202af0381910a6bfab687ef494ef1bc3e02c902202506c3
62d0e3bee15e802aa729bf378e051644648253513f1c085b264cc2a72001475221023da092f6980e58d2c037173180e9
a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c3
9fa711c152ae3e195220
num_htlcs: 0

name: commitment tx with two outputs untrimmed (maximum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 9651180
# base commitment transaction fee = 6987454
# actual commitment transaction fee = 6999454
# to_local amount 546 wscript
63210212a140cd0c6539d07cd08dfe09984dec3251ea808b892efeac3ede9402bf2b1967029000b2752103fd5960528d
c152014952efdb702a88f71e3c1653b2314431701ec77e57fde83c68ac
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402200a8544eba1d216f5c5e530597665fa9bec56943c0f66d98fc3d028df52d84f7002201e45fa5c6bc3a506cc25
53e7d1c0043a9811313fc39c954692c0d47cfce2bbd3
# local_signature =
3045022100e11b638c05c650c2f63a421d36ef8756c5ce82f2184278643520311cdf50aa200220259565fb9c8e4a87cc
af17f27a3b9ca4f20625754a0920d9c6c239d8156a11de
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
022202000000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80ec0c62d00
00000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e484040048304502210 0e11b638c05c650c2f63a421d36
ef8756c5ce82f2184278643520311cdf50aa200220259565fb9c8e4a87ccaf17f27a3b9ca4f20625754a0920d9c6c239
d8156a11de0147304402200a8544eba1d216f5c5e530597665fa9bec56943c0f66d98fc3d028df52d84f7002201e45fa
5c6bc3a506cc2553e7d1c0043a9811313fc39c954692c0d47cfce2bbd301475221023da092f6980e58d2c037173180e9
a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c3
9fa711c152ae3e195220
num_htlcs: 0

name: commitment tx with one output untrimmed (minimum feerate)
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 9651181
# base commitment transaction fee = 6987455
# actual commitment transaction fee = 7000000
# to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402202ade0142008309eb376736575ad58d03e5b115499709c6db0b46e36ff394b492022037b63d78d66404d6504d
4c4ac13be346f3d1802928a6d3ad95a6a944227161a2
# local_signature =
304402207e8d51e0c570a5868a78414f4e0cbfaed1106b171b9581542c30718ee4eb95ba02203af84194c97adf98898c
9afe2f2ed4a7f8dba05a2dfab28ac9d9c604aa49a379
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
```

01c0c62d0000000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e484040047304402207e8d51e0c570a5868a
78414f4e0cbfaed1106b171b9581542c30718ee4eb95ba02203af84194c97adf98898c9afe2f2ed4a7f8dba05a2dfab2
8ac9d9c604aa49a3790147304402202ade0142008309eb376736575ad58d03e5b115499709c6db0b46e36ff394b49202
2037b63d78d66404d6504d4c4ac13be346f3d1802928a6d3ad95a6a944227161a201475221023da092f6980e58d2c037
173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132
cec6d3c39fa711c152ae3e195220
num_htlcs: 0

name: commitment tx with fee greater than funder amount
to_local_msat: 6988000000
to_remote_msat: 3000000000
local_feerate_per_kw: 9651936
base commitment transaction fee = 6988001
actual commitment transaction fee = 7000000
to_remote amount 3000000
P2WPKH(032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991)
remote_signature =
304402202ade0142008309eb376736575ad58d03e5b115499709c6db0b46e36ff394b492022037b63d78d66404d6504d
4c4ac13be346f3d1802928a6d3ad95a6a944227161a2
local_signature =
304402207e8d51e0c570a5868a78414f4e0cbfaed1106b171b9581542c30718ee4eb95ba02203af84194c97adf98898c
9afe2f2ed4a7f8dba05a2dfab28ac9d9c604aa49a379
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80
01c0c62d0000000000160014cc1b07838e387deacd0e5232e1e8b49f4c29e484040047304402207e8d51e0c570a5868a
78414f4e0cbfaed1106b171b9581542c30718ee4eb95ba02203af84194c97adf98898c9afe2f2ed4a7f8dba05a2dfab2
8ac9d9c604aa49a3790147304402202ade0142008309eb376736575ad58d03e5b115499709c6db0b46e36ff394b49202
2037b63d78d66404d6504d4c4ac13be346f3d1802928a6d3ad95a6a944227161a201475221023da092f6980e58d2c037
173180e9a465476026ee50f96695963e8efe436f54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132
cec6d3c39fa711c152ae3e195220
num_htlcs: 0

name: commitment tx with 3 htlc outputs, 2 offered having the same amount and preimage
to_local_msat: 6987999999
to_remote_msat: 3000000000
local_feerate_per_kw: 253
HTLC #1 received amount 2000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c8201208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d41
7a46946384f88d5f3337267c5e579765875dc4daca813e21734b140639e752ae677502f501b175ac6868
HTLC #5 offered amount 5000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9142002cc93ebefbb1b73f0af055dcc27a0b504ad7688ac6868
HTLC #6 offered amount 5000 wscript
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9142002cc93ebefbb1b73f0af055dcc27a0b504ad7688ac6868
HTLC #5 and 6 have CLTV 506 and 505, respectively, and preimage
0505050505050505050505050505050505050505050505050505050505050505
remote_signature =
304402207d0870964530f97b62497b11153c551dca0a1e226815ef0a336651158da0f82402200f5378beee0e77759147
b8a0a284decd11bfd2bc55c8fafa41c134fe996d43c8
local_signature =
304402200d10bf5bc5397fc59d7188ae438d80c77575595a2d488e41bd6363a810cc8d72022012b57e714fbbfdf7a28c

47d5b370cb8ac37c8545f596216e5b21e9b236ef457c
output commit_tx:
02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a488489000000000038b02b80
05d007000000000000220020748eba944fedc8827f6b06bc44678f93c0f9e6078b35c6331ed31e75f8ce0c2d88130000
0000000220020305c12e1a0bc21e283c131cea1c66d68857d28b7b2fce0a6fbc40c164852121b881300000000000022
0020305c12e1a0bc21e283c131cea1c66d68857d28b7b2fce0a6fbc40c164852121bc0c62d0000000000160014cc1b07
838e387deacd0e5232e1e8b49f4c29e484a69f6a0000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c1
1a40d857accc862d6b7dd80e040047304402200d10bf5bc5397fc59d7188ae438d80c77575595a2d488e41bd6363a810
cc8d72022012b57e714fbbfdf7a28c47d5b370cb8ac37c8545f596216e5b21e9b236ef457c0147304402207d08709645
30f97b62497b11153c551dca0a1e226815ef0a336651158da0f82402200f5378beee0e77759147b8a0a284decd11bfd2
bc55c8fafa41c134fe996d43c801475221023da092f6980e58d2c037173180e9a465476026ee50f96695963e8efe436f
54eb21030e9f7b623d2ccc7c9bd44d66d5ce21ce504c0acf6385a132cec6d3c39fa711c152ae3e195220
num_htlcs: 3
signature for output #0 (htlc-success for htlc #1)
remote_htlc_signature =
3045022100b470fe12e5b7fea9eccb8cbff1972cea4f96758041898982a02bcc7f9d56d50b0220338a75b2afaab4ec00
cdd2d9273c68c7581ff5a28bcbb40c4d138b81f1d45ce5
local_htlc_signature =
3044022017b90c65207522a907fb6a137f9dd528b3389465a8ae72308d9e1d564f512cf402204fc917b4f0e88604a3e9
94f85bfae7c7c1f9d9e9f78e8cd112e0889720d9405b
htlc_success_tx (htlc #1):
020000000001014bdccf28653066a2c554cafeffdfe1e678e64a69b056684deb0c4fba909423ec0000000000000000000
011f07000000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100b470fe12e5b7fea9eccb8cbff1972cea4f96758041898982a02bcc7f9d56d50b0220338a75b2afaab4ec00cd
d2d9273c68c7581ff5a28bcbb40c4d138b81f1d45ce501473044022017b90c65207522a907fb6a137f9dd528b3389465
a8ae72308d9e1d564f512cf402204fc917b4f0e88604a3e994f85bfae7c7c1f9d9e9f78e8cd112e0889720d9405b0120
0101010101010101010101010101010101010101010101010101010101010101018a76a91414011f7254d96b819c76986c
277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184d88256816826d6231c068d4a5b7c82
01208763a9144b6b2e5444c2639cc0fb7bcea5afba3f3cdce23988527c21030d417a46946384f88d5f3337267c5e5797
65875dc4daca813e21734b140639e752ae677502f501b175ac686800000000
signature for output #1 (htlc-timeout for htlc #6)
remote_htlc_signature =
3045022100b575379f6d8743cb0087648f81cfd82d17a97fbf8f67e058c65ce8b9d25df9500220554a210d65b02d9f36
c6adf0f639430ca8293196ba5089bf67cc3a9813b7b00a
local_htlc_signature =
3045022100ee2e16b90930a479b13f8823a7f14b600198c838161160b9436ed086d3fc57e002202a66fa2324f342a171
29949c640bfe934cbc73a869ba7c06aa25c5a3d0bfb53d
htlc_timeout_tx (htlc #6):
020000000001014bdccf28653066a2c554cafeffdfe1e678e64a69b056684deb0c4fba909423ec0100000000000000000
01e11200000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004830
45022100b575379f6d8743cb0087648f81cfd82d17a97fbf8f67e058c65ce8b9d25df9500220554a210d65b02d9f36c6
adf0f639430ca8293196ba5089bf67cc3a9813b7b00a01483045022100ee2e16b90930a479b13f8823a7f14b600198c8
38161160b9436ed086d3fc57e002202a66fa2324f342a17129949c640bfe934cbc73a869ba7c06aa25c5a3d0bfb53d01
008576a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a
2184d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4da
ca813e21734b140639e752ae67a9142002cc93ebefbb1b73f0af055dcc27a0b504ad7688ac6868f9010000
signature for output #2 (htlc-timeout for htlc #5)
remote_htlc_signature =
30440220471c9f3ad92e49b13b7b8059f43ecf8f7887b0dccbb9fdb54bfe23d62a8ae332022024bd22fae0740e86a442
28c35330da9526fd7306dffb2b9dc362d5e78abef7cc
local_htlc_signature =
304402207157f452f2506d73c315192311893800cfb3cc235cc1185b1cfcc136b55230db022014be242dbc6c5da141fe
c4034e7f387f74d6ff1899453d72ba957467540e1ecb
htlc_timeout_tx (htlc #5):
020000000001014bdccf28653066a2c554cafeffdfe1e678e64a69b056684deb0c4fba909423ec0200000000000000000

```
01e1120000000000002200204adb4e2f00643db396dd120d4e7dc17625f5f2c11a40d857accc862d6b7dd80e05004730
440220471c9f3ad92e49b13b7b8059f43ecf8f7887b0dccbb9fdb54bfe23d62a8ae332022024bd22fae0740e86a44228
c35330da9526fd7306dffb2b9dc362d5e78abef7cc0147304402207157f452f2506d73c315192311893800cfb3cc235c
c1185b1cfcc136b55230db022014be242dbc6c5da141fec4034e7f387f74d6ff1899453d72ba957467540e1ecb010085
76a91414011f7254d96b819c76986c277d115efce6f7b58763ac67210394854aa6eab5b2a8122cc726e9dded053a2184
d88256816826d6231c068d4a5b7c820120876475527c21030d417a46946384f88d5f3337267c5e579765875dc4daca81
3e21734b140639e752ae67a9142002cc93ebefbb1b73f0af055dcc27a0b504ad7688ac6868fa010000
```

Appendix D: Per-commitment Secret Generation Test Vectors

These test the generation algorithm that all nodes use.

Generation Tests

```
name: generate_from_seed 0 final node
seed: 0x0000000000000000000000000000000000000000000000000000000000000000
I: 281474976710655
output: 0x02a40c85b6f28da08dfdbe0926c53fab2de6d28c10301f8f7c4073d5e42e3148

name: generate_from_seed FF final node
seed: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
I: 281474976710655
output: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc

name: generate_from_seed FF alternate bits 1
seed: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
I: 0xaaaaaaaaaaa
output: 0x56f4008fb007ca9acf0e15b054d5c9fd12ee06cea347914ddbaed70d1c13a528

name: generate_from_seed FF alternate bits 2
seed: 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
I: 0x555555555555
output: 0x9015daaeb06dba4ccc05b91b2f73bd54405f2be9f217fbacd3c5ac2e62327d31

name: generate_from_seed 01 last nontrivial node
seed: 0x0101010101010101010101010101010101010101010101010101010101010101
I: 1
output: 0x915c75942a26bb3a433a8ce2cb0427c29ec6c1775cfc78328b57f6ba7bfeaa9c
```

Storage Tests

These test the optional compact storage system. In many cases, an incorrect entry cannot be determined until its parent is revealed: an entry is specifically corrupted, along with all its children.

For these tests a seed of `0xFFF...FF` is used, and incorrect entries are seeded with `0x000...00`.

```
name: insert_secret correct sequence
I: 281474976710655
secret: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: OK
I: 281474976710653
```

```
secret: 0x2273e227a5b7449b6e70f1fb4652864038b1cbf9cd7c043a7d6456b7fc275ad8
output: OK
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: OK
I: 281474976710651
secret: 0xc65716add7aa98ba7acb236352d665cab17345fe45b55fb879ff80e6bd0c41dd
output: OK
I: 281474976710650
secret: 0x969660042a28f32d9be17344e09374b379962d03db1574df5a8a5a47e19ce3f2
output: OK
I: 281474976710649
secret: 0xa5a64476122ca0925fb344bdc1854c1c0a59fc614298e50a33e331980a220f32
output: OK
I: 281474976710648
secret: 0x05cde6323d949933f7f7b78776bcc1ea6d9b31447732e3802e1f7ac44b650e17
output: OK

name: insert_secret #1 incorrect
I: 281474976710655
secret: 0x02a40c85b6f28da08dfdbe0926c53fab2de6d28c10301f8f7c4073d5e42e3148
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: ERROR

name: insert_secret #2 incorrect (#1 derived from incorrect)
I: 281474976710655
secret: 0x02a40c85b6f28da08dfdbe0926c53fab2de6d28c10301f8f7c4073d5e42e3148
output: OK
I: 281474976710654
secret: 0xdddc3a8d14fddf2b68fa8c7fbad2748274937479dd0f8930d5ebb4ab6bd866a3
output: OK
I: 281474976710653
secret: 0x2273e227a5b7449b6e70f1fb4652864038b1cbf9cd7c043a7d6456b7fc275ad8
output: OK
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: ERROR

name: insert_secret #3 incorrect
I: 281474976710655
secret: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: OK
I: 281474976710653
secret: 0xc51a18b13e8527e579ec56365482c62f180b7d5760b46e9477dae59e87ed423a
output: OK
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: ERROR

name: insert_secret #4 incorrect (1,2,3 derived from incorrect)
```

```
I: 281474976710655
secret: 0x02a40c85b6f28da08dfdbe0926c53fab2de6d28c10301f8f7c4073d5e42e3148
output: OK
I: 281474976710654
secret: 0xdddc3a8d14fddf2b68fa8c7fbad2748274937479dd0f8930d5ebb4ab6bd866a3
output: OK
I: 281474976710653
secret: 0xc51a18b13e8527e579ec56365482c62f180b7d5760b46e9477dae59e87ed423a
output: OK
I: 281474976710652
secret: 0xba65d7b0ef55a3ba300d4e87af29868f394f8f138d78a7011669c79b37b936f4
output: OK
I: 281474976710651
secret: 0xc65716add7aa98ba7acb236352d665cab17345fe45b55fb879ff80e6bd0c41dd
output: OK
I: 281474976710650
secret: 0x969660042a28f32d9be17344e09374b379962d03db1574df5a8a5a47e19ce3f2
output: OK
I: 281474976710649
secret: 0xa5a64476122ca0925fb344bdc1854c1c0a59fc614298e50a33e331980a220f32
output: OK
I: 281474976710648
secret: 0x05cde6323d949933f7f7b78776bcc1ea6d9b31447732e3802e1f7ac44b650e17
output: ERROR

name: insert_secret #5 incorrect
I: 281474976710655
secret: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: OK
I: 281474976710653
secret: 0x2273e227a5b7449b6e70f1fb4652864038b1cbf9cd7c043a7d6456b7fc275ad8
output: OK
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: OK
I: 281474976710651
secret: 0x631373ad5f9ef654bb3dade742d09504c567edd24320d2fcd68e3cc47e2ff6a6
output: OK
I: 281474976710650
secret: 0x969660042a28f32d9be17344e09374b379962d03db1574df5a8a5a47e19ce3f2
output: ERROR

name: insert_secret #6 incorrect (5 derived from incorrect)
I: 281474976710655
secret: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: OK
I: 281474976710653
secret: 0x2273e227a5b7449b6e70f1fb4652864038b1cbf9cd7c043a7d6456b7fc275ad8
output: OK
```

```
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: OK
I: 281474976710651
secret: 0x631373ad5f9ef654bb3dade742d09504c567edd24320d2fcd68e3cc47e2ff6a6
output: OK
I: 281474976710650
secret: 0xb7e76a83668bde38b373970155c868a653304308f9896692f904a23731224bb1
output: OK
I: 281474976710649
secret: 0xa5a64476122ca0925fb344bdc1854c1c0a59fc614298e50a33e331980a220f32
output: OK
I: 281474976710648
secret: 0x05cde6323d949933f7f7b78776bcc1ea6d9b31447732e3802e1f7ac44b650e17
output: ERROR

name: insert_secret #7 incorrect
I: 281474976710655
secret: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: OK
I: 281474976710653
secret: 0x2273e227a5b7449b6e70f1fb4652864038b1cbf9cd7c043a7d6456b7fc275ad8
output: OK
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: OK
I: 281474976710651
secret: 0xc65716add7aa98ba7acb236352d665cab17345fe45b55fb879ff80e6bd0c41dd
output: OK
I: 281474976710650
secret: 0x969660042a28f32d9be17344e09374b379962d03db1574df5a8a5a47e19ce3f2
output: OK
I: 281474976710649
secret: 0xe7971de736e01da8ed58b94c2fc216cb1dca9e326f3a96e7194fe8ea8af6c0a3
output: OK
I: 281474976710648
secret: 0x05cde6323d949933f7f7b78776bcc1ea6d9b31447732e3802e1f7ac44b650e17
output: ERROR

name: insert_secret #8 incorrect
I: 281474976710655
secret: 0x7cc854b54e3e0dcdb010d7a3fee464a9687be6e8db3be6854c475621e007a5dc
output: OK
I: 281474976710654
secret: 0xc7518c8ae4660ed02894df8976fa1a3659c1a8b4b5bec0c4b872abeba4cb8964
output: OK
I: 281474976710653
secret: 0x2273e227a5b7449b6e70f1fb4652864038b1cbf9cd7c043a7d6456b7fc275ad8
output: OK
I: 281474976710652
secret: 0x27cddaa5624534cb6cb9d7da077cf2b22ab21e9b506fd4998a51d54502e99116
output: OK
```

```
I: 281474976710651
secret: 0xc65716add7aa98ba7acb236352d665cab17345fe45b55fb879ff80e6bd0c41dd
output: OK
I: 281474976710650
secret: 0x969660042a28f32d9be17344e09374b379962d03db1574df5a8a5a47e19ce3f2
output: OK
I: 281474976710649
secret: 0xa5a64476122ca0925fb344bdc1854c1c0a59fc614298e50a33e331980a220f32
output: OK
I: 281474976710648
secret: 0xa7efbc61aac46d34f77778bac22c8a20c6a46ca460addc49009bda875ec88fa4
output: ERROR
```

Appendix E: Key Derivation Test Vectors

These test the derivation for `localpubkey`, `remotepubkey`, `local_htlcpubkey`, `remote_htlcpubkey`, `local_delayedpubkey`, and `remote_delayedpubkey` (which use the same formula), as well as the `revocationpubkey`.

All of them use the following secrets (and thus the derived points):

```
base_secret: 0x000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
per_commitment_secret: 0x1f1e1d1c1b1a19181716151413121110f0e0d0c0b0a0908070605040302010 0
base_point: 0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2
per_commitment_point: 0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486

name: derivation of pubkey from basepoint and per_commitment_point
# SHA256(per_commitment_point || basepoint)
# => SHA256(0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486 ||
0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2)
# = 0xcbcdd70fcfad15ea8e9e5c5a12365cf00912504f08ce01593689dd426bca9ff0
# + basepoint (0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2)
# = 0x0235f2dbfaa89b57ec7b055afe29849ef7ddfeb1cefdb9ebdc43f5494984db29e5
localpubkey: 0x0235f2dbfaa89b57ec7b055afe29849ef7ddfeb1cefdb9ebdc43f5494984db29e5

name: derivation of private key from basepoint secret and per_commitment_secret
# SHA256(per_commitment_point || basepoint)
# => SHA256(0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486 ||
0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2)
# = 0xcbcdd70fcfad15ea8e9e5c5a12365cf00912504f08ce01593689dd426bca9ff0
# + basepoint_secret (0x000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f)
# = 0xcbced912d3b21bf196a766651e436aff192362621ce317704ea2f75d87e7be0f
localprivkey: 0xcbced912d3b21bf196a766651e436aff192362621ce317704ea2f75d87e7be0f

name: derivation of revocation pubkey from basepoint and per_commitment_point
# SHA256(revocation_basepoint || per_commitment_point)
# => SHA256(0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2 ||
0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486)
# = 0xefbf7ba5a074276701798376950a64a90f698997cce0dff4d24a6d2785d20963
# x revocation_basepoint = 0x02c00c4aadc536290422a807250824a8d87f19d18da9d610d45621df22510db8ce
# SHA256(per_commitment_point || revocation_basepoint)
# => SHA256(0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486 ||
0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2)
# = 0xcbcdd70fcfad15ea8e9e5c5a12365cf00912504f08ce01593689dd426bca9ff0
```

```
# x per_commitment_point = 0x0325ee7d3323ce52c4b33d4e0a73ab637711057dd8866e3b51202a04112f054c43
# 0x02c00c4aadc536290422a807250824a8d87f19d18da9d610d45621df22510db8ce +
0x0325ee7d3323ce52c4b33d4e0a73ab637711057dd8866e3b51202a04112f054c43 =>
0x02916e326636d19c33f13e8c0c3a03dd157f332f3e99c317c141dd865eb01f8ff0
revocationpubkey: 0x02916e326636d19c33f13e8c0c3a03dd157f332f3e99c317c141dd865eb01f8ff0

name: derivation of revocation secret from basepoint_secret and per_commitment_secret
# SHA256(revocation_basepoint || per_commitment_point)
# => SHA256(0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2 ||
0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486)
# = 0xefbf7ba5a074276701798376950a64a90f698997cce0dff4d24a6d2785d20963
# * revocation_basepoint_secret
(0x0001020304050607080090a0b0c0d0e0f10111213141516171819 1a1b1c1d1e1f)# =
0x44bfd55f845f885b8e60b2dca4b30272d5343be048d79ce87879d9863dedc842
# SHA256(per_commitment_point || revocation_basepoint)
# => SHA256(0x025f7117a78150fe2ef97db7cfc83bd57b2e2c0d0dd25eaf467a4a1c2a45ce1486 ||
0x036d6caac248af96f6afa7f904f550253a0f3ef3f5aa2fe6838a95b216691468e2)
# = 0xcbcdd70fcfad15ea8e9e5c5a12365cf00912504f08ce01593689dd426bca9ff0
# * per_commitment_secret (0x1f1e1d1c1b1a19181716151413121110 0f0e0d0c0b0a09080706050403020100)#
= 0x8be02a96a97b9a3c1c9f59ebb718401128b72ec009d85ee1656319b52319b8ce
# => 0xd09ffff62ddb2297ab000cc85bcb4283fdeb6aa052affbc9dddcf33b61078110
revocationprivkey: 0xd09ffff62ddb2297ab000cc85bcb4283fdeb6aa052affbc9dddcf33b61078110
```

Appendix F: Commitment and HTLC Transaction Test Vectors (anchors)

The anchor test vectors are based on the test cases as defined in appendix C. Note that in appendix C, `to_local_msat` and `to_remote_msat` are balances before subtraction of: * Commit fee (funder only) * Anchor outputs (funder only) * In-flight htlcs


```
[
    {
        "Name": "simple commitment tx with no HTLCs",
        "LocalBalance": 7000000000,
        "RemoteBalance": 3000000000,
        "DustLimitSatoshis": 546,
        "FeePerKw": 15000,
        "UseTestHtlcs": false,
        "HtlcDescs": [],
        "ExpectedCommitmentTxHex":
        "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a4884890000000000038b02b80044

        "RemoteSigHex":
        "3045022100f89034eba16b2be0e5581f750a0a6309192b75cce0f202f0ee2b4ec0cc394850022076c65dc507fe42276152b

    },
    {
        "Name": "simple commitment tx with no HTLCs and single anchor",
        "LocalBalance": 10000000000,
        "RemoteBalance": 0,
        "DustLimitSatoshis": 546,
        "FeePerKw": 15000,
        "UseTestHtlcs": false,
        "HtlcDescs": [],
        "ExpectedCommitmentTxHex":
        "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a4884890000000000038b02b80024
```

```
        "RemoteSigHex":
        "30440220655bf909fb6fa81d086f1336ac72c97906dce29d1b166e305c99152d810e26e1022051f577faa46412c46707aaa

    },
    {
        "Name": "commitment tx with seven outputs untrimmed",
        "LocalBalance": 6988000000,
        "RemoteBalance": 3000000000,
        "DustLimitSatoshis": 546,
        "FeePerKw": 644,
        "UseTestHtlcs": true,
        "HtlcDescs": [
            {
                "RemoteSigHex":
        "30440220746dc89a593e1b50915db63359b50c3c404f8324f78075c87708c866ccefda2502202a11062012dc8607b17e8c4

                "ResolutionTxHex":
        "02000000000101b8cefef62ea66f5178b9361b2371be0759cbc8c689bcfa7a8e6746d497ec221a0200000000100000001e

            },
            {
                "RemoteSigHex":
        "3045022100a847bc3b8cf2441013725ae32dc589c419835d069afd6d7f3d9834d8be7cea6e02204ce9d35ec7f0788da80e4

                "ResolutionTxHex":
        "02000000000101b8cefef62ea66f5178b9361b2371be0759cbc8c689bcfa7a8e6746d497ec221a0300000000100000001d

            },
            {
                "RemoteSigHex":
        "3044022035977f2aa6d6ae12f1dae3366440faa17558e9c88c3188bfc8df7e276c6c65410220659f1f9070c725d9d46e43b

                "ResolutionTxHex":
        "02000000000101b8cefef62ea66f5178b9361b2371be0759cbc8c689bcfa7a8e6746d497ec221a0400000000100000001d

            },
            {
                "RemoteSigHex":
        "3044022002f3ff9a31270092c214d3d5b8b4f826599404bba64b87f7536ef6324d41551b022079dc4cb25f7ecd84b49f5cc

                "ResolutionTxHex":
        "02000000000101b8cefef62ea66f5178b9361b2371be0759cbc8c689bcfa7a8e6746d497ec221a0500000000100000001b

            },
            {
                "RemoteSigHex":
        "304402205479375642 88f64fb3e3c945a1348b912471f0c1c4cc7dc8ceca15a4cbd299b6022053c4f8e30832b13dfbe31e4

                "ResolutionTxHex":
        "02000000000101b8cefef62ea66f5178b9361b2371be0759cbc8c689bcfa7a8e6746d497ec221a0600000000100000001a

            }
        ],
        "ExpectedCommitmentTxHex":
        "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a488489000000000038b02b80094
```

```
        "RemoteSigHex":
        "3045022100e0106830467a558c07544a3de7715610c1147062e7d091deeebe8b5c661cda9402202ad049c1a6d04834317a7

    },
    {
        "Name": "commitment tx with six outputs untrimmed (minimum dust limit)",
        "LocalBalance": 6988000000,
        "RemoteBalance": 3000000000,
        "DustLimitSatoshis": 1001,
        "FeePerKw": 645,
        "UseTestHtlcs": true,
        "HtlcDescs": [
            {
                "RemoteSigHex":
        "3045022100e04d160a326432659fe9fb127304c1d348dfeaba840081bdc57d8efd902a48d8022008a824e7cf5492b97e4d9

                "ResolutionTxHex":
        "02000000000101104f394af4c4fad78337f95e3e9f802f4c0d86ab231853af09b2853485613200020000000010000000 1c

            },
            {
                "RemoteSigHex":
        "3045022100fbdc3c367ce3bf30796025cc590ee1f2ce0e72ae1ac19f5986d6d0a4fc76211f02207e45ae9267e8e820d1885

                "ResolutionTxHex":
        "02000000000101104f394af4c4fad78337f95e3e9f802f4c0d86ab231853af09b2853485613200030000000010000000 1c

            },
            {
                "RemoteSigHex":
        "3044022066c5ef625cee3ddd2bc7b6bfb354b5834cf1cc6d52dd972fb41b7b225437ae4a022066cb85647df65c6b87a54e4

                "ResolutionTxHex":
        "02000000000101104f394af4c4fad78337f95e3e9f802f4c0d86ab231853af09b2853485613200040000000010000000 1b

            },
            {
                "RemoteSigHex":
        "3044022022c7e11595c53ee89a57ca76baf0aed730da035952d6ab3fe6459f5eff3b337a022075e10cc5f5fd724a35ce408

                "ResolutionTxHex":
        "02000000000101104f394af4c4fad78337f95e3e9f802f4c0d86ab231853af09b2853485613200050000000010000000 1a

            }
        ],
        "ExpectedCommitmentTxHex":
        "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80084

        "RemoteSigHex":
        "3044022025d97466c8049e955a5afce28e322f4b34d2561118e52332fb400f9b908cc0a402205dc6fba3a0d67ee142c428c

    },
    {
        "Name": "commitment tx with four outputs untrimmed (minimum dust limit)",
        "LocalBalance": 6988000000,
        "RemoteBalance": 3000000000,
        "DustLimitSatoshis": 2001,
```

```
    "FeePerKw": 2185,
    "UseTestHtlcs": true,
    "HtlcDescs": [
        {
            "RemoteSigHex":
    "304402206870514a72ad6e723ff7f1e0370d7a33c1cd2a0b9272674143ebaf6a1d02dee102205bd953c34faf5e7322e9a1c

            "ResolutionTxHex":
    "02000000000101ac13a7715f80b8e52dda43c6929cade5521bdced3a405da02b443f1ffb1e33cc0200000000100000001b

        },
        {
            "RemoteSigHex":
    "3045022100949e8dd938da56445b1cdfdebe1b7efea086edd05d89910d205a1e2e033ce47102202cbd68b5262ab144d9ec1

            "ResolutionTxHex":
    "02000000000101ac13a7715f80b8e52dda43c6929cade5521bdced3a405da02b443f1ffb1e33cc0300000000100000001a

        }
    ],
    "ExpectedCommitmentTxHex":
    "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80064

    "RemoteSigHex":
    "3044022040f63a16148cf35c8d3d41827f5ae7f7c3746885bb64d4d1b895892a83812b3e02202fcf95c2bf02c466163b3fa

},
{
    "Name": "commitment tx with three outputs untrimmed (minimum dust limit)",
    "LocalBalance": 6988000000,
    "RemoteBalance": 3000000000,
    "DustLimitSatoshis": 3001,
    "FeePerKw": 3687,
    "UseTestHtlcs": true,
    "HtlcDescs": [
        {
            "RemoteSigHex":
    "3044022017b558a3cf5f0cb94269e2e927b29ed22bd2416abb8a7ce6de4d1256f359b93602202e9ca2b1a23ea3e69f433c7

            "ResolutionTxHex":
    "0200000000101542562b326c08e3a076d9cfca2be175041366591da334d8d513ff1686fd95a6002000000000100000001a

        }
    ],
    "ExpectedCommitmentTxHex":
    "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80054

    "RemoteSigHex":
    "3045022100ad6c71569856b2d7ff42e838b4abe74a713426b37f22fa667a195a4c88908c6902202b37272b02a42dc6d9f41

},
{
    "Name": "commitment tx with two outputs untrimmed (minimum dust limit)",
    "LocalBalance": 6988000000,
    "RemoteBalance": 3000000000,
    "DustLimitSatoshis": 4001,
    "FeePerKw": 4894,
```

```
    "UseTestHtlcs": true,
    "HtlcDescs": [],
    "ExpectedCommitmentTxHex":
    "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80044

    "RemoteSigHex":
    "3045022100e784a66b1588575801e237d35e510fd92a81ae3a4a2a1b90c031ad803d07b3f3022021bc5f16501f167607d63

},
{
    "Name": "commitment tx with one output untrimmed (minimum dust limit)",
    "LocalBalance": 6988000000,
    "RemoteBalance": 3000000000,
    "DustLimitSatoshis": 4001,
    "FeePerKw": 6216010,
    "UseTestHtlcs": true,
    "HtlcDescs": [],
    "ExpectedCommitmentTxHex":
    "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a48848900000000038b02b80024

    "RemoteSigHex":
    "30450221008fd5dbff02e4b59020d4cd23a3c30d3e287065fda75a0a09b402980adf68ccda022001e0b8b620cd915ddff11

},
{
    "Name": "commitment tx with 3 htlc outputs, 2 offered having the same amount and
    preimage",
    "LocalBalance": 6987999999,
    "RemoteBalance": 3000000000,
    "DustLimitSatoshis": 546,
    "FeePerKw": 253,
    "UseTestHtlcs": true,
    "HtlcDescs": [
        {
            "RemoteSigHex":
    "30440220078fe5343dab88c348a3a8a9c1a9293259dbf35507ae971702cc39dd623ea9af022011ed0c0f35243cd0bb4d9ca

            "ResolutionTxHex":
    "020000000001013d060d0305c9616eaabc21d41fae85bcb5477b5d7f1c92aa429cf15339bbe1c402000000000100000001c

        },
        {
            "RemoteSigHex":
    "304402202df6bf0f98a42cfd0172a16bded7d1b16c14f5f42ba23f5c54648c14b647531302200fe1508626817f23925bb56

            "ResolutionTxHex":
    "020000000001013d060d0305c9616eaabc21d41fae85bcb5477b5d7f1c92aa429cf15339bbe1c403000000000100000001800

        },
        {
            "RemoteSigHex":
    "3045022100bd206b420c495f3aa714d3ea4766cbe95441deacb5d2f737f1913349aee7c2ae02200249d2c950dd3b15326bf

            "ResolutionTxHex":
    "020000000001013d060d0305c9616eaabc21d41fae85bcb5477b5d7f1c92aa429cf15339bbe1c404000000000100000001800

        }
```

```
        ],
        "ExpectedCommitmentTxHex":
        "02000000000101bef67e4e2fb9ddeeb3461973cd4c62abb35050b1add772995b820b584a488489000000000038b02b80074

        "RemoteSigHex":
        "3044022027b38dfb654c34032ffb70bb43022981652fce923cbbe3cbe7394e2ade8b34230220584195b78da6e25c2e8da6b

    }
]
```


Appendix G: Dual Funded Transaction Test Vectors

Funding Transaction Construction

Preliminaries:

```
Genesis block 0:
0100000000000000000000000000000000000000000000000000000000000000000000003ba3edfd7a7b12b27ac72c3e
67768f617fc81bc3888a51323a9fb8aa4b1e5e4adae5494dffff7f2002000000010100000001000000000000000000000
000000000000000000000000000000000000000000000ffffffff4d04ffff001d0104455468652054696d65732030332f
4a616e2f32303039204368616e63656c6c6f72206f6e206272696e6b206f66207365636f6e64206261696c6f75742066
6f722062616e6b73ffffffff0100f2052a01000000434104678afdb0fe5548271967f1a67130b7105cd6a828e03909a6
7962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec112de5c384df7ba0b8d578a4c702b6bf11d5fac00000000

Block
1:0000002006226e46111a0b59caaf126043eb5bbf28c34f3a5e332a1fc7b2b73cf188910ff86fd1d0db3ac5a72df968
622f31e6b5e6566a09e29206d7c7a55df90e181de8be86815cffff7f2000000000010200000000010100000000000000
000000000000000000000000000000000000000000000000ffffffff03510101ffffffff0200f2052a0100000017a9
14113ca7e584fe1575b6fc39abae991529f66eda5887000000000000000266a24aa21a9ede2f61c3f71d1defd3fa999
dfa36953755c690689799962b48bebd836974e8cf9012000000000000000000000000000000000000000000000000000
00000000000000000000
Coinbase address pubkey: 2MtpN8zCxTp8AWSg7VBjBX7vU6x73bVCKP8
Coinbase address privkey: cPxFtfE1w3ptFnsZvvFeWji21kTArYa9GXwMkYsoQHdaJKrjUTek

Parent transaction (spends coinbase of block 1):
02000000000101f86fd1d0db3ac5a72df968622f31e6b5e6566a09e29206d7c7a55df90e181de800000000171600141f
b9623ffd0d422eacc450fd1e967efc477b83ccfffffffff0580b2e60e0000000022002 0fd89acf65485df89797d9ba7ba
7a33624ac4452f00db08107f34257d33e5b94680b2e60e0000000017a9146a235d064786b49e7043e4a042d4cc429f7e
b6948780b2e60e00000000160014fbb4db9d85fba5e301f4399e3038928e44e37d3280b2e60e0000000017a9147ecd1b
519326bc13b0ec716e469b58ed02b112a087f0006bee0000000017a914f856a70093da3a5b5c4302ade033d4c2171705
d3870247304402206 96f6cee2929f1feb3fd6adf024ca0f9aa2f4920ed6d35fb9ec5b78c8408475302201641afae1124
2160101c6f9932aeb4fcd1f13a9c6df5d1386def000ea259a35001210381d7d5b1bc0d7600565d827242576d9cb793bf
e0754334af82289ee8b65d137600000000
```

Funding transaction (spends parent's outputs):

Locktime: 120 Feerate: 253 sat/kiloweight Opener's `funding_satoshi`: 2 0000 0000 sat Accepter's `funding_satoshi`: 2 0000 0000 sat

Inputs:

```
4303ca8ff10c6c345b9299672a66f111c5b81ae027cc5b0d4d39d09c66b032b9 0
    witness_data:
      preimage: 20 68656c6c6f2074686572652c207468697320697320612062697 4636f6e212121
```

```
      witness_script: 27
82012088a820add57dfe5277079d069ca4ad4893c96de91f88ffb981fdc6a2a34d5336c66aff87
    scriptPubKey: 0020fd89acf65485df89797d9ba7ba7a33624ac4452f00db08107f34257d33e5b946
    address: bcrt1qlky6eaj5sh0cj7tanwnm573nvf9vg3f0qrdssyrlxsjh6vl9h9rql40v2g

4303ca8ff10c6c345b9299672a66f111c5b81ae027cc5b0d4d39d09c66b032b9 2
    pubkey: 034695f5b7864c580bf11f9f8cb1a94eb336f2ce9ef872d2ae1a90ee276c772484
    privkey: cUM8Dr33wK4uFmw3Tz8sbQ7BiBNgX5BthRurU7RkgXVvNUPcWrJf
    witness_program: fbb4db9d85fba5e301f4399e3038928e44e37d32
    scriptPubKey: 0014fbb4db9d85fba5e301f4399e3038928e44e37d32
    address: bcrt1qlw6dh8v9lwj7xq058x0rqwyj3ezwxlfjxsy7er
```

Expected Opener's `tx_add_input` (input 0 above):

```
  num_inputs: 1
  tx_add_input:[
    {
      channel_id: "xxx",
      serial_id: 20,
      prevtx_len: 353,
      prevtx:
"02000000000101f86fd1d0db3ac5a72df968622f31e6b5e6566a09e29206d7c7a55df90e181de800000000171600141
fb9623ffd0d422eacc450fd1e967efc477b83ccfffffffff0580b2e60e0000000220020fd89acf65485df89797d9ba7b
a7a33624ac4452f00db08107f34257d33e5b94680b2e60e0000000017a9146a235d064786b49e7043e4a042d4cc429f7
eb6948780b2e60e00000000160014fbb4db9d85fba5e301f4399e3038928e44e37d3280b2e60e0000000017a9147ecd1
b519326bc13b0ec716e469b58ed02b112a087f0006bee0000000017a914f856a70093da3a5b5c4302ade033d4c217170
5d3870247304402206 96f6cee2929f1feb3fd6adf024ca0f9aa2f4920ed6d35fb9ec5b78c8408475302201641afae112
42160101c6f9932aeb4fcd1f13a9c6df5d1386def000ea259a35001210381d7d5b1bc0d7600565d827242576d9cb793b
fe0754334af82289ee8b65d137600000000",
      prev_vout: 0,
      sequence: 4294967293
    }
  ]
```

Expected Accepter's `tx_add_input` (input 2 above):

```
  num_inputs: 1
  tx_add_input:[
    {
      channel_id: "xxx",
      serial_id: 11,
      prevtx_len: 353,
      prevtx:
"02000000000101f86fd1d0db3ac5a72df968622f31e6b5e6566a09e29206d7c7a55df90e181de800000000171600141
fb9623ffd0d422eacc450fd1e967efc477b83ccfffffffff0580b2e60e0000000220020fd89acf65485df89797d9ba7b
a7a33624ac4452f00db08107f34257d33e5b94680b2e60e0000000017a9146a235d064786b49e7043e4a042d4cc429f7
eb6948780b2e60e00000000160014fbb4db9d85fba5e301f4399e3038928e44e37d3280b2e60e0000000017a9147ecd1
b519326bc13b0ec716e469b58ed02b112a087f0006bee0000000017a914f856a70093da3a5b5c4302ade033d4c217170
5d3870247304402206 96f6cee2929f1feb3fd6adf024ca0f9aa2f4920ed6d35fb9ec5b78c8408475302201641afae112
42160101c6f9932aeb4fcd1f13a9c6df5d1386def000ea259a35001210381d7d5b1bc0d7600565d827242576d9cb793b
fe0754334af82289ee8b65d137600000000",
      prev_vout: 2,
      sequence: 4294967293
```

```
    }
  ]
```

Outputs: (scriptPubKeys)

```
# opener's change address
pubkey: 0206e626a4c6d4392d4030bc78bd93f728d1ba61214a77c63adc17d71e32ded3df
# privkey: cSpC1KYEV1vsUFBwTdcuRkncbwfipY1m5zuQ9CjgAYwiVvbQ4fc1
scriptPubKey: 00141ca1cca8855bad6bc1ea5436edd8cff10b7e448b
address: bcrt1qrjsue2y9twkkhs022smwmkx07y9hu3ytshgjmj

# accepter's change address
pubkey: 028f3978c211f4c0bf4d20674f345ae14e08871b25b2c957b4bdbd42e9726278fc
privkey: cQ1HXnbAE4wGhuB2b9rJEydV5ayeEmMqxf1dvHPZmyMTPkwvZJyg
scriptPubKey: 001444cb0c39f93ecc372b5851725bd29d865d333b10
address: bcrt1qgn9scw0e8mxrw26c29e9h55asewnxwcsdxdp50

# the 2-of-2s
pubkey1: 0292edb5f7bbf9e900f7e024be1c1339c6d149c11930e613af3a983d2565f4e41e
pubkey2: 02e16172a41e928cbd78f761bd1c657c4afc7495a1244f7f30166b654fbf7661e3
script_def:
multi(2,0292edb5f7bbf9e900f7e024be1c1339c6d149c11930e613af3a983d2565f4e41e,02e16172a41e928cbd78f
761bd1c657c4afc7495a1244f7f30166b654fbf7661e3)
script:
52210292edb5f7bbf9e900f7e024be1c1339c6d149c11930e613af3a983d2565f4e41e2102e16172a41e928cbd78f761
bd1c657c4afc7495a1244f7f30166b654fbf7661e352ae
scriptPubKey: 0020297b92c238163e820b82486084634b4846b86a3c658d87b9384192e6bea98ec5
address: bcrt1q99ae9s3czclgyzuzfpsggc6tfprts63uvkxc0wfcgxfwd04f3mzs3asq6l
```

Expected Opener's `tx_add_output`:

```
  num_outputs: 2
  tx_add_output[
    {
      channel_id:"xxx",
      serial_id: 30,
      sats: 49999845,
      scriptlen: 22,
      script: "1600141ca1cca8855bad6bc1ea5436edd8cff10b7e448b"
    },{
      channel_id: "xxx",
      serial_id: 44,
      sats: 400000000,
      scriptlen: 34,
      script: "220020297b92c238163e820b82486084634b4846b86a3c658d87b9384192e6bea98ec5"
    }
  ]
```

Expected Accepter's `tx_add_output`:

```
  num_outputs: 1
  tx_add_output[
    {
      channel_id: xxx,
```

```
      serial_id: 33,
      sats: 49999900,
      scriptlen: 22,
      script: 16001444cb0c39f93ecc372b5851725bd29d865d333b10
    }
  ]
```

Expected Fee Calculation:

Opener's fees and change:

```
    initiator_weight = (input_count, output_count, version, locktime) * 4
                     + segwit_fields (marker + flag)
                     + inputs (txid, vout, scriptSig, sequence) * number initiator inputs * 4
                     + funding_output * 4
                     + change_output * 4
                     + max(number initiator inputs
                         * minimum witness weight,
                 actual witness weight of all inputs)

    initiator_weight = (1 + 1 + 4 + 4) * 4
                     + 2
                     + (32 + 4 + 1 + 4) * 1 * 4
                     + 43 * 4
                     + 31 * 4
                     + max(1 * 107, 71)

    initiator_weight = 609

    initiator_fees = initiator_weight * feerate
    initiator_fees = 609 * 253 / 1000
    initiator_fees = 155 sats

    change = total_funding
           - funding_sats
           - fees

    change = 2 5000 0000
           - 2 0000 0000
           -         155

    change =   4999 9845

    as hex: e5effa0200000000
```

Accepter's fees and change:

```
    contributor_weight = inputs(txid, vout, scriptSig, sequence)
              * number contributor inputs * 4
                       + change_output * 4
               + max(number contributor inputs
                     * minimum witness weight,
                 actual witness weight of all inputs)
```

```
    contributor_weight = (32 + 4 + 1 + 4) * 1 *  4
                       + 31 * 4
                       + max(1 * 107, 99)

    contributor_weight = 395

    contributor_fees = contributor_weight * feerate
    contributor_fees = 398 * 253 / 1000
    contributor_fees = 100 sats

    change = total_funding
           - funding_sats
           - fees

    change = 2 5000 0000
           - 2 0000 0000
           -         100

    change =   4999 9900

    as hex: 1cf0fa0200000000
```

Unsigned Funding Transaction:

```
0200000002b932b0669cd0394d0d5bcc27e01ab8c511f1662a6799925b346c0cf18fca03430200000000fdffffffb932
b0669cd0394d0d5bcc27e01ab8c511f1662a6799925b346c0cf18fca03430000000000fdffffff03e5effa0200000000
1600141ca1cca8855bad6bc1ea5436edd8cff10b7e448b1cf0fa020000000016001444cb0c39f93ecc372b5851725bd2
9d865d333b100084d71700000000220020297b92c238163e820b82486084634b4846b86a3c658d87b9384192e6bea98e
c578000000
```

Expected Opener's `tx_signatures`:

```
  tx_signatures{
      channel_id: xxx,
      txid: "5ca4e657c1aa9d069ea4a5d712045d233a7d7c52738cb02993637289e6386057",
      num_witnesses: 1,
      witness[{
        len: 74,
        witness_data:
"022068656c6c6f2074686572652c207468697320697320612062697 4636f6e2121212782012088a820add57dfe52770
79d069ca4ad4893c96de91f88ffb981fdc6a2a34d5336c66aff87"
      }]
  }
```

Expected Accepter's `tx_signatures`:

```
  tx_signatures{
      channel_id: xxx,
      txid: "5ca4e657c1aa9d069ea4a5d712045d233a7d7c52738cb02993637289e6386057",
      num_witnesses: 1,
      witness[{
        len: 107,
        witness_data:
"0247304402207de9ba56bb9f641372e805782575ee840a899e61021c8b1572b3ec1d5b5950e9022069e9ba998915dae
```

```
193d3c25cb89b5e64370e6a3a7755e7f31cf6d7cbc2a49f6d0121034695f5b7864c580bf11f9f8cb1a94eb336f2ce9ef
872d2ae1a90ee276c772484"
      }]
  }
```

Signed Funding Transaction:

Note locktime is set to 120.

```
02000000000102b932b0669cd0394d0d5bcc27e01ab8c511f1662a6799925b346c0cf18fca03430200000000fdffffff
b932b0669cd0394d0d5bcc27e01ab8c511f1662a6799925b346c0cf18fca03430000000000fdffffff03e5effa020000
00001600141ca1cca8855bad6bc1ea5436edd8cff10b7e448b1cf0fa02000000016001444cb0c39f93ecc372b585172
5bd29d865d333b100084d71700000000220020297b92c238163e820b82486084634b4846b86a3c658d87b9384192e6be
a98ec50247304402207de9ba56bb9f641372e805782575ee840a899e61021c8b1572b3ec1d5b5950e9022069e9ba9989
15dae193d3c25cb89b5e64370e6a3a7755e7f31cf6d7cbc2a49f6d0121034695f5b7864c580bf11f9f8cb1a94eb336f2
ce9ef872d2ae1a90ee276c772484022068656c6c6f2074686572652c207468697320697320612062697463 6f6e212121
2782012088a820add57dfe5277079d069ca4ad4893c96de91f88ffb981fdc6a2a34d5336c66aff8778000000
```

Appendix H: Commitment and HTLC Transaction Test Vectors (zero-fee-commitments)

Test vectors are provided that detail commitment and HTLC transactions created using `zero_fee_commitments` in various scenarios. All parameters are provided to ensure that implementations generate exactly the same signed transactions.

References

Authors

[FIXME:]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

Routing Payments

You don't open a channel with everyone you pay. Most Lightning payments bounce through other people's channels, and the sender is the one who picks the path.

BOLT 04 is onion routing. The sender builds a Sphinx onion: each hop can only decrypt its own layer, which tells it the next node, the amount, and the CLTV expiry. Inner hops don't learn the source, the destination, or the full route. Failures come back along the same onion, encrypted so that only the sender can make sense of them.

BOLT 07 is gossip — how you learn the graph that onions are built on. Nodes announce themselves and their channels, sign those announcements, and flood them through the network. Channel updates carry fees and expiry deltas. Without gossip, onion routing has nowhere to go; without onions, gossip is just a map.

Together these two BOLTs are how a payment finds its way. Invoices, in the next chapter, are how you know who to pay and what hash to use.

In this chapter

- BOLT 04 — Onion routing
- BOLT 07 — P2P node and channel discovery (gossip)

BOLT #4: Onion Routing Protocol

Overview

This document describes the construction of an onion routed packet that is used to route a payment from an *origin node* to a *final node*. The packet is routed through a number of intermediate nodes, called *hops*.

The routing schema is based on the Sphinx construction and is extended with a per-hop payload.

Intermediate nodes forwarding the message can verify the integrity of the packet and can learn which node they should forward the packet to. They cannot learn which other nodes, besides their predecessor or successor, are part of the packet's route; nor can they learn the length of the route or their position within it. The packet is obfuscated at each hop, to ensure that a network-level attacker cannot associate packets belonging to the same route (i.e. packets belonging to the same route do not share any correlating information). Notice that this does not preclude the possibility of packet association by an attacker via traffic analysis.

The route is constructed by the origin node, which knows the public keys of each intermediate node and of the final node. Knowing each node's public key allows the origin node to create a shared secret (using ECDH) for each intermediate node and for the final node. The shared secret is then used to generate a *pseudo-random stream* of bytes (which is used to obfuscate the packet) and a number of *keys* (which are used to encrypt the payload and compute the HMACs). The HMACs are then in turn used to ensure the integrity of the packet at each hop.

Each hop along the route only sees an ephemeral key for the origin node, in order to hide the sender's identity. The ephemeral key is blinded by each intermediate hop before forwarding to the next, making the onions unlinkable along the route.

This specification describes *version 0* of the packet format and routing mechanism.

A node: - upon receiving a higher version packet than it implements: - MUST report a route failure to the origin node. - MUST discard the packet.

Table of Contents

- Conventions
- Key Generation
- Pseudo Random Byte Stream
- Packet Structure
    - Payload Format
    - Basic Multi-Part Payments
- Route Blinding
    - Inside encrypted_recipient_data: encrypted_data_tlv
- Accepting and Forwarding a Payment
    - Payload for the Last Node

    - Non-strict Forwarding
- Shared Secret
- Blinding Ephemeral Onion Keys
- Packet Construction
- Onion Decryption
- Filler Generation
- Returning Errors
    - Failure Messages
    - Receiving Failure Codes
- Successful Payments
- Onion Messages
- `max_htlc_cltv` Selection
- Test Vector
    - Returning Errors
- References
- Authors

Conventions

There are a number of conventions adhered to throughout this document:

- HMAC: the integrity verification of the packet is based on Keyed-Hash Message Authentication Code, as defined by the FIPS 198 Standard / RFC 2104, and using a `SHA256` hashing algorithm.
- Elliptic curve: for all computations involving elliptic curves, the Bitcoin curve is used, as specified in `secp256k1`
- Pseudo-random stream: `ChaCha20` is used to generate a pseudo-random byte stream. For its generation, a fixed 96-bit null-nonce (`0x000000000000000000000000`) is used, along with a key derived from a shared secret and with a `0x00`-byte stream of the desired output size as the message.
- The terms *origin node* and *final node* refer to the initial packet sender and the final packet recipient, respectively.
- The terms *hop* and *node* are sometimes used interchangeably, but a *hop* usually refers to an intermediate node in the route rather than an end node. *origin node* –> *hop* –> … –> *hop* –> *final node*
- The term *processing node* refers to the specific node along the route that is currently processing the forwarded packet.
- The term *peers* refers only to hops that are direct neighbors (in the overlay network): more specifically, *sending peers* forward packets to *receiving peers*.
- Each hop in the route has a variable length `hop_payload`.
    - The variable length `hop_payload` is prefixed with a `bigsize` encoding the length in bytes, excluding the prefix and the trailing HMAC.
- These `hop_payloads` are used to build the combined payload in the packet, also referred to as `hop_payloads` or the mix-header.

Key Generation

A number of encryption and verification keys are derived from the shared secret:

- *rho*: used as key when generating the pseudo-random byte stream that is used to obfuscate the per-hop information

- *mu*: used during the HMAC generation
- *um*: used during error reporting
- *pad*: use to generate random filler bytes for the starting mix-header packet

The key generation function takes a key-type (*rho*=`0x72686F`, *mu*=`0x6d75`, *um*=`0x756d`, or *pad*=`0x706164`) and a 32-byte secret as inputs and returns a 32-byte key.

Keys are generated by computing an HMAC (with `SHA256` as hashing algorithm) using the appropriate key-type (i.e. *rho*, *mu*, *um*, or *pad*) as HMAC-key and the 32-byte shared secret as the message. The resulting HMAC is then returned as the key.

Notice that the key-type does not include a C-style `0x00`-termination-byte, e.g. the length of the *rho* key-type is 3 bytes, not 4.

Pseudo Random Byte Stream

The pseudo-random byte stream is used to obfuscate the packet at each hop of the path, so that each hop may only recover the address and HMAC of the next hop. The pseudo-random byte stream is generated by encrypting (using `ChaCha20`) a `0x00`-byte stream, of the required length, which is initialized with a key derived from the shared secret and a 96-bit zero-nonce (`0x000000000000000000000000`).

The use of a fixed nonce is safe, since the keys are never reused.

Packet Structure

The packet consists of four sections:

- a `version` byte
- a 33-byte compressed `secp256k1` `public_key`, used during the shared secret generation
- a 1300-byte `hop_payloads` consisting of multiple, variable length, `hop_payload` payloads
- a 32-byte `hmac`, used to verify the packet's integrity

The network format of the packet consists of the individual sections serialized into one contiguous byte-stream and then transferred to the packet recipient. Due to the fixed size of the packet, it need not be prefixed by its length when transferred over a connection.

The overall structure of the packet is as follows:

1. type: `onion_packet`
2. data:
    - [`byte:version`]
    - [`point:public_key`]
    - [`1300*byte:hop_payloads`]
    - [`32*byte:hmac`]

For this specification (*version 0*), `version` has a constant value of `0x00`.

The `hop_payloads` field is a structure that holds obfuscated routing information, and associated HMAC. It is 1300 bytes long and has the following structure:

1. type: `hop_payloads`

2. data:
    - [`bigsize:length`]
    - [`length*byte:payload`]
    - [`32*byte:hmac`]
    - ...
    - `filler`

Where, the `length`, `payload`, and `hmac` are repeated for each hop; and where, `filler` consists of obfuscated, deterministically-generated padding, as detailed in Filler Generation. Additionally, `hop_payloads` is incrementally obfuscated at each hop.

Using the `payload` field, the origin node is able to specify the path and structure of the HTLCs forwarded at each hop. As the `payload` is protected under the packet-wide HMAC, the information it contains is fully authenticated with each pair-wise relationship between the HTLC sender (origin node) and each hop in the path.

Using this end-to-end authentication, each hop is able to cross-check the HTLC parameters with the `payload`'s specified values and to ensure that the sending peer hasn't forwarded an ill-crafted HTLC.

Since no `payload` TLV value can ever be shorter than 2 bytes, `length` values of 0 and 1 are reserved. (`0` indicated a legacy format no longer supported, and `1` is reserved for future use).

`payload` format

This is formatted according to the Type-Length-Value format defined in BOLT #1.

1. `tlv_stream`: `payload`
2. types:
    1. type: 2 (`amt_to_forward`)
    2. data:
        - [`tu64:amt_to_forward`]
    3. type: 4 (`outgoing_cltv_value`)
    4. data:
        - [`tu32:outgoing_cltv_value`]
    5. type: 6 (`short_channel_id`)
    6. data:
        - [`short_channel_id:short_channel_id`]
    7. type: 8 (`payment_data`)
    8. data:
        - [`32*byte:payment_secret`]
        - [`tu64:total_msat`]
    9. type: 10 (`encrypted_recipient_data`)
    10. data:
        - [`...*byte:encrypted_recipient_data`]
    11. type: 12 (`current_path_key`)
    12. data:
        - [`point:path_key`]
    13. type: 16 (`payment_metadata`)

14. data:
    - [`...*byte:payment_metadata`]
15. type: 18 (`total_amount_msat`)
16. data:
    - [`tu64:total_msat`]

`short_channel_id` is the ID of the outgoing channel used to route the message; the receiving peer should operate the other end of this channel.

`amt_to_forward` is the amount, in millisatoshis, to forward to the next receiving peer specified within the routing information, or for the final destination.

For non-final nodes, this includes the origin node's computed *fee* for the receiving peer, calculated according to the receiving peer's advertised fee schema (as described in BOLT #7).

`outgoing_cltv_value` is the CLTV value that the *outgoing* HTLC carrying the packet should have. Inclusion of this field allows a hop to both authenticate the information specified by the origin node, and the parameters of the HTLC forwarded, and ensure the origin node is using the current `cltv_expiry_delta` value.

If the values don't correspond, this indicates that either a forwarding node has tampered with the intended HTLC values or that the origin node has an obsolete `cltv_expiry_delta` value.

The requirements ensure consistency in responding to an unexpected `outgoing_cltv_value`, whether it is the final node or not, to avoid leaking its position in the route.

Requirements

The creator of `encrypted_recipient_data` (usually, the recipient of payment):

- MUST create `encrypted_data_tlv` for each node in the blinded route (including itself).
- MUST include `encrypted_data_tlv.payment_relay` for each non-final node.
- MUST include exactly one of `encrypted_data_tlv.short_channel_id` or `encrypted_data_tlv.next_node_id` for each non-final node.
- MUST set `encrypted_data_tlv.payment_constraints` for each non-final node and MAY set it for the final node:
    - `max_cltv_expiry` to the largest block height at which the route is allowed to be used, starting from the final node's chosen `max_cltv_expiry` height at which the route should expire, adding the final node's `min_final_cltv_expiry_delta` and then adding `encrypted_data_tlv.payment_relay.cltv_expiry_delta` at each hop.
    - `htlc_minimum_msat` to the largest minimum HTLC value the nodes will allow.
- If it sets `encrypted_data_tlv.allowed_features`:
    - MUST set it to an empty array.
- MUST compute the total fees and CLTV delta of the route as follows and communicate them to the sender:
    - `total_fee_base_msat(n+1) = (fee_base_msat(n+1) * 1000000 + total_fee_base_msat(n) * (1000000 + fee_proportional_millionths(n+1)) + 1000000 - 1) / 1000000`
    - `total_fee_proportional_millionths(n+1) = ((total_fee_proportional_millionths(n) + fee_proportional_millionths(n+1)) * 1000000 + total_fee_proportional_millionths(n) * fee_proportional_millionths(n+1) + 1000000 - 1) / 1000000`

        - `total_cltv_delta = cltv_delta(0) + cltv_delta(1) + ... + cltv_delta(n) + min_final_cltv_expiry_delta`
    - MUST create the `encrypted_recipient_data` from the `encrypted_data_tlv` as required in Route Blinding.

The writer of the TLV `payload`:

- For every node inside a blinded route:
    - MUST include the `encrypted_recipient_data` provided by the recipient
    - For the first node in the blinded route:
        - MUST include the `path_key` provided by the recipient in `current_path_key`
    - If it is the final node:
        - MUST include `amt_to_forward`, `outgoing_cltv_value` and `total_amount_msat`.
        - The value set for `outgoing_cltv_value`:
            - MUST use the current block height as a baseline value.
            - if a random offset was added to improve privacy:
                - SHOULD add the offset to the baseline value.
    - MUST NOT include any other tlv field.
- For every node outside of a blinded route:
    - MUST include `amt_to_forward` and `outgoing_cltv_value`.
    - For every non-final node:
        - MUST include `short_channel_id`
        - MUST NOT include `payment_data`
    - For the final node:
        - MUST NOT include `short_channel_id`
        - if the recipient provided `payment_secret`:
            - MUST include `payment_data`
            - MUST set `payment_secret` to the one provided
            - MUST set `total_msat` to the total amount it will send
        - if the recipient provided `payment_metadata`:
            - MUST include `payment_metadata` with every HTLC
            - MUST not apply any limits to the size of `payment_metadata` except the limits implied by the fixed onion size

The reader:

- If `encrypted_recipient_data` is present:
    - If `path_key` is set in the incoming `update_add_htlc`:
        - MUST return an error if `current_path_key` is present.
        - MUST use that `path_key` as `path_key` for decryption.
    - Otherwise:
        - MUST return an error if `current_path_key` is not present.
        - MUST use that `current_path_key` as the `path_key` for decryption.
        - SHOULD add a random delay before returning errors.
    - MUST return an error if `encrypted_recipient_data` does not decrypt using the `path_key` as described in Route Blinding.

    - If `payment_constraints` is present:
        - MUST return an error if:
            - the expiry is greater than `encrypted_recipient_data.payment_constraints.max_cltv_expiry`.
            - the amount is below `encrypted_recipient_data.payment_constraints.htlc_minimum_msat`.
    - If `allowed_features` is missing:
        - MUST process the message as if it were present and contained an empty array.
    - MUST return an error if:
        - `encrypted_recipient_data.allowed_features.features` contains an unknown feature bit (even if it is odd).
        - `encrypted_recipient_data` contains both `short_channel_id` and `next_node_id`.
        - the payment uses a feature not included in `encrypted_recipient_data.allowed_features.features`.
    - If it is not the final node:
        - MUST return an error if the payload contains other tlv fields than `encrypted_recipient_data` and `current_path_key`.
        - MUST return an error if `encrypted_recipient_data` does not contain either `short_channel_id` or `next_node_id`.
        - MUST return an error if `encrypted_recipient_data` does not contain `payment_relay`.
        - MUST use values from `encrypted_recipient_data.payment_relay` to calculate `amt_to_forward` and `outgoing_cltv_value` as follows:
            - `amt_to_forward = ((amount_msat - fee_base_msat) * 1000000 + 1000000 + fee_proportional_millionths - 1) / (1000000 + fee_proportional_millionths)`
            - `outgoing_cltv_value = cltv_expiry - payment_relay.cltv_expiry_delta`
    - If it is the final node:
        - MUST return an error if the payload contains other tlv fields than `encrypted_recipient_data`, `current_path_key`, `amt_to_forward`, `outgoing_cltv_value` and `total_amount_msat`.
        - MUST return an error if `amt_to_forward`, `outgoing_cltv_value` or `total_amount_msat` are not present.
        - MUST return an error if `amt_to_forward` is below what it expects for the payment.
        - MUST return an error if incoming `cltv_expiry` < `outgoing_cltv_value`.
        - MUST return an error if incoming `cltv_expiry` < `current_block_height` + `min_final_cltv_expiry_delta`.
- Otherwise (it is not part of a blinded route):
    - MUST return an error if `path_key` is set in the incoming `update_add_htlc` or `current_path_key` is present.
    - MUST return an error if `amt_to_forward` or `outgoing_cltv_value` are not present.
    - if it is not the final node:
        - MUST return an error if:
            - `short_channel_id` is not present,
            - it cannot forward the HTLC to the peer indicated by the channel `short_channel_id`.
            - incoming `amount_msat` - `fee` < `amt_to_forward` (where `fee` is the advertised fee as described in BOLT #7)
            - `cltv_expiry` - `cltv_expiry_delta` < `outgoing_cltv_value`

- If it is the final node:
    - MUST return an error if:
        - `total_msat` is not present
        - incoming `amount_msat` < `amt_to_forward`.
        - incoming `cltv_expiry` < `outgoing_cltv_value`.
        - incoming `cltv_expiry` < `current_block_height` + `min_final_cltv_expiry_delta`.

Additional requirements are specified here for multi-part payments, and here for blinded payments.

Basic Multi-Part Payments

An HTLC may be part of a larger "multi-part" payment: such "base" atomic multipath payments will use the same `payment_hash` for all paths.

Note that `amt_to_forward` is the amount for this HTLC only: a `total_msat` field containing a greater value is a promise by the ultimate sender that the rest of the payment will follow in succeeding HTLCs; we call these outstanding HTLCs which have the same preimage, an "HTLC set".

Note that there are two distinct tlv fields that can be used to transmit `total_msat`. The last one, `total_amount_msat`, was introduced with blinded paths for which the `payment_secret` doesn't make sense.

`payment_metadata` is to be included in every payment part, so that invalid payment details can be detected as early as possible.

Requirements

The writer: - if the invoice offers the `basic_mpp` feature: - MAY send more than one HTLC to pay the invoice. - MUST use the same `payment_hash` on all HTLCs in the set. - SHOULD send all payments at approximately the same time. - SHOULD try to use diverse paths to the recipient for each HTLC. - SHOULD retry and/or re-divide HTLCs which fail. - if the invoice specifies an `amount`: - MUST set `total_msat` to at least that `amount`, and less than or equal to twice `amount`. - otherwise: - MUST set `total_msat` to the amount it wishes to pay. - MUST ensure that the total `amt_to_forward` of the HTLC set which arrives at the payee is equal to or greater than `total_msat`. - MUST NOT send another HTLC if the total `amt_to_forward` of the HTLC set is already greater or equal to `total_msat`. - MUST include `payment_secret`. - otherwise: - MUST set `total_msat` equal to `amt_to_forward`.

The final node: - MUST fail the HTLC if dictated by Requirements under Failure Messages - Note: "amount paid" specified there is the `total_msat` field. - if it does not support `basic_mpp`: - MUST fail the HTLC if `total_msat` is not exactly equal to `amt_to_forward`. - otherwise, if it supports `basic_mpp`: - MUST add it to the HTLC set corresponding to that `payment_hash`. - SHOULD fail the entire HTLC set if `total_msat` is not the same for all HTLCs in the set. - if the total `amt_to_forward` of this HTLC set is equal to or greater than `total_msat`: - SHOULD fulfill all HTLCs in the HTLC set - otherwise, if the total `amt_to_forward` of this HTLC set is less than `total_msat`: - MUST NOT fulfill any HTLCs in the HTLC set - MUST fail all HTLCs in the HTLC set after some reasonable timeout. - SHOULD wait for at least 60 seconds after the initial HTLC. - SHOULD use `mpp_timeout` for the failure message. - MUST require `payment_secret` for all HTLCs in the set. - if it fulfills any HTLCs in the HTLC set: - MUST fulfill the entire HTLC set.

Rationale

If `basic_mpp` is present it causes a delay to allow other partial payments to combine. The total amount must be sufficient for the desired payment, just as it must be for single payments. But this must be reasonably bounded to avoid a denial-of-service.

Because invoices do not necessarily specify an amount, and because payers can add noise to the final amount, the total amount must be sent explicitly. The requirements allow exceeding this slightly, as it simplifies adding noise to the amount when splitting, as well as scenarios in which the senders are genuinely independent (friends splitting a bill, for example).

Because a node may need to pay more than its desired amount (due to the `htlc_minimum_msat` value of channels in the desired path), nodes are allowed to pay more than the `total_msat` they specified. Otherwise, nodes would be constrained in which paths they can take when retrying payments along specific paths. However, no individual HTLC may be for less than the difference between the total paid and `total_msat`.

The restriction on sending an HTLC once the set is over the agreed total prevents the preimage being released before all the partial payments have arrived: that would allow any intermediate node to immediately claim any outstanding partial payments.

An implementation may choose not to fulfill an HTLC set which otherwise meets the amount criterion (eg. some other failure, or invoice timeout), however if it were to fulfill only some of them, intermediary nodes could simply claim the remaining ones.

Route Blinding

1. subtype: `blinded_path`
2. data:
    - [`sciddir_or_pubkey:first_node_id`]
    - [`point:first_path_key`]
    - [`byte:num_hops`]
    - [`num_hops*blinded_path_hop:path`]
3. subtype: `blinded_path_hop`
4. data:
    - [`point:blinded_node_id`]
    - [`u16:enclen`]
    - [`enclen*byte:encrypted_recipient_data`]

A blinded path consists of: 1. an initial introduction point (`first_node_id`) 2. an initial key to share a secret with the first node_id (`first_path_key`) 3. a series of tweaked node ids (`path.blinded_node_id`) 4. a series of binary blobs encrypted to the nodes (`path.encrypted_recipient_data`) to tell them the next hop.

For example, Dave wants Alice to reach him via public node Bob then Carol. He creates a chain of public keys ("path_keys") for Bob, Carol and finally himself, so he can share a secret with each of them. These keys are a simple chain, so each node can derive the next `path_key` without having to be told explicitly.

From these shared secrets, Dave creates and encrypts three `encrypted_data_tlv`s: 1. encrypted_data_bob: For Bob to tell him to forward to Carol 2. encrypted_data_carol: For Carol to tell her to forward to him 3. encrypted_data_dave: For himself to indicate the path was used, and any metadata he wants.

To mask the node ids, he also derives three blinding factors from the shared secrets, which turn Bob into Bob', Carol into Carol' and Dave into Dave'.

So this is the `blinded_path` he hands to Alice.

1. `first_node_id`: Bob
2. `first_path_key`: the first path key for Bob
3. `path`: [Bob', encrypted_data_bob], [Carol', encrypted_data_carol], [Dave', encrypted_data_dave]

There are two different ways for Alice to construct an onion which gets to Bob (since he's probably not a direct peer of hers) which are described in the requirements below.

But after Bob the path is always the same: he will send Carol the `path_key` he derived, along with the onion. She will use the `path_key` to derive the tweak for the onion (which Alice encrypted for Carol' not Carol) so she can decrypt it, and also to derive the key to decrypt `encrypted_data_tlv` which will tell her to forward to Dave (and possibly additional restrictions Dave specified).

Requirements

Note that the creator of the blinded path (i.e. the recipient) is creating it for the sender to use to create an onion, and for the intermediate nodes to read the instructions, hence there are two reader sections here.

The writer of a `blinded_path`:

- MUST create a viable path to itself (‘Nr‘`N_r`) i.e. ‘N0→N1→...→Nr‘`N_0 \rightarrow N_1 \rightarrow ... \rightarrow N_r`.
- MUST set `first_node_id` to ‘N0‘`N_0`
- MUST create a series of ECDH shared secrets for each node in the route using the following algorithm:
  - ‘e0←{0;1}256‘`e_0 \leftarrow \{0;1\}^{256}` (‘e0‘`e_0` SHOULD be obtained via CSPRNG)
  - ‘E0=e0·G‘`E_0 = e_0 \cdot G`
  - For every node in the route:
    - let ‘Ni=ki*G‘`N_i = k_i * G` be the `node_id` (‘ki‘`k_i` is ‘Ni‘`N_i`’s private key)
    - ‘ssi=SHA256(ei*Ni)=SHA256(ki*Ei)‘`ss_i = SHA256(e_i * N_i) = SHA256(k_i * E_i)` (ECDH shared secret known only by ‘Nr‘`N_r` and ‘Ni‘`N_i`)
    - ‘rhoi=HMAC256("rho",ssi)‘`rho_i = HMAC256(\text{"rho"}, ss_i)` (key used to encrypt `encrypted_recipient_data` for ‘Ni‘`N_i` by ‘Nr‘`N_r`)
    - ‘ei+1=SHA256(Ei||ssi)*ei‘`e_{i+1} = SHA256(E_i || ss_i) * e_i` (ephemeral private path key, only known by ‘Nr‘`N_r`)
    - ‘Ei+1=SHA256(Ei||ssi)*Ei‘`E_{i+1} = SHA256(E_i || ss_i) * E_i` (`path_key`. NB: ‘Ni‘`N_i` MUST NOT learn ‘ei‘`e_i`)
- MUST set `first_path_key` to ‘E0‘`E_0`
- MUST create a series of blinded node IDs ‘Bi‘`B_i` for each node using the following algorithm:
  - ‘Bi=HMAC256("blinded_node_id",ssi)*Ni‘`B_i = HMAC256(\text{"blinded\_node\_id"}, ss_i) * N_i` (blinded `node_id` for ‘Ni‘`N_i`, private key known only by ‘Ni‘`N_i`)
  - MUST set `blinded_node_id` for each `blinded_path_hop` in `path` to ‘Bi‘`B_i`
- MAY replace ‘Ei+1‘`E_{i+1}` with a different value, but if it does:
  - MUST set `encrypted_data_tlv[i].next_path_key_override` to ‘Ei+1‘`E_{i+1}`
- MAY store private data in `encrypted_data_tlv[r].path_id` to verify that the route is used in the right context and was created by them

- SHOULD add padding data to ensure all `encrypted_data_tlv[i]` have the same length
- MUST encrypt each `encrypted_data_tlv[i]` with ChaCha20-Poly1305 using the corresponding ‘rhoi‘`rho_i` key and an all-zero nonce to produce `encrypted_recipient_data[i]`
- MAY add additional “dummy” hops at the end of the path (which it will ignore on receipt) to obscure the path length.

The reader of the `blinded_path`: - MUST prepend its own onion payloads to reach the `first_node_id` - MUST include the corresponding `encrypted_recipient_data` in each onion payload within `path` - For the first entry in `path`: - if it is sending a payment: - SHOULD create an unblinded onion payment to `first_node_id`, and include `first_path_key` as `current_path_key`. - otherwise: - MUST encrypt the first blinded path onion to the first `blinded_node_id`. - MUST set `next_path_key_override` in the prior onion payload to `first_path_key`. - For each successive entry in `path`: - MUST encrypt the onion to the corresponding `blinded_node_id`.

The reader of the `encrypted_recipient_data`:

- MUST compute:
  - ‘ssi=SHA256(ki*Ei)‘`ss_i = SHA256(k_i * E_i)` (standard ECDH)
  - ‘bi=HMAC256("blinded_node_id",ssi)*ki‘`b_i = HMAC256(\text{"blinded\_node\_id"}, ss_i) * k_i`
  - ‘rhoi=HMAC256("rho",ssi)‘`rho_i = HMAC256(\text{"rho"}, ss_i)`
- MUST decrypt the `encrypted_recipient_data` field using ‘rhoi‘`rho_i` as a key using ChaCha20-Poly1305 and an all-zero nonce key.
- If the `encrypted_recipient_data` field is missing, cannot be decrypted into an `encrypted_data_tlv` or contains unknown even fields:
  - MUST return an error
- If the `encrypted_data_tlv` contains a `next_path_key_override`:
  - MUST use it as the next `path_key`.
- Otherwise:
  - MUST use ‘Ei+1=SHA256(Ei||ssi)*Ei‘`E_{i+1} = SHA256(E_i || ss_i) * E_i` as the next `path_key`
- MUST forward the onion and include the next `path_key` in the lightning message for the next node
- If it is the final recipient:
  - MUST ignore the message if the `path_id` does not match the blinded route it created for this purpose

Rationale

Route blinding is a lightweight technique to provide recipient anonymity. It’s more flexible than rendezvous routing because it simply replaces the public keys of the nodes in the route with random public keys while letting senders choose what data they put in the onion for each hop. Blinded routes are also reusable in some cases (e.g. onion messages).

Each node in the blinded route needs to receive ‘Ei‘`E_i` to be able to decrypt the onion and the `encrypted_recipient_data` payload.

When concatenating two blinded routes generated by different nodes, the last node of the first route needs to know the first `path_key` of the second route: the `next_path_key_override` field must be used to transmit this information. In theory this method could be used for payments (not just onion messages), but we recommend using an unblinded path to reach the `first_node_id` and using `current_path_key` there: this means that the

node can tell it is being used as an introductory point, but also does not require blinded path support on the nodes to reach that point, and gives meaningful errors on the unblinded part of the payment.

The final recipient must verify that the blinded route is used in the right context (e.g. for a specific payment) and was created by them. Otherwise a malicious sender could create different blinded routes to all the nodes that they suspect could be the real recipient and try them until one accepts the message. The recipient can protect against that by storing ‘Er‘`E_r` and the context (e.g. a `payment_hash`), and verifying that they match when receiving the onion. Otherwise, to avoid additional storage cost, it can put some private context information in the `path_id` field (e.g. the `payment_preimage`) and verify that when receiving the onion. Note that it’s important to use private information in that case, that senders cannot have access to.

Whenever the introduction point receives a failure from the blinded route, it should add a random delay before forwarding the error. Failures are likely to be probing attempts and message timing may help the attacker infer its distance to the final recipient.

Note that nodes in the blinded route return failures through `update_fail_malformed_htlc` and therefore do not and can not provide timing information via attribution data to the sender.

The `padding` field can be used to ensure that all `encrypted_recipient_data` have the same length. It’s particularly useful when adding dummy hops at the end of a blinded route, to prevent the sender from figuring out which node is the final recipient.

When route blinding is used for payments, the recipient specifies the fees and expiry that blinded nodes should apply to the payment instead of letting the sender configure them. The recipient also adds additional constraints to the payments that can go through that route to protect against probing attacks that would let malicious nodes unblind the identity of the blinded nodes. It should set `payment_constraints.max_cltv_expiry` to restrict the lifetime of a blinded route and reduce the risk that an intermediate node updates its fees and rejects payments (which could be used to unblind nodes inside the route).

Inside `encrypted_recipient_data`: `encrypted_data_tlv`

The `encrypted_recipient_data` is a TLV stream, encrypted for a given blinded node, that may contain the following TLV fields:

1. `tlv_stream`: `encrypted_data_tlv`
2. types:
    1. type: 1 (`padding`)
    2. data:
        - [`...*byte:padding`]
    3. type: 2 (`short_channel_id`)
    4. data:
        - [`short_channel_id:short_channel_id`]
    5. type: 4 (`next_node_id`)
    6. data:
        - [`point:node_id`]
    7. type: 6 (`path_id`)
    8. data:
        - [`...*byte:data`]

9. type: 8 (`next_path_key_override`)
10. data:
    - [`point:path_key`]
11. type: 10 (`payment_relay`)
12. data:
    - [`u16:cltv_expiry_delta`]
    - [`u32:fee_proportional_millionths`]
    - [`tu32:fee_base_msat`]
13. type: 12 (`payment_constraints`)
14. data:
    - [`u32:max_cltv_expiry`]
    - [`tu64:htlc_minimum_msat`]
15. type: 14 (`allowed_features`)
16. data:
    - [`...*byte:features`]

Rationale

Encrypted recipient data is created by the final recipient to give to the sender, containing instructions for the node on how to handle the message (it can also be created by the sender themselves: the node forwarding cannot tell). It's used in both payment onions and onion messages onions. See Route Blinding.

Accepting and Forwarding a Payment

Once a node has decoded the payload it either accepts the payment locally, or forwards it to the peer indicated as the next hop in the payload.

Non-strict Forwarding

A node MAY forward an HTLC along an outgoing channel other than the one specified by `short_channel_id`, so long as the receiver has the same node public key intended by `short_channel_id`. Thus, if `short_channel_id` connects nodes A and B, the HTLC can be forwarded across any channel connecting A and B. Failure to adhere will result in the receiver being unable to decrypt the next hop in the onion packet.

Rationale

In the event that two peers have multiple channels, the downstream node will be able to decrypt the next hop payload regardless of which channel the packet is sent across.

Nodes implementing non-strict forwarding are able to make real-time assessments of channel bandwidths with a particular peer, and use the channel that is locally-optimal.

For example, if the channel specified by `short_channel_id` connecting A and B does not have enough bandwidth at forwarding time, then A is able use a different channel that does. This can reduce payment latency by preventing the HTLC from failing due to bandwidth constraints across `short_channel_id`, only to have the sender attempt the same route differing only in the channel between A and B.

Non-strict forwarding allows nodes to make use of private channels connecting them to the receiving node, even if the channel is not known in the public channel graph.

Recommendation

Implementations using non-strict forwarding should consider applying the same fee schedule to all channels with the same peer, as senders are likely to select the channel which results in the lowest overall cost. Having distinct policies may result in the forwarding node accepting fees based on the most optimal fee schedule for the sender, even though they are providing aggregate bandwidth across all channels with the same peer.

Alternatively, implementations may choose to apply non-strict forwarding only to like-policy channels to ensure their expected fee revenue does not deviate by using an alternate channel.

Payload for the Last Node

When building the route, the origin node MUST use a payload for the final node with the following values:

- `payment_secret`: set to the payment secret specified by the recipient (e.g. `payment_secret` from a BOLT #11 payment invoice)
- `outgoing_cltv_value`: set to the final expiry specified by the recipient (e.g. `min_final_cltv_expiry_delta` from a BOLT #11 payment invoice)
- `amt_to_forward`: set to the final amount specified by the recipient (e.g. `amount` from a BOLT #11 payment invoice)

This allows the final node to check these values and return errors if needed, but it also eliminates the possibility of probing attacks by the second-to-last node. Such attacks could, otherwise, attempt to discover if the receiving peer is the last one by re-sending HTLCs with different amounts/expiries. The final node will extract its onion payload from the HTLC it has received and compare its values against those of the HTLC. See the Returning Errors section below for more details.

If not for the above, since it need not forward payments, the final node could simply discard its payload.

Shared Secret

The origin node establishes a shared secret with each hop along the route using Elliptic-curve Diffie-Hellman between the sender's ephemeral key at that hop and the hop's node ID key. The resulting curve point is serialized to the compressed format and hashed using `SHA256`. The hash output is used as the 32-byte shared secret.

Elliptic-curve Diffie-Hellman (ECDH) is an operation on an EC private key and an EC public key that outputs a curve point. For this protocol, the ECDH variant implemented in `libsecp256k1` is used, which is defined over the `secp256k1` elliptic curve. During packet construction, the sender uses the ephemeral private key and the hop's public key as inputs to ECDH, whereas during packet forwarding, the hop uses the ephemeral public key and its own node ID private key. Because of the properties of ECDH, they will both derive the same value.

Blinding Ephemeral Onion Keys

In order to ensure multiple hops along the route cannot be linked by the ephemeral public keys they see, the key is blinded at each hop. The blinding is done in a deterministic way that allows the sender to compute the corresponding blinded private keys during packet construction.

The blinding of an EC public key is a single scalar multiplication of the EC point representing the public key with a 32-byte blinding factor. Due to the commutative property of scalar multiplication, the blinded private key is the multiplicative product of the input's corresponding private key with the same blinding factor.

The blinding factor itself is computed as a function of the ephemeral public key and the 32-byte shared secret. Concretely, it is the `SHA256` hash value of the concatenation of the public key serialized in its compressed format and the shared secret.

Packet Construction

In the following example, it's assumed that a *sending node* (origin node), `n_0`, wants to route a packet to a *receiving node* (final node), `n_r`. First, the sender computes a route `{n_0, n_1, ..., n_{r-1}, n_r}`, where `n_0` is the sender itself and `n_r` is the final recipient. All nodes `n_i` and `n_{i+1}` MUST be peers in the overlay network route. The sender then gathers the public keys for `n_1` to `n_r` and generates a random 32-byte `sessionkey`. Optionally, the sender may pass in *associated data*, i.e. data that the packet commits to but that is not included in the packet itself. Associated data will be included in the HMACs and must match the associated data provided during integrity verification at each hop. The `payment_hash` is commonly used for this purpose.

To construct the onion, the sender initializes the ephemeral private key for the first hop `ek_1` to the `sessionkey` and derives from it the corresponding ephemeral public key `epk_1` by multiplying with the `secp256k1` base point. For each of the `k` hops along the route, the sender then iteratively computes the shared secret `ss_k` and ephemeral key for the next hop `ek_{k+1}` as follows:

- The sender executes ECDH with the hop's public key and the ephemeral private key to obtain a curve point, which is hashed using `SHA256` to produce the shared secret `ss_k`.
- The blinding factor is the `SHA256` hash of the concatenation between the ephemeral public key `epk_k` and the shared secret `ss_k`.
- The ephemeral private key for the next hop `ek_{k+1}` is computed by multiplying the current ephemeral private key `ek_k` by the blinding factor.
- The ephemeral public key for the next hop `epk_{k+1}` is derived from the ephemeral private key `ek_{k+1}` by multiplying with the base point.

Once the sender has all the required information above, it can construct the packet. Constructing a packet routed over `r` hops requires `r` 32-byte ephemeral public keys, `r` 32-byte shared secrets, `r` 32-byte blinding factors, and `r` variable length `hop_payload` payloads. The construction returns a single 1366-byte packet along with the first receiving peer's address.

The packet construction is performed in the reverse order of the route, i.e. the last hop's operations are applied first.

The packet is initialized with 1300 *random* bytes derived from a CSPRNG (ChaCha20). The *pad* key referenced above is used to extract additional random bytes from a ChaCha20 stream, using it as a CSPRNG for this

purpose. Once the `paddingKey` has been obtained, ChaCha20 is used with an all zero nonce, to generate 1300 random bytes. Those random bytes are then used as the starting state of the mix-header to be created.

A filler is generated (see Filler Generation) using the shared secret.

For each hop in the route, in reverse order, the sender applies the following operations:

- The *rho*-key and *mu*-key are generated using the hop's shared secret.
- `shift_size` is defined as the length of the `hop_payload` plus the bigsize encoding of the length and the length of that HMAC. Thus if the payload length is `l` then the `shift_size` is `1 + l + 32` for `l < 253`, otherwise `3 + l + 32` due to the bigsize encoding of `l`.
- The `hop_payloads` field is right-shifted by `shift_size` bytes, discarding the last `shift_size` bytes that exceed its 1300-byte size.
- The bigsize-serialized length, serialized `hop_payload` and `hmac` are copied into the `shift_size` bytes as the beginning of the mix-header.
- The *rho*-key is used to generate a 1300 byte pseudo-random stream which is then applied, with `XOR`, to the `hop_payloads` field.
- If this is the last hop, i.e. the first iteration, then the tail of the `hop_payloads` field is overwritten with the routing information `filler`.
- The next HMAC is computed (with the *mu*-key as HMAC-key) over the concatenated `hop_payloads` and associated data.

The resulting final HMAC value is the HMAC that will be used by the first receiving peer in the route.

The packet generation returns a serialized packet that contains the `version` byte, the ephemeral pubkey for the first hop, the HMAC for the first hop, and the obfuscated `hop_payloads`.

The following Go code is an example implementation of the packet construction:

```
const RoutingInfoSize int32 = 1300

func NewOnionPacket(paymentPath []*btcec.PublicKey, sessionKey *btcec.PrivateKey,
    hopsData []HopData, assocData []byte) (*OnionPacket, error) {

    numHops := len(paymentPath)
    hopSharedSecrets := make([][sha256.Size]byte, numHops)

    // Initialize ephemeral key for the first hop to the session key.
    var ephemeralKey big.Int
    ephemeralKey.Set(sessionKey.D)

    for i := 0; i < numHops; i++ {
        // Perform ECDH and hash the result.
        ecdhResult := scalarMult(paymentPath[i], ephemeralKey)
        hopSharedSecrets[i] = sha256.Sum256(ecdhResult.SerializeCompressed())

        // Derive ephemeral public key from private key.
        ephemeralPrivKey := btcec.PrivKeyFromBytes(btcec.S256(), ephemeralKey.Bytes())
        ephemeralPubKey := ephemeralPrivKey.PubKey()

        // Compute blinding factor.
        sha := sha256.New()
```

```
        sha.Write(ephemeralPubKey.SerializeCompressed())
        sha.Write(hopSharedSecrets[i])

        var blindingFactor big.Int
        blindingFactor.SetBytes(sha.Sum(nil))

        // Blind ephemeral key for next hop.
        ephemeralKey.Mul(&ephemeralKey, &blindingFactor)
        ephemeralKey.Mod(&ephemeralKey, btcec.S256().Params().N)
    }

    // Generate the padding, called "filler strings" in the paper.
    filler := generateHeaderPadding(numHops, hopSharedSecrets)

    // Allocate and initialize fields to zero-filled slices
    var mixHeader [RoutingInfoSize]byte
    var nextHmac [32]byte

// Our starting packet needs to be filled out with random bytes, we
// generate some deterministically using the session private key.
paddingKey := generateKey("pad", sessionKey.Serialize())
paddingBytes := generateCipherStream(paddingKey, RoutingInfoSize)
copy(mixHeader[:], paddingBytes)

    // Compute the routing information for each hop along with a
    // MAC of the routing information using the shared key for that hop.
    for i := numHops - 1; i >= 0; i-- {
        rhoKey := generateKey("rho", hopSharedSecrets[i])
        muKey := generateKey("mu", hopSharedSecrets[i])

        hopsData[i].HMAC = nextHmac

        // Shift and obfuscate routing information
        streamBytes := generateCipherStream(rhoKey, RoutingInfoSize)

        rightShift(mixHeader[:], hopDataSize)
        buf := &bytes.Buffer{}
        hopsData[i].Encode(buf)
        copy(mixHeader[:], buf.Bytes())
        xor(mixHeader[:], mixHeader[:], streamBytes[:RoutingInfoSize])

        // These need to be overwritten, so every node generates a correct padding
        if i == numHops-1 {
            copy(mixHeader[len(mixHeader)-len(filler):], filler)
        }

        packet := append(mixHeader[:], assocData...)
        nextHmac = calcMac(muKey, packet)
    }

    packet := &OnionPacket{
        Version:      0x00,
        EphemeralKey: sessionKey.PubKey(),
        RoutingInfo:  mixHeader,
        HeaderMAC:    nextHmac,
```

```
    }
    return packet, nil
}
```

Onion Decryption

There are two kinds of `onion_packet` we use:

1. `onion_routing_packet` in `update_add_htlc` for payments, which contains a `payload` TLV (see Adding an HTLC)
2. `onion_message_packet` in `onion_message` for messages, which contains an `onionmsg_tlv` TLV (see Onion Messages)

Those sections specify the `associated_data` to use, the `path_key` (if any), the extracted payload format and handling (including how to determine the next peer, if any), and how to handle errors. The processing itself is identical.

Requirements

A reader: - if `version` is not 0: - MUST abort processing the packet and fail. - if `public_key` is not a valid pubkey: - MUST abort processing the packet and fail. - if the onion is for a payment: - if `hmac` has previously been received: - if the preimage is known: - MAY immediately redeem the HTLC using the preimage. - otherwise: - MUST abort processing the packet and fail. - if `path_key` is specified: - Calculate the `blinding_ss` as ECDH(`path_key`, `node_privkey`). - Either: - Tweak `public_key` by multiplying by ‘HMAC256("blinded_node_id",blinding_ss)‘`HMAC256(\text{"blinded\_node\_id"}, blinding\_ss)`. - or (equivalently): - Tweak its own `node_privkey` below by multiplying by ‘HMAC256("blinded_node_id",blinding_ss)‘`HMAC256(\text{"blinded\_node\_id"}, blinding\_ss)`. - Derive the shared secret `ss` as ECDH(`public_key`, `node_privkey`) (see Shared Secret). - Derive `mu` as ‘HMAC256("mu",ss)‘`HMAC256(\text{"mu"}, ss)` (see Key Generation). - Derive the HMAC as ‘HMAC256(mu,hop_payloads||associated_data)‘`HMAC256(mu, hop\_payloads || associated\_data)`. - MUST use a constant time comparison of the computed HMAC and `hmac`. - If the computed HMAC and `hmac` differ: - MUST abort processing the packet and fail. - Derive `rho` as ‘HMAC256("rho",ss)‘`HMAC256(\text{"rho"}, ss)` (see Key Generation). - Derive `bytestream` of twice the length of `hop_payloads` using `rho` (see Pseudo Random Byte Stream). - Set `unwrapped_payloads` to the XOR of `hop_payloads` and `bytestream`. - Remove a `bigsize` from the front of `unwrapped_payloads` as `payload_length`. If that is malformed: - MUST abort processing the packet and fail. - If the `payload_length` is less than two: - MUST abort processing the packet and fail. - If there are fewer than `payload_length` bytes remaining in `unwrapped_payloads`: - MUST abort processing the packet and fail. - Remove `payload_length` bytes from the front of `unwrapped_payloads`, as the current `payload`. - If there are fewer than 32 bytes remaining in `unwrapped_payloads`: - MUST abort processing the packet and fail. - Remove 32 bytes as `next_hmac` from the front of `unwrapped_payloads`. - If `unwrapped_payloads` is smaller than `hop_payloads`: - MUST abort processing the packet and fail. - If `next_hmac` is not all-zero (not the final node): - Derive `blinding_tweak` as ‘SHA256(public_key||ss)‘`SHA256(public\_key || ss)` (see Blinding Ephemeral Onion Keys). - SHOULD forward an onion to the next peer with: - `version` set to 0. - `public_key` set to the incoming `public_key` multiplied by `blinding_tweak`. - `hop_payloads` set to the `unwrapped_payloads`, truncated to the incoming `hop_payloads` size. - `hmac` set to `next_hmac`. - If it cannot forward: - MUST fail. - Otherwise (all-zero `next_hmac`): - This is the final destination of the onion.

Rationale

In the case where blinded paths are used, the sender did not actually encrypt this onion for our `node_id`, but for a tweaked version: we can derive the tweak used from `path_key` which is given alongside the onion. Then we either tweak our node private key the same way to decrypt the onion, or tweak to the onion ephemeral key which is mathematically equivalent.

Filler Generation

Upon receiving a packet, the processing node extracts the information destined for it from the route information and the per-hop payload. The extraction is done by deobfuscating and left-shifting the field. This would make the field shorter at each hop, allowing an attacker to deduce the route length. For this reason, the field is pre-padded before forwarding. Since the padding is part of the HMAC, the origin node will have to pre-generate an identical padding (to that which each hop will generate) in order to compute the HMACs correctly for each hop. The filler is also used to pad the field-length, in the case that the selected route is shorter than 1300 bytes.

Before deobfuscating the `hop_payloads`, the processing node pads it with 1300 `0x00`-bytes, such that the total length is `2*1300`. It then generates the pseudo-random byte stream, of matching length, and applies it with `XOR` to the `hop_payloads`. This deobfuscates the information destined for it, while simultaneously obfuscating the added `0x00`-bytes at the end.

In order to compute the correct HMAC, the origin node has to pre-generate the `hop_payloads` for each hop, including the incrementally obfuscated padding added by each hop. This incrementally obfuscated padding is referred to as the `filler`. Keep in mind that while the mix-header is generated in reverse-route order, the filler is generated in route order.

The following example code shows how the filler is generated in Go:

```
const (
  // The mix-header in Lightning is a max of 1300 bytes
  NumMaxHops int = 20
  HopSize = 65
)

func generateFiller(numHops int, sharedSecrets [][sharedSecretSize]byte) []byte {
    fillerSize := uint((NumMaxHops + 1) * HopSize)
    filler := make([]byte, fillerSize)

    // The last hop does not obfuscate, it's not forwarding anymore.
    for i := 0; i < numHops-1; i++ {

        // Left-shift the field
        copy(filler[:], filler[HopSize:])

        // Zero-fill the last hop
        copy(filler[len(filler)-HopSize:], bytes.Repeat([]byte{0x00}, HopSize))

        // Generate pseudo-random byte stream
        streamKey := generateKey("rho", sharedSecrets[i])
        streamBytes := generateCipherStream(streamKey, fillerSize)
```

```
        // Obfuscate
        xor(filler, filler, streamBytes)
    }

    // Cut filler down to the correct length (numHops+1)*HopSize
    // bytes will be prepended by the packet generation.
    return filler[(NumMaxHops-numHops+2)*HopSize:]
}
```

Note that this example implementation is for demonstration purposes only; the `filler` can be generated much more efficiently. The last hop need not obfuscate the `filler`, since it won't forward the packet any further and thus need not extract an HMAC either.

Returning Errors

The onion routing protocol includes a mechanism for returning encrypted error messages to the origin node. The returned error messages may be failures reported by any hop, including the final node. The format of the forward packet is not usable for the return path, since no hop besides the origin has access to the information required for its generation. Note that these error messages are not reliable, as they are not placed on-chain due to the possibility of hop failure.

Return packets are limited to 32768 bytes (32 KiB), leaving room in `update_fail_htlc` for `attribution_data` and future extensions. Earlier versions permitted larger return packets, so intermediate nodes truncate them to their first 32768 bytes instead of rejecting the corresponding message.

Intermediate hops store the shared secret from the forward path and reuse it to authenticate and obfuscate any corresponding return packet during each hop. In addition, each node locally stores data regarding its own sending peer in the route, so it knows where to return-forward any eventual return packets.

Erring node

The node generating the error message builds a *return packet* consisting of the following fields:

1. data:
    - [`32*byte:hmac`]
    - [`u16:failure_len`]
    - [`failure_len*byte:failuremsg`]
    - [`u16:pad_len`]
    - [`pad_len*byte:pad`]

Where `hmac` is an HMAC authenticating the remainder of the packet, with a key generated using the above process, with key type `um`, `failuremsg` as defined below, and `pad` as the extra bytes used to conceal length.

The erring node then generates a new key, using the key type `ammag`. This key is then used to generate a pseudo-random stream, which is in turn applied to the packet using `XOR`.

Error handling for HTLCs with `path_key` is particularly fraught, since differences in implementations (or versions) may be leveraged to de-anonymize elements of the blinded path. Thus the decision turn every error into `invalid_onion_blinding` which will be converted to a normal onion error by the introduction point.

Initialization of `attribution_data`

For the layout of `attribution data`, see Removing an HTLC: `update_fulfill_htlc`, `update_fail_htlc`, and `update_fail_malformed_htlc`.

The `htlc_hold_times` field specifies the htlc hold time for each hop in units of 100 milliseconds. For example, a value of 3 represents 300 ms. Nodes along the path that lack accurate timing information may report a value of zero.

The erring node puts its hold time at the start of this array and zeroes out the rest. The size of the field is based on the maximum supported number of hops in a route (20).

The field `truncated_hmacs` contains truncated authentication codes series for each hop, with the same `um` key that is used for `hmac` in the return packet. Regular 32 byte HMACs are truncated to the first 4 bytes to save space.

In theory this truncation makes it possible for malicious nodes to guess the HMAC. However, game theory is against them because a wrong guess will get them penalized in future pathfinding.

The size of the field is based on the maximum number of hops in a route (20) and the truncated HMAC size (4 bytes). Each hop adds 20 HMACs, one for each possible position that the hop could be at in the path. This is necessary because only the sender knows the position of each hop in the path.

It is not required to store all 20 * 20 = 400 HMACs. The node in position 0 needs to store an HMAC for every one of the 20 positions. For the node in position 1, the maximum remaining route length is only 19. For that position just 19 HMACs need to be kept, and so on. This makes for a total number of HMACs of 20+19+18+… +1 = 210 HMACs.

The layout of the `hmacs` field is shown below. The actual format is much longer, but for readability the format is described as if the maximum route length would be just three hops.

`hmac_0_2 | hmac_0_1| hmac_0_0| hmac_1_1| hmac_1_0| hmac_2_0`

`hmac_x_y` is the hmac added by node `x` (counted from the node that is currently handling the failure message) assuming that this node is `y` hops away from the erring node.

Each HMAC is computed from the combination of the following elements, concatenated in the specified order.

- The return packet before applying the pseudo-random byte stream.
- The concatenation of the first `y+1` hold times in `htlc_hold_times`. For example, `hmac_0_2` would cover all three hold times.
- The concatenation of `y` downstream hmacs that correspond to downstream node positions relative to `x`. For example, `hmac_0_2` would cover `hmac_1_1` and `hmac_2_0`.

The erring node stores its 20 HMACs at the start of the array and zeroes out the rest. Strictly speaking the erring node would only need to add the single `hmac_0_0` here, because there is no downstream data to cover. However, for verification efficiency at the origin node, we still require all HMACs to be calculated. The redundant HMACs will cover portions of the zero-initialized data.

Finally a new key is generated, using the key type `ammagext`. This key is then used to generate a pseudo-random stream, which is in turn applied to the `attribution_data` field using `XOR`.

Requirements

The *erring node*: - MUST construct the return packet such that its total length is no more than 32768 bytes (32 KiB). - MUST set `pad` such that the `failure_len` plus `pad_len` is at least 256. - SHOULD set `pad` such that the `failure_len` plus `pad_len` is equal to 256. Deviating from this may cause older nodes to be unable to parse the return message. - if `option_attribution_data` is advertised: - if `path_key` is not set in the incoming `update_add_htlc`: - MUST initialize `attribution_data` and include it in `update_fail_htlc`

Intermediate nodes

Transformation of the return packet

Generate the node's `ammag` key, generate the pseudo-random byte streams, and apply the result to obfuscate the return packet. This is then stored as the `reason` field of the `update_htlc_fail` message.

This obfuscation step is identical to the obfuscation steps that the erring node carries out.

Transformation of `attribution_data`

- Shift all existing hold times to the right (4 bytes).
- Shift and prune all existing HMACs.

  At each step backwards, one HMAC for every hop can be pruned. When HMACs for all 20 positions are present, and it turns out that there is another hop upstream, each existing HMAC that now corresponds to position 21 due to the preceding hop becomes obsolete.

  For the simplified three-hop layout above, the shift/prune operation would apply a transformation that results in:

  `– | – | – | hmac_0'_1 | hmac_0'_0 | hmac_1'_0`

  The former `hmac_x'_y` now becomes `hmac_x+1_y`. The left-most HMAC for each hop is discarded.

Update of `attribution_data`

- Put the node's hold time at the start of `htlc_hold_times`. The shift operation above has opened up a slot for that.
- Calculate its own 20 truncated HMACs and put them at the start of `hmacs` in the newly opened slots.
- Generate the node's `ammagext` key, generate the pseudo-random byte stream, and apply the result to obfuscate the `attribution_data` field. This obfuscation step is identical to the obfuscation steps that the erring node carries out.

Requirements

The *intermediate node*: - if the return packet received from downstream is longer than 32768 bytes: - MUST truncate it to its first 32768 bytes before transforming the return packet or updating `attribution_data` - if `option_attribution_data` is advertised: - if `path_key` is not set in the incoming `update_add_htlc`: - if `attribution_data` is received from downstream: - MUST transform `attribution_data` as described above - otherwise: - MUST instantiate an all-zeroes `attribution_data` block - MUST update `attribution_data` as described above - MUST transform the return packet as described above. - MUST return-forward the `update_htlc_fail` message

Origin node

The origin node is able to detect that it's the intended final recipient of the return message, because of course, it was the originator of the corresponding forward packet. When an origin node receives an `update_htlc_fail` message matching a transfer it initiated (i.e. it cannot return-forward the error any further) it generates the `ammag`, `ammagext` and `um` keys for each hop in the route.

It then iteratively decrypts the message, using each hop's `ammag` and `ammagext` keys. At each hop, the following steps are carried out:

For origin nodes supporting `option_attribution_data`:

- Verify the HMAC in `attribution_data` that corresponds to the hop's position in the path using the hop's `um` key. If the HMAC is invalid, processing of the message can stop and the node should penalize this hop for future path selection. This is what makes the failure 'attributable'.

  Because HMACs cover all data including HMACs added by downstream nodes, it is not possible for a malicious node to tamper with the message without revealing themselves. Blame is still only assignable to a pair of nodes though, because it is impossible to know whether sender or receiver modified the message. This is true for other failure cases in Lightning too.

  When not every path node supports `option_attribution_data`, the origin node will still have attribution data up to the first node downstream without support.

- Record the reported htlc hold time for this hop.

  The origin node can use this information to score nodes on latency. When a zero hold time is reported, the origin node should distribute any potential latency penalty across multiple nodes. This encourages path nodes to provide timing data to avoid being held responsible for the high latency of other nodes.

For all nodes:

- Compute the HMAC for the return packet, using the hop's `um` key.
- When the computed HMAC matches `hmac` in the return packet, the origin node will know that the current hop is the sender of the failure. They can then parse `failuremsg`.

The association between the forward and return packets is handled outside of this onion routing protocol, e.g. via association with an HTLC in a payment channel.

Requirements

The *origin node*: - once the return message has been decrypted: - SHOULD store a copy of the message. - SHOULD continue decrypting, until the loop has been repeated 27 times (maximum route length of tlv payload type). - SHOULD use constant `ammag` and `um` keys to obfuscate the route length. - When the failure source cannot be identified from the return packet AND `attribution_data` is present: - SHOULD use `attribution_data` to identify the failure source

Rationale

The requirements for the *origin node* should help hide the payment sender. By continuing decrypting 27 times (dummy decryption cycles after the error is found) the erroring node cannot learn its relative position in the route by performing a timing analysis if the sender were to retry the same route multiple times.

Failure Messages

The failure message encapsulated in `failuremsg` has an identical format as a normal message: a 2-byte type `failure_code` followed by data applicable to that type. The message data is followed by an optional TLV stream.

Below is a list of the currently supported `failure_code` values, followed by their use case requirements.

Notice that the `failure_codes` are not of the same type as other message types, defined in other BOLTs, as they are not sent directly on the transport layer but are instead wrapped inside return packets. The numeric values for the `failure_code` may therefore reuse values, that are also assigned to other message types, without any danger of causing collisions.

The top byte of `failure_code` can be read as a set of flags: * 0x8000 (BADONION): unparsable onion encrypted by sending peer * 0x4000 (PERM): permanent failure (otherwise transient) * 0x2000 (NODE): node failure (otherwise channel) * 0x1000 (UPDATE): channel forwarding parameter was violated

The following `failure_codes` are defined:

1. type: NODE | 2 (`temporary_node_failure`)

General temporary failure of the processing node.

1. type: PERM | NODE | 2 (`permanent_node_failure`)

General permanent failure of the processing node.

1. type: PERM | NODE | 3 (`required_node_feature_missing`)

The processing node has a required feature which was not in this onion.

1. type: BADONION | PERM | 4 (`invalid_onion_version`)
2. data:
    - [`sha256:sha256_of_onion`]

The `version` byte was not understood by the processing node.

1. type: BADONION | PERM | 5 (`invalid_onion_hmac`)
2. data:
    - [`sha256:sha256_of_onion`]

The HMAC of the onion was incorrect when it reached the processing node.

1. type: BADONION | PERM | 6 (`invalid_onion_key`)
2. data:
    - [`sha256:sha256_of_onion`]

The ephemeral key was unparsable by the processing node.

1. type: UPDATE | 7 (`temporary_channel_failure`)
2. data:
    - [`u16:len`]
    - [`len*byte:channel_update`]

The channel from the processing node was unable to handle this HTLC, but may be able to handle it, or others, later.

1. type: PERM | 8 (`permanent_channel_failure`)

The channel from the processing node is unable to handle any HTLCs.

1. type: PERM | 9 (`required_channel_feature_missing`)

The channel from the processing node requires features not present in the onion.

1. type: PERM | 10 (`unknown_next_peer`)

The onion specified a `short_channel_id` which doesn't match any leading from the processing node.

1. type: UPDATE | 11 (`amount_below_minimum`)
2. data:
    - [`u64:htlc_msat`]
    - [`u16:len`]
    - [`len*byte:channel_update`]

The HTLC amount was below the `htlc_minimum_msat` of the channel from the processing node.

1. type: UPDATE | 12 (`fee_insufficient`)
2. data:
    - [`u64:htlc_msat`]
    - [`u16:len`]
    - [`len*byte:channel_update`]

The fee amount was below that required by the channel from the processing node.

1. type: UPDATE | 13 (`incorrect_cltv_expiry`)

2. data:
    - [u32:cltv_expiry]
    - [u16:len]
    - [len*byte:channel_update]

The cltv_expiry does not comply with the cltv_expiry_delta required by the channel from the processing node: it does not satisfy the following requirement:

```
cltv_expiry - cltv_expiry_delta >= outgoing_cltv_value
```

1. type: UPDATE|14 (expiry_too_soon)
2. data:
    - [u16:len]
    - [len*byte:channel_update]

The CLTV expiry is too close to the current block height for safe handling by the processing node.

1. type: PERM|15 (incorrect_or_unknown_payment_details)
2. data:
    - [u64:htlc_msat]
    - [u32:height]

The payment_hash is unknown to the final node, the payment_secret doesn't match the payment_hash, the amount for that payment_hash is too low, the CLTV expiry of the htlc is too close to the current block height for safe handling or payment_metadata isn't present while it should be.

The htlc_msat parameter is superfluous, but left in for backwards compatibility. The value of htlc_msat is required to be at least the value specified in the final hop onion payload. It therefore does not have any substantial informative value to the sender (though may indicate the penultimate node took a lower fee than expected). A penultimate hop sending an amount or an expiry that is too low for the htlc is handled through final_incorrect_cltv_expiry and final_incorrect_htlc_amount.

The height parameter is set by the final node to the best known block height at the time of receiving the htlc. This can be used by the sender to distinguish between sending a payment with the wrong final CLTV expiry and an intermediate hop delaying the payment so that the receiver's invoice CLTV delta requirement is no longer met.

Note: Originally PERM|16 (incorrect_payment_amount) and 17 (final_expiry_too_soon) were used to differentiate incorrect htlc parameters from unknown payment hash. Sadly, sending this response allows for probing attacks whereby a node which receives an HTLC for forwarding can check guesses as to its final destination by sending payments with the same hash but much lower values or expiry heights to potential destinations and check the response. Care must be taken by implementations to differentiate the previously non-permanent case for final_expiry_too_soon (17) from the other, permanent failures now represented by incorrect_or_unknown_payment_details (PERM|15).

1. type: 18 (final_incorrect_cltv_expiry)
2. data:
    - [u32:cltv_expiry]

The CLTV expiry in the HTLC is less than the value in the onion.

1. type: 19 (`final_incorrect_htlc_amount`)
2. data:
    - [`u64:incoming_htlc_amt`]

The amount in the HTLC is less than the value in the onion.

1. type: UPDATE | 20 (`channel_disabled`)
2. data:
    - [`u16:disabled_flags`]
    - [`u16:len`]
    - [`len*byte:channel_update`]

The channel from the processing node has been disabled. No flags for `disabled_flags` are currently defined, thus it is currently always two zero bytes.

1. type: 21 (`expiry_too_far`)

The CLTV expiry in the HTLC is too far in the future.

1. type: PERM | 22 (`invalid_onion_payload`)
2. data:
    - [`bigsize:type`]
    - [`u16:offset`]

The decrypted onion per-hop payload was not understood by the processing node or is incomplete. If the failure can be narrowed down to a specific tlv type in the payload, the erring node may include that `type` and its byte `offset` in the decrypted byte stream.

1. type: 23 (`mpp_timeout`)

The complete amount of the multi-part payment was not received within a reasonable time.

1. type: BADONION | PERM | 24 (`invalid_onion_blinding`)
2. data:
    - [`sha256:sha256_of_onion`]

An error occurred within the blinded path.

Requirements

An *erring node*: - if `path_key` is set in the incoming `update_add_htlc`: - MUST return an `invalid_onion_blinding` error. - if `current_path_key` is set in the onion payload and it is not the final node: - MUST return an `invalid_onion_blinding` error. - otherwise: - MUST select one of the above error codes when creating an error message. - MUST include the appropriate data for that particular error type. - if there is more than one error: - SHOULD select the first error it encounters from the list above.

An *erring node* MAY: - if the per-hop payload in the onion is invalid (e.g. it is not a valid tlv stream) or is missing required information (e.g. the amount was not specified): - return an `invalid_onion_payload` error. - if an otherwise unspecified transient error occurs for the entire node: - return a `temporary_node_failure` error. -

if an otherwise unspecified permanent error occurs for the entire node: - return a `permanent_node_failure` error. - if a node has requirements advertised in its `node_announcement` `features`, which were NOT included in the onion: - return a `required_node_feature_missing` error.

A *forwarding node* MUST: - if `path_key` is set in the incoming `update_add_htlc`: - return an `invalid_onion_blinding` error. - if `current_path_key` is set in the onion payload: - return an `invalid_onion_blinding` error. - otherwise: - select one of the above error codes when creating an error message.

A *forwarding node* MAY, but a *final node* MUST NOT: - if during forwarding to its receiving peer, an otherwise unspecified, transient error occurs in the outgoing channel (e.g. channel capacity reached, too many in-flight HTLCs, etc.): - return a `temporary_channel_failure` error. - if an otherwise unspecified, permanent error occurs during forwarding to its receiving peer (e.g. channel recently closed): - return a `permanent_channel_failure` error. - if the outgoing channel has requirements advertised in its `channel_announcement`'s `features`, which were NOT included in the onion: - return a `required_channel_feature_missing` error. - if the receiving peer specified by the onion is NOT known: - return an `unknown_next_peer` error. - if the HTLC amount is less than the currently specified minimum amount: - report the amount of the outgoing HTLC and the current channel setting for the outgoing channel. - return an `amount_below_minimum` error. - if the HTLC does NOT pay a sufficient fee: - report the amount of the incoming HTLC and the current channel setting for the outgoing channel. - return a `fee_insufficient` error. - if the incoming `cltv_expiry` minus the `outgoing_cltv_value` is below the `cltv_expiry_delta` for the outgoing channel: - report the `cltv_expiry` of the outgoing HTLC and the current channel setting for the outgoing channel. - return an `incorrect_cltv_expiry` error. - if the `cltv_expiry` is unreasonably near the present: - report the current channel setting for the outgoing channel. - return an `expiry_too_soon` error. - if the `cltv_expiry` is more than `max_htlc_cltv` in the future: - return an `expiry_too_far` error. - if the channel is disabled: - report the current channel setting for the outgoing channel. - return a `channel_disabled` error.

A *forwarding node* and a *final node* MAY: - if the onion `version` byte is unknown: - return an `invalid_onion_version` error. - if the onion HMAC is incorrect: - return an `invalid_onion_hmac` error. - if the ephemeral key in the onion is unparsable: - return an `invalid_onion_key` error.

A *forwarding node* MUST NOT, but a *final node* MUST: - if the payment hash has already been paid: - MAY treat the payment hash as unknown. - MAY succeed in accepting the HTLC. - if the `payment_secret` doesn't match the expected value for that `payment_hash`, or the `payment_secret` is required and is not present: - MUST fail the HTLC. - MUST return an `incorrect_or_unknown_payment_details` error. - if the amount paid is less than the amount expected: - MUST fail the HTLC. - MUST return an `incorrect_or_unknown_payment_details` error. - if the payment hash is unknown: - MUST fail the HTLC. - MUST return an `incorrect_or_unknown_payment_details` error. - if the amount paid is more than twice the amount expected: - SHOULD fail the HTLC. - SHOULD return an `incorrect_or_unknown_payment_details` error. - Note: this allows the origin node to reduce information leakage by altering the amount while not allowing for accidental gross overpayment. - if the `cltv_expiry` value is unreasonably near the present: - MUST fail the HTLC. - MUST return an `incorrect_or_unknown_payment_details` error. - if the `cltv_expiry` from the final node's HTLC is below `outgoing_cltv_value`: - MUST return `final_incorrect_cltv_expiry` error. - if `amount_msat` from the final node's HTLC is below `amt_to_forward`: - MUST return a `final_incorrect_htlc_amount` error. - if it returns a `channel_update`: - MUST set `short_channel_id` to the `short_channel_id` used by the incoming onion.

Rationale

In the case of multiple short_channel_id aliases, the `channel_update` `short_channel_id` should refer to the one the original sender is expecting, to both avoid confusion and to avoid leaking information about other aliases (or the real location of the channel UTXO).

The `channel_update` field used to be mandatory in messages whose `failure_code` includes the `UPDATE` flag. However, because nodes applying an update contained in the onion to their gossip data is a massive fingerprinting vulnerability, the `channel_update` field is no longer mandatory and nodes are expected to transition away from including it. Nodes which do not provide a `channel_update` are expected to set the `channel_update` `len` field to zero.

Some nodes may still use the `channel_update` for retries of the same payment, however.

Receiving Failure Codes

Requirements

The *origin node*: - MUST ignore any extra bytes in `failuremsg`. - if the *final node* is returning the error: - if the PERM bit is set: - SHOULD fail the payment. - otherwise: - if the error code is understood and valid: - MAY retry the payment. In particular, `final_expiry_too_soon` can occur if the block height has changed since sending, and in this case `temporary_node_failure` could resolve within a few seconds. - otherwise, an *intermediate hop* is returning the error: - if the NODE bit is set: - SHOULD remove all channels connected with the erring node from consideration. - if the PERM bit is NOT set: - SHOULD restore the channels as it receives new `channel_update`s from its peers. - otherwise: - if UPDATE is set, AND the `channel_update` is valid and more recent than the `channel_update` used to send the payment: - MAY consider the `channel_update` when calculating routes to retry the payment which failed - MUST NOT expose the `channel_update` to third-parties in any other context, including applying the `channel_update` to the local network graph, send the `channel_update` to peers as gossip, etc. - SHOULD then retry routing and sending the payment. - MAY use the data specified in the various failure types for debugging purposes.

Successful Payments

The `update_fulfill_htlc` message contains an optional `attribution_data` field, identical to the one used in the failure case described above. With attribution data, the sender can obtain hold times as reported and committed to by each hop along the payment path.

In addition, the final node can return data to the origin node in an optional `fulfillment_payload` field. It is the success counterpart of the failure message that an erring node returns in the `reason` field of `update_fail_htlc`. The final node builds the plaintext as a `fulfillment_payload_tlvs` TLV stream:

1. `tlv_stream`: `fulfillment_payload_tlvs`
2. types:
    1. type: 1 (`padding`)
    2. data:
        - `[...*byte:padding]`

`padding` conceals the length of the conveyed data and is ignored by the origin. Apart from `padding`, no record types are defined yet.

The final node generates a key using key type `fulfillment` and its shared secret with the origin node, then encrypts this plaintext with ChaCha20-Poly1305 using that key and an all-zero nonce, as for `encrypted_recipient_data` in route blinding. The ciphertext and its 16-byte tag are the `fulfillment_payload`. Each intermediate node then obfuscates the `fulfillment_payload` with its `ammag` key, exactly as it obfuscates a failure return packet; the final node does not, as the ciphertext is already confidential and forms the innermost layer. The `attribution_data` HMACs added by intermediate nodes, which already cover the hold times, now additionally cover the `fulfillment_payload`. For each intermediate HMAC, the covered fields are, in order: the `fulfillment_payload` as received from the downstream node, before the current node applies its own `ammag` obfuscation; the hold times; and the downstream HMACs. The final node's `attribution_data` HMACs do not cover the `fulfillment_payload`. With no payload present, there is nothing extra for intermediate nodes to cover. The final node still initializes and updates `attribution_data`, and the origin still verifies the final node's `attribution_data` HMACs for its reported hold time.

A successful payment is relayed back with `update_fulfill_htlc`, so a `fulfillment_payload` propagates back through every hop, even a blinded path: a blinded final node can originate it, and blinded hops obfuscate and relay it. The origin can still recover it, because it shares a secret with every hop, blinded ones included, computed from each hop's public key (the blinded public key for a blinded hop) when it built the onion. Returning a `fulfillment_payload` is therefore not gated by the `path_key` condition that applies to `attribution_data`.

The origin knows the number of hops in the path, so the final node is its last hop: the origin peels each intermediate hop's `ammag` obfuscation, including obfuscation for blinded hops whose blinded public keys it used when constructing the route, and decrypts the `fulfillment_payload` with the last hop's `fulfillment` key, one operation per hop. The origin verifies the final node's `attribution_data` HMACs without a `fulfillment_payload` input. For each intermediate hop, the origin removes that hop's `ammag` obfuscation from the `fulfillment_payload` and verifies the hop's `attribution_data` HMACs over the resulting `fulfillment_payload` bytes. A recipient that conceals its position with dummy hops in a blinded path must therefore originate the `fulfillment_payload` as though it were the last hop, applying the obfuscation for the concealed hops itself, so that this decoding succeeds without revealing its position.

The two integrity checks cover different extents. The Poly1305 tag is verified end-to-end and detects tampering by any hop, blinded or not. The `attribution_data` HMACs are contributed only by hops with `path_key` not set, so they provide per-hop attribution where available, as in the failure case. This split is intentional: `attribution_data` is per-hop data for scoring path nodes, which the origin cannot do for blinded hops it cannot identify and whose hold times would aid de-anonymization, whereas a `fulfillment_payload` is end-to-end data that reveals nothing about the intermediate hops.

Like return packets, `fulfillment_payload` is limited to 32768 bytes (32 KiB) to leave room for `attribution_data` and future extensions. Because `fulfillment_payload` was introduced with this limit, receiving a larger payload is a protocol violation as specified in BOLT #2.

Requirements

`attribution_data` is initialized, transformed, updated and verified exactly as in the failure case.

The *final node*: - if `option_attribution_data` is advertised: - MAY include a `fulfillment_payload`, even when reached through a blinded path - if it includes a `fulfillment_payload`: - MUST ensure the `fulfillment_payload` is no more than 32768 bytes (32 KiB) - MUST set `fulfillment_payload_tlvs` to a

serialized TLV stream - MUST pad with `padding` such that the serialized `fulfillment_payload_tlvs` stream is at least 256 bytes and a multiple of 256 bytes, excluding the 16-byte Poly1305 tag. The size calculation includes the type and length bytes of the `padding` record; this can require padding to the next multiple of 256 bytes - SHOULD pad such that the serialized `fulfillment_payload_tlvs` stream is exactly 256 bytes if possible - MUST encrypt the plaintext with ChaCha20-Poly1305 as above - MUST NOT include the `fulfillment_payload` in its `attribution_data` HMACs - MUST still initialize and update `attribution_data` for its reported hold time

An *intermediate node*: - if it receives a `fulfillment_payload` from the downstream node: - MUST obfuscate it with its `ammag` key and include it in the outgoing `update_fulfill_htlc`, as it transforms a return packet - MUST include the `fulfillment_payload` as received from the downstream node in its `attribution_data` HMACs

The *origin node*: - MUST recover a `fulfillment_payload` by peeling each intermediate hop's `ammag` obfuscation and decrypting it with the last hop's `fulfillment` key - MUST verify the final node's `attribution_data` HMACs as in the failure case, except that the HMAC input excludes the `fulfillment_payload`, and process its reported hold time - MUST remove an intermediate hop's `ammag` obfuscation from the `fulfillment_payload` before verifying that hop's `attribution_data` HMACs over the resulting `fulfillment_payload` bytes - MUST ignore a `fulfillment_payload` whose Poly1305 tag is invalid, or whose decrypted `fulfillment_payload_tlvs` stream has malformed lengths, duplicate or unordered types, or unknown even types - MUST ignore `padding` records in a valid `fulfillment_payload_tlvs` stream

Onion Messages

Onion messages allow peers to use existing connections to query for invoices (see BOLT 12). Like gossip messages, they are not associated with a particular local channel. Like HTLCs, they use onion messages protocol for end-to-end encryption.

Onion messages use the same form as HTLC `onion_packet`, with a slightly more flexible format: instead of 1300 byte payloads, the payload length is implied by the total length (minus 66 bytes for the header and trailing bytes). The `onionmsg_payloads` themselves are the same as the `hop_payloads` format, except there is no "legacy" length: a 0 `length` would mean an empty `onionmsg_payload`.

Onion messages are unreliable: in particular, they are designed to be cheap to process and require no storage to forward. As a result, there is no error returned from intermediary nodes.

For consistency, all onion messages use Route Blinding.

The `onion_message` Message

1. type: 513 (`onion_message`) (`option_onion_messages`)
2. data:
    - [`point:path_key`]
    - [`u16:len`]
    - [`len*byte:onion_message_packet`]
3. type: `onion_message_packet`
4. data:
    - [`byte:version`]
    - [`point:public_key`]

        - [...*byte:onionmsg_payloads]
        - [32*byte:hmac]
    5. type: onionmsg_payloads
    6. data:
        - [bigsize:length]
        - [length*u8:onionmsg_tlv]
        - [32*byte:hmac]
        - ...
        - filler

The onionmsg_tlv itself is a TLV: an intermediate node expects an encrypted_recipient_data which it can decrypt into an encrypted_data_tlv using the path_key which it is handed along with the onion message.

Field numbers 64 and above are reserved for payloads for the final hop.

1. tlv_stream: onionmsg_tlv
2. types:
    1. type: 2 (reply_path)
    2. data:
        - [blinded_path:path]
    3. type: 4 (encrypted_recipient_data)
    4. data:
        - [...*byte:encrypted_recipient_data]
    5. type: 64 (invoice_request)
    6. data:
        - [tlv_invoice_request:invreq]
    7. type: 66 (invoice)
    8. data:
        - [tlv_invoice:inv]
    9. type: 68 (invoice_error)
    10. data:
        - [tlv_invoice_error:inverr]

Requirements

The creator of encrypted_recipient_data (usually, the recipient of the onion):

- MUST create the encrypted_recipient_data from the encrypted_data_tlv as required in Route Blinding.
- MUST NOT include payment_relay or payment_constraints in any encrypted_data_tlv
- MUST include either next_node_id or short_channel_id in the encrypted_data_tlv for each non-final node.

The writer:

- MUST set the onion_message_packet version to 0.
- MUST construct the onion_message_packet onionmsg_payloads as detailed above using Sphinx.
- MUST NOT use any associated_data in the Sphinx construction.
- SHOULD set onion_message_packet len to 1366 or 32834.

- SHOULD retry via a different path if it expects a response and doesn't receive one after a reasonable period.
- For the non-final nodes' `onionmsg_tlv`:
    - MUST NOT set fields other than `encrypted_recipient_data`.
- For the final node's `onionmsg_tlv`:
    - if the final node is permitted to reply:
        - MUST set `reply_path` `path_key` to the initial path key for the `first_node_id`
        - MUST set `reply_path` `first_node_id` to the unblinded node id of the first node in the reply path.
        - For every `reply_path` `path`:
            - MUST set `blinded_node_id` to the blinded node id to encrypt the onion hop for.
            - MUST set `encrypted_recipient_data` to a valid encrypted `encrypted_data_tlv` stream which meets the requirements of the `onionmsg_tlv` when used by the recipient.
            - MAY use `path_id` to contain a secret so it can recognize use of this `reply_path`.
    - otherwise:
        - MUST NOT set `reply_path`.

The reader:

- if it advertises `option_onion_messages_only_channels`:
    - MUST NOT accept onion messages from peers without an established channel.
- otherwise:
    - SHOULD accept onion messages from peers without an established channel.
- MAY rate-limit messages by dropping them.
- MUST decrypt `onion_message_packet` using an empty `associated_data`, and `path_key`, as described in Onion Decryption to extract an `onionmsg_tlv`.
- If decryption fails, the result is not a valid `onionmsg_tlv`, or it contains unknown even types:
    - MUST ignore the message.
- if `encrypted_data_tlv` contains `allowed_features`:
    - MUST ignore the message if:
        - `encrypted_data_tlv.allowed_features.features` contains an unknown feature bit (even if it is odd).
        - the message uses a feature not included in `encrypted_data_tlv.allowed_features.features`.
- if it is not the final node according to the onion encryption:
    - if the `onionmsg_tlv` contains other tlv fields than `encrypted_recipient_data`:
        - MUST ignore the message.
    - if the `encrypted_data_tlv` contains `path_id`:
        - MUST ignore the message.
    - otherwise:
        - if `next_node_id` is present:
            - the *next peer* is the peer with that node id.
        - otherwise, if `short_channel_id` is present and corresponds to an announced short_channel_id or a local alias for a channel:
            - the *next peer* is the peer at the other end of that channel.

        - otherwise:
            - MUST ignore the message.
        - SHOULD forward the message using `onion_message` to the *next peer*.
        - if it forwards the message:
            - MUST set `path_key` in the forwarded `onion_message` to the next `path_key` as calculated in Route Blinding.
- otherwise (it is the final node):
    - if `path_id` is set and corresponds to a path the reader has previously published in a `reply_path`:
        - if the onion message is not a reply to that previous onion:
            - MUST ignore the onion message
    - otherwise (unknown or unset `path_id`):
        - if the onion message is a reply to an onion message which contained a `path_id`:
            - MUST respond (or not respond) exactly as if it did not send the initial onion message.
    - if the `onionmsg_tlv` contains more than one payload field:
        - MUST ignore the message.
    - if it wants to send a reply:
        - MUST create an onion message using `reply_path`.
        - MUST send the reply via `onion_message` to the node indicated by the `first_node_id`, using `reply_path` `path_key` to send along `reply_path` `path`.

Rationale

Accepting onion messages from channelless peers is the baseline: a node that advertises `option_onion_messages` is expected to accept onion messages from any connected peer. This is a SHOULD rather than a MUST because acceptance is always best effort: a node may still drop messages due to rate-limiting.

A node that wants to restrict acceptance to channel peers (e.g. to limit the DoS surface of free bandwidth/CPU for any node that can connect) advertises `option_onion_messages_only_channels` as an explicit opt-out. A restrictive node should not silently drop these messages while advertising only the baseline: it has to advertise the bit, which makes the bit a reliable signal. A sender can therefore detect it and simply not attempt a relay it knows will fail, rather than sending a message that is silently dropped.

Care must be taken that replies are only accepted using the exact reply_path given, otherwise probing is possible. That means checking both ways: non-replies don't use the reply path, and replies always use the reply path.

The requirement to discard messages with `onionmsg_tlv` fields which are not strictly required ensures consistency between current and future implementations. Even odd fields can be a problem since they are parsed (and thus may be rejected!) by nodes which understand them, and ignored by those which don't.

All onion messages are blinded, even though this overhead is not always necessary (33 bytes here, the 16-byte MAC for each encrypted_data_tlv in the onion). This blinding allows nodes to use a path provided by others without knowing its contents. Using it universally simplifies implementations a little, and makes it more difficult to distinguish onion messages.

`len` allows larger messages to be sent than the standard 1300 bytes allowed for an HTLC onion, but this should be used sparingly as it reduces the anonymity set, hence the recommendation that it either looks like an HTLC onion, or if larger, be a fixed size.

Onion messages don't explicitly require a channel, but for spam-reduction a node may choose to ratelimit such peers, especially messages it is asked to forward.

`max_htlc_cltv` Selection

This `max_htlc_cltv` value is defined as 2016 blocks, based on historical value deployed by Lightning implementations.

Test Vector

Returning Errors

The test vectors use the following parameters:

```
pubkey[0] = 0x02eec7245d6b7d2ccb30380bfbe2a3648cd7a942653f5aa340edcea1f283686619
htlc_hold_time[0] = 1

pubkey[1] = 0x0324653eac434488002cc06bbfb7f10fe18991e35f9fe4302dbea6d2353dc0ab1c
htlc_hold_time[1] = 2

pubkey[2] = 0x027f31ebc5462c1fdce1b737ecff52d37d75dea43ce11c74d25aa297165faa2007
htlc_hold_time[2] = 3

pubkey[3] = 0x032c0b7cf95324a07d05398b240174dc0c2be444d96b159aa6c7f7b1e668680991
htlc_hold_time[3] = 4

pubkey[4] = 0x02edabbd16b41c8371b92ef2f04c1185b4f03b6dcd52ba9b78d9d7c89c8f221145
htlc_hold_time[4] = 5

nhops = 5
sessionkey = 0x4141414141414141414141414141414141414141414141414141414141414141

failure_source  = node 4
failure_message = `incorrect_or_unknown_payment_details`
  htlc_msat = 100
  height    = 800000
  tlv data
    type  = 34001
    value = [128, 128, ..., 128] (300 bytes)
```

The following is an in-depth trace of an example of error message creation:

```
# creating error message
encoded_failure_message =
400f000000000000064000c3500fd84d1fd012c8080808080808080808080808080808080808080808080808
8080808080808080808080808080808080808080808080808080808080808080808080808080808080808080
8080808080808080808080808080808080808080808080808080808080808080808080808080808080808080
8080808080808080808080808080808080808080808080808080808080808080808080808080808080808080
8080808080808080808080808080808080808080808080808080808080808080808080808080808080808080
```

80808080808080808080808080808080808080808080808080808080808080808080808080808080808080808080808080808080808080
80808080808080808080808080808080808080808080808080808080808080808080808080808080808080808080808002c0000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000

0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000

forwarding error packet
shared_secret = b5756b9b542727dbafc6765a49488b023a725d631af688fc031217e90770c328
ammag_key = 2f36bb8822e1f0d04c27b7d8bb7d7dd586e032a3218b8d414afbba6f169a4d68
stream =
e9c975b07c9a374ba64fd9be3aae955e917d34d1fa33f2e90f53bbf4394713c6a8c9b16ab5f12fd45edd73c1b0c8b330
02df376801ff58aaa94000bf8a86f92620f343baef38a580102395ae3abf9128d1047a0736ff9b83d456740ebbb4aeb3
aa9737f18fb4afb4aa074fb26c4d702f42968888550a3bded8c05247e045b866baef0499f079fdaeef6538f31d44deaf
ffdfd3afa2fb4ca9082b8f1c465371a9894dd8c243fb4847e004f5256b3e90e2edde4c9fb3082ddfe4d1e734cacd96ef
0706bf63c9984e22dc98851bcccd1c3494351feb458c9c6af41c0044bea3c47552b1d992ae542b17a2d0bba1a096c78d
169034ecb55b6e3a7263c26017f033031228833c1daefc0dedb8cf7c3e37c9c37ebfe42f3225c326e8bcfd338804c145
b16e34e4f5984bc119af09d471a61f39e9e389c4120cadabc5d9b7b1355a8ccef050ca8ad72f642fc26919927b347808
bade4b1c321b08bc363f20745ba2f97f0ced2996a232f55ba28fe7dfa70a9ab0433a085388f25cce8d53de6a2fbd7546
377d6ede9027ad173ba1f95767461a3689ef405ab608a21086165c64b02c1782b04a6dba2361a7784603069124e12f2f
6dcb1ec7612a4fbf94c0e14631a2bef6190c3d5f35e0c4b32aa85201f449d830fd8f782ec758b0910428e3ec3ca1dba3
b6c7d89f69e1ee1b9df3dfbbf6d361e1463886b38d52e8f43b73a3bd48c6f36f5897f514b93364a31d49d1d506340b13
15883d425cb36f4ea553430d538fd6f3596d4afc518db2f317dd051abc0d4bfb0a7870c3db70f19fe78d6604bbf088fc
b4613f54e67b038277fedcd9680eb97bdffc3be1ab2cbcbafd625b8a7ac34d8c190f98d3064ecd3b95b8895157c6a37f
31ef4de094b2cb9dbf8ff1f419ba0ecacb1bb13df0253b826bec2ccca1e745dd3b3e7cc6277ce284d649e7b828572773
5ff4ef6cca6c18e2714f4e2a1ac67b25213d3bb49763b3b94e7ebf72507b71fb2fe0329666477ee7cb7ebd6b88ad5add
8b217188b1ca0fa13de1ec09cc674346875105be6e0e0d6c8928eb0df23c39a639e04e4aedf535c4e093f08b2c905a14
f25c0c0fe47a5a1535ab9eae0d9d67bdd79de13a08d59ee05385c7ea4af1ad3248e61dd22f8990e9e99897d653dd7b1b
1433a6d464ea9f74e377f2d8ce99ba7dbc753297644234d25ecb5bd528e2e2082824681299ac30c05354baaa9c3967d8
6d7c07736f87fc0f63e5036d47235d7ae12178ced3ae36ee5919c093a02579e4fc9edad2c446c656c790704bfc8e2c49
1a42500aa1d75c8d4921ce29b753f883e17c79b09ea324f1f32ddf1f3284cd70e847b09d90f6718c42e5c94484cc9cbb
0df659d255630a3f5a27e7d5dd14fa6b974d1719aa98f01a20fb4b7b1c77b42d57fab3c724339d459ee4a1c6b5d3bd4e
08624c786a257872acc9ad3ff62222f2265a658d9f2a007229a5293b67ec91c84c4b4407c228434bad8a815ca9b256c7
76bd2c9f
error packet for node 4:
146e94a9086dbbed6a0ab6932d00c118a7195dbf69b7d7a12b0e6956fc54b5e0a989f165b5f12fd45edd73a5b0c48630
ff5be69500d3d82a29c0803f0a0679a6a073c33a6fb8250090a3152eba3f11a85184fa87b67f1b0354d6f48e3b342e33
2a17b7710f342f342a87cf32eccdf0afc2160808d58abb5e5840d2c760c538e63a6f841970f97d2e6fe5b8739dc45e2f
7f5f532f227bcc2988ab0f9cc6d3f12909cd5842c37bc8c7608475a5ebbe10626d5ecc1f3388ad5f645167b44a4d166f
87863fe34918cea25c18059b4c4d9cb414b59f6bc50c1cea749c80c43e2344f5d23159122ed4ab9722503b212016470d
9610b46c35dbeebaf2e342e09770b38392a803bc9d2e7c8d6d384ffcbeb74943fe3f64afb2a543a6683c7db3088441c5
31eeb4647518cb41992f8954f1269fb969630944928c2d2b45593731b5da0c4e70d04a0a57afe4af42e99912fbb4f888
3a5ecb9cb29b883cb6bfa0f4db2279ff8c6d2b56a232f55ba28fe7dfa70a9ab0433a085388f25cce8d53de6a2fbd7546
377d6ede9027ad173ba1f95767461a3689ef405ab608a21086165c64b02c1782b04a6dba2361a7784603069124e12f2f
6dcb1ec7612a4fbf94c0e14631a2bef6190c3d5f35e0c4b32aa85201f449d830fd8f782ec758b0910428e3ec3ca1dba3
b6c7d89f69e1ee1b9df3dfbbf6d361e1463886b38d52e8f43b73a3bd48c6f36f5897f514b93364a31d49d1d506340b13
15883d425cb36f4ea553430d538fd6f3596d4afc518db2f317dd051abc0d4bfb0a7870c3db70f19fe78d6604bbf088fc
b4613f54e67b038277fedcd9680eb97bdffc3be1ab2cbcbafd625b8a7ac34d8c190f98d3064ecd3b95b8895157c6a37f
31ef4de094b2cb9dbf8ff1f419ba0ecacb1bb13df0253b826bec2ccca1e745dd3b3e7cc6277ce284d649e7b828572773
5ff4ef6cca6c18e2714f4e2a1ac67b25213d3bb49763b3b94e7ebf72507b71fb2fe0329666477ee7cb7ebd6b88ad5add

8b217188b1ca0fa13de1ec09cc674346875105be6e0e0d6c8928eb0df23c39a639e04e4aedf535c4e093f08b2c905a14
f25c0c0fe47a5a1535ab9eae0d9d67bdd79de13a08d59ee05385c7ea4af1ad3248e61dd22f8990e9e99897d653dd7b1b
1433a6d464ea9f74e377f2d8ce99ba7dbc753297644234d25ecb5bd528e2e2082824681299ac30c05354baaa9c3967d8
6d7c07736f87fc0f63e5036d47235d7ae12178ced3ae36ee5919c093a02579e4fc9edad2c446c656c790704bfc8e2c49
1a42500aa1d75c8d4921ce29b753f883e17c79b09ea324f1f32ddf1f3284cd70e847b09d90f6718c42e5c94484cc9cbb
0df659d255630a3f5a27e7d5dd14fa6b974d1719aa98f01a20fb4b7b1c77b42d57fab3c724339d459ee4a1c6b5d3bd4e
08624c786a257872acc9ad3ff62222f2265a658d9f2a007229a5293b67ec91c84c4b4407c228434bad8a815ca9b256c7
76bd2c9f
attribution data for node 4:
d77d0711b5f71d1d1be56bd88b3bb7ebc1792bb739ea7ebc1bc3b031b8bc2df3a50e25aeb99f47d7f7ab39e24187d3f4
df9c4333463b053832ee9ac07274a5261b8b2a01fc09ce9ea7cd04d7b585dfb8cf5958e3f3f2a4365d1ec0df1d83c6a6
221b5b7d1ff30156a2289a1d3ee559e7c7256bda444bb8e046f860e00b3a59a85e1e1a43de215fd5e6bf646a5deab97b
1912c934e31b1cfd344764d6ca7e14ea7b3f2a951aba907c964c0f5d19a44e6d1d7279637321fa598adde927b3087d23
8f8b426ecde500d318617cdb7a56e6ce3520fc95be41a549973764e4dc483853ecc313947709f1b5199cb077d46e701f
a633e11d3e13b03e9212c115ca6fa004b2f3dd912814693b705a561a06da54cdf603677a3abecdc22c7358c2de3cef77
1b366a568150aeecc86ad1990bb0f4e2865933b03ea0df87901bff467908273dc6cea31cbab0e2b8d398d10b001058c2
59ed221b7b55762f4c7e49c8c11a45a107b7a2c605c26dc5b0b10d719b1c844670102b2b6a36c43fe4753a78a483fc39
166ae28420f112d50c10ee64ca69569a2f690712905236b7c2cb7ac8954f02922d2d918c56d42649261593c47b14b324
a65038c3c5be8d3c403ce0c8f19299b1664bf077d7cf1636c4fb9685a8e58b7029fd0939fa07925a60bed339b23f9732
93598f595e75c8f9d455d7cebe4b5e23357c8bd47d66d6628b39427e37e0aecbabf46c11be6771f7136e108a143ae9ba
fba0fc47a51b6c7deef4cba54bae906398ee3162a41f2191ca386b628bde7e1dd63d1611aa01a95c456df337c763cb8c
3a81a6013aa633739d8cd554c688102211725e6adad165adc1bcd429d020c51b4b25d2117e8bb27eb0cc7020f9070d4a
d19ac31a76ebdf5f9246646aeadbfb9a3f1d75bd8237961e786302516a1a781780e8b73f58dc06f307e58bd0eb1d8f5c
9111f01312974c1dc777a6a2d3834d8a2a40014e9818d0685cb3919f6b3b788ddc640b0ff9b1854d7098c7dd6f35196e
902b26709640bc87935a3914869a807e8339281e9cedaaca99474c3e7bdd35050bb998ab4546f9900904e0e39135e861
ff7862049269701081ebce32e4cca992c6967ff0fd239e38233eaf614af31e186635e9439ec5884d798f9174da6ff569
d68ed5c092b78bd3f880f5e88a7a8ab36789e1b57b035fb6c32a6358f51f83e4e5f46220bcad072943df8bd9541a61b7
dae8f30fa3dd5fb39b1fd9a0b8e802552b78d4ec306ecee15bfe6da14b29ba6d19ce5be4dd478bca74a52429cd5309d4
04655c3dec85c252

forwarding error packet
shared_secret = 21e13c2d7cfe7e18836df50872466117a295783ab8aab0e7ecc8c725503ad02d
ammag_key = cd9ac0e09064f039fa43a31dea05f5fe5f6443d40a98be4071af4a9d704be5ad
stream =
617ca1e4624bc3f04fece3aa5a2b615110f421ec62408d16c48ea6c1b7c33fe7084a2bd9d4652fc5068e5052bf6d0aca
e2176018a3d8c75f37842712913900263cff92f39f3c18aa1f4b20a93e70fc429af7b2b1967ca81a761d40582daf0eb4
9cef66e3d6fbca0218d3022d32e994b41c884a27c28685ef1eb14603ea80a204b2f2f474b6ad5e71c6389843e3611ebe
afc62390b717ca53b3670a33c517ef28a659c251d648bf4c966a4ef187113ec9848bf110816061ca4f2f68e76ceb88bd
6208376460b916fb2ddeb77a65e8f88b2e71a2cbf4ea4958041d71c17d05680c051c3676fb0dc8108e5d78fb1e2c44d7
9a202e9d14071d536371ad47c39a05159e8d6c41d17a1e858faaaf572623aa23a38ffc73a4114cb1ab1cd7f906c6bd4e
21b29694f9830d12e8ad4b1ac320b3d5bfb4e534f02cefe9a983d66939581412acb1927eb93e8ed73145cddf24266bdc
c95923ecb38c8c9c5f4465335b0f18bf9f2d03fa02d57f258db27983d94378bc796cbe7737180dd7e39a36e461ebcb7e
c82b6dcdf9d3f209381f7b3a23e798c4f92e13b4bb972ee977e24f4b83bb59b577c210e1a612c2b035c8271d9bc1fb91
5776ac6560315b124465866830473aa238c35089cf2adb9c6e9f05ab113c1d0a4a18ba0cb9951b928c0358186532c36d
4c3daa65657be141cc22e326f88e445e898893fd5f0a7dd231ee5bc972077b1e12a8e382b75d4b557e895a2adc757f2e
451e33e0ae3fb54566ee09155da6ada818aa4a4a2546832a8ba22f0ef9ec6a1c78e03a7c29cb126bcaf81aea61cd8b07
ab9f4e5e0ad0d9a3a0c66d2d0a00cc05884d183a68e816e76b75842d55895f5b91c5c1b9f7052763aae8a647aa079921
4275b6e781f0816fc9ffb802a0101eb5a2de6b3375d3e3478f892b2de7f1900d8ca9bf188fcba89fc49d03c38fa2587a
8ae119abfb295b15fa11cb188796bedc4fdfceef296e44fbfa7e84569cc6346389a421782e40a298e1e2b6f9cae3103c
3f39d24541e4ab7b61dafe1a5f2fe936a59d87cccdaf7c226acc451ceec3e81bc4828b4925feeae3526d5e2bf93bd5f4
fdc0e069010aea1ae7e0d480d438918598896b776bf08fea124f91b3a13414b56934857707902612fc97b0e5d02cbe6e
5901ad304c7e8656390efccf3a1b22e18a2935181b78d5d2c89540ede8b0e6194d0d3945780bf577f622cb12deedbf82
10eb1450c298b9ee19f3c7082aabc2bdbd3384f3539dc3766978567135549df0d48287735854a6098fa40a9e48eaa27e
0d159beb65dd871e4c0b3fffa65f0375f0d3253f582193135ece60d5b9d8ba6739d87964e992cbec674b728d9eaaed59
5462c41d15fb497d4baa062368005d13fc99e1402563117a6c140c10363b05196a4cbb6b84ae807d62c748485c15e331

6841e4a98c3aac81e3bc996b4baca77eac8cdbe99cc7c5ebeb85c907cefb4abe15cbe87fdc5dc2326019196235ac2059
34fcf8e3
error packet for node 3:
7512354d6a26781d25e65539772ba049b7ed7c530bf75ab7ef80cf974b978a07a1c3dabc61940011585323f70fa98cfa
1d4c868da30b1f751e44a72d9b3f79809c8c51c9f0843daa8fe83587844fedeacb7348362003b31922cbb4d6169b2087
b6f8d192d9cfe5363254cd1fde24641bde9e422f170c3eb146f194c48a459ae2889d706dc654235fa9dd20307ea54091
d09970bf956c067a3bcc05af03c41e01af949a131533778bf6ee3b546caf2eabe9d53d0fb2e8cc952b7e0f5326a69ed2
e58e088729a1d85971c6b2e129a5643f3ac43da031e655b27081f10543262cf9d72d6f64d5d96387ac0d43da3e3a03da
0c309af121dcf3e99192efa754eab6960c256ffd4c546208e292e0ab9894e3605db098dc16b40f17c320aa4a0e42fc8b
105c22f08c9bc6537182c24e32062c6cd6d7ec7062a0c2c2ecdae1588c82185cdc61d874ee916a7873ac54cddf929354
f307e870011704a0e9fbc5c7802d6140134028aca0e78a7e2f3d9e5c7e49e20c3a56b624bfea51196ec9e88e4e56be38
ff56031369f45f1e03be826d44a182f270c153ee0d9f8cf9f1f4132f33974e37c7887d5b857365c873cb218cbf20d4be
3abdb2a2011b14add0a5672e01e5845421cf6dd6faca1f2f443757aae575c53ab797c2227ecdab03882bbbf4599318ce
fafa72fa0c9a0f5a51d13c9d0e5d25bfcfb0154ed25895260a9df8743ac188714a3f16960e6e2ff663c08bffda41743d
50960ea2f28cda0bc3bd4a180e297b5b41c700b674cb31d99c7f2a1445e121e772984abff2bbe3f42d757ceeda3d03fb
1ffe710aecabda21d738b1f4620e757e57b123dbc3c4aa5d9617dfa72f4a12d788ca596af14bea583f502f16fdc13a5e
739afb0715424af2767049f6b9aa107f69c5da0e85f6d8c5e46507e14616d5d0b797c3dea8b74a1b12d4e47ba7f57f09
d515f6c7314543f78b5e85329d50c5f96ee2f55bbe0df742b4003b24ccbd4598a64413ee4807dc7f2a9c0b92424e4ae1
b418a3cdf02ea4da5c3b12139348aa7022cc8272a3a1714ee3e4ae111cffd1bdfd62c503c80bdf27b2feaea0d5ab8fe0
0f9cec66e570b00fd24b4a2ed9a5f6384f148a4d6325110a41ca5659ebc5b98721d298a52819b6fb150f273383f1c575
4d320be428941922da790e17f482989c365c078f7f3ae100965e1b38c052041165295157e1a7c5b7a57671b842d4d85a
7d971323ad1f45e17a16c4656d889fc75c12fc3d8033f598306196e29571e414281c5da19c12605f48347ad5b4648e37
1757cbe1c40adb93052af1d6110cfbf611af5c8fc682b7e2ade3bfca8b5c7717d19fc9f97964ba6025aebbc91a6671e2
59949dcf40984342118de1f6b514a7786bd4f6598ffbe1604cef476b2a4cb1343db608aca09d1d38fc23e98ee9c65e7f
6023a8d1e61fd4f34f753454bd8e858c8ad6be6403edc599c220e03ca917db765980ac781e758179cd93983e9c1e769e
4241d47c
attribution data for node 3:
1571e10db7f8aa9f8e7e99caaf9c892e106c817df1d8e3b7b0e39d1c48f631e473e17e205489dd7b3c634cac3be0825c
bf01418cd46e83c24b8d9c207742db9a0f0e5bcd888086498159f08080ba7bf36dee297079eb841391ccd3096da76461
e314863b6412efe0ffe228d51c6097db10d3edb2e50ea679820613bfe9db11ba02920ab4c1f2a79890d997f1fc022f3a
b78f0029cc6de0c90be74d55f4a99bf77a50e20f8d076fe61776190a61d2f41c408871c0279309cba3b60fcdc7efc4a0
e90b47cb4a418fc78f362ecc7f15ebbce9f854c09c7be300ebc1a40a69d4c7cb7a19779b6905e82bec221a709c1dab8c
bdcde7b527aca3f54bde651aa9f3f2178829cee3f1c0b9292758a40cc63bd998fcd0d3ed4bdcaf1023267b8f8e44130a
63ad15f76145936552381eabb6d684c0a3af6ba8efcf207cebaea5b7acdbb63f8e7221102409d10c23f0514dc9f4d0ef
b2264161a193a999a23e992632710580a0d320f676d367b9190721194514457761af05207cdab2b6328b1b3767eacb36
a7ef4f7bd2e16762d13df188e0898b7410f62459458712a44bf594ae662fd89eb300abb6952ff8ad40164f2bcd7f86db
5c7650b654b79046de55d51aa8061ce35f867a3e8f5bf98ad920be827101c64fb871d86e53a4b3c0455bfac578416821
8aa72cbee86d9c750a9fa63c363a8b43d7bf4b2762516706a306f0aa3be1ec788b5e13f8b24837e53ac414f211e11c7a
093cd9653dfa5fba4e377c79adfa5e841e2ddb6afc054fc715c05ddc6c8fc3e1ee3406e1ffceb2df77dc2f02652614d1
bfcfaddebaa53ba919c7051034e2c7b7cfaabdf89f26e7f8e3f956d205dfab747ad0cb505b85b54a68439621b25832cb
c2898919d0cd7c0a64cfd235388982dd4dd68240cb668f57e1d2619a656ed326f8c92357ee0d9acead3c20008bc5f04c
a8059b55d77861c6d04dfc57cfba57315075acbe1451c96cf28e1e328e142890248d18f53b5d3513ce574dea7156cf59
6fdb3d909095ec287651f9cf1bcdc791c5938a5dd9b47e84c004d24ab3ae74492c7e8dcc1da15f65324be2672947ec82
074cac8ce2b925bc555facbbf1b55d63ea6fbea6a785c97d4caf2e1dad9551b7f66c31caae5ebc7c0047e892f201308f
cf452c588be0e63d89152113d87bf0dbd01603b4cdc7f0b724b0714a9851887a01f709408882e18230fe810b9fafa58a
666654576d8eba3005f07221f55a6193815a672e5db56204053bc4286fa3db38250396309fd28011b5708a26a2d76c4a
333b69b6bfd272fb

forwarding error packet
shared_secret = 3a6b412548762f0dbccce5c7ae7bb8147d1caf9b5471c34120b30bc9c04891cc
ammag_key = 1bf08df8628d452141d56adfd1b25c1530d7921c23cecfc749ac03a9b694b0d3
stream =
6149f48b5a7e8f3d6f5d870b7a698e204cf64452aab4484ff1dee671fe63fd4b5f1b78ee2047dfa61e3d576b149bedaf
83058f85f06a3172a3223ad6c4732d96b32955da7d2feb4140e58d86fc0f2eb5d9d1878e6f8a7f65ab9212030e8e9155
73ebbd7f35e1a430890be7e67c3fb4bbf2def662fa625421e7b411c29ebe81ec67b77355596b05cc155755664e59c16e

21410aabe53e80404a615f44ebb31b365ca77a6e91241667b26c6cad24fb2324cf64e8b9dd6e2ce65f1f098cfd1ef41b
a2d4c7def0ff165a0e7c84e7597c40e3dffe97d417c144545a0e38ee33ebaae12cc0c14650e453d46bfc48c0514f3547
73435ee89b7b2810606eb73262c77a1d67f3633705178d79a1078c3a01b5fadc9651feb63603d19decd3a00c1f69af2d
ab2595931ca50d8280758b1cc91ba2dc43dbbc3d91bf25c08b46c2ecef7a32cec64d4b61ee3a629ef563afe058b71e71
bcb69033948bc8728c5ebe65ec596e4f305b9fc159d53f723dfc95b57f3d51717f1c89af97a6d587e89e62efcc92198a
1b2bd66e2d875505ea4046c04389f8cb0ee98f0af03af2652e2f3d9a9c48430f2891a4d9b16e7d18099e4a3dd334c24a
ba1e2450792c2f22092c170da549d43a440021e699bd6c20d8bbf1961100a01ebcce06a4609f5ad93066287acf68294c
fa9ea7cea03a508983b134a9f0118b16409a61c06aaa95897d2067cb7cd59123f3e2ccf0e16091571d616c44818f118b
b7835a679f5c0eea8cf1bd5479882b2c2a341ec26dbe5da87b3d37d66b1fbd176f71ab203a3b6eaf7f214d579e7d0e4a
3e59089ebd26ba04a62403ae7a793516ec16d971d51c5c0107a917d1a70221e6de16edca7cb057c7d06902b5191f298a
a4d478a0c3a6260c257eae504ebbf2b591688e6f3f77af770b6f566ae9868d2f26c12574d3bf9323af59f0fe0072ff94
ae597c2aa6fbcbf0831989e02f9d3d1b9fd6dd97f509185d9ecbf272e38bd621ee94b97af8e1cd43853a8f6aa6e83725
85c71bf88246d064ade524e1e0bd8496b620c4c2d3ae06b6b064c97536aaf8d515046229f72bee8aa398cd0cc21afd54
49595016bef4c77cb1e2e9d31fe1ca3ffde06515e6a4331ccc84edf702e5777b10fc844faf17601a4be3235931f6feca
4582a8d247c1d6e4773f8fb6de320cf902bbb1767192782dc550d8e266e727a2aa2a414b816d1826ea46af7170153719
3c22bbcc0123d7ff5a23b0aa8d7967f36fef27b14fe1866ff3ab215eb29e07af49e19174887d71da7e7fe1b7aa1b3c80
5c063e0fafedf125fa6c57e38cce33a3f7bb35fd8a9f0950de3c22e49743c05f40bc55f960b8a8b5e2fde4bb229f1255
38438de418cb318d13968532499118cb7dcaaf8b6d635ac4001273bdafd12c8ea0702fb2f0dac81dbaaf68c1c3226638
2b293fa3951cb952ed5c1bdc41750cdbc0bd62c51bb685616874e251f031a929c06faef5bfcb0857f815ae20620b823f
0abecfb5
error packet for node 2:
145bc1c63058f7204abbd2320d422e69fb1b3801a14312f81e5e29e6b5f4774cfed8a25241d3dfb7466e749c1b326155
9e49090853612e07bd669dfb5f4c54162fa504138dabd6ebcf0db8017840c35f12a2cfb84f89cc7c8959a6d51815b1d2
c5136cedec2e4106bb5f2af9a21bd0a02c40b44ded6e6a90a145850614fb1b0eef2a03389f3f2693bc8a755630fc81ff
f1d87a147052863a71ad5aebe8770537f333e07d841761ec448257f948540d8f26b1d5b66f86e073746106dfdbb86ac9
475acf59d95ece037fba360670d924dce53aaa74262711e62a8fc9eb70cd8618fbedae22853d3053c7f10b1a6f75369d
7f73c419baa7dbf9f1fc5895362dcc8b6bd60cca4943ef7143956c91992119bccbe1666a20b7de8a2ff30a46112b53a6
bb79b763903ecbd1f1f74952fb1d8eb0950c504df31fe702679c23b463f82a921a2c931500ab08e686cffb2d87258d25
4fb17843959cccd265a57ba26c740f0f231bb76df932b50c12c10be90174b37d454a3f8b284c849e86578a6182c4a7b2
e47dd57d44730a1be9fec4ad07287a397e28dce4fda57e9cdfdb2eb5afdf0d38ef19d982341d18d07a556bb16c1416f4
80a396f278373b8fd9897023a4ac506e65cf4c306377730f9c8ca63cf47565240b59c4861e52f1dab84d938e96fb3182
0064d534aca05fd3d2600834fe4caea98f2a748eb8f200af77bd9fbf46141952b9ddda66ef0ebea17ea1e7bb5bce65b6
e71554c56dd0d4e14f4cf74c77a150776bf31e7419756c71e7421dc22efe9cf01de9e19fc8808d5b525431b944400db1
21a77994518d6025711cb25a18774068bba7faaa16d8f65c91bec8768848333156dcb4a08dfbbd9fef392da3e4de13d4
d74e83a7d6e46cfe530ee7a6f711e2caf8ad5461ba8177b2ef0a518baf9058ff9156e6aa7b08d938bd8d1485a787809d
7b4c8aed97be880708470cd2b2cdf8e2f13428cc4b04ef1f2acbc9562f3693b948d0aa94b0e6113cafa684f8e4a67dc4
31dfb835726874bef1de36f273f52ee694ec46b0700f77f8538067642a552968e866a72a3f2031ad116663ac17b172b4
46c5bc705b84777363a9a3fdc6443c07b2f4ef58858122168d4ebbaee920cefc312e1cea870ed6e15eec046ab2073bbf
08b0a3366f55cfc6ad4681a12ab0946534e7b6f90ea8992d530ec3daa6b523b3cf03101c60cadd914f30dec932c1ef43
41b5a8efac3c921e203574cfe0f1f83433fddb8ccfd273f7c3cab7bc27efe3bb61fdccd5146f1185364b9b621e7fb2b7
4b51f5ee6be72ab6ff46a6359dc2c855e61469724c1dbeb273df9d2e1c1fb74891239c0019dc12d5c7535f7238f963b7
61d7102b585372cf021b64c4fc85bfb3161e59d2e298bba44cfd34d6859d9dba9dc6271e5047d525468c814f2ae43847
4b0a977273036da1a2292f88fcfb89574a6bdca1185b40f8aa54026d5926725f99ef028da1be892e3586361efe15f4a1
48ff1bc9
attribution data for node 2:
34e34397b8621ec2f2b54dbe6c14073e267324cd60b152bce76aec8729a6ddefb61bc263be4b57bd592aae604a32bea6
9afe6ef4a6b573c26b17d69381ec1fc9b5aa769d148f2f1f8b5377a73840bb6dc641f68e356323d766fff0aaca5039fe
7fc27038195844951a97d5a5b26698a4ca1e9cd4bca1fcca0aac5fee91b18977d2ad0e399ba159733fc98f6e96898ebc
39bf0028c9c81619233bab6fad0328aa183a635fac20437fa6e00e899b2527c3697a8ab7342e42d55a679b176ab76671
fcd480a9894cb897fa6af0a45b917a162bed6c491972403185df7235502f7ada65769d1bfb12d29f10e25b0d3cc08bbf
6de8481ac5c04df32b4533b4f764c2aefb7333202645a629fb16e4a208e9045dc36830759c852b31dd613d8b2b10bbea
d1ed4eb60c85e8a4517deba5ab53e39867c83c26802beee2ee545bdd713208751added5fc0eb2bc89a5aa2decb18ee37
dac39f22a33b60cc1a369d24de9f3d2d8b63c039e248806de4e36a47c7a0aed30edd30c3d62debdf1ad82bf7aedd7ede
c413850d91c261e12beec7ad1586a9ad25b2db62c58ca17119d61dcc4f3e5c4520c42a8e384a45d8659b338b3a08f9e1
23a1d3781f5fc97564ccff2c1d97f06fa0150cfa1e20eacabefb0c339ec109336d207cc63d9170752fc58314c43e6d4a

528fd0975afa85f3aa186ff1b6b8cb12c97ed4ace295b0ef5f075f0217665b8bb180246b87982d10f43c9866b2287810
6f5214e99188781180478b07764a5e12876ddcb709e0a0a8dd42cf004c695c6fc1669a6fd0e4a1ca54b024d0d80eac49
2a9e5036501f36fb25b72a054189294955830e43c18e55668337c8c6733abb09fc2d4ade18d5a853a2b82f7b4d77151a
64985004f1d9218f2945b63c56fdebd1e96a2a7e49fa70acb4c39873947b83c191c10e9a8f40f60f3ad5a2be47145c22
ea59ed3f5f4e61cb069e875fb67142d281d784bf925cc286eacc2c43e94d08da4924b83e58dbf2e43fa625bdd620eba6
d9ce960ff17d14ed1f2dbee7d08eceb540fdc75ff06dabc767267658fad8ce99e2a3236e46d2deedcb51c3c6f8158935
7edebac9772a70b3d910d83cd1b9ce6534a011e9fa557b891a23b5d88afcc0d9856c6dabeab25eea55e9a248182229e4
927f268fe5431672fcce52f434ca3d27d1a2136bae5770bb36920df12fbc01d0e8165610efa04794f414c1417f1d4059
435c5385bfe2de83ce0e238d6fd2dbd3c0487c69843298577bfa480fe2a16ab2a0e4bc712cd8b5a14871cda61c993b68
35303d9043d7689a

forwarding error packet
shared_secret = a6519e98832a0b179f62123b3567c106db99ee37bef036e783263602f3488fae
ammag_key = 59ee5867c5c151daa31e36ee42530f429c433836286e63744f2020b980302564
stream =
0f10c86f05968dd91188b998ee45dcddfbf89fe9a99aa6375c42ed5520a257e048456fe417c15219ce39d921555956ae
2ff795177c63c819233f3bcb9b8b28e5ac6e33a3f9b87ca62dff43f4cc4a2755830a3b7e98c326b278e2bd31f4a9973e
e99121c62873f5bfb2d159d3d48c5851e3b341f9f6634f51939188c3b9ff45feeb11160bb39ce3332168b8e744a92107
db575ace7866e4b8f390f1edc4acd726ed106555900a0832575c3a7ad11bb1fe388ff32b99bcf2a0d0767a83cf293a22
0a983ad014d404bfa20022d8b369fe06f7ecc9c74751dcda0ff39d8bca74bf9956745ba4e5d299e0da8f68a9f660040b
eac03e795a046640cf8271307a8b64780b0588422f5a60ed7e36d60417562938b400802dac5f87f267204b6d5bcfd8a0
5b221ec294d883271b06ca709042ed5dbb64a7328d2796195eab4512994fb88919c73b3e5dd7bf68b2136d34cff39b3b
e266b71e004509bf975a240800bb8ae5eed248423a991ae80ef751b2d03b67fb93ffdd7969d5b500fe446a4ffb4cd04d
0767a5d367ebd3f8f260f38ae1e9d9f9a7bd1a99ca1e10ee36bd241f06fc2b481c9b7450d9c97042046668077832 64a0
e93468e22db4dc4a7a4db2963ddf4366d08e225cf94848aac794bcecb7e850113e38cc3647a03a5dfaa3442b1bb58b1d
e7fa7f436feb4d7c23cbd2de6d55d4025fcd383cc9d49c0b130e2fd5a9097c216683c842f898a8a2159761cca9aa1c81
8194e3b7bea6da6652d5189f3b6b0ca1d5398b6d14e311d9c7f00399c29e94deb98496f4cd97c5d7d6a65cabc3791f60
d728d6422a422c0cff5f7dfd4ce2d7e8d38dd71ae18763acc832c57275497f61b2620cca13cc64c0c48353f3817016f9
1448d6fc1cc451ee1f4a429e43292bbcd54fcd807e2c47675bac1781d9d81e9e6dc69028d428f5ee261750f626bcaf41
6a0e7badadf73fe1922207ae6c5209d16849e4a108f4a6f38694075f55177105ac4c2b97f6a474b94c03257d8d12b019
6e2905d914b8c2213a1b9dc9608e1a2a1e03fe0820a813275de83be5e9734875787a9e006eb8574c23ddd49e2347d1ec
fcedf3caa0a5dd45666368525b48ac14225d6422f82dbf59860ee4dc78e845d3c57668ce9b9e7a8d012491cef242078b
458a956ad67c360fb6d8b86ab201d6217e49b55fa02a1dea2dbe88d0b08d30670d1b93c35cc5e41e088fccb267e41d61
51cf8560496e1beeefe680744d9dabb383a4957466b4dc3e2bce7b135211da483d998a22fa687cc609641126c5dee3ed
87291067916b5b065f40582163291d48e81ecd975d0d6fd52a31754f8ef15e43a560bd30ea5bf21915bd2e7007e607ab
bc6261edc8430cc7f789675b1fe83e807c5c475bd5178eba2fc40674706b0a68c6a428e5dec36e413e653c6db1178923
ff87e2389a78bf9e93b713de4f4753f9f9d6a361369b609e1970c91ff9bd191c472e0bf2e8681412260ad0ef5855dc39
f2084d45
error packet for node 1:
1b4b09a935ce7af95b336baae307f2b400e3a7e808d9b4cf421cc4b3955620acb69dcdb656128dae8857adbd4e6b37fb
b1be9c1f2f02e61e9e59a630c4c77cf383cb37b07413aa4de2f2fbf5b40ae40a91a8f4c6d74aeacef1bb1be4ecbc26ec
2c824d2bc45db4b9098e732a769788f1cff3f5b41b0d25c132d40dc5ad045ef0043b15332ca3c5a09de2cdb17455a0f8
2a8f20da08346282823dab062cdbd2111e238528141d69de13de6d83994fbc711e3e269df63a12d3a4177c5c149150eb
4dc2f589cd8acabcddba14dec3b0dada12d663b36176cd3c257c5460bab93981ad99f58660efa9b31d7e63b399153296
95b3fa60e0a3bdb93e7e29a54ca6a8f360d3848866198f9c3da3ba958e7730847fe1e6478ce8597848d3412b4ae48b06
e05ba9a104e648f6eaf183226b5f63ed2e68f77f7e38711b393766a6fab7921b03eba82b5d7cb78e34dc961948d6161e
add7cf5d95d9c56df2ff5faa6ccf85eacdc9ff2fc3abafe41c365a5bd14fd486d6b5e2f24199319e7813e02e798877ff
e31a70ae2398d9e31b9e3727e6c1a3c0d995c67d37bb6e72e9660aaaa9232670f382add2edd468927e3303b614267254
6997fe105583e7c5a3c4c2b599731308b5416e6c9a3f3ba55b181ad0439d3535356108b059f2cb8742eed7a58d4eba9f
e79eaa77c34b12aff1abdaea93197aabd0e74cb271269ca464b3b06aef1d6573df5e1224179616036b368677f2647937
6681b772d3760e871d99efd34cca5cd6beca95190d967da820b21e5bec60082ea46d776b0517488c84f26d12873912d1
f68fafd67bcf4c298e43cfa754959780682a2db0f75f95f0598c0d04fd014c50e4beb86a9e37d95f2bba7e5065ae052d
c306555bca203d104c44a538b438c9762de299e1c4ad30d5b4a6460a76484661fc907682af202cd69b9a4473813b2fdc
1142f1403a49b7e69a650b7cde9ff133997dcc6d43f049ecac5fce097a21e2bce49c810346426585e3a5a18569b4cddd
5ff6bdec66d0b69fcbc5ab3b137b34cc8aefb8b850a764df0e685c81c326611d901c392a519866e132bbb73234f6a358

ba284fbafb21aa3605cacbaf9d0c901390a98b7a7dac9d4f0b405f7291c88b2ff45874241c90ac6c5fc895a440453c34
4d3a365cb929f9c91b9e39cb98b142444aae03a6ae8284c77eb04b0a163813d4c21883df3c0f398f47bf127b5525f222
107a2d8fe55289f0cfd3f4bbad6c5387b0594ef8a966afc9e804ccaf75fe39f35c6446f7ee076d433f2f8a44dba1515a
cc78e589fa8c71b0a006fe14feebd51d0e0aa4e51110d16759eee86192eee90b34432130f387e0ccd2ee71023f1f641c
ddb571c690107e08f592039fe36d81336a421e89378f351e633932a2f5f697d25b620ffb8e84bb6478e9bd229bf3b164
b48d754ae97bd23f319e3c56b3bcdaaeb3bd7fc02ec02066b324cb72a09b6b43dec1097f49d69d3c138ce6f1a6402898
baf7568c
attribution data for node 1:
74a4ea61339463642a2182758871b2ea724f31f531aa98d80f1c3043febca41d5ee52e8b1e127e61719a0d078db89097
48d57839e58424b91f063c4fbc8a221bef261140e66a9b596ca6d420a973ad54fef30646ae53ccf0855b61f291a81e0e
c6dc0f6bf69f0ca0e5889b7e23f577ba67d2a7d6a2aa91264ab9b20630ed52f8ed56cc10a869807cd1a4c2cd802d8433
fee5685d6a04edb0bff248a480b93b01904bed3bb31705d1ecb7332004290cc0cd9cc2f7907cf9db28eec02985301668
f53fbc28c3e095c8f3a6cd8cab28e5e442fd9ba608b8b12e098731bbfda755393bd403c62289093b40390b2bae337fc8
7d2606ca028311d73a9ffbdffef56020c735ada30f54e577c6a9ec515ae2739290609503404b118d7494499ecf0457d7
5015bb60a16288a4959d74cf5ac5d8d6c113de39f748a418d2a7083b90c9c0a09a49149fd1f2d2cde4412e5aa2421eca
6fd4f6fe6b2c362ff37d1a0608c931c7ca3b8fefcfd4c44ef9c38357a0767b14f83cb49bd1989fb3f8e2ab202ac98bd8
439790764a40bf309ea2205c163261095649572003 0a25dc7118e0c868fdfa78c3e9ecce58215579a0581b3bafdb7dbb
e53be9e904567fdc0ce1236aab5d22f1ebc18997e3ea83d362d891e04c5785fd5238326f767bce499209f8db211a50e1
402160486e98e7235cf397dbb9ae19fd9b79ef589c821c6f99f28be33452405a003b33f4540fe0a41dfcc286f4d7cc10
b70552ba7850869abadcd4bb7f256823face853633d6e2a999ac9fcd259c71d08e266db5d744e1909a62c0db673745ad
9585949d108ab96640d2bc27fb4acac7fa8b170a30055a5ede90e004df9a44bdc29aeb4a6bec1e85dde1de6aaf01c6a5
d12405d0bec22f49026cb23264f8c04b8401d3c2ab6f2e109948b6193b3bec27adfe19fb8afb8a92364d6fc5b219e873
7d583e7ff3a4bcb75d53edda3bf3f52896ac36d8a877ad9f296ea6c045603fc62ac4ae41272bde85ef7c3b3fd3538aac
fd5b025fefbe277c2906821ecb20e6f75ea479fa3280f9100fb0089203455c56b6bc775e5c2f0f58c63edd63fa3eec0b
40da4b276d0d41da2ec0ead865a98d12bc694e23d8eaadd2b4d0ee88e9570c88fb878930f492e036d27998d593e47763
927ff7eb80b188864a3846dd2238f7f95f4090ed399ae95deaeb37abca1cf37c397cc12189affb42dca46b4ff6988eb8
c060691d155302d448f50ff70a794d97c0408f8cee9385d6a71fa412e36edcb22dbf433db9db4779f27b682ee17fc05e
70c8e794b9f7f6d1

forwarding error packet
shared_secret = 53eb63ea8a3fec3b3cd433b85cd62a4b145e1dda09391b348c4e1cd36a03ea66
ammag_key = 3761ba4d3e726d8abb16cba5950ee976b84937b61b7ad09e741724d7dee12eb5
stream =
3699fd352a948a05f604763c0bca2968d5eaca2b0118602e52e59121f050936c8dd90c24df7dc8cf8f1665e39a6c75e9
e2c0900ea245c9ed3b0008148e0ae18bbfaea0c711d67eade980c6f5452e91a06b070bbde68b5494a92575c114660fb5
3cf04bf686e67ffa4a0f5ae41a59a39a8515cb686db553d25e71e7a97cc2febcac55df2711b6209c502b2f8827b13d3a
d2f491c45a0cafe7b4d8d8810e805dee25d676ce92e0619b9c206f922132d806138713a8f69589c18c3fdc5acee41c12
34b17ecab96b8c56a46787bba2c062468a13919afc18513835b472a79b2c35f9a91f38eb3b9e998b1000cc4a0dbd62ac
1a5cc8102e373526d7e8f3c3a1b4bfb2f8a3947fe350cb89f73aa1bb054edfa9895c0fc971c2b5056dc8665902b51fce
d6dff80c4d247db977c15a710ce280fbd0ae3ca2a245b1c967aeb5a1a4a441c50bc9ceb33ca64b5ca93bd8b50060520f
35a54a148a4112e8762f9d0b0f78a7f46a5f06c7a4b0845d020eb505c9e527aabab71009289a6919520d32af1f9f51ce
4b3655c6f1aae1e26a16dc9aae55e9d4a6f91d4ba76e96fcb851161da3fc39d0d97ce30a5855c75ac2f613ff36a24801
bcbd33f0ce4a3572b9a2fca21efb3b07897fc07ee71e8b1c0c6f8dbb7d2c4ed13f11249414fc94047d1a4a0be94d45db
56af4c1a3bf39c9c5aa18209eaebb9e025f670d4c8cc1ee598c912db154eaa3d0c93cb3957e126c50486bf98c852ad32
6b5f80a19df6b2791f3d65b8586474f4c5dcb2aca0911d2257d1bb6a1e9fc1435be879e75d23290f9feb93ed40baaeca
1c399fc91fb1da3e5f0f5d63e543a8d12fe6f7e654026d3a118ab58cb14bef9328d4af254215eb1f639828cc6405a3ab
02d90bb70a798787a52c29b3a28fc67b0908563a65f08112abd4e9115cb01db09460c602aba3ddc375569dc3abe42c61
c5ea7feb39ad8b05d8e2718e68806c0e1c34b0bc85492f985f8b3e76197a50d63982b780187078f5c59ebd814afaeffc
7b2c6ee39d4f9c8c45fb5f685756c563f4b9d028fe7981b70752f5a31e44ba051ab40f3604c8596f1e95dc9b0911e7ed
e63d69b5eecd245fbecbcf233cf6eba842c0fec795a5adeab2100b1a1bc62c15046d48ec5709da4af64f59a2e552ddbb
dcda1f543bb4b687e79f2253ff0cd9ba4e6bfae8e510e5147273d288fd4336dbd0b6617bf0ef71c0b4f1f9c1dc999c17
ad32fe196b1e2b27baf4d59bba8e5193a9595bd786be00c32bae89c5dbed1e994fddffbec49d0e2d270bcc1068850e5d
7e7652e274909b3cf5e3bc6bf64def0bbeac974a76d835e9a10bdd7896f27833232d907b7405260e3c986569bb8fdd65
a55b020b91149f27bda9e63b4c2cc5370bcc81ef044a68c40c1b178e4265440334cc40f59ab5f82a022532805bfa6592
57c8d8ab9b4aef6abbd05de284c2eb165ef35737e3d387988c566f7b1ca0b1fc3e7b4ed991b77f23775e1c36a09a9913

```
84a33b78
error packet for node 0:
2dd2f49c1f5af0fcad371d96e8cddbdcd5096dc309c1d4e110f955926506b3c03b44c192896f45610741c85ed4074212
537e0c118d472ff3a559ae244acd9d783c65977765c5d4e00b723d00f12475aafaafff7b31c1be5a589e6e25f8da2959
107206dd42bbcb43438129ce6cce2b6b4ae63edc76b876136ca5ea6cd1c6a04ca86eca143d15e53ccdc9e23953e49dc2
f87bb11e5238cd6536e57387225b8fff3bf5f3e686fd08458ffe0211b87d64770db9353500af9b122828a006da754cf9
79738b4374e146ea79dd93656170b89c98c5f2299d6e9c0410c826c721950c780486cd6d5b7130380d7eaff994a8503a
8fef3270ce94889fe996da66ed121741987010f785494415ca991b2e8b39ef2df6bde98efd2aec7d251b2772485194c8
368451ad49c2354f9d30d95367bde316fec6cbdddc7dc0d25e99d3075e13d3de0822669861dafcd29de74eac48b64411
987285491f98d78584d0c2a163b7221ea796f9e8671b2bb91e38ef5e18aaf32c6c02f2fb690358872a1ed28166172631
a82c2568d23238017188ebbd48944a147f6cdb3690d5f88e51371cb70adf1fa02afe4ed8b581afc8bcc5104922843a55
d52acde09bc9d2b71a663e178788280f3c3eae127d21b0b95777976b3eb17be40a702c244d0e5f833ff49dae6403ff44
b131e66df8b88e33ab0a58e379f2c34bf5113c66b9ea8241fc7aa2b1fa53cf4ed3cdd91d407730c66fb039ef3a36d405
0dde37d34e80bcfe02a48a6b14ae28227b1627b5ad07608a7763a531f2ffc96dff850e8c583461831b19feffc783bc1b
eab6301f647e9617d14c92c4b1d63f5147ccda56a35df8ca4806b8884c4aa3c3cc6a174fdc2232404822569c01aba686
c1df5eecc059ba97e9688c8b16b70f0d24eacfdba15db1c71f72af1b2af85bd168f0b0800483f115eeccd9b02adf03bd
d4a88eab03e43ce342877af2b61f9d3d85497cd1c6b96674f3d4f07f635bb26add1e36835e321d70263b1c04234e2221
24dad30ffb9f2a138e3ef453442df1af7e566890aedee568093aa922dd62db188aa8361c55503f8e2c2e6ba93de744b5
5c15260f15ec8e69bb01048ca1fa7bbbd26975bde80930a5b95054688a0ea73af0353cc84b997626a987cc06a517e18f
91e02908829d4f4efc011b9867bd9bfe04c5f94e4b9261d30cc39982eb7b250f12aee2a4cce0484ff34eebba89bc6e35
bd48d3968e4ca2d77527212017e202141900152f2fd8af0ac3aa456aae13276a13b9b9492a9a636e18244654b3245f07
b20eb76b8e1cea8c55e5427f08a63a16b0a633af67c8e48ef8e53519041c9138176eb14b8782c6c2ee76146b8490b979
78ee73cd0104e12f483be5a4af414404618e9f6633c55dda6f22252cb793d3d16fae4f0e1431434e7acc8fa2c009d4f6
e345ade172313d558a4e61b4377e31b8ed4e28f7cd13a7fe3f72a409bc3bdabfe0ba47a6d861e21f64d2fac706dab18b
3e546df4
attribution data for node 0:
84986c936d26bfd3bb2d34d3ec62cfdb63e0032fdb3d9d75f3e5d456f73dffa7e35aab1db4f1bd3b98ff585caf004f65
6c51037a3f4e810d275f3f6aea0c8e3a125ebee5f374b6440bcb9bb2955ebf706f42be9999a62ed49c7a81fc73c0b4a1
6419fd6d334532f40bf179dd19afec21bd8519d5e6ebc3802501ef373bc378eee1f14a6fc5fab5b697c91ce31d592219
9d1b0ad5ee12176aacafc7c81d54bc5b8fb7e63f3bfd40a3b6e21f985340cbd1c124c7f85f0369d1aa86ebc66def4171
07a7861131c8bcd73e8946f4fb54bfac87a2dc15bd7af642f32ae583646141e8875ef81ec9083d7e32d5f135131eab7a
43803360434100ff67087762bbe3d6afe2034f5746b8c50e0c3c20dd62a4c174c38b1df7365dccebc7f24f19406649fb
f48981448abe5c858bbd4bef6eb983ae7a23e9309fb33b5e7c0522554e88ca04b1d65fc190947dead8c0ccd329329765
37d869b5ca53ed4945bccafab2a014ea4cbdc6b0250b25be66ba0afff2ff19c0058c68344fd1b9c472567147525b13b1
bc27563e61310110935cf89fda0e34d0575e2389d57bdf2869398ca2965f64a6f04e1d1c2edf2082b97054264a47824d
d1a9691c27902b39d57ae4a94dd6481954a9bd1b5cff4ab29ca221fa2bf9b28a362c9661206f896fc7cec563fb80aa5e
accb26c09fa4ef7a981e63028a9c4dac12f82ccb5bea090d56bbb1a4c431e315d9a169299224a8dbd099fb67ea61dfc6
04edf8a18ee742550b636836bb552dabb28820221bf8546331f32b0c143c1c89310c4fa2e1e0e895ce1a1eb0f43278fd
b528131a3e32bfffe0c6de9006418f5309cba773ca38b6ad8507cc59445ccc0257506ebc16a4c01d4cd97e03fcf7a204
9fea0db28447858f73b8e9fe98b391b136c9dc510288630a1f0af93b26a8891b857bfe4b818af99a1e011e6dbaa53982
d29cf74ae7dffef45545279f19931708ed3eede5e82280eab908e8eb80abff3f1f023ab66869297b40da8496861dc455
ac3abe1efa8a6f9e2c4eda48025d43a486a3f26f269743eaa30d6f0e1f48db6287751358a41f5b07aee0f098862e3493
731fe2697acce734f004907c6f11eef189424fee52cd30ad708707eaf2e441f52bcf3d0c5440c1742458653c0c8a27b5
ade784d9e09c8b47f1671901a29360e7e5e94946b9c75752a1a8d599d2a3e14ac81b84d42115cd688c8383a64fc6e7e1
dc5568bb4837358ebe63207a4067af66b2027ad2ce8fb7ae3a452d40723a51fdf9f9c9913e8029a222cf81d12ad41e58
860d75deb6de30ad
```

Returning success

A successful payment without a `fulfillment_payload` using the parameters above would result in the following attribution data values:

attribution data for node 4:
d77d0711b5f71d1d1be56bd88b3bb7ebc1792bb739ea7ebc1bc3b031b8bc2df3a50e25aeb99f47d7f7ab39e24187d3f4df9c4333463b05

attribution data for node 3:
1571e10db7f8aa9f8e7e99caaf9c892e106c817df1d8e3b7b0e39d1c48f631e473e17e205489dd7b3c634cac3be0825cbf01418cd46e83c2

attribution data for node 2:
34e34397b8621ec2f2b54dbe6c14073e267324cd60b152bce76aec8729a6ddefb61bc263be4b57bd592aae604a32bea69afe6ef4a6b573c

attribution data for node 1:
74a4ea61339463642a2182758871b2ea724f31f531aa98d80f1c3043febca41d5ee52e8b1e127e61719a0d078db8909748d57839e58424b

attribution data for node 0:
84986c936d26bfd3bb2d34d3ec62cfdb63e0032fdb3d9d75f3e5d456f73dffa7e35aab1db4f1bd3b98ff585caf004f656c51037a3f4e810d

A successful payment with a `fulfillment_payload` using the same route parameters contains the following `fulfillment_payload_tlvs` records:

```
type  = 1 (`padding`)
value = 245 zero bytes

type  = 65537
value = 070809
```

These two records serialize to 256 bytes: 247 bytes for the `padding` record and 9 bytes for the type `65537` record. With the 16-byte Poly1305 tag, this results in a 272-byte `fulfillment_payload`.

The following are the `fulfillment_payload` and attribution data values at each hop:

fulfillment payload for node 4:
b5b1fed711469adc0da510494818ce54624b537e7a8aba4b190403d90439798a694bdca0d0d53c18487d9940664a8416eb855290621e1
attribution data for node 4:
d77d0711b5f71d1d1be56bd88b3bb7ebc1792bb739ea7ebc1bc3b031b8bc2df3a50e25aeb99f47d7f7ab39e24187d3f4df9c4333463b05

fulfillment payload for node 3:
d4cd5f33730d592c4249f3e31233af0572bf729218ca375ddd8aa518b3fa466d6101f77904b013dd4ef3c912d9278edc09923288c1c6df3
attribution data for node 3:
1571e10db7f8aa9f8e7e99caaf9c892e106c817df1d8e3b7b0e39d1c48f631e473e17e205489dd7b3c634cac3be0825cbf01418cd46e83c2

fulfillment payload for node 2:
b584abb82973d6112d1474e8685a21253e4936c0b27e7f122c5443694d99bb263e1a8f9724f7cc7b50ce9e79cdbc63738a97bd0d31acee4
attribution data for node 2:
34e34397b8621ec2f2b54dbe6c14073e267324cd60b152bce76aec8729a6ddefb61bc263be4b57bd592aae604a32bea69afe6ef4a6b573c

fulfillment payload for node 1:
ba9463d72ce55bc83c9ccd70861ffdf8c5b1a9291be4d9257016ae3c6d3becc6765fe07333369e629ef7475898e535dda560281a4dcf2654
attribution data for node 1:
74a4ea61339463642a2182758871b2ea724f31f531aa98d80f1c3043febca41d5ee52e8b1e127e61719a0d078db8909748d57839e58424b

fulfillment payload for node 0:
8c0d9ee20671d1cdca98bb4c8dd5d490105b63021afcb90b22f33f1d9d6b7faafb86ec57ec4b56ad11e122bb0289403447a0b814ef8aefb
attribution data for node 0:
84986c936d26bfd3bb2d34d3ec62cfdb63e0032fdb3d9d75f3e5d456f73dffa7e35aab1db4f1bd3b98ff585caf004f656c51037a3f4e810d

References

Authors

[FIXME:]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

BOLT #7: P2P Node and Channel Discovery

This specification describes simple node discovery, channel discovery, and channel update mechanisms that do not rely on a third-party to disseminate the information.

Node and channel discovery serve two different purposes:

- Node discovery allows nodes to broadcast their ID, host, and port, so that other nodes can open connections and establish payment channels with them.
- Channel discovery allows the creation and maintenance of a local view of the network's topology, so that a node can discover routes to desired destinations.

To support channel and node discovery, three *gossip messages* are supported:

- For node discovery, peers exchange `node_announcement` messages, which supply additional information about the nodes. There may be multiple `node_announcement` messages, in order to update the node information.

- For channel discovery, peers in the network exchange `channel_announcement` messages containing information regarding new channels between the two nodes. They can also exchange `channel_update` messages, which update information about a channel. There can only be one valid `channel_announcement` for any channel, but at least two `channel_update` messages are expected.

Table of Contents

- Definition of `short_channel_id`
- The `announcement_signatures` Message
- The `channel_announcement` Message
- The `node_announcement` Message
- The `channel_update` Message
- Query Messages
- Rebroadcasting
- HTLC Fees
- Pruning the Network View
- Recommendations for Routing
- References

Definition of `short_channel_id`

The `short_channel_id` is the unique description of the funding transaction. It is constructed as follows: 1. the most significant 3 bytes: indicating the block height 2. the next 3 bytes: indicating the transaction index within the block 3. the least significant 2 bytes: indicating the output index that pays to the channel.

The standard human readable format for `short_channel_id` is created by printing the above components, in the order: block height, transaction index, and output index. Each component is printed as a decimal number, and separated from each other by the small letter `x`. For example, a `short_channel_id` might be written as `539268x845x1`, indicating a channel on the output 1 of the transaction at index 845 of the block at height 539268.

Rationale

The `short_channel_id` human readable format is designed so that double-clicking or double-tapping it will select the entire ID on most systems. Humans prefer decimal when reading numbers, so the ID components are written in decimal. The small letter `x` is used since on most fonts, the `x` is visibly smaller than decimal digits, making it easy to visibly group each component of the ID.

The `announcement_signatures` Message

This is a direct message between the two endpoints of a channel and serves as an opt-in mechanism to allow the announcement of the channel to the rest of the network. It contains the necessary signatures, by the sender, to construct the `channel_announcement` message.

1. type: 259 (`announcement_signatures`)
2. data:
    - [`channel_id`:`channel_id`]
    - [`short_channel_id`:`short_channel_id`]
    - [`signature`:`node_signature`]
    - [`signature`:`bitcoin_signature`]

The willingness of the initiating node to announce the channel is signaled during channel opening by setting the `announce_channel` bit in `channel_flags` (see BOLT #2).

Requirements

The `announcement_signatures` message is created by constructing a `channel_announcement` message, corresponding to the newly confirmed channel funding transaction, and signing it with the secrets matching an endpoint's `node_id` and `bitcoin_key`.

A node: - If the `open_channel` message has the `announce_channel` bit set AND a `shutdown` message has not been sent: - After `channel_ready` has been sent and received AND the funding transaction has enough confirmations to ensure that it won't be reorganized: - MUST send `announcement_signatures` for the funding transaction. - After `splice_locked` has been sent and received AND the splice transaction has enough confirmations to ensure that it won't be reorganized: - MUST send `announcement_signatures` for the matching splice transaction. - Otherwise: - MUST NOT send the `announcement_signatures` message. - Upon reconnection (once the above timing requirements have been met): - If it has NOT previously received `announcement_signatures` for the funding transaction: - MUST send its own `announcement_signatures` message. - If it receives

`announcement_signatures` for the funding transaction: - MUST respond with its own `announcement_signatures` message. - If it has NOT previously received `announcement_signatures` for a splice transaction: - MUST SET the `announcement_signatures` bit in the `retransmit_flags` of `my_current_funding_locked`. - If the `announcement_signatures` bit is set in the *remote* `retransmit_flags`: - MUST retransmit its `announcement_signatures` message.

A recipient node: - If the `short_channel_id` doesn't match one of its funding transactions: - SHOULD send a `warning`. - If the `node_signature` OR the `bitcoin_signature` is NOT correct: - MAY send a `warning` and close the connection, or send an `error` and fail the channel. - If it has sent AND received a valid `announcement_signatures` message: - If the funding transaction has at least 6 confirmations: - SHOULD queue the `channel_announcement` message for its peers. - If it has not sent `channel_ready`: - SHOULD defer handling the `announcement_signatures` until after it has sent `channel_ready`. - If it has not sent `splice_locked` for the transaction matching this `short_channel_id`: - SHOULD defer handling the `announcement_signatures` until after it has sent `splice_locked`.

Rationale

Channels must not be announced before the funding transaction has enough confirmations, because a blockchain reorganization would otherwise invalidate the `short_channel_id`.

When splicing is used, a `channel_announcement` is generated for every splice transaction once both sides have sent `splice_locked`. This lets the network know that the transaction spending a currently active channel is a splice and not a closing transaction, and this channel can still be used with its updated `short_channel_id`.

The `channel_announcement` Message

This gossip message contains ownership information regarding a channel. It ties each on-chain Bitcoin key to the associated Lightning node key, and vice-versa. The channel is not practically usable until at least one side has announced its fee levels and expiry, using `channel_update`.

Proving the existence of a channel between `node_1` and `node_2` requires:

1. proving that the funding transaction pays to `bitcoin_key_1` and `bitcoin_key_2`
2. proving that `node_1` owns `bitcoin_key_1`
3. proving that `node_2` owns `bitcoin_key_2`

Assuming that all nodes know the unspent transaction outputs, the first proof is accomplished by a node finding the output given by the `short_channel_id` and verifying that it is indeed a P2WSH funding transaction output for those keys specified in BOLT #3.

The last two proofs are accomplished through explicit signatures: `bitcoin_signature_1` and `bitcoin_signature_2` are generated for each `bitcoin_key` and each of the corresponding `node_id`s are signed.

It's also necessary to prove that `node_1` and `node_2` both agree on the announcement message: this is accomplished by having a signature from each `node_id` (`node_signature_1` and `node_signature_2`) signing the message.

1. type: 256 (`channel_announcement`)
2. data:
    - [`signature:node_signature_1`]

- [signature:node_signature_2]
- [signature:bitcoin_signature_1]
- [signature:bitcoin_signature_2]
- [u16:len]
- [len*byte:features]
- [chain_hash:chain_hash]
- [short_channel_id:short_channel_id]
- [point:node_id_1]
- [point:node_id_2]
- [point:bitcoin_key_1]
- [point:bitcoin_key_2]

Requirements

The origin node: - MUST set `chain_hash` to the 32-byte hash that uniquely identifies the chain that the channel was opened within: - for the *Bitcoin blockchain*: - MUST set `chain_hash` value (encoded in hex) equal to `6fe28c0ab6f1b372c1a6a246ae63f74f931e8365e15a089c68d6190000000000`. - When announcing a channel creation: - MUST set `short_channel_id` to refer to the confirmed funding transaction, as specified in BOLT #2. - When announcing a splice transaction: - MUST set `short_channel_id` to refer to the confirmed splice transaction for which `splice_locked` has been sent and received, as specified in BOLT #2. - SHOULD keep relaying payments that use the `short_channel_id`s of its previous `channel_announcements`. - SHOULD send a new `channel_update` using the `short_channel_id` that matches the latest `channel_announcement`. - Note: the corresponding output MUST be a P2WSH, as described in BOLT #3. - MUST set `node_id_1` and `node_id_2` to the public keys of the two nodes operating the channel, such that `node_id_1` is the lexicographically-lesser of the two compressed keys sorted in ascending lexicographic order. - MUST set `bitcoin_key_1` and `bitcoin_key_2` to `node_id_1` and `node_id_2`'s respective `funding_pubkeys`. - MUST compute the double-SHA256 hash `h` of the message, beginning at offset 256, up to the end of the message. - Note: the hash skips the 4 signatures but hashes the rest of the message, including any future fields appended to the end. - MUST set `node_signature_1` and `node_signature_2` to valid signatures of the hash `h` (using `node_id_1` and `node_id_2`'s respective secrets). - MUST set `bitcoin_signature_1` and `bitcoin_signature_2` to valid signatures of the hash `h` (using `bitcoin_key_1` and `bitcoin_key_2`'s respective secrets). - MUST set `features` based on what features were negotiated for this channel, according to BOLT #9 - MUST set `len` to the minimum length required to hold the `features` bits it sets. - If the funding transaction has less than 6 confirmations: - MUST NOT send `channel_announcement`.

The receiving node: - MUST verify the integrity AND authenticity of the message by verifying the signatures. - if `node_id_1` is not lexicographically less than `node_id_2`: - SHOULD send a `warning`. - MAY close the connection. - MUST ignore the message. - if there is an unknown even bit in the `features` field: - MUST NOT attempt to route messages through the channel. - if the `short_channel_id`'s output does NOT correspond to a P2WSH (using `bitcoin_key_1` and `bitcoin_key_2`, as specified in BOLT #3) OR the output is spent: - MUST ignore the message. - if the specified `chain_hash` is unknown to the receiver: - MUST ignore the message. - if the `short_channel_id`'s output does NOT have at least 6 confirmations: - MAY accept the message if the output is close to 6 confirmations, in case the receiving node hasn't received the latest block(s) yet. - otherwise: - SHOULD ignore the message. - otherwise: - if `bitcoin_signature_1`, `bitcoin_signature_2`, `node_signature_1` OR `node_signature_2` are invalid OR NOT correct: - SHOULD send a `warning`. - MAY close the connection. - MUST ignore the message. - otherwise: - if `node_id_1` OR `node_id_2` are blacklisted: - SHOULD ignore the message. - otherwise: - if the transaction referred to was NOT previously announced as a

channel: - SHOULD queue the message for rebroadcasting. - MAY choose NOT to for messages longer than the minimum expected length. - if it has previously received a valid `channel_announcement`, for the same transaction, in the same block, but for a different `node_id_1` or `node_id_2`: - SHOULD blacklist the previous message's `node_id_1` and `node_id_2`, as well as this `node_id_1` and `node_id_2` AND forget any channels connected to them. - otherwise: - SHOULD store this `channel_announcement`. - once its funding output has been spent OR reorganized out: - SHOULD forget a channel after a 72-block delay. - SHOULD NOT rebroadcast this `channel_announcement` to its peers.

Rationale

Both nodes are required to sign to indicate they are willing to route other payments via this channel (i.e. be part of the public network); requiring their Bitcoin signatures proves that they control the channel.

The blacklisting of conflicting nodes disallows multiple different announcements. Such conflicting announcements should never be broadcast by any node, as this implies that keys have leaked.

While channels should not be advertised before they are sufficiently deep, the requirement against rebroadcasting only applies if the transaction has not moved to a different block.

In order to avoid storing excessively large messages, yet still allow for reasonable future expansion, nodes are permitted to restrict rebroadcasting (perhaps statistically).

New channel features are possible in the future: backwards compatible (or optional) features will have *odd* feature bits, while incompatible features will have *even* feature bits ("It's OK to be odd!").

A delay of 72 blocks is used when forgetting a channel after detecting that it has been spent: this can allow a new `channel_announcement` to propagate to indicate that this channel was spliced and not closed. Thanks to this delay, payments can still be relayed on the channel while the splice transaction is waiting for enough confirmations.

The `node_announcement` Message

This gossip message allows a node to indicate extra data associated with it, in addition to its public key. To avoid trivial denial of service attacks, nodes not associated with an already known channel are ignored.

1. type: 257 (`node_announcement`)
2. data:
    - [`signature:signature`]
    - [`u16:flen`]
    - [`flen*byte:features`]
    - [`u32:timestamp`]
    - [`point:node_id`]
    - [`3*byte:rgb_color`]
    - [`32*byte:alias`]
    - [`u16:addrlen`]
    - [`addrlen*byte:addresses`]

`timestamp` allows for the ordering of messages, in the case of multiple announcements. `rgb_color` and `alias` allow intelligence services to assign nodes colors like black and cool monikers like 'IRATEMONK' and 'WISTFULTOLL'.

`addresses` allows a node to announce its willingness to accept incoming network connections: it contains a series of `address descriptors` for connecting to the node. The first byte describes the address type and is followed by the appropriate number of bytes for that type.

The following `address descriptor` types are defined:

- `1`: ipv4; data = `[4:ipv4_addr][2:port]` (length 6)
- `2`: ipv6; data = `[16:ipv6_addr][2:port]` (length 18)
- `3`: Deprecated (length 12). Used to contain Tor v2 onion services.
- `4`: Tor v3 onion service; data = `[35:onion_addr][2:port]` (length 37)
    - version 3 (prop224) onion service addresses; Encodes: `[32:32_byte_ed25519_pubkey] || [2:checksum] || [1:version]`, where `checksum = sha3(".onion checksum" || pubkey || version)[:2]`.
- `5`: DNS hostname; data = `[1:hostname_len][hostname_len:hostname][2:port]` (length up to 258)
    - `hostname` bytes MUST be ASCII characters.
    - Non-ASCII characters MUST be encoded using Punycode: https://en.wikipedia.org/wiki/Punycode

Requirements

The origin node: - MUST set `timestamp` to be greater than that of any previous `node_announcement` it has previously created. - MAY base it on a UNIX timestamp. - MUST set `signature` to the signature of the double-SHA256 of the entire remaining packet after `signature` (using the key given by `node_id`). - MAY set `alias` AND `rgb_color` to customize its appearance in maps and graphs. - Note: the first byte of `rgb_color` is the red value, the second byte is the green value, and the last byte is the blue value. - MUST set `alias` to a valid UTF-8 string, with any `alias` trailing-bytes equal to 0. - SHOULD fill `addresses` with an address descriptor for each public network address that expects incoming connections. - MUST set `addrlen` to the number of bytes in `addresses`. - MUST place address descriptors in ascending order. - SHOULD NOT place any zero-typed address descriptors anywhere. - SHOULD use placement only for aligning fields that follow `addresses`. - MUST NOT create an address descriptor with `port` equal to 0. - SHOULD ensure `ipv4_addr` AND `ipv6_addr` are routable addresses. - MUST set `features` according to BOLT #9 - SHOULD set `flen` to the minimum length required to hold the `features` bits it sets. - SHOULD not announce a Tor v2 onion service. - MUST NOT announce more than one `type 5` DNS hostname.

The receiving node: - if `node_id` is NOT a valid compressed public key: - SHOULD send a `warning`. - MAY close the connection. - MUST NOT process the message further. - if `signature` is NOT a valid signature (using `node_id` of the double-SHA256 of the entire message following the `signature` field, including any future fields appended to the end): - SHOULD send a `warning`. - MAY close the connection. - MUST NOT process the message further. - if `features` field contains *unknown even bits*: - SHOULD NOT connect to the node. - Unless paying a BOLT #11 invoice which does not have the same bit(s) set, MUST NOT attempt to send payments *to* the node. - MUST NOT route a payment *through* the node. - SHOULD ignore the first `address descriptor` that does NOT match the types defined above. - if `addrlen` is insufficient to hold the address descriptors of the known types: - SHOULD send a `warning`. - MAY close the connection. - if `port` is equal to 0: - SHOULD ignore that address descriptor. - if `node_id` is NOT previously known from a `channel_announcement` message, OR if

`timestamp` is NOT greater than the last-received `node_announcement` from this `node_id`: - SHOULD ignore the message. - otherwise: - if `timestamp` is greater than the last-received `node_announcement` from this `node_id`: - SHOULD queue the message for rebroadcasting. - MAY choose NOT to queue messages longer than the minimum expected length. - MAY use `rgb_color` AND `alias` to reference nodes in interfaces. - SHOULD insinuate their self-signed origins. - SHOULD ignore Tor v2 onion services. - if more than one `type 5` address is announced: - SHOULD ignore the additional data. - MUST not forward the `node_announcement`.

Rationale

New node features are possible in the future: backwards compatible (or optional) ones will have *odd* `feature` *bits*, incompatible ones will have *even* `feature` *bits*. These will be propagated normally; incompatible feature bits here refer to the nodes, not the `node_announcement` message itself.

New address types may be added in the future; as address descriptors have to be ordered in ascending order, unknown ones can be safely ignored. Additional fields beyond `addresses` may also be added in the future—with optional padding within `addresses`, if they require certain alignment.

Security Considerations for Node Aliases

Node aliases are user-defined and provide a potential avenue for injection attacks, both during the process of rendering and during persistence.

Node aliases should always be sanitized before being displayed in HTML/Javascript contexts or any other dynamically interpreted rendering frameworks. Similarly, consider using prepared statements, input validation, and escaping to protect against injection vulnerabilities and persistence engines that support SQL or other dynamically interpreted querying languages.

- Stored and Reflected XSS Prevention
- DOM-based XSS Prevention
- SQL Injection Prevention

Don't be like the school of Little Bobby Tables.

The `channel_update` Message

After a channel has been initially announced, each side independently announces the fees and minimum expiry delta it requires to relay HTLCs through this channel. Each uses the 8-byte channel shortid that matches the `channel_announcement` and the 1-bit `channel_flags` field to indicate which end of the channel it's on (origin or final). A node can do this multiple times, in order to change fees.

Note that the `channel_update` gossip message is only useful in the context of *relaying* payments, not *sending* payments. When making a payment `A -> B -> C -> D`, only the `channel_updates` related to channels `B -> C` (announced by `B`) and `C -> D` (announced by `C`) will come into play. When building the route, amounts and expiries for HTLCs need to be calculated backward from the destination to the source. The exact initial value for `amount_msat` and the minimal value for `cltv_expiry`, to be used for the last HTLC in the route, are provided in the payment request (see BOLT #11).

1. type: 258 (`channel_update`)

2. data:
    - [signature:signature]
    - [chain_hash:chain_hash]
    - [short_channel_id:short_channel_id]
    - [u32:timestamp]
    - [byte:message_flags]
    - [byte:channel_flags]
    - [u16:cltv_expiry_delta]
    - [u64:htlc_minimum_msat]
    - [u32:fee_base_msat]
    - [u32:fee_proportional_millionths]
    - [u64:htlc_maximum_msat]

The `channel_flags` bitfield is used to indicate the direction of the channel: it identifies the node that this update originated from and signals various options concerning the channel. The following table specifies the meaning of its individual bits:

Bit Position	Name	Meaning
0	`direction`	Direction this update refers to.
1	`disable`	Disable the channel.

The `message_flags` bitfield is used to provide additional details about the message:

Bit Position	Name
0	`must_be_one`
1	`dont_forward`

The `node_id` for the signature verification is taken from the corresponding `channel_announcement`: `node_id_1` if the least-significant bit of flags is 0 or `node_id_2` otherwise.

Requirements

The origin node: - MUST NOT send a created `channel_update` before `channel_ready` has been received. - MAY create a `channel_update` to communicate the channel parameters to the channel peer, even though the channel has not yet been announced (i.e. the `announce_channel` bit was not set or the `channel_update` is sent before the peers exchanged announcement signatures). - MUST set the `short_channel_id` to either an `alias` it has received from the peer, or the real channel `short_channel_id`. - MUST set `dont_forward` to 1 in `message_flags` - MUST NOT forward such a `channel_update` to other peers, for privacy reasons. - Note: such a `channel_update`, one not preceded by a `channel_announcement`, is invalid to any other peer and would be discarded. - MUST set `signature` to the signature of the double-SHA256 of the entire remaining packet after `signature`, using its own `node_id`. - MUST set `chain_hash` AND `short_channel_id` to match the 32-byte hash AND 8-byte channel ID that uniquely identifies the channel specified in the `channel_announcement` message. - if the origin node is `node_id_1` in the message: - MUST set the `direction` bit of `channel_flags` to 0. - otherwise: - MUST set the `direction` bit of `channel_flags` to 1. - MUST set `htlc_maximum_msat` to the maximum value it will send through this channel for a single HTLC. - MUST set this to less than or equal to the channel capacity. - MUST set this to less than or equal to `max_htlc_value_in_flight_msat` it received from the peer. - MUST set

this to greater than or equal to `htlc_minimum_msat`. - MUST set `must_be_one` in `message_flags` to 1. - MUST set bits in `channel_flags` and `message_flags` that are not assigned a meaning to 0. - MAY create and send a `channel_update` with the `disable` bit set to 1, to signal a channel's temporary unavailability (e.g. due to a loss of connectivity) OR permanent unavailability (e.g. prior to an on-chain settlement). - MAY send a subsequent `channel_update` with the `disable` bit set to 0 to re-enable the channel. - MUST set `timestamp` to greater than 0, AND to greater than any previously-sent `channel_update` for this `short_channel_id`. - SHOULD base `timestamp` on a UNIX timestamp. - MUST set `cltv_expiry_delta` to the number of blocks it will subtract from an incoming HTLC's `cltv_expiry`. - MUST set `htlc_minimum_msat` to the minimum HTLC value (in millisatoshi) that the channel peer will accept. - MUST set `htlc_minimum_msat` to less than or equal to `htlc_maximum_msat`. - MUST set `fee_base_msat` to the base fee (in millisatoshi) it will charge for any HTLC. - MUST set `fee_proportional_millionths` to the amount (in millionths of a satoshi) it will charge per transferred satoshi. - SHOULD NOT create redundant `channel_updates` - If it creates a new `channel_update` with updated channel parameters: - SHOULD keep accepting the previous channel parameters for 10 minutes

The receiving node: - if the `short_channel_id` does NOT match a previous `channel_announcement`: - MUST ignore `channel_updates` that do NOT correspond to one of its own channels. - if the channel output has been spent: - MUST ignore `channel_updates`, unless they have the `disable` bit set to 1. - SHOULD NOT rebroadcast `channel_updates` to its peers, unless they have the `disable` bit set to 1. - SHOULD accept `channel_updates` for its own channels (even if non-public), in order to learn the associated origin nodes' forwarding parameters. - if `signature` is not a valid signature, using `node_id` of the double-SHA256 of the entire message following the `signature` field (including unknown fields following `fee_proportional_millionths`): - SHOULD send a `warning` and close the connection. - MUST NOT process the message further. - if the specified `chain_hash` value is unknown (meaning it isn't active on the specified chain): - MUST ignore the channel update. - if the `timestamp` is equal to the last-received `channel_update` for this `short_channel_id` AND `node_id`: - if the fields below `timestamp` differ: - MAY blacklist this `node_id`. - MAY forget all channels associated with it. - if the fields below `timestamp` are equal: - SHOULD ignore this message - if `timestamp` is lower than that of the last-received `channel_update` for this `short_channel_id` AND for `node_id`: - SHOULD ignore the message. - otherwise: - if the `timestamp` is unreasonably far in the future: - MAY discard the `channel_update`. - otherwise: - SHOULD queue the message for rebroadcasting. - MAY choose NOT to for messages longer than the minimum expected length. - if `htlc_maximum_msat` < `htlc_minimum_msat`: - SHOULD ignore this channel during route considerations. - if `htlc_maximum_msat` is greater than channel capacity: - MAY blacklist this `node_id` - SHOULD ignore this channel during route considerations. - otherwise: - SHOULD consider the `htlc_maximum_msat` when routing.

Rationale

The `timestamp` field is used by nodes for pruning `channel_updates` that are either too far in the future or have not been updated in two weeks; so it makes sense to have it be a UNIX timestamp (i.e. seconds since UTC 1970-01-01). This cannot be a hard requirement, however, given the possible case of two `channel_updates` within a single second.

It is assumed that more than one `channel_update` message changing the channel parameters in the same second may be a DoS attempt, and therefore, the node responsible for signing such messages may be blacklisted. However, a node may send a same `channel_update` message with a different signature (changing the nonce in signature signing), and hence fields apart from signature are checked to see if the channel parameters have changed for the same timestamp. It is also important to note that ECDSA signatures are malleable. So, an intermediate node who received the `channel_update` message can rebroadcast it just by

changing the `s` component of signature with `-s`. This should however not result in the blacklist of the `node_id` from where the message originated.

The recommendation against redundant `channel_update`s minimizes spamming the network, however it is sometimes inevitable. For example, a channel with a peer which is unreachable will eventually cause a `channel_update` to indicate that the channel is disabled, with another update re-enabling the channel when the peer reestablishes contact. Because gossip messages are batched and replace previous ones, the result may be a single seemingly-redundant update.

When a node creates a new `channel_update` to change its channel parameters, it will take some time to propagate through the network and payers may use older parameters. It is recommended to keep accepting older parameters for at least 10 minutes to improve payment latency and reliability.

The `must_be_one` field in `message_flags` was previously used to indicate the presence of the `htlc_maximum_msat` field. This field must now always be present, so `must_be_one` is a constant value, and ignored by receivers.

Query Messages

Understanding of messages used to be indicated with the `gossip_queries` feature bit; now these messages are universally supported, that feature has now been slightly repurposed. Not offering this feature means a node is not worth querying for gossip: either they do not store the entire gossip map, or they are only connected to a single peer (this one).

There are several messages which contain a long array of `short_channel_id`s (called `encoded_short_ids`) so we include an encoding byte which allows for different encoding schemes to be defined in the future, if they provide benefit.

Encoding types: * `0`: uncompressed array of `short_channel_id` types, in ascending order. * `1`: Previously used for zlib compression, this encoding MUST NOT be used.

This encoding is also used for arrays of other types (timestamps, flags, …), and specified with an `encoded_` prefix. For example, `encoded_timestamps` is an array of timestamps with a `0` prefix.

Query messages can be extended with optional fields that can help reduce the number of messages needed to synchronize routing tables by enabling:

- timestamp-based filtering of `channel_update` messages: only ask for `channel_update` messages that are newer than the ones you already have.
- checksum-based filtering of `channel_update` messages: only ask for `channel_update` messages that carry different information from the ones you already have.

Nodes can signal that they support extended gossip queries with the `gossip_queries_ex` feature bit.

The `query_short_channel_ids`/`reply_short_channel_ids_end` Messages

1. type: 261 (`query_short_channel_ids`)
2. data:
    - [`chain_hash`:`chain_hash`]
    - [`u16`:`len`]

        - [len*byte:encoded_short_ids]
        - [query_short_channel_ids_tlvs:tlvs]
    3. tlv_stream: query_short_channel_ids_tlvs
    4. types:
        1. type: 1 (query_flags)
        2. data:
            - [byte:encoding_type]
            - [...*byte:encoded_query_flags]

encoded_query_flags is an array of bitfields, one bigsize per bitfield, one bitfield for each short_channel_id. Bits have the following meaning:

Bit Position	Meaning
0	Sender wants channel_announcement
1	Sender wants channel_update for node 1
2	Sender wants channel_update for node 2
3	Sender wants node_announcement for node 1
4	Sender wants node_announcement for node 2

Query flags must be minimally encoded, which means that one flag will be encoded with a single byte.

1. type: 262 (reply_short_channel_ids_end)
2. data:
    - [chain_hash:chain_hash]
    - [byte:full_information]

This is a general mechanism which lets a node query for the channel_announcement and channel_update messages for specific channels (identified via short_channel_ids). This is usually used either because a node sees a channel_update for which it has no channel_announcement or because it has obtained previously unknown short_channel_ids from reply_channel_range.

Requirements

The sender: - SHOULD NOT send this to a peer which does not offer gossip_queries. - MUST NOT send query_short_channel_ids if it has sent a previous query_short_channel_ids to this peer and not received reply_short_channel_ids_end. - MUST set chain_hash to the 32-byte hash that uniquely identifies the chain that the short_channel_ids refer to. - MUST set the first byte of encoded_short_ids to the encoding type. - MUST encode a whole number of short_channel_ids to encoded_short_ids - MAY send this if it receives a channel_update for a short_channel_id for which it has no channel_announcement. - SHOULD NOT send this if the channel referred to is not an unspent output. - MAY include an optional query_flags. If so: - MUST set encoding_type, as for encoded_short_ids. - Each query flag is a minimally-encoded bigsize. - MUST encode one query flag per short_channel_id.

The receiver: - if the first byte of encoded_short_ids is not a known encoding type: - MAY send a warning. - MAY close the connection. - if encoded_short_ids does not decode into a whole number of short_channel_id: - MAY send a warning. - MAY close the connection. - if it has not sent reply_short_channel_ids_end to a previously received query_short_channel_ids from this sender: - MAY send a warning. - MAY close the

connection. - if the incoming message includes `query_short_channel_ids_tlvs`: - if `encoding_type` is not a known encoding type: - MAY send a `warning`. - MAY close the connection. - if `encoded_query_flags` does not decode to exactly one flag per `short_channel_id`: - MAY send a `warning`. - MAY close the connection. - MUST respond to each known `short_channel_id`: - if the incoming message does not include `encoded_query_flags`: - with a `channel_announcement` and the latest `channel_update` for each end - MUST follow with any `node_announcements` for each `channel_announcement` - otherwise: - We define `query_flag` for the Nth `short_channel_id` in `encoded_short_ids` to be the Nth bigsize of the decoded `encoded_query_flags`. - if bit 0 of `query_flag` is set: - MUST reply with a `channel_announcement` - if bit 1 of `query_flag` is set and it has received a `channel_update` from `node_id_1`: - MUST reply with the latest `channel_update` for `node_id_1` - if bit 2 of `query_flag` is set and it has received a `channel_update` from `node_id_2`: - MUST reply with the latest `channel_update` for `node_id_2` - if bit 3 of `query_flag` is set and it has received a `node_announcement` from `node_id_1`: - MUST reply with the latest `node_announcement` for `node_id_1` - if bit 4 of `query_flag` is set and it has received a `node_announcement` from `node_id_2`: - MUST reply with the latest `node_announcement` for `node_id_2` - SHOULD NOT wait for the next outgoing gossip flush to send these. - SHOULD avoid sending duplicate `node_announcements` in response to a single `query_short_channel_ids`. - MUST follow these responses with `reply_short_channel_ids_end`. - if does not maintain up-to-date channel information for `chain_hash`: - MUST set `full_information` to 0. - otherwise: - SHOULD set `full_information` to 1.

Rationale

Future nodes may not have complete information; they certainly won't have complete information on unknown `chain_hash` chains. While this `full_information` field (previously and confusingly called `complete`) cannot be trusted, a 0 does indicate that the sender should search elsewhere for additional data.

The explicit `reply_short_channel_ids_end` message means that the receiver can indicate it doesn't know anything, and the sender doesn't need to rely on timeouts. It also causes a natural ratelimiting of queries.

The `query_channel_range` and `reply_channel_range` Messages

1. type: 263 (`query_channel_range`)
2. data:
    - [`chain_hash`:`chain_hash`]
    - [`u32`:`first_blocknum`]
    - [`u32`:`number_of_blocks`]
    - [`query_channel_range_tlvs`:`tlvs`]
3. `tlv_stream`: `query_channel_range_tlvs`
4. types:
    1. type: 1 (`query_option`)
    2. data:
        - [`bigsize`:`query_option_flags`]

`query_option_flags` is a bitfield represented as a minimally-encoded bigsize. Bits have the following meaning:

Bit Position	Meaning
0	Sender wants timestamps
1	Sender wants checksums

Though it is possible, it would not be very useful to ask for checksums without asking for timestamps too: the receiving node may have an older `channel_update` with a different checksum, asking for it would be useless. And if a `channel_update` checksum is actually 0 (which is quite unlikely) it will not be queried.

1. type: 264 (`reply_channel_range`)
2. data:
    - [`chain_hash:chain_hash`]
    - [`u32:first_blocknum`]
    - [`u32:number_of_blocks`]
    - [`byte:sync_complete`]
    - [`u16:len`]
    - [`len*byte:encoded_short_ids`]
    - [`reply_channel_range_tlvs:tlvs`]
3. `tlv_stream`: `reply_channel_range_tlvs`
4. types:
    1. type: 1 (`timestamps_tlv`)
    2. data:
        - [`byte:encoding_type`]
        - [`...*byte:encoded_timestamps`]
    3. type: 3 (`checksums_tlv`)
    4. data:
        - [`...*channel_update_checksums:checksums`]

For a single `channel_update`, timestamps are encoded as:

1. subtype: `channel_update_timestamps`
2. data:
    - [`u32:timestamp_node_id_1`]
    - [`u32:timestamp_node_id_2`]

Where: * `timestamp_node_id_1` is the timestamp of the `channel_update` for `node_id_1`, or 0 if there was no `channel_update` from that node. * `timestamp_node_id_2` is the timestamp of the `channel_update` for `node_id_2`, or 0 if there was no `channel_update` from that node.

For a single `channel_update`, checksums are encoded as:

1. subtype: `channel_update_checksums`
2. data:
    - [`u32:checksum_node_id_1`]
    - [`u32:checksum_node_id_2`]

Where: * `checksum_node_id_1` is the checksum of the `channel_update` for `node_id_1`, or 0 if there was no `channel_update` from that node. * `checksum_node_id_2` is the checksum of the `channel_update` for `node_id_2`, or 0 if there was no `channel_update` from that node.

The checksum of a `channel_update` is the CRC32C checksum as specified in RFC3720 of this `channel_update` without its `signature` and `timestamp` fields.

This allows querying for channels within specific blocks.

Requirements

The sender of `query_channel_range`: - SHOULD NOT send this to a peer which does not offer `gossip_queries`. - MUST NOT send this if it has sent a previous `query_channel_range` to this peer and not received all `reply_channel_range` replies. - MUST set `chain_hash` to the 32-byte hash that uniquely identifies the chain that it wants the `reply_channel_range` to refer to - MUST set `first_blocknum` to the first block it wants to know channels for - MUST set `number_of_blocks` to 1 or greater. - MAY append an additional `query_channel_range_tlv`, which specifies the type of extended information it would like to receive.

The receiver of `query_channel_range`: - if it has not sent all `reply_channel_range` to a previously received `query_channel_range` from this sender: - MAY send a `warning`. - MAY close the connection. - MUST respond with one or more `reply_channel_range`: - MUST set with `chain_hash` equal to that of `query_channel_range`, - MUST limit `number_of_blocks` to the maximum number of blocks whose results could fit in `encoded_short_ids` - MAY split block contents across multiple `reply_channel_range`. - the first `reply_channel_range` message: - MUST set `first_blocknum` less than or equal to the `first_blocknum` in `query_channel_range` - MUST set `first_blocknum` plus `number_of_blocks` greater than `first_blocknum` in `query_channel_range`. - successive `reply_channel_range` message: - MUST have `first_blocknum` equal or greater than the previous `first_blocknum`. - MUST set `sync_complete` to `false` if this is not the final `reply_channel_range`. - the final `reply_channel_range` message: - MUST have `first_blocknum` plus `number_of_blocks` equal or greater than the `query_channel_range` `first_blocknum` plus `number_of_blocks`. - MUST set `sync_complete` to `true`.

If the incoming message includes `query_option`, the receiver MAY append additional information to its reply: - if bit 0 in `query_option_flags` is set, the receiver MAY append a `timestamps_tlv` that contains `channel_update` timestamps for all `short_channel_ids` in `encoded_short_ids` - if bit 1 in `query_option_flags` is set, the receiver MAY append a `checksums_tlv` that contains `channel_update` checksums for all `short_channel_ids` in `encoded_short_ids`

Rationale

A single response might be too large for a single packet, so multiple replies may be required. We want to allow a peer to store canned results for (say) 1000-block ranges, so replies can exceed the requested range. However, we require that each reply be relevant (overlapping the requested range).

By insisting that replies be in increasing order, the receiver can easily determine if replies are done: simply check if `first_blocknum` plus `number_of_blocks` equals or exceeds the `first_blocknum` plus `number_of_blocks` it asked for.

The addition of timestamp and checksum fields allow a peer to omit querying for redundant updates.

The `gossip_timestamp_filter` Message

1. type: 265 (`gossip_timestamp_filter`)
2. data:
    - [`chain_hash:chain_hash`]
    - [`u32:first_timestamp`]
    - [`u32:timestamp_range`]

This message allows a node to constrain future gossip messages to a specific range. A node which wants any gossip messages has to send this, otherwise no gossip messages would be received.

Note that this filter replaces any previous one, so it can be used multiple times to change the gossip from a peer.

Requirements

The sender: - MUST set `chain_hash` to the 32-byte hash that uniquely identifies the chain that it wants the gossip to refer to. - If the receiver does not offer `gossip_queries`: - SHOULD set `first_timestamp` to 0xFFFFFFFF and `timestamp_range` to 0.

The receiver: - SHOULD send all gossip messages whose `timestamp` is greater or equal to `first_timestamp`, and less than `first_timestamp` plus `timestamp_range`. - MAY wait for the next outgoing gossip flush to send these. - SHOULD send gossip messages as it generates them regardless of `timestamp`. - Otherwise (relayed gossip): - SHOULD restrict future gossip messages to those whose `timestamp` is greater or equal to `first_timestamp`, and less than `first_timestamp` plus `timestamp_range`. - If a `channel_announcement` has no corresponding `channel_updates`: - MUST NOT send the `channel_announcement`. - If the funding output of the `channel_announcement` has been spent: - SHOULD NOT send the `channel_announcement`. - Otherwise: - MUST consider the `timestamp` of the `channel_announcement` to be the `timestamp` of a corresponding `channel_update`. - MUST consider whether to send the `channel_announcement` after receiving the first corresponding `channel_update`. - If a `channel_announcement` is sent: - MUST send the `channel_announcement` prior to any corresponding `channel_updates` and `node_announcements`.

Rationale

Since `channel_announcement` doesn't have a timestamp, we generate a likely one. If there's no `channel_update` then it is not sent at all, which is most likely in the case of pruned channels.

Otherwise the `channel_announcement` is usually followed immediately by a `channel_update`. Ideally we would specify that the first (oldest) `channel_update`'s timestamp is to be used as the time of the `channel_announcement`, but new nodes on the network will not have this, and further would require the first `channel_update` timestamp to be stored. Instead, we allow any update to be used, which is simple to implement.

In the case where the `channel_announcement` is nonetheless missed, `query_short_channel_ids` can be used to retrieve it.

Nodes can use `timestamp_filter` to reduce their gossip load when they have many peers (eg. setting `first_timestamp` to `0xFFFFFFFF` after the first few peers, in the assumption that propagation is adequate). This assumption of adequate propagation does not apply for gossip messages generated directly by the node itself, so they should ignore filters.

Requirements

A node: - MUST NOT relay any gossip messages it did not generate itself, unless explicitly requested.

Rebroadcasting

Requirements

A receiving node: - upon receiving a new `channel_announcement` or a `channel_update` or `node_announcement` with an updated `timestamp`: - SHOULD update its local view of the network's topology accordingly. - after applying the changes from the announcement: - if there are no channels associated with the corresponding origin node: - MAY purge the origin node from its set of known nodes. - otherwise: - SHOULD update the appropriate metadata AND store the signature associated with the announcement. - Note: this will later allow the node to rebuild the announcement for its peers.

A node: - MUST not send gossip it did not generate itself, until it receives `gossip_timestamp_filter`. - SHOULD flush outgoing gossip messages once every 60 seconds, independently of the arrival times of the messages. - Note: this results in staggered announcements that are unique (not duplicated). - SHOULD NOT forward gossip messages to peers who sent `networks` in `init` and did not specify the `chain_hash` of this gossip message. - MAY re-announce its channels regularly. - Note: this is discouraged, in order to keep the resource requirements low.

Rationale

Once the gossip message has been processed, it's added to a list of outgoing messages, destined for the processing node's peers, replacing any older updates from the origin node. This list of gossip messages will be flushed at regular intervals; such a store-and-delayed-forward broadcast is called a *staggered broadcast*. Also, such batching forms a natural rate limit with low overhead.

HTLC Fees

Requirements

The origin node: - SHOULD accept HTLCs that pay a fee equal to or greater than: - fee_base_msat + (amount_to_forward * fee_proportional_millionths / 1000000) - SHOULD accept HTLCs that pay an older fee, for some reasonable time after sending `channel_update`. - Note: this allows for any propagation delay.

Pruning the Network View

Requirements

A node: - SHOULD monitor the funding transactions in the blockchain, to identify channels that are being closed. - if the funding output of a channel is spent and received 72 block confirmations: - SHOULD be removed from the local network view AND be considered closed. - if the announced node no longer has any associated open channels: - MAY prune nodes added through `node_announcement` messages from their local view. - Note: this is a direct result of the dependency of a `node_announcement` being preceded by a `channel_announcement`.

Recommendation on Pruning Stale Entries

Requirements

A node: - if the `timestamp` of the latest `channel_update` in either direction is older than two weeks (1209600 seconds): - MAY prune the channel. - MAY ignore the channel. - Note: this is an individual node policy and MUST NOT be enforced by forwarding peers, e.g. by closing channels when receiving outdated gossip messages.

Rationale

Several scenarios may result in channels becoming unusable and its endpoints becoming unable to send updates for these channels. For example, this occurs if both endpoints lose access to their private keys and can neither sign `channel_updates` nor close the channel on-chain. In this case, the channels are unlikely to be part of a computed route, since they would be partitioned off from the rest of the network; however, they would remain in the local network view would be forwarded to other peers indefinitely.

The oldest `channel_update` is used to prune the channel since both sides need to be active in order for the channel to be usable. Doing so prunes channels even if one side continues to send fresh `channel_updates` but the other node has disappeared.

Recommendations for Routing

When calculating a route for an HTLC, both the `cltv_expiry_delta` and the fee need to be considered: the `cltv_expiry_delta` contributes to the time that funds will be unavailable in the event of a worst-case failure. The relationship between these two attributes is unclear, as it depends on the reliability of the nodes involved.

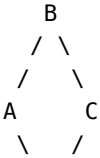
If a route is computed by simply routing to the intended recipient and summing the `cltv_expiry_delta`s, then it's possible for intermediate nodes to guess their position in the route. Knowing the CLTV of the HTLC, the surrounding network topology, and the `cltv_expiry_delta`s gives an attacker a way to guess the intended recipient. Therefore, it's highly desirable to add a random offset to the CLTV that the intended recipient will receive, which bumps all CLTVs along the route.

In order to create a plausible offset, the origin node MAY start a limited random walk on the graph, starting from the intended recipient and summing the `cltv_expiry_delta`s, and use the resulting sum as the offset. This effectively creates a *shadow route extension* to the actual route and provides better protection against this attack vector than simply picking a random offset would.

Other more advanced considerations involve diversification of route selection, to avoid single points of failure and detection, and balancing of local channels.

Routing Example

Consider four nodes:



```
\ /
 D
```

Each advertises the following `cltv_expiry_delta` on its end of every channel:

1. A: 10 blocks
2. B: 20 blocks
3. C: 30 blocks
4. D: 40 blocks

C also uses a `min_final_cltv_expiry_delta` of 18 (the default) when requesting payments.

Also, each node has a set fee scheme that it uses for each of its channels:

1. A: 100 base + 1000 millionths
2. B: 200 base + 2000 millionths
3. C: 300 base + 3000 millionths
4. D: 400 base + 4000 millionths

The network will see eight `channel_update` messages:

1. A->B: `cltv_expiry_delta` = 10, `fee_base_msat` = 100, `fee_proportional_millionths` = 1000
2. A->D: `cltv_expiry_delta` = 10, `fee_base_msat` = 100, `fee_proportional_millionths` = 1000
3. B->A: `cltv_expiry_delta` = 20, `fee_base_msat` = 200, `fee_proportional_millionths` = 2000
4. D->A: `cltv_expiry_delta` = 40, `fee_base_msat` = 400, `fee_proportional_millionths` = 4000
5. B->C: `cltv_expiry_delta` = 20, `fee_base_msat` = 200, `fee_proportional_millionths` = 2000
6. D->C: `cltv_expiry_delta` = 40, `fee_base_msat` = 400, `fee_proportional_millionths` = 4000
7. C->B: `cltv_expiry_delta` = 30, `fee_base_msat` = 300, `fee_proportional_millionths` = 3000
8. C->D: `cltv_expiry_delta` = 30, `fee_base_msat` = 300, `fee_proportional_millionths` = 3000

B->C. If B were to send 4,999,999 millisatoshi directly to C, it would neither charge itself a fee nor add its own `cltv_expiry_delta`, so it would use C's requested `min_final_cltv_expiry_delta` of 18. Presumably it would also add a *shadow route* to give an extra CLTV of 42. Additionally, it could add extra CLTV deltas at other hops, as these values represent a minimum, but chooses not to do so here, for the sake of simplicity:

- `amount_msat`: 4999999
- `cltv_expiry`: current-block-height + 18 + 42
- `onion_routing_packet`:
    - `amt_to_forward` = 4999999
    - `outgoing_cltv_value` = current-block-height + 18 + 42

A->B->C. If A were to send 4,999,999 millisatoshi to C via B, it needs to pay B the fee it specified in the B->C `channel_update`, calculated as per HTLC Fees:

```
fee_base_msat + ( amount_to_forward * fee_proportional_millionths / 1000000 )

200 + ( 4999999 * 2000 / 1000000 ) = 10199
```

Similarly, it would need to add B->C's `channel_update` `cltv_expiry_delta` (20), C's requested `min_final_cltv_expiry_delta` (18), and the cost for the *shadow route* (42). Thus, A->B's `update_add_htlc` message would be:

- `amount_msat`: 5010198
- `cltv_expiry`: current-block-height + 20 + 18 + 42
- `onion_routing_packet`:
    - `amt_to_forward` = 4999999
    - `outgoing_cltv_value` = current-block-height + 18 + 42

B->C's `update_add_htlc` would be the same as B->C's direct payment above.

A->D->C. Finally, if for some reason A chose the more expensive route via D, A->D's `update_add_htlc` message would be:

- `amount_msat`: 5020398
- `cltv_expiry`: current-block-height + 40 + 18 + 42
- `onion_routing_packet`:
    - `amt_to_forward` = 4999999
    - `outgoing_cltv_value` = current-block-height + 18 + 42

And D->C's `update_add_htlc` would again be the same as B->C's direct payment above.

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

Paying People

Lightning payments are still HTLCs under the hood, but humans don't paste onion packets into a wallet. They scan invoices, or they fetch offers.

BOLT 11 is the classic invoice. It's a bech32 string (`lnbc...` on mainnet) that encodes amount, payment hash, expiry, a description, a fallback on-chain address, and routing hints for unannounced channels. Almost every Lightning wallet you've used is speaking BOLT 11.

BOLT 12 is offers. Instead of a one-shot invoice, a receiver publishes a reusable offer. The payer requests an invoice over the Lightning network itself (onion messages), gets a fresh invoice back, and pays it. Offers can be static, they can request payer information, and they can be used for refunds. It's the more recent, more flexible cousin of BOLT 11.

If you're building a merchant flow, start with BOLT 11 — it's everywhere — then read BOLT 12 to see where the protocol is going.

In this chapter

- BOLT 11 — Invoice protocol
- BOLT 12 — Offers

BOLT #11: Invoice Protocol for Lightning Payments

A simple, extendable, QR-code-ready protocol for requesting payments over Lightning.

Table of Contents

- Encoding Overview
- Human-Readable Part
- Data Part
    - Tagged Fields
    - Feature Bits
- Payer / Payee Interactions
    - Payer / Payee Requirements
- Implementation
- Examples
- Authors

Encoding Overview

The format for a Lightning invoice uses bech32 encoding, which is already used for Bitcoin Segregated Witness. It can be simply reused for Lightning invoices even though its 6-character checksum is optimized for manual entry, which is unlikely to happen often given the length of Lightning invoices.

If a URI scheme is desired, the current recommendation is to either use 'lightning:' as a prefix before the BOLT-11 encoding (note: not 'lightning://'), or for fallback to Bitcoin payments, to use 'bitcoin:', as per BIP-21, with the key 'lightning' and the value equal to the BOLT-11 encoding.

Requirements

A writer: - MUST encode the payment request in Bech32 (see BIP-0173) - SHOULD use upper case for QR codes (see BIP-0173) - MAY exceed the 90-character limit specified in BIP-0173.

A reader: - MUST parse the address as Bech32, as specified in BIP-0173 (also without the character limit). - Note: this includes handling uppercase as specified by BIP-0173 - if the checksum is incorrect: - MUST fail the payment.

Human-Readable Part

The human-readable part of a Lightning invoice consists of two sections: 1. `prefix`: `ln` + BIP-0173 currency prefix (e.g. `lnbc` for Bitcoin mainnet, `lntb` for Bitcoin testnet, `lntbs` for Bitcoin signet, and `lnbcrt` for Bitcoin regtest) 1. `amount`: optional number in that currency, followed by an optional `multiplier` letter. The unit encoded here is the 'social' convention of a payment unit – in the case of Bitcoin the unit is 'bitcoin' NOT satoshis.

The following `multiplier` letters are defined:

- `m` (milli): multiply by 0.001
- `u` (micro): multiply by 0.000001

- `n` (nano): multiply by 0.000000001
- `p` (pico): multiply by 0.000000000001

Requirements

A writer: - MUST encode `prefix` using the currency required for successful payment. - if a specific minimum `amount` is required for successful payment: - MUST include that `amount`. - MUST encode `amount` as a positive decimal integer with no leading 0s. - If the `p` multiplier is used the last decimal of `amount` MUST be `0`. - SHOULD use the shortest representation possible, by using the largest multiplier or omitting the multiplier.

A reader: - if it does NOT understand the `prefix`: - MUST fail the payment. - if the `amount` is empty: - SHOULD indicate to the payer that amount is unspecified. - otherwise: - if `amount` contains a non-digit OR is followed by anything except a `multiplier` (see table above): - MUST fail the payment. - if the `multiplier` is present: - MUST multiply `amount` by the `multiplier` value to derive the amount required for payment. - if multiplier is `p` and the last decimal of `amount` is not 0: - MUST fail the payment.

Rationale

The `amount` is encoded into the human readable part, as it's fairly readable and a useful indicator of how much is being requested.

Donation addresses often don't have an associated amount, so `amount` is optional in that case. Usually a minimum payment is required for whatever is being offered in return.

The `p` multiplier would allow to specify sub-millisatoshi amounts, which cannot be transferred on the network, since HTLCs are denominated in millisatoshis. Requiring a trailing `0` decimal ensures that the `amount` represents an integer number of millisatoshis.

Note that non-largest multipliers have been encountered in the wild, and as such invoice parsers should handle them.

Data Part

The data part of a Lightning invoice consists of multiple sections:

1. `timestamp`: seconds-since-1970 (35 bits, big-endian)
2. zero or more tagged parts
3. `signature`: compact ECDSA/secp256k1 signature of the above (520 bits: 64-byte R||S + 1-byte recovery id)

Requirements

A writer: - MUST set `timestamp` to the number of seconds since Midnight 1 January 1970, UTC in big-endian. - MUST set `signature` to a valid compact ECDSA signature over secp256k1 of the SHA-256 hash of: the human-readable part (as UTF-8 bytes) concatenated with the data part (excluding the signature), with 0 bits appended to pad to a byte boundary. The signature is encoded as 64 bytes (R || S), followed by a trailing 1-byte recovery id in {0,1,2,3}.

A reader: - MUST check that the `signature` is valid (see the `n` tagged field specified below).

Rationale

`signature` covers an exact number of bytes even though the SHA2 standard actually supports hashing in bit boundaries, because it's not widely implemented. The recovery ID allows public-key recovery, so the identity of the payee node can be implied.

Tagged Fields

Each Tagged Field is of the form:

1. `type` (5 bits)
2. `data_length` (10 bits, big-endian)
3. `data` (`data_length` x 5 bits)

Note that the maximum length of a Tagged Field's `data` is constricted by the maximum value of `data_length`. This is 1023 x 5 bits, or 639 bytes.

Currently defined tagged fields are:

- `p` (1): `data_length` 52. 256-bit SHA256 payment_hash. Preimage of this provides proof of payment.
- `s` (16): `data_length` 52. This 256-bit secret prevents forwarding nodes from probing the payment recipient.
- `d` (13): `data_length` variable. Short description of purpose of payment (UTF-8), e.g. '1 cup of coffee' or 'ナンセンス 1杯'
- `m` (27): `data_length` variable. Additional metadata to attach to the payment. Note that the size of this field is limited by the maximum hop payload size. Long metadata fields reduce the maximum route length.
- `n` (19): `data_length` 53. 33-byte public key of the payee node
- `h` (23): `data_length` 52. 256-bit description of purpose of payment (SHA256). This is used to commit to an associated description that is over 639 bytes, but the transport mechanism for the description in that case is transport specific and not defined here.
- `x` (6): `data_length` variable. `expiry` time in seconds (big-endian). Default is 3600 (1 hour) if not specified.
- `c` (24): `data_length` variable. `min_final_cltv_expiry_delta` to use for the last HTLC in the route. Default is 18 if not specified.
- `f` (9): `data_length` variable, depending on version. Fallback on-chain address: for Bitcoin, this starts with a 5-bit `version` and contains a witness program or P2PKH or P2SH address.
- `r` (3): `data_length` variable. One or more entries containing extra routing information for a private route; there may be more than one `r` field
    - `pubkey` (264 bits)
    - `short_channel_id` (64 bits)
    - `fee_base_msat` (32 bits, big-endian)
    - `fee_proportional_millionths` (32 bits, big-endian)
    - `cltv_expiry_delta` (16 bits, big-endian)
- `9` (5): `data_length` variable. One or more 5-bit values containing features supported or required for receiving this payment. See Feature Bits.

Requirements

A writer: - MUST include exactly one `p` field. - MUST include exactly one `s` field. - MUST set `payment_hash` to the SHA2 256-bit hash of the `payment_preimage` that will be given in return for payment. - MUST include either exactly one `d` or exactly one `h` field. - if `d` is included: - MUST set `d` to a valid UTF-8 string. - SHOULD use a complete description of the purpose of the payment. - if `h` is included: - MUST make the preimage of the hashed description in `h` available through some unspecified means. - SHOULD use a complete description of the purpose of the payment. - MAY include one `x` field. - if `x` is included: - MUST use the minimum `data_length` possible, i.e. no leading 0 field-elements. - SHOULD include one `c` field (`min_final_cltv_expiry_delta`). - MUST set `c` to the minimum `cltv_expiry` it will accept for the last HTLC in the route. - MUST use the minimum `data_length` possible, i.e. no leading 0 field-elements. - MAY include one `n` field. (Otherwise performing public-key recovery is required) - MUST set `n` to the public key used to create the `signature`. - MAY include one or more `f` fields. - for Bitcoin payments: - MUST set an `f` field to a valid witness version and program, OR to `17` followed by a public key hash, OR to `18` followed by a script hash. - if there is NOT a public channel associated with its public key: - MUST include at least one `r` field. - `r` field MUST contain one or more ordered entries, indicating the forward route from a public node to the final destination. - Note: for each entry, the `pubkey` is the node ID of the start of the channel; `short_channel_id` is the short channel ID field to identify the channel; and `fee_base_msat`, `fee_proportional_millionths`, and `cltv_expiry_delta` are as specified in BOLT #7. - MAY include more than one `r` field to provide multiple routing options. - if `9` contains non-zero bits: - MUST use the minimum `data_length` possible to encode the non-zero bits with no 0 field-elements at the start. - otherwise: - MUST omit the `9` field altogether. - MUST pad field data to a multiple of 5 bits, using 0s. - if a writer offers more than one of any field type, it: - MUST specify the most-preferred field first, followed by less-preferred fields, in order.

A reader: - MUST skip over `f` fields that use an unknown `version`. - MUST fail the payment if any field with fixed `data_length` (`p`, `h`, `s`, `n`) does not have the correct length (52, 52, 52, 53). - MUST fail the payment if neither a `d` field nor a `h` field is present, or if both are present. - if the `9` field contains unknown *odd* bits that are non-zero: - MUST ignore the bit. - if the `9` field contains unknown *even* bits that are non-zero: - MUST fail the payment. - SHOULD indicate the unknown bit to the user. - MUST check that the SHA2 256-bit hash in the `h` field exactly matches the hashed description. - if a valid `n` field is provided: - MUST use the `n` field to validate the signature instead of performing public-key recovery. - If the signature is not compliant with the low-S standard rule[low-S]: - MUST fail the payment - otherwise: - MUST perform ECDSA public-key recovery and accept both high-S and low-S signatures. - if a valid `s` field is not provided: - MUST fail the payment. - otherwise: - MUST use the `s` field as `payment_secret` - if the `c` field (`min_final_cltv_expiry_delta`) is not provided: - MUST use an expiry delta of at least 18 when making the payment - if an `m` field is provided: - MUST use that as `payment_metadata` - if a `c`, `x`, or `9` field is provided which has a non-minimal `data_length` (i.e. begins with 0 field elements): - SHOULD treat the invoice as invalid.

Rationale

The type-and-length format allows future extensions to be backward compatible. `data_length` is always a multiple of 5 bits, for easy encoding and decoding.

The `d` field allows inline descriptions, but may be insufficient for complex orders. Thus, the `h` field allows a summary: though the method by which the description is served is as-yet unspecified and will probably be transport dependent.

The `m` field allows metadata to be attached to the payment. This supports applications where the recipient doesn't keep any context for the payment.

The `n` field can be used to explicitly specify the destination node ID, instead of requiring public-key recovery.

The `x` field gives warning as to when a payment will be refused: mainly to avoid confusion. The default was chosen to be reasonable for most payments and to allow sufficient time for on-chain payment, if necessary.

The `c` field allows a way for the destination node to require a specific minimum CLTV expiry for its incoming HTLC. Destination nodes may use this to require a higher, more conservative value than the default one (depending on their fee estimation policy and their sensitivity to time locks). Note that remote nodes in the route specify their required `cltv_expiry_delta` in the `channel_update` message, which they can update at all times.

The `f` field allows on-chain fallback; however, this may not make sense for tiny or time-sensitive payments. It's possible that new address forms will appear; thus, multiple `f` fields (in an implied preferred order) help with transition, and `f` fields with versions 19-31 will be ignored by readers.

The `r` field allows limited routing assistance: as specified, it only allows minimum information to use private channels, however, it could also assist in future partial-knowledge routing.

Security Considerations for Payment Descriptions

Payment descriptions are user-defined and provide a potential avenue for injection attacks: during the processes of both rendering and persistence.

Payment descriptions should always be sanitized before being displayed in HTML/Javascript contexts (or any other dynamically interpreted rendering frameworks). Implementers should be extra perceptive to the possibility of reflected XSS attacks, when decoding and displaying payment descriptions. Avoid optimistically rendering the contents of the payment request until all validation, verification, and sanitization processes have been successfully completed.

Furthermore, consider using prepared statements, input validation, and/or escaping, to protect against injection vulnerabilities in persistence engines that support SQL or other dynamically interpreted querying languages.

- Stored and Reflected XSS Prevention
- DOM-based XSS Prevention
- SQL Injection Prevention

Don't be like the school of Little Bobby Tables.

Feature Bits

Feature bits allow forward and backward compatibility, and follow the *it's ok to be odd* rule. Features appropriate for use in the `9` field are marked in BOLT 9.

The field is big-endian. The least-significant bit is numbered 0, which is *even*, and the next most significant bit is numbered 1, which is *odd*.

Requirements

A writer: - MUST set the `9` field to a feature vector compliant with the BOLT 9 origin node requirements.

A reader: - if the feature vector does not set all known, transitive feature dependencies: - MUST NOT attempt the payment. - if the `basic_mpp` feature is offered in the invoice: - MAY pay using Basic multi-part payments. - otherwise: - MUST NOT use Basic multi-part payments.

Payer / Payee Interactions

These are generally defined by the rest of the Lightning BOLT series, but it's worth noting that BOLT #4 specifies that the payee SHOULD accept up to twice the expected `amount`, so the payer can make payments harder to track by adding small variations.

The intent is that the payer recovers the payee's node ID from the signature, and after checking that conditions such as fees, expiry, and block timeout are acceptable, attempts a payment. It can use `r` fields to augment its routing information, if necessary to reach the final node.

If the payment succeeds but there is a later dispute, the payer can prove both the signed offer from the payee and the successful payment.

Payer / Payee Requirements

A payer: - after the `timestamp` plus `expiry` has passed: - SHOULD NOT attempt a payment. - otherwise: - if a Lightning payment fails: - MAY attempt to use the address given in the first `f` field that it understands for payment. - MAY use the sequence of channels, specified by the `r` field, to route to the payee. - SHOULD consider the fee amount and payment timeout before initiating payment. - SHOULD use the first `p` field as the payment hash.

A payee: - after the `timestamp` plus `expiry` has passed: - SHOULD NOT accept a payment.

Implementation

https://github.com/rustyrussell/lightning-payencode

Examples

NB: all the following examples are signed with `priv_key=e126f68f7eafcc8b74f54d269fe206be715000f94dac067d1c04a8ca3b2db734`. All invoices contain a `payment_secret=1111111111111111111111111111111111111111111111111111111111111111` unless otherwise noted. Signatures are deterministic and generated using RFC6979 (using HMAC-SHA256). Note that even though using a `low R` would save 1 byte in the DER-encoded signature (by avoiding the need for a leading zero byte when the most significant bit is set), it is not enforced in this specification.

> **Please make a donation of any amount using payment_hash 0001020304050607080900010203040506070809000102030405060708090102 to me @03e7156ae33b0a208d0744199163177e909e80176e55d97a2f221ede0f934dd9ad**
>
> lnbc1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwz

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret
  - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
  - `zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygs`: payment secret 1111111111111111111111111111111111111111111111111111111111111111
- `p`: payment hash
  - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
  - `qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypq`: payment hash 0001020304050607080900010203040506070809000102030405060708090102
- `d`: short description
  - `pl`: `data_length` (`p` = 1, `l` = 31; 1 * 32 + 31 == 63)
  - `2pkx2ctnv5sxxmmwwd5kgetjypeh2ursdae8g6twvus8g6rfwvs8qun0dfjkxaq`: 'Please consider supporting this project'
- `9`: features
  - `qr`: `data_length` (`q` = 0, `r` = 3; 0 * 32 + 3 == 3)
  - `sgq`: b100000100000000
- `357wnc5r2ueh7ck6q93dj32dlqnls087fxdwk8qakdyafkq3yap9us6v52vjjsrvywa6rt52cm9r9zqt8r2t7mlcwspyetp5h2tztu` signature
- `9lfyql`: Bech32 checksum
- Signature breakdown:
  - `8d3ce9e28357337f62da0162d9454df827f83cfe499aeb1c1db349d4d81127425e434ca29929406c23bba1ae8ac6ca32` hex of signature data (32-byte r, 32-byte s)
  - `1` (int) recovery flag contained in `signature`
  - `6c6e62630b25fe64500d04444444444444444444444444444444444444444444444444444444444444444444444444021a0000810` hex of data for signing (prefix + data after separator up to the start of the signature)
  - `6daf4d488be41ce7cbb487cab1ef2975e5efcea879b20d421f0ef86b07cbb987` hex of SHA256 of the preimage

Please send $3 for a cup of coffee to the same peer, within one minute

lnbc2500u1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `2500u`: amount (2500 micro-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `p`: payment hash…
- `d`: short description
  - `q5`: `data_length` (`q` = 0, `5` = 20; 0 * 32 + 20 == 20)
  - `xysxxatsyp3k7enxv4js`: '1 cup coffee'

- x: expiry time
    - qz: data_length (q = 0, z = 2; 0 * 32 + 2 == 2)
    - pu: 60 seconds (p = 1, u = 28; 1 * 32 + 28 == 60)
- 9: features
    - qr: data_length (q = 0, r = 3; 0 * 32 + 3 == 3)
    - sgq: b100000100000000
- uk0rl77nj30yxdy8j9vdx85fkpmdla2087ne0xh8nhedh8w27kyke0lp53ut353s06fv3qfegext0eh0ymjpf39tuven09sam30g4v signature
- fna3rh: Bech32 checksum
- Signature breakdown:
    - e59e3ffbd3945e4334879158d31e89b076dff54f3fa7979ae79df2db9dcaf5896cbfe1a478b8d2307e92c88139464cb7 hex of signature data (32-byte r, 32-byte s)
    - 1 (int) recovery flag contained in signature
    - 6c6e626332353030750b25fe64500d044444444444444444444444444444444444444444444444444444444444444444444444444440 hex of data for signing (prefix + data after separator up to the start of the signature)
    - 047e24bf270b25d42a56d57b2578faa3a10684641bab817c2851a871cb41dbc0 hex of SHA256 of the preimage

Please send 0.0025 BTC for a cup of nonsense (ナンセンス 1杯) to the same peer, within one minute

lnbc2500u1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq5

Breakdown:

- lnbc: prefix, Lightning on Bitcoin mainnet
- 2500u: amount (2500 micro-bitcoin)
- 1: Bech32 separator
- pvjluez: timestamp (1496314658)
- s: payment secret…
- p: payment hash…
- d: short description
    - pq: data_length (p = 1, q = 0; 1 * 32 + 0 == 32)
    - uwpc4curk03c9wlrswe78q4eyqc7d8d0: 'ナンセンス 1杯'
- x: expiry time
    - qz: data_length (q = 0, z = 2; 0 * 32 + 2 == 2)
    - pu: 60 seconds (p = 1, u = 28; 1 * 32 + 28 == 60)
- 9: features
    - qr: data_length (q = 0, r = 3; 0 * 32 + 3 == 3)
    - sgq: b100000100000000
- htjpauu9ur7fw2thcl4y9vfvh4m9wlfyz2gem29g5ghe2aak2pm3ps8fdhtceqsaagty2vph7utlgj48u0ged6a337aewvraedends signature
- 573dxr: Bech32 checksum
- Signature breakdown:
    - bae41ef385e0fc972977c7ea42b12cbd76577d2412919da8a8a22f9577b6507710c0e96dd78c821dea16453037f717f4 hex of signature data (32-byte r, 32-byte s)
    - 1 (int) recovery flag contained in signature

    - `6c6e626332353030750b25fe64500d044444444444444444444444444444444444444444444444444444444444444444444444444444444444444440` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `f140d992ba419578ba9cfe1af85f92df90a76f442fb5e6e09b1f0582534ba87d` hex of SHA256 of the preimage

Now send $24 for an entire list of things (hashed)

lnbc20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq5r

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `p`: payment hash…
- `h`: tagged field: hash of description
    - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
    - `8yjmdan79s6qqdhdzgynm4zwqd5d7xmw5fk98klysy043l2ahrqs`: SHA256 of 'One piece of chocolate cake, one icecream cone, one pickle, one slice of swiss cheese, one slice of salami, one lollypop, one piece of cherry pie, one sausage, one cupcake, and one slice of watermelon' (SHA256 hex: `3925b6f67e2c340036ed12093dd44e0368df1b6ea26c53dbe4811f58fd5db8c1`)
- `9`: features
    - `qr`: `data_length` (`q` = 0, `r` = 3; 0 * 32 + 3 == 3)
    - `sgq`: b100000100000000
- `7ea976txfraylvgzuxs8kgcw23ezlrszfnh8r6qtfpr6cxga50aj6txm9rxrydzd06dfeawfk6swupvz4erwnyutnjq7x39ymw6j38` signature
- `7ynn44`: Bech32 checksum
- Signature breakdown:
    - `f67a5f696648fa4fb102e1a07b230e54722f8e024cee71e80b4847ac191da3fb2d2cdb28cc32344d7e9a9cf5c9b6a0ee` hex of signature data (32-byte r, 32-byte s)
    - `1` (int) recovery flag contained in `signature`
    - `6c6e626332306d0b25fe64500d044444444444444444444444444444444444444444444444444444444444444444444444021a0` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `e2ffa444e2979edb639fbdaa384638683ba1a5240b14dd7a150e45a04eea261d` hex of SHA256 of the preimage

The same, on testnet, with a fallback address mk2QpYatsKicvFVuTAQLBryyccRXMUaGHP

lntb20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygshp58yjmdan79s6qqdhdzgynm4zwq

Breakdown:

- `lntb`: prefix, Lightning on Bitcoin testnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)

- `h`: tagged field: hash of description…
- `s`: payment secret…
- `p`: payment hash…
- `f`: tagged field: fallback address
    - `pp`: `data_length` (`p` = 1; 1 * 32 + 1 == 33)
    - `3` = 17, so P2PKH address
    - `x9et2e20v6pu37c5d9vax37wxq72un98`: 160-bit P2PKH address
- `9`: features…
- `dj545axuxtnfemtpwkc45hx9d2ft7x04mt8q7y6t0k2dge9e7h8kpy9p34ytyslj3yu569aalz2xdk8xkd7ltxqld94u8h2esmsmac` signature
- `ghe9k8`: Bech32 checksum
- Signature breakdown:
    - `6ca95a74dc32e69ced6175b15a5cc56a92bf19f5dace0f134b7d94d464b9f5cf6090a18d48b243f289394d17bdf89466` hex of signature data (32-byte r, 32-byte s)
    - `1` (int) recovery flag contained in `signature`
    - `6c6e746232306d0b25fe64500d04444444444444444444444444444444444444444444444444444444444444444444442e1a1` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `33bc6642a336097c74299cadfdfdd2e4884a555cf1b4fda72b095382d473d795` hex of SHA256 of the preimage

On mainnet, with fallback address 1RustyRX2oai4EYYDpQGWvEL62BBGqN9T with extra routing info to go via nodes 029e03a901b85534ff1e92c43c74431f7ce72046060fcf7a95c37e148f78c77255 then 039e03a901b85534ff1e92c43c74431f7ce72046060fcf7a95c37e148f78c77255

lnbc20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq5r

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `p`: payment hash…
- `h`: tagged field: hash of description…
- `f`: tagged field: fallback address
    - `pp`: `data_length` (`p` = 1; 1 * 32 + 1 == 33)
    - `3` = 17, so P2PKH address
    - `qjmp7lwpagxun9pygexvgpjdc4jdj85f`: 160-bit P2PKH address
- `r`: tagged field: route information
    - `9y`: `data_length` (`9` = 5, `y` = 4; 5 * 32 + 4 == 164)
        - `q20q82gphp2nflc7jtzrcazrra7wwgzxqc8u7754cdlpfrmccae92qgzqvzq2ps8pqqqqqqpqqqqq9qqqvpeuqafqx`
            - pubkey: `029e03a901b85534ff1e92c43c74431f7ce72046060fcf7a95c37e148f78c77255`
            - `short_channel_id`: 66051x263430x1800
            - `fee_base_msat`: 1 millisatoshi

        - `fee_proportional_millionths`: 20
        - `cltv_expiry_delta`: 3
        - pubkey: `039e03a901b85534ff1e92c43c74431f7ce72046060fcf7a95c37e148f78c77255`
        - `short_channel_id`: 197637x395016x2314
        - `fee_base_msat`: 2 millisatoshi
        - `fee_proportional_millionths`: 30
        - `cltv_expiry_delta`: 4
- `9`: features…
- `dfjcdk6w3ak5pca9hwfwfh63zrrz06wwfya0ydlzpgzxkn5xagsqz7x9j4jwe7yj7vaf2k9lqsdk45kts2fd0fkr28am0u4w95tt2n` signature
- `76cqw0`: Bech32 checksum
- Signature breakdown:
    - `6a6586db4e8f6d40e3a5bb92e4df5110c627e9ce493af237e20a046b4e86ea200178c59564ecf892f33a9558bf041b6a` hex of signature data (32-byte r, 32-byte s)
    - `0` (int) recovery flag contained in `signature`
    - `6c6e626332306d0b25fe64500d04444444444444444444444444444444444444444444444444444444444444444444444444444021a0` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `b342d4655b984e53f405fe4d872fb9b7cf54ba538fcd170ed4a5906a9f535064` hex of SHA256 of the preimage

On mainnet, with fallback (P2SH) address 3EktnHQD7RiAE6uzMj2ZifT9YgRrkSgzQX

lnbc20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygshp58yjmdan79s6qqdhdzgynm4zwq

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `h`: tagged field: hash of description…
- `p`: payment hash…
- `f`: tagged field: fallback address
    - `pp`: `data_length` (`p` = 1; 1 * 32 + 1 == 33)
    - `j` = 18, so P2SH address
    - `3a24vwu6r8ejrss3axul8rxldph2q7z9`: 160-bit P2SH address
- `9`: features…
- `z6qsgww34xlatfj6e3sngrwfy3ytkt29d2qttr8qz2mnedfqysuqypgqex4haa2h8fx3wnypranf3pdwyluftwe680jjcfp438u82x` signature
- `hf75ym`: Bech32 checksum
- Signature breakdown:
    - `16810439d1a9bfd5a65acc61340dc92448bb2d456a80b58ce012b73cb5202438020500c9ab7ef5573a4d174c811f6698` hex of signature data (32-byte r, 32-byte s)
    - `1` (int) recovery flag contained in `signature`

    - `6c6e626332306d0b25fe64500d04444444444444444444444444444444444444444444444444444444444444444442e1a1` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `9e93321a775f7dffdca03e61d1ac6e0e356cc63cecd3835271200c1e5b499d29` hex of SHA256 of the preimage

On mainnet, with fallback (P2WPKH) address bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t4

lnbc20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygshp58yjmdan79s6qqdhdzgynm4zwq

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `h`: tagged field: hash of description…
- `p`: payment hash…
- `f`: tagged field: fallback address
    - `pp`: `data_length` (`p` = 1; 1 * 32 + 1 == 33)
    - `q`: 0, so witness version 0
    - `w508d6qejxtdg4y5r3zarvary0c5xw7k`: 160 bits = P2WPKH.
- `9`: features…
- `t29a0wturnys2hhxpner2e3plp6jyj8qx7548zr2z7ptgjjc7hljm98xhjym0dg52sdrvqamxdezkmqg4gdrvwwnf0kv2jdfnl4xat` signature
- `mrnsse`: Bech32 checksum
- Signature breakdown:
    - `5a8bd7b97c1cc9055ee60cf2356621f8752248e037a953886a1782b44a58f5ff2d94e6bc89b7b514541a3603bb33722b` hex of signature data (32-byte r, 32-byte s)
    - `0` (int) recovery flag contained in `signature`
    - `6c6e626332306d0b25fe64500d04444444444444444444444444444444444444444444444444444444444444444442e1a1` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `44fbec32cdac99a1a3cd638ec507dad633a1e5bba514832fd3471e663a157f7b` hex of SHA256 of the preimage

On mainnet, with fallback (P2WSH) address bc1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3qccfmv3

lnbc20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygshp58yjmdan79s6qqdhdzgynm4zwq

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `h`: tagged field: hash of description…
- `p`: payment hash…
- `f`: tagged field: fallback address
    - `p4`: `data_length` (`p` = 1, `4` = 21; 1 * 32 + 21 == 53)
    - `q`: 0, so witness version 0

    - `rp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q`: 260 bits = P2WSH.
- `9`: features…
- `9vlvyj8cqvq6ggvpwd53jncp9nwc47xlrsnenq2zp70fq83qlgesn4u3uyf4tesfkkwwfg3qs54qe426hp3tz7z6sweqdjg05axsrj` signature
- `9yrrwc`: Bech32 checksum
- Signature breakdown:
    - `2b3ec248f80301a421817369194f012cdd8af8df1c279981420f9e901e20fa3309d791e11355e609b59ce4a220852a0c` hex of signature data (32-byte r, 32-byte s)
    - `1` (int) recovery flag contained in `signature`
    - `6c6e626332306d0b25fe64500d04444444444444444444444444444444444444444444444444444444444444442e1a1` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `865a2cc6730e1eeeacd30e6da8e9ab0e9115828d27953ec0c0f985db05da5027` hex of SHA256 of the preimage

On mainnet, with fallback (P2TR) address bc1pptdvg0d2nj99568qn6ssdy4cygnwuxgw2ukmnwgwz7jpqjz2kszse2s3lm

lnbc20m1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq5r

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `20m`: amount (20 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `s`: payment secret…
- `p`: payment hash…
- `h`: tagged field: hash of description…
- `f`: tagged field: fallback address
    - `p4`: `data_length` (`p` = 1, `4` = 21; 1 * 32 + 21 == 53)
    - `p`: 1, so witness version 1
    - `ptdvg0d2nj99568qn6ssdy4cygnwuxgw2ukmnwgwz7jpqjz2kszs`: 260 bits = P2TR.
- `9`: features…
- `y606dznq28exnydt2r4c29y56xjtn3sk4mhgjtl4pg2y4ar3249rq4ajlmj9jy8zvlzw7cr8mggqzm842xfr0v72rswzq9xvr4hknf` signature
- `wmn6xd`: Bech32 checksum
- Signature breakdown:
    - `269fa68a6051f26991ab50eb851494d1a4b9c616aeee892ff50a144af471554a3057b2fee45910e267c4ef6067da1001` hex of signature data (32-byte r, 32-byte s)
    - `0` (int) recovery flag contained in `signature`
    - `6c6e626332306d0b25fe64500d04444444444444444444444444444444444444444444444444444444444444444021a0` hex of data for signing (prefix + data after separator up to the start of the signature)
    - `116fdb0f18352c886deb263f6466eb40e5e6518b80231a1f9df86088bfa48043` hex of SHA256 of the preimage

Please send 0.00967878534 BTC for a list of items within one week, amount in pico-BTC

lnbc9678785340p1pwmna7lpp5gc3xfm08u9qy06djf8dfflhugl6p7lgza6dsjxq454gxhj9t7a0sd8dgfkx7cmtwd68yetpd5s9xa

Breakdown:

- `lnbc`: prefix, Lightning on bitcoin mainnet
- `9678785340p`: amount (9678785340 pico-bitcoin = 967878534 milli satoshi)
- `1`: Bech32 separator
- `pwmna7l`: timestamp (1572468703)
- `s`: payment secret…
- `p`: payment hash
    - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
    - `gc3xfm08u9qy06djf8dfflhugl6p7lgza6dsjxq454gxhj9t7a0s`: payment hash 462264ede7e14047e9b249da94fefc47f41f7d02ee9b091815a5506bc8abf75f
- `d`: short description
    - `8d`: `data_length` (`8` = 7, `d` = 13; 7 * 32 + 13 == 237)
    - `gfkx7cmtwd68yetpd5s9xar0wfjn5gpc8qhrsdfq24f5ggrxdaezqsnvda3kkum5wfjkzmfqf3jkgem9wgsyuctwdus9xgrc`
      'Blockstream Store: 88.85 USD for Blockstream Ledger Nano S x 1, "Back In My Day" Sticker x 2, "I Got Lightning Working" Sticker x 2 and 1 more items'
- `x`: expiry time
    - `qy`: `data_length` (`q` = 0, `y` = 2; 0 * 32 + 4 == 4)
    - `jw5q`: 604800 seconds (`j` = 18, `w` = 14, `5` = 20, `q` = 0; 18 * 32^3 + 14 * 32^2 + 20 * 32 + 0 == 604800)
- `c`: `min_final_cltv_expiry_delta`
    - `qp`: `data_length` (`q` = 0, `p` = 1; 0 * 32 + 1 == 1)
    - `2`: min_final_cltv_expiry_delta = 10
- `r`: tagged field: route information
    - `zj`: `data_length` (`z` = 2, `j` = 18; 2 * 32 + 18 == 82)
    - `q0gxwkzc8w6323m55m4jyxcjwmy7stt9hwkwe2qxmy8zpsgg7jcuwz87fcqqeuqqqyqqqqlgqqqqn3qq9q`:
        - pubkey: 03d06758583bb5154774a6eb221b1276c9e82d65bbaceca806d90e20c108f4b1c7
        - short_channel_id: 589390x3312x1
        - fee_base_msat = 1000
        - fee_proportional_millionths = 2500
        - cltv_expiry_delta = 40
- `9`: features…
- `rvgkpnmps664wgkp43l22qsgdw4ve24aca4nymnxddlnp8vh9v2sdxlu5ywdxefsfvm0fq3sesf08uf6q9a2ke0hc9j6z6wlxg5z5k` signature
- `u2v9wz`: Bech32 checksum

Please send $30 for coffee beans to the same peer, which supports features 8, 14 and 99, using secret 0x1111111111111111111111111111111111111111111111111111111111111111

lnbc25m1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdq5vdhkven9v5sxyetpdeessp5zyg3

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `25m`: amount (25 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `p`: payment hash…

- `d`: short description
    - `q5`: `data_length` (`q` = 0, `5` = 20; 0 * 32 + 20 == 20)
    - `vdhkven9v5sxyetpdees`: 'coffee beans'
- `s`: payment secret
    - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
    - `zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygs`: 0x1111111111111111111111111111111111111111111111111111111111111111
- `9`: features
    - `q5`: `data_length` (`q` = 0, `5` = 20; 0 * 32 + 20 == 20)
    - `sqqqqqqqqqqqqqqqqsgq`: b1000….00000100000100000000
- `2a25dxl5hrntdtn6zvydt7d66hyzsyhqs4wdynavys42xgl6sgx9c4g7me86a27t07mdtfry458rtjr0v92cnmswpsjscgt2vcse3s` signature
- `z3uapa`: Bech32 checksum

Same, but all upper case.

LNBC25M1PVJLUEZPP5QQQSYQCYQ5RQWZQFQQQSYQCYQ5RQWZQFQQQSYQCYQ5RQWZQFQYPQDQ5VDI

Same, but including fields which must be ignored.

lnbc25m1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdq5vdhkven9v5sxyetpdeessp5zyg3

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `25m`: amount (25 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `p`: payment hash…
- `d`: short description
    - `q5`: `data_length` (`q` = 0, `5` = 20; 0 * 32 + 20 == 20)
    - `vdhkven9v5sxyetpdees`: 'coffee beans'
- `s`: payment secret
    - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
    - `zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygs`: 0x1111111111111111111111111111111111111111111111111111111111111111
- `9`: features
    - `q5`: `data_length` (`q` = 0, `5` = 20; 0 * 32 + 20 == 20)
    - `sqqqqqqqqqqqqqqqqsgq`: b1000….00000100000100000000
- `2`: unknown field
    - `qr`: `data_length` (`q` = 0, `r` = 3; 0 * 32 + 3 == 3)
    - `qqq`: zeroes
- `f`: tagged field: fallback address
    - `pp`: `data_length` (`p` = 1, `p` = 1; 1 * 32 + 1 == 33)
    - `nqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq`: fallback address type 19 (ignored)
- `p`: payment hash
    - `pn`: `data_length` (`p` = 1, `n` = 19; 1 * 32 + 19 == 51) (ignored)
    - `qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq`

- p: payment hash
  - p4: data_length (p = 1, 4 = 21; 1 * 32 + 21 == 53) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- h: hash of description
  - pn: data_length (p = 1, n = 19; 1 * 32 + 19 == 51) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- h: hash of description
  - p4: data_length (p = 1, 4 = 21; 1 * 32 + 21 == 53) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- s: payment secret
  - pn: data_length (p = 1, n = 19; 1 * 32 + 19 == 51) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- s: payment secret
  - p4: data_length (p = 1, 4 = 21; 1 * 32 + 21 == 53) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- n: node id
  - p5: data_length (p = 1, 5 = 20; 1 * 32 + 20 == 52) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- n: node id
  - pk: data_length (p = 1, k = 22; 1 * 32 + 22 == 54) (ignored)
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
- z599y53s3ujmcfjp5xrdap68qxymkqphwsexhmhr8wdz5usdzkzrse33chw6dlp3jhuhge9ley7j2ayx36kawe7kmgg8sv5ugdyusd signature
- zn8z9x: Bech32 checksum

Please send 0.01 BTC with payment metadata 0x01fafaf0

lnbc10m1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdp9wpshjmt9de6zqmt9w3skgct5vy

Breakdown:

- lnbc: prefix, Lightning on Bitcoin mainnet
- 10m: amount (10 milli-bitcoin)
- 1: Bech32 separator
- pvjluez: timestamp (1496314658)
- p: payment hash
  - p5: data_length (p = 1, 5 = 20; 1 * 32 + 20 == 52)
  - qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypq: payment hash 0001020304050607080900010203040506070809000102030405060708090102
- d: short description
  - p9: data_length (p = 1, 9 = 5; 1 * 32 + 5 == 37)
  - wpshjmt9de6zqmt9w3skgct5vysxjmnnd9jx2: 'payment metadata inside'
- m: metadata
  - q8: data_length (q = 0, 8 = 7; 0 * 32 + 7 == 7)
  - q8a04uq: 0x01fafaf0
- s: payment secret
  - p5: data_length (p = 1, 5 = 20; 1 * 32 + 20 == 52)

    - zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygs: 0x1111111111111111111111111111111111111111111111111111111111111111
- 9: features
    - q2: data_length (q = 0, 2 = 10; 0 * 32 + 10 == 10)
    - gqqqqqqsgq: [b01000000000000000000000000000000000100000100000000] = 8 + 14 + 48
- 7hf8he7ecf7n4ffphs6awl9t6676rrclv9ckg3d3ncn7fct63p6s365duk5wrk202cfy3aj5xnnp5gs3vrdvruverwwq7yzhkf5a3x signature
- d05wjc: Bech32 checksum

Public-key recovery with high-S signature

lnbc1pvjluezsp5zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygspp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwz

Breakdown:

- lnbc: prefix, Lightning on Bitcoin mainnet
- 1: Bech32 separator
- pvjluez: timestamp (1496314658)
- s: payment secret
    - p5: data_length (p = 1, 5 = 20; 1 * 32 + 20 == 52)
    - zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygs: payment secret 1111111111111111111111111111111111111111111111111111111111111111
- p: payment hash
    - p5: data_length (p = 1, 5 = 20; 1 * 32 + 20 == 52)
    - qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypq: payment hash 0001020304050607080900010203040506070809000102030405060708090102
- d: short description
    - pl: data_length (p = 1, l = 31; 1 * 32 + 31 == 63)
    - 2pkx2ctnv5sxxmmwwd5kgetjypeh2ursdae8g6twvus8g6rfwvs8qun0dfjkxaq: 'Please consider supporting this project'
- 9: features
    - qr: data_length (q = 0, r = 3; 0 * 32 + 3 == 3)
    - sgq: b100000100000000
- 357wnc5r2ueh7ck6q93dj32dlqnls087fxdwk8qakdyafkq3yap2r09nt4ndd0unm3z9u5t48y6ucv4r5sg7lk98c77ctvjczkspk5 signature
- rc90gx: Bech32 checksum
- Signature breakdown:
    - 8d3ce9e28357337f62da0162d9454df827f83cfe499aeb1c1db349d4d8112742a1bcb35d66d6bf93dc445e51753935cc hex of signature data (32-byte r, 32-byte s)
    - 1 (int) recovery flag contained in signature
    - 6c6e62630b25fe64500d04444444444444444444444444444444444444444444444444444444444444444444444444444021a0000810 hex of data for signing (prefix + data after separator up to the start of the signature)
    - 6daf4d488be41ce7cbb487cab1ef2975e5efcea879b20d421f0ef86b07cbb987 hex of SHA256 of the preimage

Examples of Invalid Invoices

Same, but adding invalid unknown feature 100

lnbc25m1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdq5vdhkven9v5sxyetpdeessp5zyg3

Breakdown:

- `lnbc`: prefix, Lightning on Bitcoin mainnet
- `25m`: amount (25 milli-bitcoin)
- `1`: Bech32 separator
- `pvjluez`: timestamp (1496314658)
- `p`: payment hash…
- `d`: short description
  - `q5`: `data_length` (`q` = 0, `5` = 20; 0 * 32 + 20 == 20)
  - `vdhkven9v5sxyetpdees`: 'coffee beans'
- `s`: payment secret
  - `p5`: `data_length` (`p` = 1, `5` = 20; 1 * 32 + 20 == 52)
  - `zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zyg3zygs`: 0x1111111111111111111111111111111111111111111111111111111111111111
- `9`: features
  - `q4`: `data_length` (`q` = 0, `4` = 21; 0 * 32 + 21 == 21)
  - `psqqqqqqqqqqqqqqqqsgq`: b000011000….00000100000100000000
- `tqyx5vggfcsll4wu246hz02kp85x4katwsk9639we5n5yngc3yhqkm35jnjw4len8vrnqnf5ejh0mzj9n3vz2px97evektfm2l6wqc` signature
- `3y7372`: Bech32 checksum

Bech32 checksum is invalid.

lnbc2500u1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdpquwpc4curk03c9wlrswe78q4ey

Malformed bech32 string (no 1)

pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdpquwpc4curk03c9wlrswe78q4eyqc7d8d0xq

Malformed bech32 string (mixed case)

LNBC2500u1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdpquwpc4curk03c9wlrswe78q4

Signature is not recoverable.

lnbc2500u1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdq5xysxxatsyp3k7enxv4jsxqzpus

String is too short.

lnbc1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdpl2pkx2ctnv5sxxmmwwd5kgetjypeh2

Invalid multiplier

lnbc2500x1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdq5xysxxatsyp3k7enxv4jsxqzpus

Invalid sub-millisatoshi precision.

lnbc2500000001p1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdq5xysxxatsyp3k7enxv4js

Missing required s field.

lnbc20m1pvjluezpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqhp58yjmdan79s6qqdhdzgynm4zw

Non canonical signature (high-S) with 'n' field defined

lnbc25m1p70xwfzpp5qqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqqqsyqcyq5rqwzqfqypqdpl2pkx2ctnv5sxxmmwwd5kgetj

Authors

[FIXME:]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

BOLT #12: Negotiation Protocol for Lightning Payments

Table of Contents

- Limitations of BOLT 11
- Payment Flow Scenarios
- Encoding
- Signature calculation
- Offers
- Invoice Requests
- Invoices
- Invoice Errors
- Payer Proofs

Limitations of BOLT 11

The BOLT 11 invoice format has proven popular but has several limitations:

1. The entangling of bech32 encoding makes it awkward to send in other forms (e.g. inside the lightning network itself).
2. The signature applying to the entire invoice makes it impossible to prove an invoice without revealing its entirety.
3. Fields cannot generally be extracted for external use: the `h` field was a boutique extraction of the `d` field only.
4. The lack of the 'it's OK to be odd' rule makes backward compatibility harder.

5. The 'human-readable' idea of separating amounts proved fraught: `p` was often mishandled, and amounts in pico-bitcoin are harder than the modern satoshi-based counting.
6. Developers found the bech32 encoding to have an issue with extensions, which means we want to replace or discard it anyway.
7. The `payment_secret` designed to prevent probing by other nodes in the path was only useful if the invoice remained private between the payer and payee.
8. Invoices must be given per user and are actively dangerous if two payment attempts are made for the same user.

Payment Flow Scenarios

Here we use "user" as shorthand for the individual user's lightning node and "merchant" as the shorthand for the node of someone who is selling or has sold something.

There are two basic payment flows supported by BOLT 12:

The general user-pays-merchant flow is: 1. A merchant publishes an *offer*, such as on a web page or a QR code. 2. Every user requests a unique *invoice* over the lightning network using an *invoice_request* message, which contains the offer fields. 3. The merchant replies with the *invoice*. 4. The user makes a payment to the merchant as indicated by the invoice.

The merchant-pays-user flow (e.g. ATM or refund): 1. The merchant publishes an *invoice_request* which includes an amount it wishes to send to the user. 2. The user sends an *invoice* over the lightning network for the amount in the *invoice_request*, using a (possibly temporary) *invoice_node_id*. 3. The merchant confirms the *invoice_node_id* to ensure it's about to pay the correct person, and makes a payment to the invoice.

Payment Proofs and Payer Proofs

Note that the normal lightning "proof of payment" can only demonstrate that an invoice was paid (by showing the preimage of the `payment_hash`), not who paid it. The merchant can claim an invoice was paid, and once revealed, anyone can claim they paid the invoice, too.[1]

Providing a key in *invoice_request* allows the payer to prove that they were the one to request the invoice. In addition, the Merkle construction of the BOLT 12 invoice signature allows the user to reveal invoice fields in case of a dispute selectively.

Encoding

Each of the forms documented here are in TLV format.

The supported ASCII encoding is the human-readable prefix, followed by a `1`, followed by a bech32-style data string of the TLVs in order, optionally interspersed with `+` (for indicating additional data is to come). There is no checksum, unlike bech32m.

Requirements

Writers of a bolt12 string: - MUST either use all lowercase or all UPPERCASE. - SHOULD use uppercase for QR codes. - SHOULD use lower case otherwise. - MAY use `+`, optionally followed by whitespace, to separate large bolt12 strings.

Readers of a bolt12 string: - MUST handle strings which are all lowercase, or all uppercase. - if it encounters a + followed by zero or more whitespace characters between two bech32 characters: - MUST remove the + and whitespace.

Rationale

The use of bech32 is arbitrary but already exists in the bitcoin world. We currently omit the six-character trailing checksum: QR codes have their own checksums anyway, and errors don't result in loss of funds, simply an invalid offer (or inability to parse).

The use of + (which is ignored) allows use over limited text fields like Twitter:

```
lno1xxxxxxxx+

yyyyyyyyyyyy+

zzzzz
```

See format-string-test.json.

Signature Calculation

All signatures are created as per BIP-340 and tagged as recommended there. Thus we define H(`tag`,`msg`) as SHA256(SHA256(`tag`) || SHA256(`tag`) || `msg`), and SIG(`tag`,`msg`,`key`) as the signature of H(`tag`,`msg`) using `key`.

Each form is signed using one or more *signature TLV elements*: TLV types 240 through 1000 (inclusive). For these, the tag is "lightning" || `messagename` || `fieldname`, and `msg` is the Merkle-root; "lightning" is the literal 9-byte ASCII string, `messagename` is the name of the TLV stream being signed (i.e. "invoice_request", "invoice" or "payer_proof") and the `fieldname` is the TLV field containing the signature (e.g. "signature").

The formulation of the Merkle tree is similar to that proposed in BIP-341, with each TLV leaf paired with a nonce leaf to avoid revealing adjacent nodes in proofs.

The Merkle tree's leaves are, in TLV-ascending order for each tlv: 1. The H("LnLeaf",tlv). 2. The H("LnNonce"||first-tlv,tlv-type) where first-tlv is the numerically-first TLV entry in the stream, and tlv-type is the "type" field (1-9 bytes) of the current tlv.

The Merkle tree inner nodes are H("LnBranch", lesser-SHA256||greater-SHA256); this ordering means proofs are more compact since left/right is inherently determined.

If there is not exactly a power of 2 leaves, then the tree depth will be uneven, with the deepest tree on the lowest-order leaves.

e.g. consider the encoding of an `invoice` `signature` with TLVs TLV0, TLV1, and TLV2 (of types 0, 1, and 2 respectively):

```
L1=H("LnLeaf",TLV0)
L1nonce=H("LnNonce"||TLV0,0)
L2=H("LnLeaf",TLV1)
L2nonce=H("LnNonce"||TLV0,1)
L3=H("LnLeaf",TLV2)
L3nonce=H("LnNonce"||TLV0,2)
```

```
Assume L1 < L1nonce, L2 > L2nonce and L3 > L3nonce.

   L1    L1nonce                      L2   L2nonce               L3   L3nonce
     \   /                             \   /                     \   /
      v v                               v v                       v v
L1A=H("LnBranch",L1||L1nonce) L2A=H("LnBranch",L2nonce||L2)  L3A=H("LnBranch",L3nonce||L3)

Assume L1A < L2A:

       L1A   L2A                                 L3A=H("LnBranch",L3nonce||L3)
         \   /                                    |
          v v                                     v
  L1A2A=H("LnBranch",L1A||L2A)                    L3A=H("LnBranch",L3nonce||L3)

Assume L1A2A > L3A:

  L1A2A=H("LnBranch",L1A||L2A)          L3A
                          \            /
                           v          v
               Root=H("LnBranch",L3A||L1A2A)

Signature = SIG("lightninginvoicesignature", Root, nodekey)
```

Offers

Offers are a precursor to an invoice_request: readers will request an invoice (or multiple) based on the offer. An offer can be much longer-lived than a particular invoice, so it has some different characteristics; in particular the amount can be in a non-lightning currency. It's also designed for compactness to fit inside a QR code easily.

Note that the non-signature TLV elements get mirrored into invoice_request and invoice messages, so they each have specific and distinct TLV ranges.

The human-readable prefix for offers is `lno`.

TLV Fields for Offers

1. `tlv_stream`: `offer`
2. types:
    1. type: 2 (`offer_chains`)
    2. data:
        - [`...*chain_hash`:`chains`]
    3. type: 4 (`offer_metadata`)
    4. data:
        - [`...*byte`:`data`]
    5. type: 6 (`offer_currency`)
    6. data:
        - [`...*utf8`:`iso4217`]
    7. type: 8 (`offer_amount`)
    8. data:
        - [`tu64`:`amount`]

9. type: 10 (`offer_description`)
10. data:
    - [`...*utf8:description`]
11. type: 12 (`offer_features`)
12. data:
    - [`...*byte:features`]
13. type: 14 (`offer_absolute_expiry`)
14. data:
    - [`tu64:seconds_from_epoch`]
15. type: 16 (`offer_paths`)
16. data:
    - [`...*blinded_path:paths`]
17. type: 18 (`offer_issuer`)
18. data:
    - [`...*utf8:issuer`]
19. type: 20 (`offer_quantity_max`)
20. data:
    - [`tu64:max`]
21. type: 22 (`offer_issuer_id`)
22. data:
    - [`point:id`]

Requirements For Offers

A writer of an offer: - MUST NOT set any TLV fields outside the inclusive ranges: 1 to 79 and 1000000000 to 1999999999. - if the chain for the invoice is not solely bitcoin: - MUST specify `offer_chains` the offer is valid for. - otherwise: - SHOULD omit `offer_chains`, implying that bitcoin is only chain. - if a specific minimum `offer_amount` is required for successful payment: - MUST set `offer_amount` to the amount expected (per item). - MUST set `offer_amount` greater than zero. - if the currency for `offer_amount` is that of all entries in `chains`: - MUST specify `offer_amount` in multiples of the minimum lightning-payable unit (e.g. milli-satoshis for bitcoin). - otherwise: - MUST specify `offer_currency` `iso4217` as an ISO 4217 three-letter code. - MUST specify `offer_amount` in the currency unit adjusted by the ISO 4217 exponent (e.g. USD cents). - MUST set `offer_description` to a complete description of the purpose of the payment. - otherwise: - MUST NOT set `offer_amount` - MUST NOT set `offer_currency` - MAY set `offer_description` - MAY set `offer_metadata` for its own use. - if it supports bolt12 offer features: - MUST set `offer_features.features` to the bitmap of bolt12 features. - if the offer expires: - MUST set `offer_absolute_expiry` `seconds_from_epoch` to the number of seconds after midnight 1 January 1970, UTC that invoice_request should not be attempted. - if it is connected only by private channels: - MUST include `offer_paths` containing one or more paths to the node from publicly reachable nodes. - otherwise: - MAY include `offer_paths`. - if it includes `offer_paths`: - MAY set `offer_issuer_id`. - otherwise: - MUST set `offer_issuer_id` to the node's public key to request the invoice from. - if it sets `offer_issuer`: - SHOULD set it to identify the issuer of the invoice clearly. - if it includes a domain name: - SHOULD begin it with either user@domain or domain - MAY follow with a space and more text - if it can supply more than one item for a single invoice: - if the maximum quantity is known: - MUST set that maximum in `offer_quantity_max`. - MUST NOT set `offer_quantity_max` to 0. - otherwise: - MUST set `offer_quantity_max` to 0. - otherwise: - MUST NOT set `offer_quantity_max`.

A reader of an offer: - if the offer contains any TLV fields outside the inclusive ranges: 1 to 79 and 1000000000 to 1999999999: - MUST NOT respond to the offer. - if `offer_features` contains unknown *odd* bits that are non-zero: - MUST ignore the bit. - if `offer_features` contains unknown *even* bits that are non-zero: - MUST NOT respond to the offer. - SHOULD indicate the unknown bit to the user. - if `offer_chains` is not set: - if the node does not accept bitcoin invoices: - MUST NOT respond to the offer - otherwise: (`offer_chains` is set): - if the node does not accept invoices for at least one of the `chains`: - MUST NOT respond to the offer - if `offer_amount` is set and `offer_description` is not set: - MUST NOT respond to the offer. - if `offer_amount` is set and is not greater than zero: - MUST NOT respond to the offer. - if `offer_currency` is set and `offer_amount` is not set: - MUST NOT respond to the offer. - if neither `offer_issuer_id` nor `offer_paths` are set: - MUST NOT respond to the offer. - if `num_hops` is 0 in any `blinded_path` in `offer_paths`: - MUST NOT respond to the offer. - if it uses `offer_amount` to provide the user with a cost estimate: - MUST take into account the currency units for `offer_amount`: - `offer_currency` field if set - otherwise, the minimum lightning-payable unit (e.g. milli-satoshis for bitcoin). - MUST warn the user if the received `invoice_amount` differs significantly from that estimate. - if the current time is after `offer_absolute_expiry`: - MUST NOT respond to the offer. - if it chooses to send an invoice request, it sends an onion message: - if `offer_paths` is set: - MUST send the onion message via any path in `offer_paths` to the final `onion_msg_hop.blinded_node_id` in that path - otherwise: - MUST send the onion message to `offer_issuer_id` - MAY send more than one invoice request onion message at once.

Rationale

The entire offer is reflected in the invoice_request, both for completeness (so all information will be returned in the invoice), and so that the offer node can be stateless. This makes `offer_metadata` particularly useful, since it can contain an authentication cookie to validate the other fields.

Because offer fields are copied into the invoice request (and then the invoice), they require distinct ranges. The range 1-79 is the normal range, with another billion for self-assigning experimental ranges.

A signature is unnecessary, and makes for a longer string (potentially limiting QR code use on low-end cameras); if the offer has an error, no invoice will be given since the request includes all the non-signature fields.

The `offer_issuer_id` can be omitted for brevity, if `offer_paths` is set, as each of the final `blinded_node_id` in the paths can serve as a valid public key for the destination.

Because `offer_amount` can be in a different currency (using the `offer_currency` field) it is merely a guide: the issuer will convert it into a number of millisatoshis for `invoice_amount` at the time they generate an invoice, or the invoice request can specify the exact amount in `invreq_amount`, but the issuer may then reject it if it disagrees.

`offer_quantity_max` is allowed to be 1, which seems useless, but useful in a system which bases it on available stock. It would be painful to have to special-case the "only one left" offer generation.

Offers can be used to simply send money without expecting anything in return (tips, kudos, donations, etc), which means the description field is optional (the `offer_issuer` field is very useful for this case!); if you are charging for something specific, the description is vital for the user to know what it was they paid for.

An empty `offer_chains` (present but with zero entries) is explicitly invalid because it would make invoice requests impossible. The payer cannot set `invreq_chain` to "one of `offer_chains`" when there are no chains

listed. Rejecting such offers early provides clear feedback rather than leaving implementations to fail at the invoice request stage.

Invoice Requests

Invoice Requests are a request for an invoice; the human-readable prefix for invoice requests is `lnr`.

There are two similar-looking uses for invoice requests, which are almost identical from a workflow perspective, but are quite different from a user's point of view.

One is a response to an offer; this contains the `offer_issuer_id` or `offer_paths` and all other offer details, and is generally received over an onion message: if it's valid and refers to a known offer, the response is generally to reply with an `invoice` using the `reply_path` field of the onion message.

The second case is publishing an invoice request without an offer, such as via QR code. It contains neither `offer_issuer_id` nor `offer_paths`, setting the `invreq_payer_id` (and possibly `invreq_paths`) instead, as it is the one paying: the other offer fields are filled by the creator of the `invoice_request`, forming a kind of offer-to-send-money.

Note: the `invreq_metadata` is numbered 0 (not in the 80-159 range for other invreq fields) making it the "numerically-first TLV entry" for Signature Calculation. This ensures that merkle leaves are unguessable, allowing a future compact representation to hide fields while still allowing signature validation.

TLV Fields for `invoice_request`

1. `tlv_stream`: `invoice_request`
2. types:
    1. type: 0 (`invreq_metadata`)
    2. data:
        - [`...*byte:blob`]
    3. type: 2 (`offer_chains`)
    4. data:
        - [`...*chain_hash:chains`]
    5. type: 4 (`offer_metadata`)
    6. data:
        - [`...*byte:data`]
    7. type: 6 (`offer_currency`)
    8. data:
        - [`...*utf8:iso4217`]
    9. type: 8 (`offer_amount`)
    10. data:
        - [`tu64:amount`]
    11. type: 10 (`offer_description`)
    12. data:
        - [`...*utf8:description`]
    13. type: 12 (`offer_features`)
    14. data:
        - [`...*byte:features`]

15. type: 14 (`offer_absolute_expiry`)
16. data:
    - [`tu64:seconds_from_epoch`]
17. type: 16 (`offer_paths`)
18. data:
    - [`...*blinded_path:paths`]
19. type: 18 (`offer_issuer`)
20. data:
    - [`...*utf8:issuer`]
21. type: 20 (`offer_quantity_max`)
22. data:
    - [`tu64:max`]
23. type: 22 (`offer_issuer_id`)
24. data:
    - [`point:id`]
25. type: 80 (`invreq_chain`)
26. data:
    - [`chain_hash:chain`]
27. type: 82 (`invreq_amount`)
28. data:
    - [`tu64:msat`]
29. type: 84 (`invreq_features`)
30. data:
    - [`...*byte:features`]
31. type: 86 (`invreq_quantity`)
32. data:
    - [`tu64:quantity`]
33. type: 88 (`invreq_payer_id`)
34. data:
    - [`point:key`]
35. type: 89 (`invreq_payer_note`)
36. data:
    - [`...*utf8:note`]
37. type: 90 (`invreq_paths`)
38. data:
    - [`...*blinded_path:paths`]
39. type: 91 (`invreq_bip_353_name`)
40. data:
    - [`u8:name_len`]
    - [`name_len*byte:name`]
    - [`u8:domain_len`]
    - [`domain_len*byte:domain`]
41. type: 240 (`signature`)
42. data:
    - [`bip340sig:sig`]

Requirements for Invoice Requests

The writer: - if it is responding to an offer: - MUST copy all fields from the offer (including unknown fields). - if `offer_chains` is set: - MUST set `invreq_chain` to one of `offer_chains` unless that chain is bitcoin, in which case it SHOULD omit `invreq_chain`. - otherwise: - if it sets `invreq_chain` it MUST set it to bitcoin. - MUST set `signature.sig` as detailed in Signature Calculation using the `invreq_payer_id`. - if `offer_amount` is not present: - MUST specify `invreq_amount`. - otherwise: - MAY omit `invreq_amount`. - if it sets `invreq_amount`: - MUST specify `invreq_amount.msat` as greater or equal to amount expected by `offer_amount` (and, if present, `offer_currency` and `invreq_quantity`). - MUST set `invreq_payer_id` to a transient public key. - MUST remember the secret key corresponding to `invreq_payer_id`. - if `offer_quantity_max` is present: - MUST set `invreq_quantity` to greater than zero. - if `offer_quantity_max` is non-zero: - MUST set `invreq_quantity` less than or equal to `offer_quantity_max`. - otherwise: - MUST NOT set `invreq_quantity` - otherwise (not responding to an offer): - MUST set `offer_description` to a complete description of the purpose of the payment. - MUST set (or not set) `offer_absolute_expiry` and `offer_issuer` as it would for an offer. - MUST set `invreq_payer_id` (as it would set `offer_issuer_id` for an offer). - MUST set `invreq_paths` as it would set (or not set) `offer_paths` for an offer. - MUST NOT include `signature`, `offer_metadata`, `offer_chains`, `offer_amount`, `offer_currency`, `offer_features`, `offer_quantity_max`, `offer_paths` or `offer_issuer_id` - if the chain for the invoice is not solely bitcoin: - MUST specify `invreq_chain` the offer is valid for. - MUST set `invreq_amount`. - MUST NOT set any non-signature TLV fields outside the inclusive ranges: 0 to 159 and 1000000000 to 2999999999 - MUST set `invreq_metadata` to an unpredictable series of bytes. - if it sets `invreq_amount`: - MUST set `msat` in multiples of the minimum lightning-payable unit (e.g. milli-satoshis for bitcoin) for `invreq_chain` (or for bitcoin, if there is no `invreq_chain`). - if it supports bolt12 invoice request features: - MUST set `invreq_features.features` to the bitmap of features. - if it received the offer from which it constructed this `invoice_request` using BIP 353 resolution: - MUST include `invreq_bip_353_name` with, - `name` set to the post-₿, pre-@ part of the BIP 353 HRN, - `domain` set to the post-@ part of the BIP 353 HRN.

The reader: - MUST reject the invoice request if `invreq_payer_id` or `invreq_metadata` are not present. - MUST reject the invoice request if any non-signature TLV fields are outside the inclusive ranges: 0 to 159 and 1000000000 to 2999999999 - if `invreq_features` contains unknown *odd* bits that are non-zero: - MUST ignore the bit. - if `invreq_features` contains unknown *even* bits that are non-zero: - MUST reject the invoice request. - MUST reject the invoice request if `signature` is not correct as detailed in Signature Calculation using the `invreq_payer_id`. - if `num_hops` is 0 in any `blinded_path` in `invreq_paths`: - MUST reject the invoice request. - if `offer_issuer_id` is present, and `invreq_metadata` is identical to a previous `invoice_request`: - MAY simply reply with the previous invoice. - otherwise: - MUST NOT reply with a previous invoice. - if `offer_issuer_id` or `offer_paths` are present (response to an offer): - MUST reject the invoice request if the offer fields do not exactly match a valid, unexpired offer. - if `offer_paths` is present: - MUST ignore the invoice_request if it did not arrive via one of those paths. - otherwise: - MUST ignore any invoice_request if it arrived via a blinded path. - if `offer_quantity_max` is present: - MUST reject the invoice request if there is no `invreq_quantity` field. - if `offer_quantity_max` is non-zero: - MUST reject the invoice request if `invreq_quantity` is zero, OR greater than `offer_quantity_max`. - otherwise: - MUST reject the invoice request if there is an `invreq_quantity` field. - if `offer_amount` is present: - MUST calculate the *expected amount* using the `offer_amount`: - if `offer_currency` is not the `invreq_chain` currency, convert to the `invreq_chain` currency. - if `invreq_quantity` is present, multiply by `invreq_quantity.quantity`. - if `invreq_amount` is present: - MUST reject the invoice request if `invreq_amount.msat` is less than the *expected amount*. - MAY reject the invoice request if `invreq_amount.msat` greatly exceeds the *expected amount*. - otherwise (no `offer_amount`): - MUST reject the invoice request if it does not contain `invreq_amount`. - SHOULD send an invoice in response using the `onionmsg_tlv reply_path`. - otherwise (no `offer_issuer_id` or `offer_paths`, not a response to our offer): - MUST reject the invoice request

if any of the following are present: - `offer_chains`, `offer_features` or `offer_quantity_max`. - MUST reject the invoice request if `invreq_amount` is not present. - MAY use `offer_amount` (or `offer_currency`) for informational display to user. - if it sends an invoice in response: - MUST use `invreq_paths` if present, otherwise MUST use `invreq_payer_id` as the node id to send to. - if `invreq_chain` is not present: - MUST reject the invoice request if bitcoin is not a supported chain. - otherwise: - MUST reject the invoice request if `invreq_chain.chain` is not a supported chain. - if `invreq_bip_353_name` is present: - MUST reject the invoice request if `name` or `domain` contain any bytes which are not `0`-`9`, `a`-`z`, `A`-`Z`, `-`, `_` or `.`.

Rationale

`invreq_metadata` might typically contain information about the derivation of the `invreq_payer_id`. This should not leak any information (such as using a simple BIP-32 derivation path); a valid system might be for a node to maintain a base payer key and encode a 128-bit tweak here. The payer_id would be derived by tweaking the base key with SHA256(payer_base_pubkey || tweak). It's also the first entry (if present), ensuring an unpredictable nonce for hashing.

`invreq_payer_note` allows you to compliment, taunt, or otherwise engrave graffiti into the invoice for all to see.

Users can give a tip (or obscure the amount sent) by specifying an `invreq_amount` in their invoice request, even though the offer specifies an `offer_amount`. The recipient will only accept this if the invoice request amount exceeds the amount it's expecting (i.e. its `offer_amount` after any currency conversion, multiplied by `invreq_quantity`, if any).

Non-offer-response invoice requests are currently required to explicitly state the `invreq_amount` in the chain currency, so `offer_amount` and `offer_currency` are redundant (but may be informative for the payer to know how the sender claims `invreq_amount` was derived).

The requirement to use `offer_paths` if present, ensures a node does not reveal it is the source of an offer if it is asked directly. Similarly, the requirement that the correct path is used for the offer ensures that cannot be made to reveal that it is the same node that created some other offer.

Invoices

Invoices are a payment request, and when the payment is made, the payment preimage can be combined with the invoice to form a cryptographic receipt.

The recipient sends an `invoice` in response to an `invoice_request` using the `onion_message` `invoice` field.

1. `tlv_stream`: `invoice`
2. types:
    1. type: 0 (`invreq_metadata`)
    2. data:
        - [`...*byte:blob`]
    3. type: 2 (`offer_chains`)
    4. data:
        - [`...*chain_hash:chains`]
    5. type: 4 (`offer_metadata`)

6. data:
    - [...*byte:data]
7. type: 6 (offer_currency)
8. data:
    - [...*utf8:iso4217]
9. type: 8 (offer_amount)
10. data:
    - [tu64:amount]
11. type: 10 (offer_description)
12. data:
    - [...*utf8:description]
13. type: 12 (offer_features)
14. data:
    - [...*byte:features]
15. type: 14 (offer_absolute_expiry)
16. data:
    - [tu64:seconds_from_epoch]
17. type: 16 (offer_paths)
18. data:
    - [...*blinded_path:paths]
19. type: 18 (offer_issuer)
20. data:
    - [...*utf8:issuer]
21. type: 20 (offer_quantity_max)
22. data:
    - [tu64:max]
23. type: 22 (offer_issuer_id)
24. data:
    - [point:id]
25. type: 80 (invreq_chain)
26. data:
    - [chain_hash:chain]
27. type: 82 (invreq_amount)
28. data:
    - [tu64:msat]
29. type: 84 (invreq_features)
30. data:
    - [...*byte:features]
31. type: 86 (invreq_quantity)
32. data:
    - [tu64:quantity]
33. type: 88 (invreq_payer_id)
34. data:
    - [point:key]
35. type: 89 (invreq_payer_note)
36. data:
    - [...*utf8:note]

37. type: 90 (`invreq_paths`)
38. data:
    - [`...*blinded_path:paths`]
39. type: 91 (`invreq_bip_353_name`)
40. data:
    - [`u8:name_len`]
    - [`name_len*byte:name`]
    - [`u8:domain_len`]
    - [`domain_len*byte:domain`]
41. type: 160 (`invoice_paths`)
42. data:
    - [`...*blinded_path:paths`]
43. type: 162 (`invoice_blindedpay`)
44. data:
    - [`...*blinded_payinfo:payinfo`]
45. type: 164 (`invoice_created_at`)
46. data:
    - [`tu64:timestamp`]
47. type: 166 (`invoice_relative_expiry`)
48. data:
    - [`tu32:seconds_from_creation`]
49. type: 168 (`invoice_payment_hash`)
50. data:
    - [`sha256:payment_hash`]
51. type: 170 (`invoice_amount`)
52. data:
    - [`tu64:msat`]
53. type: 172 (`invoice_fallbacks`)
54. data:
    - [`...*fallback_address:fallbacks`]
55. type: 174 (`invoice_features`)
56. data:
    - [`...*byte:features`]
57. type: 176 (`invoice_node_id`)
58. data:
    - [`point:node_id`]
59. type: 240 (`signature`)
60. data:
    - [`bip340sig:sig`]

3. subtype: `blinded_payinfo`
4. data:
    - [`u32:fee_base_msat`]
    - [`u32:fee_proportional_millionths`]
    - [`u16:cltv_expiry_delta`]
    - [`u64:htlc_minimum_msat`]
    - [`u64:htlc_maximum_msat`]
    - [`u16:flen`]

        - [`flen*byte:features`]
5. subtype: `fallback_address`
6. data:
        - [`byte:version`]
        - [`u16:len`]
        - [`len*byte:address`]

Invoice Features

Bits	Description	Name
16	Multi-part-payment support	MPP/compulsory
17	Multi-part-payment support	MPP/optional

The 'MPP support' invoice feature indicates that the payer MUST (16) or MAY (17) use multiple part payments to pay the invoice.

Some implementations may not support MPP (e.g. for small payments), or may (due to capacity limits on a single channel) require it.

Requirements

A writer of an invoice: - MUST set `invoice_created_at` to the number of seconds since Midnight 1 January 1970, UTC when the invoice was created. - MUST set `invoice_amount` to the minimum amount it will accept, in units of the minimal lightning-payable unit (e.g. milli-satoshis for bitcoin) for `invreq_chain`. - if the invoice is in response to an `invoice_request`: - MUST copy all non-signature fields from the invoice request (including unknown fields). - if `invreq_amount` is present: - MUST set `invoice_amount` to `invreq_amount` - otherwise: - MUST set `invoice_amount` to the *expected amount*. - MUST set `invoice_payment_hash` to the SHA256 hash of the `payment_preimage` that will be given in return for payment. - if `offer_issuer_id` is present: - MUST set `invoice_node_id` to the `offer_issuer_id` - otherwise, if `offer_paths` is present: - MUST set `invoice_node_id` to the final `blinded_node_id` on the path it received the invoice request - MUST specify exactly one signature TLV element: `signature`. - MUST set `sig` to the signature using `invoice_node_id` as described in Signature Calculation. - if it requires multiple parts to pay the invoice: - MUST set `invoice_features.features` bit `MPP/compulsory` - or if it allows multiple parts to pay the invoice: - MUST set `invoice_features.features` bit `MPP/optional` - if the expiry for accepting payment is not 7200 seconds after `invoice_created_at`: - MUST set `invoice_relative_expiry.seconds_from_creation` to the number of seconds after `invoice_created_at` that payment of this invoice should not be attempted. - if it accepts onchain payments: - MAY specify `invoice_fallbacks` - SHOULD specify `invoice_fallbacks` in order of most-preferred to least-preferred if it has a preference. - for the bitcoin chain, it MUST set each `fallback_address` with `version` as a valid witness version and `address` as a valid witness program - MUST include `invoice_paths` containing one or more paths to the node. - MUST specify `invoice_paths` in order of most-preferred to least-preferred if it has a preference. - MUST include `invoice_blindedpay` with exactly one `blinded_payinfo` for each `blinded_path` in `paths`, in order. - MUST set `features` in each `blinded_payinfo` to match `encrypted_data_tlv.allowed_features` (or empty, if no `allowed_features`). - SHOULD ignore any payment which does not use one of the paths. - MUST NOT set any non-signature TLV fields outside the inclusive ranges: 0 to 239 and 1000000000 to 3999999999.

A reader of an invoice: - MUST reject the invoice if `invoice_amount` is not present. - MUST reject the invoice if `invoice_created_at` is not present. - MUST reject the invoice if `invoice_payment_hash` is not present. - MUST

reject the invoice if `invoice_node_id` is not present. - if `invreq_chain` is not present: - MUST reject the invoice if bitcoin is not a supported chain. - otherwise: - MUST reject the invoice if `invreq_chain.chain` is not a supported chain. - if `invoice_features` contains unknown *odd* bits that are non-zero: - MUST ignore the bit. - if `invoice_features` contains unknown *even* bits that are non-zero: - MUST reject the invoice. - if `invoice_relative_expiry` is present: - MUST reject the invoice if the current time since 1970-01-01 UTC is greater than `invoice_created_at` plus `seconds_from_creation`. - otherwise: - MUST reject the invoice if the current time since 1970-01-01 UTC is greater than `invoice_created_at` plus 7200. - MUST reject the invoice if `invoice_paths` is not present or is empty. - MUST reject the invoice if `num_hops` is 0 in any `blinded_path` in `invoice_paths`. - MUST reject the invoice if `invoice_blindedpay` is not present. - MUST reject the invoice if `invoice_blindedpay` does not contain exactly one `blinded_payinfo` per `invoice_paths.blinded_path`. - For each `invoice_blindedpay.payinfo`: - MUST NOT use the corresponding `invoice_paths.path` if `payinfo.features` has any unknown even bits set. - MUST reject the invoice if this leaves no usable paths. - if the invoice is a response to an `invoice_request`: - MUST reject the invoice if all fields in ranges 0 to 159 and 1000000000 to 2999999999 (inclusive) do not exactly match the invoice request. - if `offer_issuer_id` is present (invoice_request for an offer): - MUST reject the invoice if `invoice_node_id` is not equal to `offer_issuer_id` - otherwise, if `offer_paths` is present (invoice_request for an offer without id): - MUST reject the invoice if `invoice_node_id` is not equal to the final `blinded_node_id` it sent the invoice request to. - otherwise (invoice_request without an offer): - MAY reject the invoice if it cannot confirm that `invoice_node_id` is correct, out-of-band. - MUST reject the invoice if `signature` is not a valid signature using `invoice_node_id` as described in Signature Calculation. - SHOULD prefer to use earlier `invoice_paths` over later ones if it has no other reason for preference. - if `invoice_features` contains the MPP/compulsory bit: - MUST pay the invoice via multiple separate blinded paths. - otherwise, if `invoice_features` contains the MPP/optional bit: - MAY pay the invoice via multiple separate payments. - otherwise: - MUST NOT use multiple parts to pay the invoice. - if `invreq_amount` is present: - MUST reject the invoice if `invoice_amount` is not equal to `invreq_amount` - otherwise: - SHOULD confirm authorization if `invoice_amount.msat` is not within the amount range authorized. - for the bitcoin chain, if the invoice specifies `invoice_fallbacks`: - MUST ignore any `fallback_address` for which `version` is greater than 16. - MUST ignore any `fallback_address` for which `address` is less than 2 or greater than 40 bytes. - MUST ignore any `fallback_address` for which `address` does not meet known requirements for the given `version` - if `invreq_paths` is present: - MUST reject the invoice if it did not arrive via one of those paths. - otherwise, neither `offer_issuer_id` nor `offer_paths` are present (not derived from an offer): - MUST reject the invoice if it arrived via a blinded path. - otherwise (derived from an offer): - MUST reject the invoice if it did not arrive via invoice request `onionmsg_tlv` `reply_path`.

Rationale

Because the messaging layer is unreliable, it's quite possible to receive multiple requests for the same offer. As it's the caller's responsibility to make `invreq_metadata` both unpredictable and unique, the writer doesn't have to check all the fields are duplicates before simply returning a previous invoice. Note that such caching is optional, and should be carefully limited when e.g. currency conversion is involved, or if the invoice has expired.

The invoice duplicates fields rather than committing to the previous invreq. This flattened format simplifies storage at some space cost, as the payer need only remember the invoice for any refunds or proof.

The reader of the invoice cannot trust the invoice correctly reflects the invreq fields, hence the requirements to check that they are correct, although allowance is made for simply sending an unrequested invoice directly.

Note that the recipient of the invoice can determine the expected amount from either the offer it received, or the invreq it sent, so often already has authorization for the expected amount.

The default `invoice_relative_expiry` of 7200 seconds, which is generally a sufficient time for payment, even if new channels need to be opened.

Blinded paths provide an equivalent to `payment_secret` and `payment_metadata` used in BOLT 11. Even if `invoice_node_id` or `invreq_payer_id` is public, we force the use of blinding paths to keep these features. If the recipient does not care about the added privacy offered by blinded paths, they can create a path of length 1 with only themselves.

Rather than provide detailed per-hop-payinfo for each hop in a blinded path, we aggregate the fees and CLTV deltas. This avoids trivially revealing any distinguishing non-uniformity which may distinguish the path.

In the case of an invoice where there was no offer (just an invoice request), the payer needs to ensure that the invoice is from the intended payment recipient. This is the basis for the suggestion to confirm the invoice_node_id for this case.

Raw invoices (not based on an invoice_request) are generally not supported, though an implementation is allowed to support them, and we may define the behavior in the future. The redundant requirement to check `invreq_chain` explicitly is a nod to this: if the invoice is a response to an invoice request, that field must have existed due to the invoice request requirements, and we also require it to be mirrored here.

Invoice Errors

Informative errors can be returned in an onion message `invoice_error` field (via the onion `reply_path`) for either `invoice_request` or `invoice`.

TLV Fields for `invoice_error`

1. `tlv_stream`: `invoice_error`
2. types:
    1. type: 1 (`erroneous_field`)
    2. data:
        - [`tu64`:`tlv_fieldnum`]
    3. type: 3 (`suggested_value`)
    4. data:
        - [`...*byte`:`value`]
    5. type: 5 (`error`)
    6. data:
        - [`...*utf8`:`msg`]

Requirements

A writer of an invoice_error: - MUST set `error` to an explanatory string. - MAY set `erroneous_field` to a specific field number in the `invoice` or `invoice_request` which had a problem. - if it sets `erroneous_field`: - MAY set `suggested_value`. - if it sets `suggested_value`: - MUST set `suggested_value` to a valid field for that `tlv_fieldnum`. - otherwise: - MUST NOT set `suggested_value`.

A reader of an invoice_error: FIXME!

Rationale

Usually an error message is sufficient for diagnostics, however future enhancements may make automated handling useful.

In particular, we could allow non-offer-response `invoice_requests` to omit `invreq_amount` in the future and use offer fields to indicate alternate currencies. ("I will send you 10c!"). Then the sender of the invoice would have to guess how many msat that was, and could use the `invoice_error` to indicate if the recipient disagreed with the conversion so the sender can send a new invoice.

Payer Proofs

Payer proofs are proofs of invoice payment; the human-readable prefix for payer proofs is `lnp`.

The payer proof is based on the `invoice` TLV stream, with the exception that `invreq_metadata` cannot be included. Various other invoice fields may be omitted for privacy: numbers corresponding to (but not identical to) their position in the invoice are included, as well as the minimal hashes for missing merkle branches, to allow verification of the invoicing node's signature.

To prove that this `payer_proof` was created by someone who has the secret key used to request the invoice in the first place, it includes a signature using the `invreq_payer_id`: this signs the proof fields and invoice fields.

TLV Fields for `payer_proof`

1. `tlv_stream`: `payer_proof`
2. types:
    1. type: 2 (`offer_chains`)
    2. data:
        - [`...*chain_hash:chains`]
    3. type: 4 (`offer_metadata`)
    4. data:
        - [`...*byte:data`]
    5. type: 6 (`offer_currency`)
    6. data:
        - [`...*utf8:iso4217`]
    7. type: 8 (`offer_amount`)
    8. data:
        - [`tu64:amount`]
    9. type: 10 (`offer_description`)
    10. data:
        - [`...*utf8:description`]
    11. type: 12 (`offer_features`)
    12. data:
        - [`...*byte:features`]
    13. type: 14 (`offer_absolute_expiry`)

14. data:
    - [tu64:seconds_from_epoch]
15. type: 16 (offer_paths)
16. data:
    - [...*blinded_path:paths]
17. type: 18 (offer_issuer)
18. data:
    - [...*utf8:issuer]
19. type: 20 (offer_quantity_max)
20. data:
    - [tu64:max]
21. type: 22 (offer_issuer_id)
22. data:
    - [point:id]
23. type: 80 (invreq_chain)
24. data:
    - [chain_hash:chain]
25. type: 82 (invreq_amount)
26. data:
    - [tu64:msat]
27. type: 84 (invreq_features)
28. data:
    - [...*byte:features]
29. type: 86 (invreq_quantity)
30. data:
    - [tu64:quantity]
31. type: 88 (invreq_payer_id)
32. data:
    - [point:key]
33. type: 89 (invreq_payer_note)
34. data:
    - [...*utf8:note]
35. type: 90 (invreq_paths)
36. data:
    - [...*blinded_path:paths]
37. type: 91 (invreq_bip_353_name)
38. data:
    - [u8:name_len]
    - [name_len*byte:name]
    - [u8:domain_len]
    - [domain_len*byte:domain]
39. type: 160 (invoice_paths)
40. data:
    - [...*blinded_path:paths]
41. type: 162 (invoice_blindedpay)
42. data:
    - [...*blinded_payinfo:payinfo]

43. type: 164 (`invoice_created_at`)
44. data:
    - [`tu64`:`timestamp`]
45. type: 166 (`invoice_relative_expiry`)
46. data:
    - [`tu32`:`seconds_from_creation`]
47. type: 168 (`invoice_payment_hash`)
48. data:
    - [`sha256`:`payment_hash`]
49. type: 170 (`invoice_amount`)
50. data:
    - [`tu64`:`msat`]
51. type: 172 (`invoice_fallbacks`)
52. data:
    - [`...*fallback_address`:`fallbacks`]
53. type: 174 (`invoice_features`)
54. data:
    - [`...*byte`:`features`]
55. type: 176 (`invoice_node_id`)
56. data:
    - [`point`:`node_id`]
57. type: 240 (`signature`)
58. data:
    - [`bip340sig`:`sig`]
59. type: 241 (`proof_signature`)
60. data:
    - [`bip340sig`:`sig`]
61. type: 1001 (`proof_preimage`)
62. data:
    - [`32*byte`:`preimage`]
63. type: 1002 (`proof_omitted_tlvs`)
64. data:
    - [`...*bigsize`:`missing`]
65. type: 1003 (`proof_missing_hashes`)
66. data:
    - [`...*sha256`:`hashes`]
67. type: 1004 (`proof_leaf_hashes`)
68. data:
    - [`...*sha256`:`hashes`]
69. type: 1005 (`proof_note`)
70. data:
    - [`...*utf8`:`note`]

Requirements

A writer of a payer_proof: - MUST NOT include `invreq_metadata`. - MUST include `invreq_payer_id`, `invoice_payment_hash`, `invoice_node_id`, `signature` and (if present) `invoice_features` from the invoice. -

MUST include `proof_preimage` containing the `payment_preimage` returned from successful payment of this invoice. - For each non-signature TLV in the invoice in ascending-type order: - If the field is to be included in the payer_proof: - MUST copy it into the payer_proof. - MUST append the nonce (H("LnNonce"||TLV0,type)) to `proof_leaf_hashes`. - otherwise, if the TLV type is not zero: - MUST append a *marker number* to `proof_omitted_tlvs` - If the previous TLV type was included: - The *marker number* is that previous tlv type, plus one. - Otherwise, if `proof_omitted_tlvs` is empty: - The *marker number* is 1. - Otherwise, if the last `proof_omitted_tlvs` entry is 239: - The *marker number* is 1000000000. - Otherwise: - The *marker number* is one greater than the last `proof_omitted_tlvs` entry. - If `proof_omitted_tlvs` is empty: - MAY omit `proof_omitted_tlvs` from the payer_proof. - MAY include an annotation on the proof in `proof_note`. - MUST populate `proof_missing_hashes` with the merkle hash of the omitted branch of each internal node that has exactly one branch entirely omitted, in post-order depth-first smallest-to-largest TLV order. - MUST copy `signature` into the payer_proof. - MUST set `proof_signature` as detailed in Signature Calculation using the `invreq_payer_id`.

A reader of a payer_proof: - MUST reject the payer_proof if: - `invreq_payer_id`, `invoice_payment_hash`, `invoice_node_id`, `signature`, `proof_preimage`, `proof_missing_hashes`, `proof_leaf_hashes` or `proof_signature` are missing. - SHA256(`proof_preimage`) does not equal `invoice_payment_hash`. - `proof_omitted_tlvs` are not in strict ascending order (no duplicates). - `proof_omitted_tlvs` contains 0. - `proof_omitted_tlvs` contains number outside both ranges 1 to 239 and 1000000000 to 3999999999. - `proof_omitted_tlvs` contains the number of an included TLV field. - `proof_omitted_tlvs` contains a number which is not: - one greater than an included TLV number, - one greater than the previous `proof_omitted_tlvs` entry, or 0 if it is the first number, or - 1000000000, if the previous `proof_omitted_tlvs` entry is 239. - `proof_leaf_hashes` does not contain exactly one hash for each field in ranges 1 to 239 and 1000000000 to 3999999999. - There are not exactly enough `proof_missing_hashes` to reconstruct the merkle tree root using the `proof_omitted_tlvs` values (with `0` implied as the first omitted TLV). - `signature` is not a valid signature using `invoice_node_id` as described in Signature Calculation (with `messagename` "invoice") of the reconstructed merkle-root of the invoice (i.e. without fields 1001 through 999999999 inclusive). - `proof_signature` is not a valid signature using `invreq_payer_id` as described in Signature Calculation.

Rationale

Using the invoice as a base enshrines information about the payment including important offer and invoice_request fields. However, many fields are not useful (such as payment paths), or may compromise privacy (such as invreq_payer_note containing delivery address information), so being able to elide them while still allowing signature validation is vital.

We disallow including `invreq_metadata`: that is the hashing nonce, thus allowing brute-force of omitted fields.

`invreq_payer_id` is the key whose signature we have to attach to the proof, and `invoice_node_id` and `signature` are needed to validate the original invoice. `invoice_features` may indicate additional details in the future which would require additional fields to be in the proof. Note that `invoice_amount` is not compulsory, though it would probably be very useful in most cases.

The requirement to include minimal hashes (rather than one for every unknown leaf) minimizes the size, especially when many consecutive fields are omitted. As the exact TLV types of omitted TLVs are unimportant (as long as ordering is maintained), we renumber them to be minimal, as further obfuscation of values.

The proof fields are outside the established offer, invoice request and invoice TLV ranges, and above the signature range (240-1000), so they are committed to by the `proof_signature`.

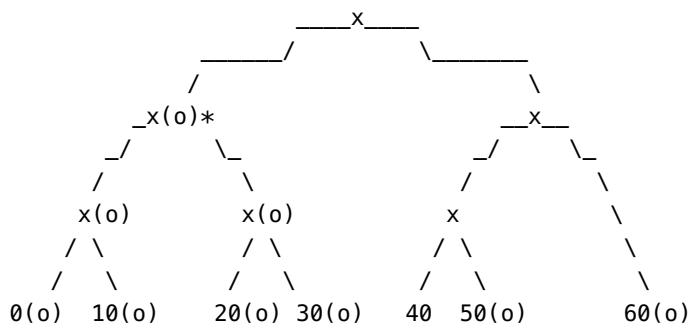
The optional `proof_note` field allows a challenge-response system to be implemented: someone requiring proof can ask for a signature with a particular note. It can also be missing.

Example for Payer Proofs

Consider a trivial TLV construct (not a valid invoice) which we are trying to prove, with the following fields:

0 - Omitted 10 - Omitted 20 - Omitted 30 - Omitted 40 - Included 50 - Omitted 60 - Omitted 240 - Omitted (signature field)

Here is the full signature Merkle tree, with omitted nodes marked with `(o)`:



Note that the signature TLV 240 is not included in the merkle tree.

`proof_leaf_hashes` contains the nonce hashes for the present non-signature TLVs:

1. H("LnNonce"||TLV0,40)

Since four adjacent nodes (0, 10 20 and 30) are omitted, we can (and must) simply provide the hash of the node above them, marked with an asterisk.

Thus, `proof_missing_hashes` contains the following hashes in order:

1. Merkle of H("LnLeaf",TLV50) and H("LnNonce"||TLV0,50)
2. Merkle of H("LnLeaf",TLV60) and H("LnNonce"||TLV0,60)
3. Merkle of (Merkle of (Merkle of H("LnLeaf",TLV0) and H("LnNonce"||TLV0,0)) (Merkle of H("LnLeaf",TLV10) and H("LnNonce"||TLV0,10))) and (Merkle of (Merkle of H("LnLeaf",TLV20) and H("LnNonce"||TLV0,20)) (Merkle of H("LnLeaf",TLV30) and H("LnNonce"||TLV0,30)))

In this example the correct proof_missing_hashes order is not ascending TLV order: the omitted subtree containing TLVs 0, 10, 20, and 30 is emitted after the omitted TLVs 50 and 60, because it is the missing sibling of their parent's parent.

The `proof_omitted_tlvs` array is based on the omitted tlvs: [0, 10, 20, 30, 50, 60]. It uses the minimal values which hide the real field numbers without changing their order, `0` is implied (as it's always omitted), giving an array of [1, 2, 3, 41, 42].

The algorithm for creating `proof_missing_hashes` is most easily implemented in a recursive fashion, traversing smallest-to-largest TLV (left-to-right in the above representation). When you need to combine two hashes where one side is entirely omitted and the other is not, append that hash to `proof_missing_hashes`.

Reconstruction is the exact opposite: when you need to combine a hash where one side is entirely omitted and the other is not, pull a hash from `proof_missing_hashes`. If there are insufficient `proof_missing_hashes`, or it isn't empty when you have completed the merkle tree, the number of `proof_missing_hashes` was incorrect.

See the Payer Proof Test Vectors for more examples.

FIXME: Possible future extensions:

1. The offer can require delivery info in the `invoice_request`.
2. An offer can be updated: the response to an `invoice_request` is another offer, perhaps with a signature from the original `offer_issuer_id`
3. Any empty TLV fields can mean the value is supposed to be known by other means (i.e. transport-specific), but is still hashed for sig.
4. We could upgrade to allow multiple offers in one invreq and invoice, to make a shopping list.
5. All-zero offer_id == gratuitous payment.
6. Streaming invoices?
7. Re-add recurrence.
8. Re-add `invoice_replace` for requesting replacement of a (stuck-payment) invoice with a new one.
9. Allow non-offer `invoice_request` with alternate currencies?
10. Add `offer_quantity_unit` to indicate stepping for quantity (e.g. 100 grams).

[1] https://www.youtube.com/watch?v=4SYc_flMnMQ

When Channels Close

Channels are great until they aren't. A peer disappears, a revocation secret gets broadcast, an HTLC is about to expire. Then you're on-chain, and the chain doesn't care about your encrypted transport.

BOLT 05 is the playbook for that. It tells an implementation how to watch for commitment transactions, when to broadcast a penalty, how to resolve HTLCs on-chain, and how to sweep outputs so you actually get your coins back. It's less about messages and more about timeouts, scripts, and not racing yourself.

Read this after you understand commitment transactions in BOLT 03. The on-chain rules only make sense once you know what those transactions look like.

Most of the time a cooperative close from BOLT 02 is enough. BOLT 05 is for the rest of the time — which is exactly when you need a spec.

In this chapter

- BOLT 05 — On-chain closing and recovery

BOLT #5: Recommendations for On-chain Transaction Handling

Abstract

Lightning allows for two parties (a local node and a remote node) to conduct transactions off-chain by giving each of the parties a *cross-signed commitment transaction*, which describes the current state of the channel (basically, the current balance). This *commitment transaction* is updated every time a new payment is made and is spendable at all times.

There are three ways a channel can end:

1. The good way (*mutual close*): at some point the local and remote nodes agree to close the channel. They generate a *closing transaction* (which is similar to a commitment transaction, but without any pending payments) and publish it on the blockchain (see BOLT #2: Channel Close).
2. The bad way (*unilateral close*): something goes wrong, possibly without evil intent on either side. Perhaps one party crashed, for instance. One side publishes its *latest commitment transaction*.
3. The ugly way (*revoked transaction close*): one of the parties deliberately tries to cheat, by publishing an *outdated commitment transaction* (presumably, a prior version, which is more in its favor).

Because Lightning is designed to be trustless, there is no risk of loss of funds in any of these three cases; provided that the situation is properly handled. The goal of this document is to explain exactly how a node should react when it encounters any of the above situations, on-chain.

Table of Contents

- General Nomenclature
- Commitment Transaction
- Failing a Channel
- Mutual Close Handling
- Unilateral Close Handling: Local Commitment Transaction
    - HTLC Output Handling: Local Commitment, Local Offers
    - HTLC Output Handling: Local Commitment, Remote Offers
- Unilateral Close Handling: Remote Commitment Transaction
    - HTLC Output Handling: Remote Commitment, Local Offers
    - HTLC Output Handling: Remote Commitment, Remote Offers
- Revoked Transaction Close Handling
    - Penalty Transactions Weight Calculation
- Generation of HTLC Transactions
- General Requirements
    - Limitations of v3 transactions
- Appendix A: Expected Weights
    - Expected Weight of the `to_local` Penalty Transaction Witness
    - Expected Weight of the `offered_htlc` Penalty Transaction Witness
    - Expected Weight of the `accepted_htlc` Penalty Transaction Witness
- Authors

General Nomenclature

Any unspent output is considered to be *unresolved* and can be *resolved* as detailed in this document. Usually this is accomplished by spending it with another *resolving* transaction. Although, sometimes simply noting the output for later wallet spending is sufficient, in which case the transaction containing the output is considered to be its own *resolving* transaction.

Outputs that are *resolved* are considered *irrevocably resolved* once the remote's *resolving* transaction is included in a block at least 100 deep, on the most-work blockchain. 100 blocks is far greater than the longest known Bitcoin fork and is the same wait time used for confirmations of miners' rewards (see Reference Implementation).

Requirements

A node: - once it has broadcast a funding transaction OR sent a commitment signature for a commitment transaction that contains an HTLC output: - until all outputs are *irrevocably resolved*: - MUST monitor the blockchain for transactions that spend any output that is NOT *irrevocably resolved*. - MUST *resolve* all outputs, as specified below. - MUST be prepared to resolve outputs multiple times, in case of blockchain reorganizations. - upon the funding transaction being spent, if the channel is NOT already closed: - MAY send a descriptive `error`. - SHOULD fail the channel. - SHOULD ignore invalid transactions.

Rationale

Once a local node has some funds at stake, monitoring the blockchain is required to ensure the remote node does not close unilaterally.

Invalid transactions (e.g. bad signatures) can be generated by anyone, (and will be ignored by the blockchain anyway), so they should not trigger any action.

Commitment Transaction

The local and remote nodes each hold a *commitment transaction*. Each of these commitment transactions has up to seven types of outputs:

1. *local node's main output*: Zero or one output, to pay to the *local node's* delayed_pubkey.
2. *remote node's main output*: Zero or one output, to pay to the *remote node's* delayed_pubkey.
3. *local node's anchor output*: one output paying to the *local node's* funding_pubkey.
4. *remote node's anchor output*: one output paying to the *remote node's* funding_pubkey.
5. *shared anchor*: one output spendable by anyone (pay-to-anchor).
6. *local node's offered HTLCs*: Zero or more pending payments (*HTLCs*), to pay the *remote node* in return for a payment preimage.
7. *remote node's offered HTLCs*: Zero or more pending payments (*HTLCs*), to pay the *local node* in return for a payment preimage.

To incentivize the local and remote nodes to cooperate, an `OP_CHECKSEQUENCEVERIFY` relative timeout encumbers the *local node's outputs* (in the *local node's commitment transaction*) and the *remote node's outputs* (in the *remote node's commitment transaction*). So for example, if the local node publishes its commitment transaction, it will have to wait to claim its own funds, whereas the remote node will have immediate access to its own funds. As a consequence, the two commitment transactions are not identical, but they are (usually) symmetrical.

See BOLT #3: Commitment Transaction for more details.

Failing a Channel

Although closing a channel can be accomplished in several ways, the most efficient is preferred.

Various error cases involve closing a channel. The requirements for sending error messages to peers are specified in BOLT #1: The `error` Message.

Requirements

A node: - if a *local commitment transaction* has NOT ever contained a `to_local` or HTLC output: - MAY simply forget the channel. - otherwise: - if the *current commitment transaction* does NOT contain `to_local` or other HTLC outputs: - MAY simply wait for the remote node to close the channel. - until the remote node closes: - MUST NOT forget the channel. - otherwise: - if it has received a valid `closing_signed` message that includes a sufficient fee: - SHOULD use this fee to perform a *mutual close*. - otherwise: - if the node knows or assumes its channel state is outdated: - MUST NOT broadcast its *last commitment transaction*. - otherwise: - MUST broadcast the *last commitment transaction*, for which it has a signature, to perform a *unilateral close*. - MUST spend any `to_local_anchor` or `shared_anchor` output, providing sufficient fees as incentive to include the commitment transaction in a block. For `to_local_anchor`, special care must be taken when spending to a third-party, because this re-introduces the pinning vulnerability that was addressed by adding the CSV delay to the non-anchor outputs. - SHOULD use replace-by-fee or other mechanism on the spending transaction if it proves insufficient for timely inclusion in a block.

Rationale

Since `dust_limit_satoshis` is supposed to prevent creation of uneconomic outputs (which would otherwise remain forever, unspent on the blockchain), all commitment transaction outputs MUST be spent.

In the early stages of a channel, it's common for one side to have little or no funds in the channel; in this case, having nothing at stake, a node need not consume resources monitoring the channel state.

There exists a bias towards preferring mutual closes over unilateral closes, because outputs of the former are unencumbered by a delay and are directly spendable by wallets. In addition, mutual close fees tend to be less exaggerated than those of commitment transactions (or in the case of commitments using anchor outputs, the commitment transaction may require a child transaction to cause it to be mined). So, the only reason not to use the mutual close transaction would be if the fee offered was too small for it to be processed.

Mutual Close Handling

A closing transaction *resolves* the funding transaction output.

In the case of a mutual close, a node need not do anything else, as it has already agreed to the output, which is sent to its specified `scriptpubkey` (see BOLT #2: Closing initiation: `shutdown`).

Unilateral Close Handling: Local Commitment Transaction

This is the first of two cases involving unilateral closes. In this case, a node discovers its *local commitment transaction*, which *resolves* the funding transaction output.

However, a node cannot claim funds from the outputs of a unilateral close that it initiated, until the `OP_CHECKSEQUENCEVERIFY` delay has passed (as specified by the remote node's `to_self_delay` field). Where relevant, this situation is noted below.

Requirements

A node: - upon discovering its *local commitment transaction*: - SHOULD spend the `to_local` output to a convenient address. - MUST wait until the `OP_CHECKSEQUENCEVERIFY` delay has passed (as specified by the remote node's `to_self_delay` field) before spending the output. - Note: if the output is spent (as recommended), the output is *resolved* by the spending transaction, otherwise it is considered *resolved* by the commitment transaction itself. - MAY ignore the `to_remote` output. - Note: No action is required by the local node, as `to_remote` is considered *resolved* by the commitment transaction itself. - MUST handle HTLCs offered by itself as specified in HTLC Output Handling: Local Commitment, Local Offers. - MUST handle HTLCs offered by the remote node as specified in HTLC Output Handling: Local Commitment, Remote Offers.

Rationale

Spending the `to_local` output avoids having to remember the complicated witness script, associated with that particular channel, for later spending.

The `to_remote` output is entirely the business of the remote node, and can be ignored.

HTLC Output Handling: Local Commitment, Local Offers

Each HTLC output can only be spent by either the *local offerer*, by using the HTLC-timeout transaction after it's timed out, or the *remote recipient*, if it has the payment preimage.

There can be HTLCs which are not represented by any outputs: either because they were trimmed as dust, or because the transaction has only been partially committed.

The HTLC output has *timed out* once the height of the latest block is equal to or greater than the HTLC `cltv_expiry`.

Requirements

A node: - if the commitment transaction HTLC output is spent using the payment preimage, the output is considered *irrevocably resolved*: - MUST extract the payment preimage from the transaction input witness. - if the commitment transaction HTLC output has *timed out* and hasn't been *resolved*: - MUST *resolve* the output by spending it using the HTLC-timeout transaction. - once the resolving transaction has reached reasonable depth: - MUST fail the corresponding incoming HTLC (if any). - MUST resolve the output of that HTLC-timeout transaction. - SHOULD resolve the HTLC-timeout transaction by spending it to a convenient address. - Note: if the output is spent (as recommended), the output is *resolved* by the spending transaction, otherwise it is considered *resolved* by the HTLC-timeout transaction itself. - MUST wait until the `OP_CHECKSEQUENCEVERIFY` delay has passed (as specified by the remote node's `open_channel` `to_self_delay` field) before spending that HTLC-timeout output. - for any committed HTLC that does NOT have an output in this commitment transaction: - if the payment preimage is known: - MUST fulfill the corresponding incoming HTLC (if any). - otherwise: - once the commitment transaction has reached reasonable depth: - MUST fail the corresponding

incoming HTLC (if any). - if no *valid* commitment transaction contains an output corresponding to the HTLC: - MAY fail the corresponding incoming HTLC sooner.

Rationale

The payment preimage either serves to prove payment (when the offering node originated the payment) or to redeem the corresponding incoming HTLC from another peer (when the offering node is forwarding the payment). Once a node has extracted the payment, it no longer cares about the fate of the HTLC-spending transaction itself.

In cases where both resolutions are possible (e.g. when a node receives payment success after timeout), either interpretation is acceptable; it is the responsibility of the recipient to spend it before this occurs.

The local HTLC-timeout transaction needs to be used to time out the HTLC (to prevent the remote node fulfilling it and claiming the funds) before the local node can back-fail any corresponding incoming HTLC, using `update_fail_htlc` (presumably with reason `permanent_channel_failure`), as detailed in BOLT #2. If the incoming HTLC is also on-chain, a node must simply wait for it to timeout: there is no way to signal early failure.

There are several reasons a committed HTLC may not have an output in the confirmed commitment transaction: the HTLC may be smaller than `dust_limit_satoshis`, the HTLC may not have been added to the commitment transaction yet, or the HTLC may have already been failed or fulfilled. In any case, if the payment preimage is known for the HTLC, the upstream HTLC needs to be fulfilled to avoid loss of funds.

If the payment preimage is not known for the missing HTLC, the correct action depends on the possibility of a blockchain reorganization that swaps out the confirmed commitment transaction for one with the HTLC present. If the HTLC is too small to appear in *any commitment transaction*, such a reorganization is not possible, and the HTLC can be safely failed immediately. Otherwise, a reorganization delay is required before failing the incoming HTLC. The requirement that the incoming HTLC be failed before its own timeout still applies as an upper bound.

HTLC Output Handling: Local Commitment, Remote Offers

Each HTLC output can only be spent by the recipient, using the HTLC-success transaction, which it can only populate if it has the payment preimage. If it doesn't have the preimage (and doesn't discover it), it's the offerer's responsibility to spend the HTLC output once it's timed out.

There are several possible cases for an offered HTLC:

1. The offerer is NOT irrevocably committed to it. The recipient will usually not know the preimage, since it will not forward HTLCs until they're fully committed. So using the preimage would reveal that this recipient is the final hop; thus, in this case, it's best to allow the HTLC to time out.
2. The offerer is irrevocably committed to the offered HTLC, but the recipient has not yet committed to an outgoing HTLC. In this case, the recipient can either forward or timeout the offered HTLC.
3. The recipient has committed to an outgoing HTLC, in exchange for the offered HTLC. In this case, the recipient must use the preimage, once it receives it from the outgoing HTLC; otherwise, it will lose funds by sending an outgoing payment without redeeming the incoming payment.

Requirements

A local node: - if it receives (or already possesses) a payment preimage for an unresolved HTLC output that it has been offered AND for which it has committed to an outgoing HTLC: - MUST *resolve* the output by spending it, using the HTLC-success transaction. - MUST NOT reveal its own preimage when it's not the final recipient.[Preimage-Extraction] - MUST resolve the output of that HTLC-success transaction. - otherwise: - if the *remote node* is NOT irrevocably committed to the HTLC: - MUST NOT *resolve* the output by spending it. - SHOULD resolve that HTLC-success transaction output by spending it to a convenient address. - MUST wait until the `OP_CHECKSEQUENCEVERIFY` delay has passed (as specified by the *remote node's* `open_channel`'s `to_self_delay` field), before spending that HTLC-success transaction output.

If the output is spent (as is recommended), the output is *resolved* by the spending transaction, otherwise it's considered *resolved* by the HTLC-success transaction itself.

If it's NOT otherwise resolved, once the HTLC output has expired, it is considered *irrevocably resolved*.

Unilateral Close Handling: Remote Commitment Transaction

The *remote node's* commitment transaction *resolves* the funding transaction output.

There are no delays constraining node behavior in this case, so it's simpler for a node to handle than the case in which it discovers its local commitment transaction (see Unilateral Close Handling: Local Commitment Transaction).

Requirements

A local node: - upon discovering a *valid* commitment transaction broadcast by a *remote node*: - if possible: - MUST handle each output as specified below. - MAY take no action in regard to the associated `to_remote`, which is simply a P2WPKH output to the *local node*. - Note: `to_remote` is considered *resolved* by the commitment transaction itself. - MAY take no action in regard to the associated `to_local`, which is a payment output to the *remote node*. - Note: `to_local` is considered *resolved* by the commitment transaction itself. - MUST handle HTLCs offered by itself as specified in HTLC Output Handling: Remote Commitment, Local Offers - MUST handle HTLCs offered by the remote node as specified in HTLC Output Handling: Remote Commitment, Remote Offers - otherwise (it is NOT able to handle the broadcast for some reason): - MUST inform the user of potentially lost funds.

Rationale

There may be more than one valid, *unrevoked* commitment transaction after a signature has been received via `commitment_signed` and before the corresponding `revoke_and_ack`. As such, either commitment may serve as the *remote node's* commitment transaction; hence, the local node is required to handle both.

In the case of data loss, a local node may reach a state where it doesn't recognize all of the *remote node's* commitment transaction HTLC outputs. It can detect the data loss state, because it has signed the transaction, and the commitment number is greater than expected. It can derive its own `remotepubkey` for the transaction, in order to salvage its own funds. Note: in this scenario, the node will be unable to salvage the HTLCs.

HTLC Output Handling: Remote Commitment, Local Offers

Each HTLC output can only be spent by either the *local offerer*, after it's timed out, or by the *remote recipient*, by using the HTLC-success transaction if it has the payment preimage.

There can be HTLCs which are not represented by any outputs: either because the outputs were trimmed as dust, or because the remote node has two *valid* commitment transactions with differing HTLCs.

The HTLC output has *timed out* once the depth of the latest block is equal to or greater than the HTLC `cltv_expiry`.

Requirements

A local node: - if the commitment transaction HTLC output is spent using the payment preimage: - MUST extract the payment preimage from the HTLC-success transaction input witness. - Note: the output is considered *irrevocably resolved*. - if the commitment transaction HTLC output has *timed out* AND NOT been *resolved*: - MUST *resolve* the output, by spending it to a convenient address. - for any committed HTLC that does NOT have an output in this commitment transaction: - if the payment preimage is known: - MUST fulfill the corresponding incoming HTLC (if any). - otherwise: - once the commitment transaction has reached reasonable depth: - MUST fail the corresponding incoming HTLC (if any). - if no *valid* commitment transaction contains an output corresponding to the HTLC: - MAY fail the corresponding incoming HTLC sooner.

Rationale

If the commitment transaction belongs to the *remote* node, the only way for it to spend the HTLC output (using a payment preimage) is for it to use the HTLC-success transaction.

The payment preimage either serves to prove payment (when the offering node is the originator of the payment) or to redeem the corresponding incoming HTLC from another peer (when the offering node is forwarding the payment). After a node has extracted the payment, it no longer need be concerned with the fate of the HTLC-spending transaction itself.

In cases where both resolutions are possible (e.g. when a node receives payment success after timeout), either interpretation is acceptable: it's the responsibility of the recipient to spend it before this occurs.

Once it has timed out, the local node needs to spend the HTLC output (to prevent the remote node from using the HTLC-success transaction) before it can back-fail any corresponding incoming HTLC, using `update_fail_htlc` (presumably with reason `permanent_channel_failure`), as detailed in BOLT #2. If the incoming HTLC is also on-chain, a node simply waits for it to timeout, as there's no way to signal early failure.

There are several reasons a committed HTLC may not have an output in the confirmed commitment transaction: the HTLC may be smaller than `dust_limit_satoshis`, the HTLC may not have been added to the commitment transaction yet, or the HTLC may have already been failed or fulfilled. In any case, if the payment preimage is known for the HTLC, the upstream HTLC needs to be fulfilled to avoid loss of funds.

If the payment preimage is not known for the missing HTLC, the correct action depends on the possibility of a blockchain reorganization that swaps out the confirmed commitment transaction for one with the HTLC present. If the HTLC is too small to appear in *any commitment transaction*, such a reorganization is not possible, and the HTLC can be safely failed immediately. Otherwise, a reorganization delay is required before failing the

incoming HTLC. The requirement that the incoming HTLC be failed before its own timeout still applies as an upper bound.

HTLC Output Handling: Remote Commitment, Remote Offers

The remote HTLC outputs can only be spent by the local node if it has the payment preimage. If the local node does not have the preimage (and doesn't discover it), it's the remote node's responsibility to spend the HTLC output once it's timed out.

There are actually several possible cases for an offered HTLC:

1. The offerer is not irrevocably committed to it. In this case, the recipient usually won't know the preimage, since it won't forward HTLCs until they're fully committed. As using the preimage would reveal that this recipient is the final hop, it's best to allow the HTLC to time out.
2. The offerer is irrevocably committed to the offered HTLC, but the recipient hasn't yet committed to an outgoing HTLC. In this case, the recipient can either forward it or wait for it to timeout.
3. The recipient has committed to an outgoing HTLC in exchange for an offered HTLC. In this case, the recipient must use the preimage, if it receives it from the outgoing HTLC; otherwise, it will lose funds by sending an outgoing payment without redeeming the incoming one.

Requirements

A local node: - if it receives (or already possesses) a payment preimage for an unresolved HTLC output that it was offered AND for which it has committed to an outgoing HTLC: - MUST *resolve* the output by spending it to a convenient address. - otherwise: - if the remote node is NOT irrevocably committed to the HTLC: - MUST NOT *resolve* the output by spending it.

If not otherwise resolved, once the HTLC output has expired, it is considered *irrevocably resolved*.

Revoked Transaction Close Handling

If any node tries to cheat by broadcasting an outdated commitment transaction (any previous commitment transaction besides the most current one), the other node in the channel can use its revocation private key to claim all the funds from the channel's original funding transaction.

Requirements

Once a node discovers a commitment transaction for which *it* has a revocation private key, the funding transaction output is *resolved*.

A local node: - MUST NOT broadcast a commitment transaction for which *it* has exposed the `per_commitment_secret`. - MAY take no action regarding the *local node's main output*, as this is a simple P2WPKH output to itself. - Note: this output is considered *resolved* by the commitment transaction itself. - MUST *resolve* the *remote node's main output* by spending it using the revocation private key. - MUST *resolve* the *remote node's offered HTLCs* in one of three ways: * spend the *commitment tx* using the payment revocation private key. * spend the *commitment tx* using the payment preimage (if known). * spend the *HTLC-timeout tx*, if the remote node has published it. - MUST *resolve* the *local node's offered HTLCs* in one of three ways: * spend the *commitment tx* using the payment revocation private key. * spend the *commitment tx* once the HTLC timeout has passed. * spend the *HTLC-success tx*, if the remote node has published it. - MUST *resolve* the *remote node's HTLC-*

timeout transaction by spending it using the revocation private key. - MUST *resolve* the *remote node's HTLC-success transaction* by spending it using the revocation private key. - SHOULD extract the payment preimage from the transaction input witness, if it's not already known. - if `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` is used for HTLC transactions (i.e. `option_anchors` or `zero_fee_commitments` applies): - MAY use a single transaction to *resolve* all the outputs. - If `zero_fee_commitments` applies: - MUST NOT exceed the 10kvB limit of v3 transactions, otherwise the transaction may not reach miners (as explained in the section about v3 limitations below). - if confirmation doesn't happen before reaching `security_delay` blocks from expiry: - SHOULD *resolve* revoked outputs in their own, separate penalty transactions. A previous penalty transaction claiming multiple revoked outputs at once may be blocked from confirming because of a transaction pinning attack. - otherwise: - MAY use a single transaction to *resolve* all the outputs. - MUST handle its transactions being invalidated by HTLC transactions.

Rationale

A single transaction that resolves all the outputs will be under the standard size limit because of the 483 HTLC-per-party limit (see BOLT #2).

Note: when `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` is used for HTLC transactions (i.e. `option_anchors` or `zero_fee_commitments`), the cheating node can pin spends of its HTLC-timeout/HTLC-success outputs. Using a single penalty transaction for all revoked outputs is thus unsafe as it could be blocked to propagate long enough for the *local node's* `to_local` *output* 's relative locktime to expire and the cheating party escaping the penalty on this output. Though this situation doesn't prevent faithful punishment of the second-level revoked output if the pinning transaction confirms.

The `security_delay` is a fixed-point relative to the absolute expiration of the revoked output at which the punishing node must broadcast a single-spend transaction for the revoked output and actively fee-bump it until its confirmation. The exact value of `security_delay` is left as a matter of node policy, though we recommend 18 blocks (similar to incoming HTLC deadline).

Penalty Transactions Weight Calculation

There are three different scripts for penalty transactions, with the following witness weights (details of weight computation are in Appendix A):

```
to_local_penalty_witness: 160 bytes
offered_htlc_penalty_witness: 243 bytes
accepted_htlc_penalty_witness: 249 bytes
```

The penalty *txinput* itself takes up 41 bytes and has a weight of 164 bytes, which results in the following weights for each input:

```
to_local_penalty_input_weight: 324 bytes
offered_htlc_penalty_input_weight: 407 bytes
accepted_htlc_penalty_input_weight: 413 bytes
```

The rest of the penalty transaction takes up 4+1+1+8+1+34+4=53 bytes of non-witness data: assuming it has a pay-to-witness-script-hash (the largest standard output script), in addition to a 2-byte witness header.

In addition to spending these outputs, a penalty transaction may optionally spend the commitment transaction's `to_remote` output (e.g. to reduce the total amount paid in fees). Doing so requires the inclusion of a P2WPKH witness and an additional *txinput*, resulting in an additional 108 + 164 = 272 bytes.

In the worst case scenario, the node holds only incoming HTLCs, and the HTLC-timeout transactions are not published, which forces the node to spend from the commitment transaction.

With a maximum standard weight of 400000 bytes, the maximum number of HTLCs that can be swept in a single transaction is as follows:

```
max_num_htlcs = (400000 - 324 - 272 - (4 * 53) - 2) / 413 = 966
```

Thus, 483 bidirectional HTLCs (containing both `to_local` and `to_remote` outputs) can be resolved in a single penalty transaction. Note: even if the `to_remote` output is not swept, the resulting `max_num_htlcs` is 967; which yields the same unidirectional limit of 483 HTLCs.

Generation of HTLC Transactions

If `option_anchors` and `zero_fee_commitments` do not apply, then HTLC-timeout and HTLC-success transactions are signed with `SIGHASH_ALL` and are complete transactions with (hopefully!) reasonable fees and must be used directly.

Otherwise, `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` MUST be used on the HTLC signatures received from the peer, as this allows HTLC transactions to be combined with other transactions. The local signature MUST use `SIGHASH_ALL`, otherwise anyone can attach additional inputs and outputs to the tx.

If `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` is used, then the HTLC-timeout and HTLC-success transactions are signed with the input and output having the same value. This means they have a zero fee and MUST be combined with other inputs to arrive at a reasonable fee.

Requirements

A node which broadcasts an HTLC-success or HTLC-timeout transaction for a commitment transaction: - if `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` is used: - MUST combine it with inputs contributing sufficient fee to ensure timely inclusion in a block. - MAY combine it with other transactions.

General Requirements

A node: - upon discovering a transaction that spends a funding transaction output which does not fall into one of the above categories (mutual close, unilateral close, or revoked transaction close): - MUST warn the user of potentially lost funds. - Note: the existence of such a rogue transaction implies that its private key has leaked and that its funds may be lost as a result. - MAY simply monitor the contents of the most-work chain for transactions. - Note: on-chain HTLCs should be sufficiently rare that speed need not be considered critical. - MAY monitor (valid) broadcast transactions (a.k.a the mempool). - Note: watching for mempool transactions should result in lower latency HTLC redemptions.

Limitations of v3 transactions

When `zero_fee_commitments` applies, we use v3 transactions, also known as TRUC transactions, which have different policy rules than v2 transactions.

Implementers should read BIP-431 to make sure they don't create transactions that aren't relayed by bitcoin nodes.

The main limitation compared to v2 transactions (but not the only one) is that transactions are restricted to 10kvB (instead of 400kvB): when batching transactions (e.g. HTLC transactions), nodes MUST NOT exceed that limit, otherwise the transaction will not reach miners and funds may be at risk.

Appendix A: Expected Weights

Expected Weight of the `to_local` Penalty Transaction Witness

As described in BOLT #3, the witness for this transaction is:

```
<sig> 1 { OP_IF <revocationpubkey> OP_ELSE to_self_delay OP_CSV OP_DROP <local_delayedpubkey>
OP_ENDIF OP_CHECKSIG }
```

The *expected weight* of the `to_local` penalty transaction witness is calculated as follows:

```
to_local_script: 83 bytes
    - OP_IF: 1 byte
        - OP_DATA: 1 byte (revocationpubkey length)
        - revocationpubkey: 33 bytes
    - OP_ELSE: 1 byte
        - OP_DATA: 1 byte (delay length)
        - delay: 8 bytes
        - OP_CHECKSEQUENCEVERIFY: 1 byte
        - OP_DROP: 1 byte
        - OP_DATA: 1 byte (local_delayedpubkey length)
        - local_delayedpubkey: 33 bytes
    - OP_ENDIF: 1 byte
    - OP_CHECKSIG: 1 byte

to_local_penalty_witness: 160 bytes
    - number_of_witness_elements: 1 byte
    - revocation_sig_length: 1 byte
    - revocation_sig: 73 bytes
    - one_length: 1 byte
    - witness_script_length: 1 byte
    - witness_script (to_local_script)
```

Expected Weight of the `offered_htlc` Penalty Transaction Witness

The *expected weight* of the `offered_htlc` penalty transaction witness is calculated as follows (some calculations have already been made in BOLT #3):

```
offered_htlc_script: 133 bytes

offered_htlc_penalty_witness: 243 bytes
    - number_of_witness_elements: 1 byte
    - revocation_sig_length: 1 byte
    - revocation_sig: 73 bytes
    - revocation_key_length: 1 byte
    - revocation_key: 33 bytes
```

```
    - witness_script_length: 1 byte
    - witness_script (offered_htlc_script)
```

Expected Weight of the `accepted_htlc` Penalty Transaction Witness

The *expected weight* of the `accepted_htlc` penalty transaction witness is calculated as follows (some calculations have already been made in BOLT #3):

```
accepted_htlc_script: 139 bytes

accepted_htlc_penalty_witness: 249 bytes
    - number_of_witness_elements: 1 byte
    - revocation_sig_length: 1 byte
    - revocation_sig: 73 bytes
    - revocationpubkey_length: 1 byte
    - revocationpubkey: 33 bytes
    - witness_script_length: 1 byte
    - witness_script (accepted_htlc_script)
```

Authors

[FIXME:]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

Finding the Network

Gossip tells you about nodes you've already heard of. It doesn't tell you how to find the first one.

BOLT 10 is DNS bootstrap. Lightning nodes can publish SRV and node records under a seed domain so a new peer can resolve a handful of entry points, connect, and start receiving gossip. It's a small document, and it's optional in spirit — you can always paste a peer address by hand — but it's how a node joins the network without a phone book.

If you're running a seed, this BOLT is the contract you have with the rest of the network. If you're writing a client, it's the one-time lookup before BOLT 07 takes over.

In this chapter

- BOLT 10 — DNS bootstrap

BOLT #10: DNS Bootstrap and Assisted Node Location

Overview

This specification describes a node discovery mechanism based on the Domain Name System (DNS). Its purpose is twofold:

- Bootstrap: providing the initial node discovery for nodes that have no known contacts in the network

- Assisted Node Location: supporting nodes in discovery of the current network address of previously known peers

A domain name server implementing this specification is referred to as a *DNS Seed* and answers incoming DNS queries of type `A`, `AAAA`, or `SRV`, as specified in RFCs 1035[1], 3596[2], and 2782[3], respectively. The DNS server is authoritative for a subdomain, referred to as a *seed root domain*, and clients may query it for subdomains.

The subdomains consist of a number of dot-separated *conditions* that further narrow the desired results.

Table of Contents

- DNS Seed Queries
    - Query Semantics
- Reply Construction
- Policies
- Examples
- References
- Authors

DNS Seed Queries

A client MAY issue queries using the `A`, `AAAA`, or `SRV` query types, specifying conditions for the desired results the seed should return.

Queries distinguish between *wildcard* queries and *node* queries, depending on whether the `l`-key is set or not.

Query Semantics

The conditions are key-value pairs: the key is a single-letter, while the remainder of the key-value pair is the value. The following key-value pairs MUST be supported by a DNS seed:

- `r`: realm byte
    - used to specify what realm the returned nodes must support
    - default value: 0 (Bitcoin)
- `a`: address types
    - a bitfield that uses the types from BOLT #7 as bit index
    - used to specify what address types should be returned for `SRV` queries
    - MAY only be used for `SRV` queries
    - default value: 6 (i.e. `2 || 4`, since bit 1 and bit 2 are set for IPv4 and IPv6, respectively)
- `l`: `node_id`
    - a bech32-encoded `node_id` of a specific node
    - used to ask for a single node instead of a random selection
    - default value: null
- `n`: number of desired reply records
    - default value: 25

Conditions are passed in the DNS seed query as individual, dot-separated subdomain components.

For example, a query for `r0.a2.n10.lseed.bitcoinstats.com` would imply: return 10 (`n10`) IPv4 (`a2`) records for nodes supporting Bitcoin (`r0`).

Requirements

The DNS seed: - MUST evaluate the conditions from the *seed root domain* by 'going up-the-tree', i.e. evaluating right-to-left in a fully qualified domain name. - E.g. to evaluate the above case: first evaluate `n10`, then `a2`, and finally `r0`. - if a condition (key) is specified more than once: - MUST discard any earlier value for that condition AND use the new value instead. - E.g. for `n5.r0.a2.n10.lseed.bitcoinstats.com`, the result is: ~~`n10`~~, `a2`, `r0`, `n5`. - SHOULD return results that match all conditions. - if it does NOT implement filtering by a given condition: - MAY ignore the condition altogether (i.e. the seed filtering is best effort only). - for `A` and `AAAA` queries: - MUST return only nodes listening on the default port 9735, as defined in BOLT #1. - for `SRV` queries: - MAY return nodes that are listening on non-default ports, since `SRV` records return a *(hostname,port)*-tuple. - upon receiving a *wildcard* query: - MUST select a random subset of up to `n` IPv4 or IPv6 addresses of nodes that are listening for incoming connections. - upon receiving a *node* query: - MUST select the record matching the `node_id`, if any, AND return all addresses associated with that node.

Querying clients: - MUST NOT rely on any given condition being met by the results.

Reply Construction

The results are serialized in a reply with a query type matching the client's query type. For example, `A`, `AAAA`, and `SRV` queries respectively result in `A`, `AAAA`, and `SRV` replies. Additionally, replies may be augmented with additional records (e.g. to add `A` or `AAAA` records matching the returned `SRV` records).

For `A` and `AAAA` queries, the reply contains the domain name and the IP address of the results.

The domain name MUST match the domain in the query, in order not to be filtered by intermediate resolvers.

For `SRV` queries, the reply consists of (*virtual hostnames*, port)-tuples. A virtual hostname is a subdomain of the seed root domain that uniquely identifies a node in the network. It is constructed by prepending the `node_id` condition to the seed root domain.

The DNS seed: - MAY additionally return the corresponding `A` and `AAAA` records that indicate the IP address for the `SRV` entries in the additional section of the reply. - MAY omit these additional records upon detecting a repeated query. - Reason: due to the large size of the resulting reply, the reply may be dropped by intermediate resolvers. - if no entries match all the conditions: - MUST return an empty reply.

Policies

The DNS seed: - MUST NOT return replies with a TTL less than 60 seconds. - MAY filter nodes from its local views for various reasons, including faulty nodes, flaky nodes, or spam prevention. - MUST reply to random queries (i.e. queries to the seed root domain and to the `_nodes._tcp.` alias for `SRV` queries) with *random and unbiased* samples from the set of all known good nodes, in accordance with the Bitcoin DNS Seed policy[4].

Examples

Querying for `AAAA` records:

```
$ dig lseed.bitcoinstats.com AAAA
lseed.bitcoinstats.com. 60      IN      AAAA    2a02:aa16:1105:4a80:1234:1234:37c1:9c9
```

Querying for SRV records:

```
$ dig lseed.bitcoinstats.com SRV
lseed.bitcoinstats.com. 59   IN      SRV     10 10 6331
ln1qwktpe6jxltmpphyl578eax6fcjc2m807qalr76a5gfmx7k9qqfjwy4mctz.lseed.bitcoinstats.com.
lseed.bitcoinstats.com. 59   IN      SRV     10 10 9735
ln1qv2w3tledmzczw227nnkqrrltvmydl8gu4w4d70g9td7avke6nmz2tdefqp.lseed.bitcoinstats.com.
lseed.bitcoinstats.com. 59   IN      SRV     10 10 9735
ln1qtynyymv99pqf0r9cuexvvqtxrlgejuecf8myfsa96vcpflgll5cqmr2xsu.lseed.bitcoinstats.com.
lseed.bitcoinstats.com. 59   IN      SRV     10 10 4280
ln1qdfvlysfpyh96apy3w3qdwlu8jjkdhnuxa689ka540tnde6gnx86cf7ga2d.lseed.bitcoinstats.com.
lseed.bitcoinstats.com. 59   IN      SRV     10 10 4281
ln1qwf789tlcpe4n34649xrqllxt97whsvfk5pm07ggqms3vrjwdj3cu6332zs.lseed.bitcoinstats.com.
```

Querying for the `A` for the first virtual hostname from the previous example:

```
$ dig ln1qwktpe6jxltmpphyl578eax6fcjc2m807qalr76a5gfmx7k9qqfjwy4mctz.lseed.bitcoinstats.com A
ln1qwktpe6jxltmpphyl578eax6fcjc2m807qalr76a5gfmx7k9qqfjwy4mctz.lseed.bitcoinstats.com. 60 IN A
139.59.143.87
```

Querying for only IPv4 nodes (`a2`) via seed filtering:

```
$dig a2.lseed.bitcoinstats.com SRV
a2.lseed.bitcoinstats.com. 59   IN  SRV 10 10 9735
ln1q2jy22cg2nckgxttjf8txmamwe9rtw325v4m04ug2dm9sxlrh9cagrrpy86.lseed.bitcoinstats.com.
a2.lseed.bitcoinstats.com. 59   IN  SRV 10 10 9735
ln1qfrkq32xayuq63anmc2zp5vtd2jxafhdzzudmuws0hvxshtgd2zd7jsqv7f.lseed.bitcoinstats.com.
```

Querying for only IPv6 nodes (`a4`) supporting Bitcoin (`r0`) via seed filtering:

```
$dig r0.a4.lseed.bitcoinstats.com SRV
r0.a4.lseed.bitcoinstats.com. 59 IN SRV 10 10 9735
ln1qwx3prnvmxuwsnaqhzwsrrpwy4pjf5m8fv4m8kcjkdvyrzymlcmj5dakwrx.lseed.bitcoinstats.com.
r0.a4.lseed.bitcoinstats.com. 59 IN SRV 10 10 9735
ln1qwr7x7q2gvj7kwzzr7urqq9x7mq0lf9xn6svs8dn7q8gu5q4e852znqj3j7.lseed.bitcoinstats.com.
```

References

- RFC 1035 - Domain Names
- RFC 3596 - DNS Extensions to Support IP Version 6
- RFC 2782 - A DNS RR for specifying the location of services (DNS SRV)
- Expectations for DNS Seed operators

Authors

[FIXME: Insert Author List]

Creative Commons License
This work is licensed under a Creative Commons Attribution 4.0 International License.

Newer Work

Lightning's original channels use a 2-of-2 `OP_CHECKMULTISIG` funding output. It works. It also looks like a Lightning channel on-chain, and it predates Taproot.

Simple Taproot Channels move funding into a Taproot output. Cooperative closes look like a single-key spend. The commitment format changes, musig2 shows up, and a few BOLT 02 / BOLT 03 messages grow extra fields. The rest of the protocol — invoices, onions, gossip — stays the same.

This document is newer than the numbered BOLTs, and implementations are still catching up. Read it after you're comfortable with BOLT 02 and BOLT 03, because it's a variation on those two, not a replacement for the rest of the book.

Treat it as a look at where channel construction is heading, not as required reading for a first implementation.

In this chapter

- Simple Taproot Channels

Extension BOLT XX: Simple Taproot Channels

Authors: * Olaoluwa Osuntokun roasbeef@lightning.engineering * Eugene Siegel eugene@lightning.engineering

Created: 2022-04-20

Table of Contents

- Introduction
  - Abstract
  - Motivation
  - Preliminaries
    - Pay-To-Taproot-Outputs
      - Tapscript Tree Semantics
      - BIP 86
      - Taproot Key Path Spends
      - Taproot Script Path Spends
    - MuSig2 & Associated Changes
      - Key Aggregation
      - Nonce Generation
      - Nonce Handling
      - Signing
    - Nothing Up My Sleeves Points
  - Design Overview
  - Specification
    - Feature Bits
    - New TLV Types

        - Channel Funding
            - `open_channel` Extensions
            - `accept_channel` Extensions
            - `funding_created` Extensions
            - `funding_signed` Extensions
            - `channel_ready` Extensions
        - RBF Cooperative Close
            - `shutdown` Extensions
            - JIT (Just-In-Time) Nonce Pattern
            - `closing_complete` Extensions
            - `closing_sig` Extensions
            - RBF Nonce Rotation Protocol
        - Channel Operation
            - `commitment_signed` Extensions
            - `revoke_and_ack` Extensions
        - Message Retransmission
            - `channel_reestablish` Extensions
        - Funding Transactions
        - Commitment Transactions
            - To Local Outputs
            - To Remote Outputs
            - Anchor Outputs
        - HTLC Scripts & Transactions
            - Offered HTLCs
            - Accepted HTLCs
            - HTLC Second Level Transactions
                - HTLC-Success Transactions
                - HTLC-Timeout Transactions
- Appendix
- Footnotes
- Test Vectors
- Acknowledgements

Introduction

Abstract

The activation of the Taproot soft-fork suite enables a number of updates to the Lightning Network, allowing developers to improve the privacy, security, and flexibility of the system. This document specifies extensions to BOLTs 2, 3, and 5 which describe new, Taproot based channels to update to the Lightning Network to take advantage of *some* of these new capabilities. Namely, we mechanically translate the current funding and commitment design to utilize `musig2` and the new tapscript tree capabilities.

Motivation

The activation of Taproot grants protocol developers with a number of new tools including: schnorr, musig2, scriptless scripts, PTLCs, merkalized script trees and more. While it's technically possible to craft a *single*

update to the Lightning Network to take advantage of *all* these new capabilities, we instead propose a step-wise update process, with each step layering on top of the prior with new capabilities. While the ultimate realization of a more advanced protocol may be delayed as a result of this approach, packing up smaller incremental updates will be easier to implement and review, and may also hasten the timeline to deploy initial taproot based channels.

In this document, we start by revising the most fundamental component of a Lightning Channel: the funding output. By first porting the funding output to Segwit V1 (P2TR) `musig2` based output, we're able to re-anchor all channels in the network with the help of dynamic commitments. Once those channels are re-anchored, dynamic commitments allows developers to ship incremental changes to the commitment transaction, HTLC structure, and overall channel structure – all without additional on-chain transactions.

By restricting the surface area of the *initial* update, we aim to expedite the initial roll-out while also providing a solid base for future updates without blocking off any interesting design paths. Implementing and shipping a constrained update also gives implementers an opportunity to get more familiar with the new tooling and capabilities, before tackling more advanced protocol designs.

Preliminaries

In this section we lay out some preliminary concepts, protocol flows, and notation that'll be used later in the core channel specification.

Pay-To-Taproot-Outputs

The Taproot soft fork package (BIPs 340, 341, and 342) introduced a new Segwit witness version: v1. This new witness version utilizes a new standardized public key script template:

```
OP_1 <taproot_output_key>
```

The `taproot_output_key` is a 32-byte x-only `secp256k1` public key, with all digital signatures based off of the schnorr signature scheme described in BIP 340.

A `taproot_output_key` commits to an internal key, and optional script root via the following mapping:

```
taproot_output_key = taproot_internal_key + tagged_hash("TapTweak", taproot_internal_key || script_root)*G
```

It's important to note that while `taproot_output_key` is serialized as a 32-byte public key, in order to properly spend the script path, the parity (even or odd) of the output public key must be remembered.

The `taproot_internal_key` is also a BIP 340 public key, ideally derived anew for each output. The `tagged_hash` scheme is described in BIP 340, we briefly define the function as:

```
tagged_hash(tag, msg) = SHA256(SHA256(tag) || SHA256(tag) || msg)
```

Tapscript Tree Semantics

The `script_root` is the root of a Tapscript tree as defined in BIP 341. A tree is composed of `tap_leaves` and `tap_branches`.

A `tap_leaf` is a two tuple of (`leaf_version`, `leaf_script`). A `tap_leaf` is serialized as:

```
leaf_version || compact_size_of(leaf_script) || leaf_script
```

where `compact_size_of` is the variable length integer used in the Bitcoin P2P protocol.

The digest of a `tap_leaf` is computed as:

```
tagged_hash("TapLeaf", leaf_encoding)
```

where `leaf_encoding` is serialized as:

```
leaf_version || compact_size_of(leaf_script) || leaf_script
```

A `tap_branch` can commit to either a `tap_leaf` or `tap_branch`. Before hashing, a `tap_branch` sorts the two `tap_node` arguments based on lexicographical ordering:

```
tagged_hash("TapBranch", sort(node_1, node_2))
```

A tapscript tree is constructed by hashing each pair of leaves into a `tap_branch`, then combining these branches with each other until only a single hash remains. There're no strict requirements at the consensus level for tree construction, allowing systems to "weight" the tree by placing items that occur more often at the top of the tree. A simple algorithm that attempts to produce a balanced script tree can be found here.

BIP 86

A `taproot_output_key` doesn't necessarily *need* to commit to a `script_root`, unless the `script_root` is being revealed, a normal key (skipping the tweaking operation) can be used. In many cases it's also nice to prove to a 3rd party that the output can only be spent with a top-level signature and not also a script path.

BIP 86 defines a taproot output key derivation scheme, wherein only the internal key is committed to without a script root:

```
taproot_output_key = taproot_internal_key + tagged_hash("TapTweak", taproot_internal_key)*G
```

We use BIP 86 whenever we want to ensure that a script path spend isn't possible, and an output can only be spent via a key path.

We recommend that in any instances where a normal unencumbered output is used (cooperative closures, etc) BIP 86 is used as well.

Taproot Key Path Spends

A taproot key path spend a transaction to spend a taproot output without revealing the script root, which may or may not exist.

The witness of a keypath spend is simply a 64 or 65 byte schnorr signature. A signature is 64 bytes if the new `SIGHASH_DEFAULT` alias for `SIGHASH_ALL` is being used. Otherwise a single byte for the sighash type is used as normal:

```
<key_path_sig>
```

If an output commits to a script root (or uses BIP 86), a private key will need to be tweaked in order to generate a valid signature, and negated if the resulting output key has an odd y coordinate (see BIP 341 for details).

For any revocation path based on a key spend (the HTLC revocation paths), the tap tweak value MUST be known in order to spend the out unilaterally. As a result, it's recommend that for a given revoked state, the tap tweak value is also stored.

Taproot Script Path Spends

A script path spend is executed when a tapscript leaf is revealed. A script path spends has three components: a control block, the script leaf being revealed, and the valid witness.

A control block has the following serialization:

```
(output_key_y_parity | leaf_version) || internal_key || inclusion_proof
```

The first segment uses a spare bit of the `leaf_version` to encode the parity of the output key, which is checked during control block verification. The `internal_key` is just that. The `inclusion_proof` is the series of sibling hashes in the path from the revealed leaf all the way up to the root of the tree. In practice, if one hashes the revealed leaf, and each 32-byte hash of the inclusion proof together, they'll reach the script root if the proof was valid.

If only a single leaf is committed to in a tree, then the `inclusion_proof` will be absent.

The final witness stack to spend a script path output is:

```
<witness1> ... <witnessN> <leaf_script> <control_block>
```

Note that whenever a script path spend is required, in order to ensure an output is unilaterally spendable by a party, the finalized control block MUST either be stored as is, or the components needed to reconstruct the control block are persisted.

SIGHASH_ALL vs SIGHASH_DEFAULT

Where ever applicable, signers SHOULD use the `SIGHASH_DEFAULT` sighash over `SIGHASH_ALL`. When using taproot semantics, both algorithm refer to the very same digest, but `SIGHASH_DEFAULT` allows us to save a byte for any signature, as omitting the explicit sighash byte denotes that the default signing algorithm was used.

MuSig2 & Associated Changes

Musig2 is a multi-signature scheme based on schnorr signatures. It allows N parties to construct an aggregated public key, and then generate a single multi-signature that can only be generated with all the parties of the aggregated key. In practice, we use this to allow the funding output of channel to look just like any other P2TR output.

A musig2 signing flow has 4 stages:

1. First, each party generates a public nonce, and exchanges it with every other party.

2. Next, public keys are exchanged by each party, and combined into a single aggregated public key.

3. Next, each party generates a *partial* signature and exchanges it with every other party.

4. Finally, once all partial signatures are known, they can be combined into a valid schnorr Signature, which can be verified using BIP 340.

Steps 1&2 may be performed out of order, or concurrently. In our case only two parties exist, so as soon as one party knows both partial signatures, they can be combined into a final signature.

Thought this document `musig2` refers to the finalized (v1.0.0) [BIP-0327](https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki specification.

Key Aggregation

We refer to the algorithm of `KeyAgg` algorithm bip-musig2 v1.0.0 for key aggregation. In order to avoid sending extra ordering information, we always assume that both keys are sorted first with the `KeySort` algorithm before aggregating (`KeyAgg(KeySort(p1, p2))`).

Nonce Generation

A `musig2` secret nonce is the concatenation of two random, 32-byte integers.

A `musig2` public nonce is technically the concatenation of two public keys, each representing the EC-point corresponding to its secret integer, thus resulting in a 66-byte value:

```
point_1 || point_2
```

For nonce generation, we refer to the `NonceGen` algorithm defined in the `musig2` BIP. We only specify the set of required arguments, though a signer can also opt to specify the additional argument, in order to strengthen their nonce.

Counter Based Nonce Generation & JIT Nonce Strengthening

Each new commitment state requires fresh nonces from both parties, as each commitment state is signed using a fresh musig2 session. Nonces sent by either party in the `commitment_signed` message can be generated Just-In-Time (JIT), thus requiring no additional state. These nonces can also be further strengthened by mixing in the commitment sighash information into the nonce generation algorithm.

Upon channel creation, and subsequent re-establishment, a fresh set of verification nonces are exchanged by each party. This is the nonce the party will use to *verify* a new incoming commitment state sent by the remote party. In order to force close a channel, the holder of a verification nonce must use that same nonce to counter sign the commitment transaction with the other half of the musig2 partial signature. Rather than force an implementation to retain additional signing state (the verification nonce) to avoid holding a “hot” (fully broadcast able) commitment on disk, we instead recommend a counter based approach.

In this scenario are nonce generated via a counter is deemed to be safe as:

1. Each commitment state only has a single signature exchanged to counter sign the commitment transaction.

2. Each commitment state has a unique number which is currently encoded as a 48-bit integer across the sequence and lock time of the commitment transaction.

3. The shachain scheme is used today to generate a fresh nonce-like value for the revocation scheme of today's penalty based channels.

4. A verification nonce is only used to *sign* a local party's commitment transaction when they go to broadcast a force close transaction.

5. The remote party never sees the unaggregated partial signature the local party will use to broadcast, only the aggregated signature is seen on chain.

6. As a result of all the above, a verification nonce will never be used to sign the exact same commitment transaction, thereby avoiding nonce reuse.

As a result, we recommend the following counter based scheme to generate verification nonce. With this scheme, signing can remain stateless, as given the commitment height/number of the state to sign, and the original shachain root, the verification nonce sent by that state can be deterministically reproduced:

1. Given the shachain root used to generate revocation pre-images, `shachain_root`, derive the `sha256` hash of this value as: `shachain_root_hash = sha256(shachain_root)`.

2. Derive a *new* shachain root to be used to generate `musig2` secret nonces via a `HMAC` invocation as: `musig2_shachain_root = hmac(msg, shachain_root_hash)`, where `msg` is any string that can serve to uniquely bind the produced secret to this dedicated context. A recommended value is the ASCII string `taproot-rev-root` concatenated with the `funding_txid` (which allows deriving distinct deterministic shachains when splicing).

3. Given a commitment height/number (`N`), the verification nonce to send to the remote party party can be derived by obtaining the `Nth` shachain leaf preimage `k_i`. The verification nonce to be derived by calling the `musig2.NonceGen` algorithm with the required values, and the `rand'` value set to `k_i`.

Nonce Handling

For commitment transactions, Due to the asymmetric state of the current revocation channels, two sets of public nonces need to be exchanged by each party: one for the local commitment, and one for the remote commitment.

The nonce for the local commitment MUST be sent a priori, thus enabling the counterparty to create a valid partial signature.

The nonce for the remote commitment *could* be sent a priori, too, but for the purposes of simplicity and disambiguation, MUST be sent alongside the partial signature. To that end, we will create a new data type that will be reused in multiple TLV extensions described further down in this document.

The exception to this is the co-op close flow: as there's only a single message to sign, we only require a single pair of nonces. These nonces can also be sent right as the co-op flow begins.

Note that if nonces for commitment updates are not generated using the counter based scheme described above, then the nonces MUST be persisted to disk. As a result, in order to minimize additional signing state, it is recommended that the shachain counter nonce scheme is used.

Nonce Terminology

At all times, there exist *two* nonces associated with a given commitment state: the signing nonce, and the verification nonce.

A peer's signing nonce is used to generate new commitments for the remote party. Each time a party signs a commitment, they send another signing nonce to the remote party.

A peer's verification nonce is used to verify incoming commitments sent by the remote party. Once a peer verifies and revoke a new commitment, another verification nonce is sent to the remote party.

Both sides will maintain a `musig2` sessions for their local commitment and the commitment of the remote party. A peer's local commitment uses their verification nonce, and the signing nonce of the remote party. The remote commitment of the channel peer uses the peer's signing nonce, and their verification nonce.

In order to sign and broadcast a new commitment to force close a channel, a peer uses their *verification* nonce to generate their final signature.

Signing

Once public nonces have been exchanged, the `NonceAgg` algorithm is used to combine the public nonces into the aggregate nonce which will be a part of the final signature.

The `Sign` method is used to generate a valid partial signature, with the `PartialSigVerify` algorithm used to verify each signature.

Once all partial signatures are known, the `PartialSigAgg` algorithm is used to aggregate the partial signature into a final, valid BIP 340 Schnorr signature.

Partial Signature Encoding

A standard `musig2` partial signature is simply the encoding of the `s` value, which is a 32-byte-element modulo the order of the group.

Throughout this specification, we will be using a custom type, `PartialSignatureWithNonce`, comprised of the aforementioned `s` value, along with the 66-byte-encoding of the public nonce used to sign remote commitments:

```
s || point_1 || point_2
```

Nothing Up My Sleeves Points

Whenever we want to ensure that a given P2TR can *only* be spent via a script path, we utilize a Nothing Up My Sleeve (NUMS) point. A NUMS point is an EC point that no one knows the private key to. If no one knows the private key, then it can't be used for key path signing, forcing the script path to always be taken.

We refer to the `simple_taproot_nums` as the following value:

```
02dca094751109d0bd055d03565874e8276dd53e926b44e3bd1bb6bf4bc130a279
```

The value was generated using this tool, with the seed phrase "Lightning Simple Taproot".

Design Overview

With the preliminaries out of the way, we provide a brief overview of the Simple Taproot Channel design.

The multisig output becomes a single P2TR key, with `musig2` key aggregation used to combine two keys into one.

For commitment transactions, we inherit the anchor output semantics, meaning there are two outputs used for CPFP, with all other scripts inheriting a `1 CSV` restriction.

The local output of the commitment transaction uses a script-path based revocation scheme in order to ensure that the information needed by 3rd parties to sweep the anchor outputs is always revealed on chain.

The remote output of the commitment transaction uses the `combined_funding_key` as the top-level internal key, and then commits to a normal remote script.

Anchor outputs use the `local_delayedpubkey` and the `remotepubkey` of both parties as the top-level internal key committing to the script `16 CSV`. Unless active HTLCs exist on the commitment transaction, if a local or remote output is missing from the commitment, its corresponding anchor must not be present, either.

All HTLC scripts use the revocation key as the top-level internal key. This allows the revocation to be executed without additional on-chain bytes, and also reduces the amount of state that nodes need to keep in order to properly handle channel breaches.

Specification

Feature Bits

Inheriting the structure put forth in BOLT 9, we define a new feature bit to be placed in the `init` message, and the `node_announcement` message.

Bits	Name	Description	Context	Dependencies	Link
80/81	`option_simple_taproot`	Node supports simple taproot channels	IN	`option_channel_type`, `option_simple_close`	TODO(roasbeef): link
180/181	`option_simple_taproot_staging`	Node supports simple taproot channels	IN	`option_channel_type`	TODO(roasbeef): link

Note that we allocate *two* pairs of feature bits: one the final version of this protocol proposal, and the higher bits (+100) for preliminary experimental deployments before this protocol extension has been finalized. This +100 feature bit is thus refereed to as the staging feature bit.

The Context column decodes as follows: * `I`: presented in the `init` message. * `N`: presented in the `node_announcement` messages * `C`: presented in the `channel_announcement` message. * `C–`: presented in the

`channel_announcement` message, but always odd (optional). * `C+`: presented in the `channel_announcement` message, but always even (required). * `9`: presented in BOLT 11 invoices.

The `option_simple_taproot` feature bit also becomes a defined channel type feature bit for explicit channel negotiation.

It's important to note that given the early version of this channel type *cannot* be announced on the public network (gossip protocol changes are required), the taproot channels type cannot be used as an interchangeable default channel type. As a result, this channel type SHOULD only be used with *explicit* channel negotiation. Otherwise, two parties that advertise the feature bit would not be able to open publicly advertised channels.

Throughout this document, we assume that `option_simple_taproot` was negotiated, and also the `option_simple_taproot` channel type is used.

New TLV Types

Note that these TLV types exist across different messages, but their type IDs are always the same.

partial_signature_with_nonce

- type: 2
- data:
    - [`32*byte`: `partial_signature`]
    - [`66*byte`: `public_nonce`]

next_local_nonce

- type: 4
- data:
    - [`66*byte`: `public_nonce`]

partial_signature

- type: 6
- data:
    - [`32*byte`: `partial_signature`]

shutdown_nonce

- type: 8
- data:
    - [`66*byte`: `public_nonce`]

next_local_nonces

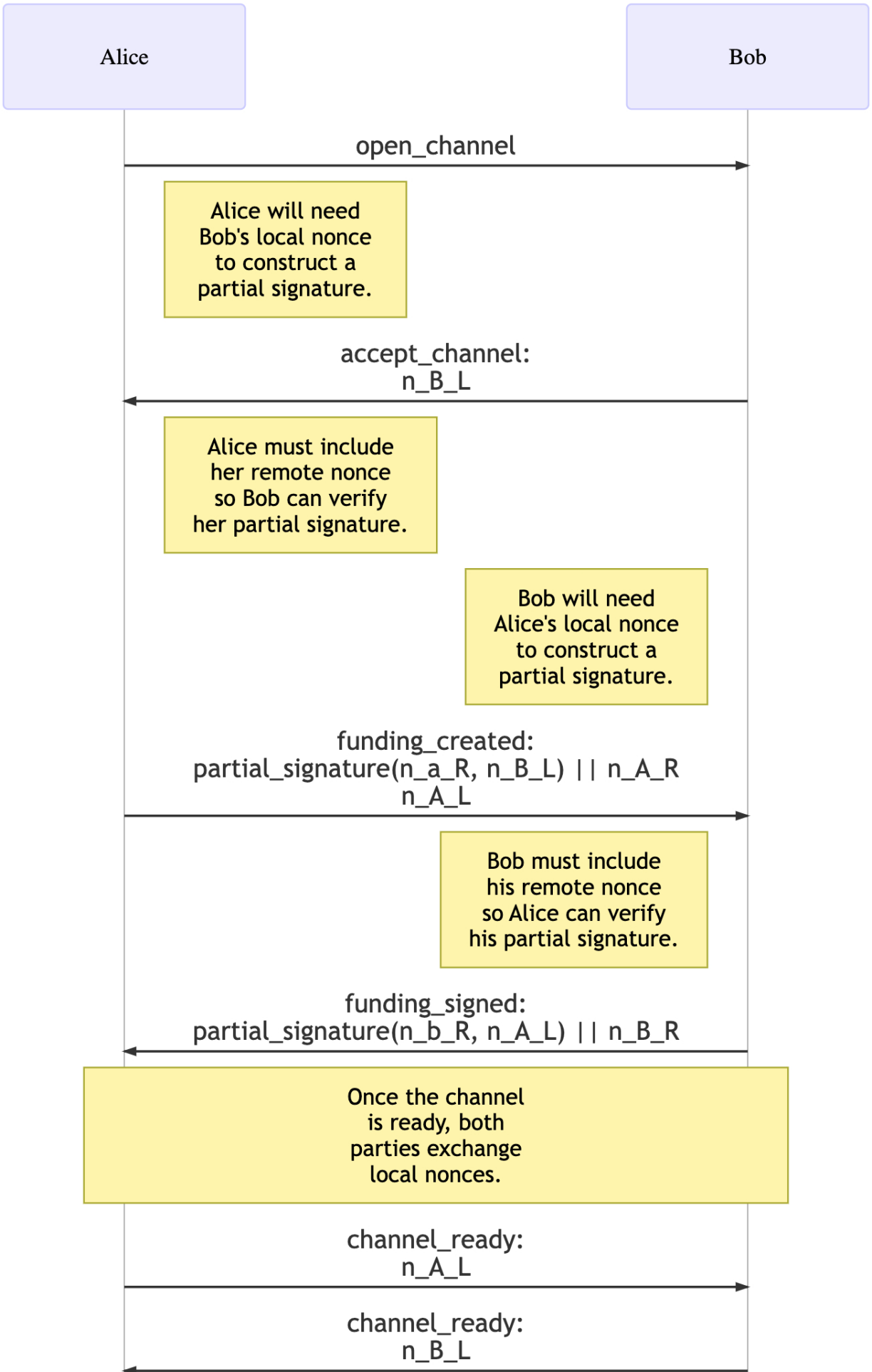
- type: 22
- data:
    - [`...*nonce_entry`: `entries`]

where `nonce_entry` is: * [`32*byte`: `funding_txid`] * [`66*byte`: `public_nonce`]

Channel Funding

`n_a_L`: Alice's local secret nonce

`n_B_R`: Bob's remote public nonce


Alice
Bob
open_channel
Alice will need Bob's local nonce to construct a partial signature.
accept_channel: n_B_L
Alice must include her remote nonce so Bob can verify her partial signature.
Bob will need Alice's local nonce to construct a partial signature.
funding_created: partial_signature(n_a_R, n_B_L) || n_A_R n_A_L
Bob must include his remote nonce so Alice can verify his partial signature.
funding_signed: partial_signature(n_b_R, n_A_L) || n_B_R
Once the channel is ready, both parties exchange local nonces.
channel_ready: n_A_L
channel_ready: n_B_L

The `open_channel` and `accept_channel` messages are extended to specify a new TLV type that houses the `musig2` public nonces.

We add `option_simple_taproot` to the set of defined channel types.

`open_channel` Extensions

1. `tlv_stream`: `open_channel_tlvs`
2. types:
    1. type: 4 (`next_local_nonce`)
    2. data:
        - [`66*byte`:`public_nonce`]

Requirements

The sending node:

- MUST specify the `next_local_nonce` field.
- MUST use the `NonceGen` algorithm defined in `bip-musig2` to generate `next_local_nonce` to ensure it generates nonces in a safe manner.
- MUST not set the `announce_channel` bit.

The receiving node MUST fail the channel if:

- the message doesn't include a `next_local_nonce` value.
- the specified public nonce cannot be parsed as two compressed secp256k1 points

Rationale

For the initial Taproot channel type (`option_simple_taproot`), musig2 nonces must be exchanged in the first round trip (open->, <-accept). This is done to ensure to ensure that the initiator is able to generate a valid musig2 partial signature at the next step once the transaction to be signed is fully specified.

The `next_local_nonce` field will be used to verify incoming commitment signatures. Each incoming signature will carry the newly generated nonce used to sign the commitment transaction. At force close broadcast time, the verification nonce is then used to sign the local party's commitment transaction.

`accept_channel` Extensions

1. `tlv_stream`: `accept_channel_tlvs`
2. types:
    1. type: 4 (`next_local_nonce`)
    2. data:
        - [`66*byte`:`public_nonce`]

Requirements

The sender:

- MUST set `next_local_nonce` to the `musig2` public nonce used to sign local commitments as specified by the `NonceGen` algorithm of `bip-musig2`.

The recipient:

- MUST reject the channel if `next_local_nonce` is absent, or cannot be parsed as two compressed secp256k1 points

`funding_created` Extensions

1. `tlv_stream`: `funding_created_tlvs`
2. types:
    1. type: 2 (`partial_signature_with_nonce`)
    2. data:
        - [`98*byte`: `partial_signature || public_nonce`]

Requirements

The sender:

- MUST set the original, non-TLV `signature` field to a 0-byte-array of length

  64.

- MUST sort the exchanged `funding_pubkeys` using the `KeySort` algorithm from `bip-musig2`.
- MUST compute the aggregated `musig2` public key from the sorted `funding_pubkeys` using the `KeyAgg` algorithm from `bip-musig2`.
- MUST generate a unique nonce to combine with the `next_local_nonce` previously received in `accept_channel` using `NonceAgg` from `bip-musig2`.
- MUST use the generated secret nonce and the calculated aggregate nonce to construct a `musig2` partial signature for the sender's remote commitment using the `Sign` algorithm from `bip-musig2`.
- MUST include the partial signature and the public counterpart of the generated nonce in the `partial_signature_with_nonce` field.

The recipient:

- MUST fail the channel if `partial_signature_with_nonce` is absent.
- MUST fail the channel if `next_local_nonce` is absent.
- MUST compute the aggregate nonce from:
    - the `next_local_nonce` field the recipient previously sent in `accept_channel`

    - the `public_nonce` included as part of the `partial_signature_with_nonce` field

- MUST verify the `partial_signature_with_nonce` field using the `PartialSigVerifyInternal` algorithm of `bip-musig2`:
    - if the partial signature is invalid, MUST fail the channel

`funding_signed` Extensions

1. `tlv_stream`: `funding_signed_tlvs`
2. types:
    1. type: 2 (`partial_signature_with_nonce`)
    2. data:
        - [`98*byte`: `partial_signature || public_nonce`]

Requirements

The sender:

- MUST set the original, non-TLV `signature` field to a 0-byte-array of length

  64.

- MUST sort the exchanged `funding_pubkeys` using the `KeySort` algorithm from `bip-musig2`.
- MUST compute the aggregated `musig2` public key from the sorted `funding_pubkeys` using the `KeyAgg` algorithm from `bip-musig2`.
- MUST generate a unique nonce to combine with the `next_local_nonce` previously received in `open_channel` using `NonceAgg` from `bip-musig2`.
- MUST use the generated secret nonce and the calculated aggregate nonce to construct a `musig2` partial signature for the sender's remote commitment using the `Sign` algorithm from `bip-musig2`.
- MUST include the partial signature and the public counterpart of the generated nonce in the `partial_signature_with_nonce` field.

The recipient:

- MUST fail the channel if `partial_signature_with_nonce` is absent.
- MUST compute the aggregate nonce from:
    - the `next_local_nonce` field the recipient previously sent in `open_channel`
    - the `public_nonce` included as part of the `partial_signature_with_nonce` field
- MUST verify the `partial_signature_with_nonce` field using the `PartialSigVerifyInternal` algorithm of `bip-musig2`:
    - if the partial signature is invalid, MUST fail the channel

`channel_ready` Extensions

We add a new TLV field to the `channel_ready` message:

1. `tlv_stream`: `channel_ready_tlvs`
2. types:
    1. type: 4 (`next_local_nonce`)
    2. data:
        - [`66*byte`: `public_nonce`]

Similar to the `next_per_commitment_point`, by sending the `next_local_nonce` value in this message, we ensure that the remote party has our public nonce which is required to generate a new commitment signature.

Requirements

The sender:

- MUST set `next_local_nonce` to a fresh, unique `musig2` nonce as specified by `bip-musig2`

The recipient:

- MUST fail the channel if `next_local_nonce` is absent.

RBF Cooperative Close

For simple taproot channels, the cooperative close protocol uses `closing_complete` and `closing_sig` messages (as specified in BOLT #2) with extensions for MuSig2 signature generation and RBF (Replace-By-Fee) iterations.

`shutdown` Extensions

For taproot channels, the shutdown message includes a single nonce:

1. type: 38 (`shutdown`)
2. data:
    - …
    - [`shutdown_tlvs:tlvs`]
3. `tlv_stream`: `shutdown_tlvs`
4. types:
    1. type: 8 (`shutdown_nonce`)
    2. data:
        - [`66*byte:public_nonce`]

The `shutdown_nonce` represents the sender's "closee nonce" - the nonce they will use when sending `closing_sig` messages.

Additional nonces are provided just-in-time (JIT) with signatures:

- `closing_complete` uses `PartialSigWithNonce` which includes the sender's closer nonce

- `closing_sig` uses just `PartialSig` (no nonce) since the receiver already knows the nonce

Requirements

For taproot channels:

A sending node: - MUST set the `shutdown_nonce` to a valid `musig2` public nonce. - MUST use this nonce when sending `closing_sig` messages.

A receiving node: - MUST verify that the `shutdown_nonce` value is a valid `musig2` public nonce. - MUST store this nonce for use when verifying the peer's `closing_sig` messages.

JIT (Just-In-Time) Nonce Pattern

Final nonces are delivered just-in-time with signatures using an asymmetric pattern:

- `closing_complete` uses `PartialSigWithNonce` (98 bytes) to bundle the signature with the next closer nonce
- `closing_sig` uses `PartialSig` (32 bytes) with a separate `NextCloseeNonce` field

With this pattern, we exchange the set of `closee_nonces` up front in the `shutdown` message. When either party wants to initiate a new update, they send `closing_complete` which includes their JIT `closer_nonce`. Sending this signatures serves to consume the `closee_nonce` of the remote party, so then they respond with `closing_sig`, they include a new `next_closee_nonce`.

`closing_complete` Extensions

For taproot channels, the `closing_complete` message uses `PartialSigWithNonce` to support MuSig2 signatures with JIT nonce delivery:

1. `tlv_stream`: `closing_tlvs`
2. types:
    1. type: 5 (`closer_no_closee`)
    2. data:
        - [`98*byte:partial_sig_with_nonce`]
    3. type: 6 (`no_closer_closee`)
    4. data:
        - [`98*byte:partial_sig_with_nonce`]
    5. type: 7 (`closer_and_closee`)
    6. data:
        - [`98*byte:partial_sig_with_nonce`]

Note: The TLV type numbers (5, 6, 7) are different from the non-taproot types (1, 2, 3) to distinguish taproot signatures. Each `partial_sig_with_nonce` contains:

- 32 bytes: MuSig2 partial signature
- 66 bytes: The sender's closer nonce used to produce the partial signature

Requirements

The sender of `closing_complete` (aka. "the closer"):

- For taproot channels (when `option_simple_taproot` was negotiated):
  - MUST provide `PartialSigWithNonce` (98 bytes) in the appropriate TLV field based on output presence
  - MUST compute the aggregated `musig2` public key from the sorted `funding_pubkeys` using the `KeyAgg` algorithm from `bip-musig2`
  - For the first closing transaction:
    - MUST use the `shutdown_nonce` received from the peer as their closee nonce
    - MUST use a locally generated nonce as the closer nonce
  - For RBF iterations:
    - MUST use the nonce extracted from the peer's previous `closing_sig` message as their closee nonce
    - MUST use a fresh locally generated nonce as the closer nonce
  - MUST generate the partial signature using the `Sign` algorithm from `bip-musig2`

The receiver of `closing_complete` (aka. "the closee"):

- For taproot channels:
  - MUST verify that signature fields contain `PartialSigWithNonce` (98 bytes)
  - MUST extract the partial signature (first 32 bytes) and the sender's closer nonce (remaining 66 bytes)
  - MUST compute the aggregate nonce by combining the closer nonce received with:
    - For the first closing transaction:
      - the `shutdown_nonce` the receiver previously sent (their closee nonce)
    - For RBF iterations:
      - the `next_closee_nonce` the receiver sent in their previous `closing_sig` message
  - MUST verify the partial signature using the `PartialSigVerifyInternal` algorithm of `bip-musig2`
  - MUST store (in memory) the extracted nonce for potential future RBF iterations

`closing_sig` Extensions

For taproot channels, the `closing_sig` message uses `PartialSig` with a separate next nonce field:

1. `tlv_stream`: `closing_tlvs`
2. types:
   1. type: 5 (`closer_no_closee`)
   2. data:
      - [`32*byte:partial_sig`]
   3. type: 6 (`no_closer_closee`)
   4. data:
      - [`32*byte:partial_sig`]
   5. type: 7 (`closer_and_closee`)
   6. data:
      - [`32*byte:partial_sig`]
   7. type: 22 (`next_closee_nonce`)

8. data:
    - [66*byte:public_nonce]

Note: Unlike `closing_complete`, `closing_sig` separates the signature from the next nonce (which will be used for the *next* RBF attempt signature):

- The partial signature (32 bytes) is sent in the appropriate TLV field based on output presence
- The `next_closee_nonce` (66 bytes) is sent separately in TLV type 22
- The receiver (the closer) already knows the current closee signing nonce from either the `shutdown` message or the previous `next_closee_nonce`

Requirements

The sender of `closing_sig`:

- For taproot channels:
    - MUST provide a 32-byte partial signature in the appropriate TLV field based on output presence
    - For the first closing transaction:
        - MUST use the `shutdown_nonce` sent in their own `shutdown` message as their closee nonce
    - For RBF iterations:
        - MUST use the `next_closee_nonce` sent in their own previous `closing_sig` message as their closee nonce
    - MUST use the nonce provided in the `PartialSigWithNonce` message of the received `closing_complete` message as the closer nonce.
    - MUST generate the partial signature using the `Sign` algorithm from `bip-musig2`
    - MUST include `next_closee_nonce` for potential future RBF iterations
    - MUST generate `next_closee_nonce` using the `NonceGen` algorithm from `bip-musig2` if included

The receiver of `closing_sig`:

- For taproot channels:
    - MUST verify that signature fields contain 32-byte partial signatures
    - MUST use the appropriate nonces for verification:
        - The sender's closee nonce (from their `shutdown` message or previous `closing_sig`)
        - The receiver's own closer nonce included in `closing_complete`
    - MUST verify the partial signature using `PartialSigVerifyInternal` from `bip-musig2`
    - MUST combine both partial signatures using `PartialSigAgg` to create the final schnorr signature
    - MUST broadcast the closing transaction with the aggregated signature
    - SHOULD store `next_closee_nonce` for potential future RBF iterations

RBF Nonce Rotation Protocol

For taproot channels supporting RBF cooperative close, nonces are delivered just-in-time (JIT) with signatures using an asymmetric pattern:

1. **Initial Exchange**: Nonces are exchanged in `shutdown` messages
    - Each party sends their "closee nonce" - the nonce they'll use when signing as the closee
    - This nonce is used by the remote party when they act as closer

2. **RBF Iterations**: Subsequent nonces are delivered with signatures
    - `closing_complete` (from closer): Uses `PartialSigWithNonce` (98 bytes)
        - Bundles the partial signature with the sender's closer nonce
        - The bundling is necessary because the closee doesn't know this nonce yet
    - `closing_sig` (from closee): Uses `PartialSig` (32 bytes) + `NextCloseeNonce` field
        - Separates the signature from the next closee nonce because this nonce is for the *next* RBF attempt, not the one currently being signed
3. **Asymmetric Roles**:
    - **Closer**: The party sending `closing_complete`, proposing a new fee
    - **Closee**: The party sending `closing_sig`, accepting the fee proposal
    - Each party alternates between these roles during RBF iterations

Nonce State Management

With this JIT nonce approach for coop close, an implementation only needs to store (in memory) the current closee nonce for the remote and local party. When either side is ready to sign, it'll generate a new closer nonce, and send that along with the sig.

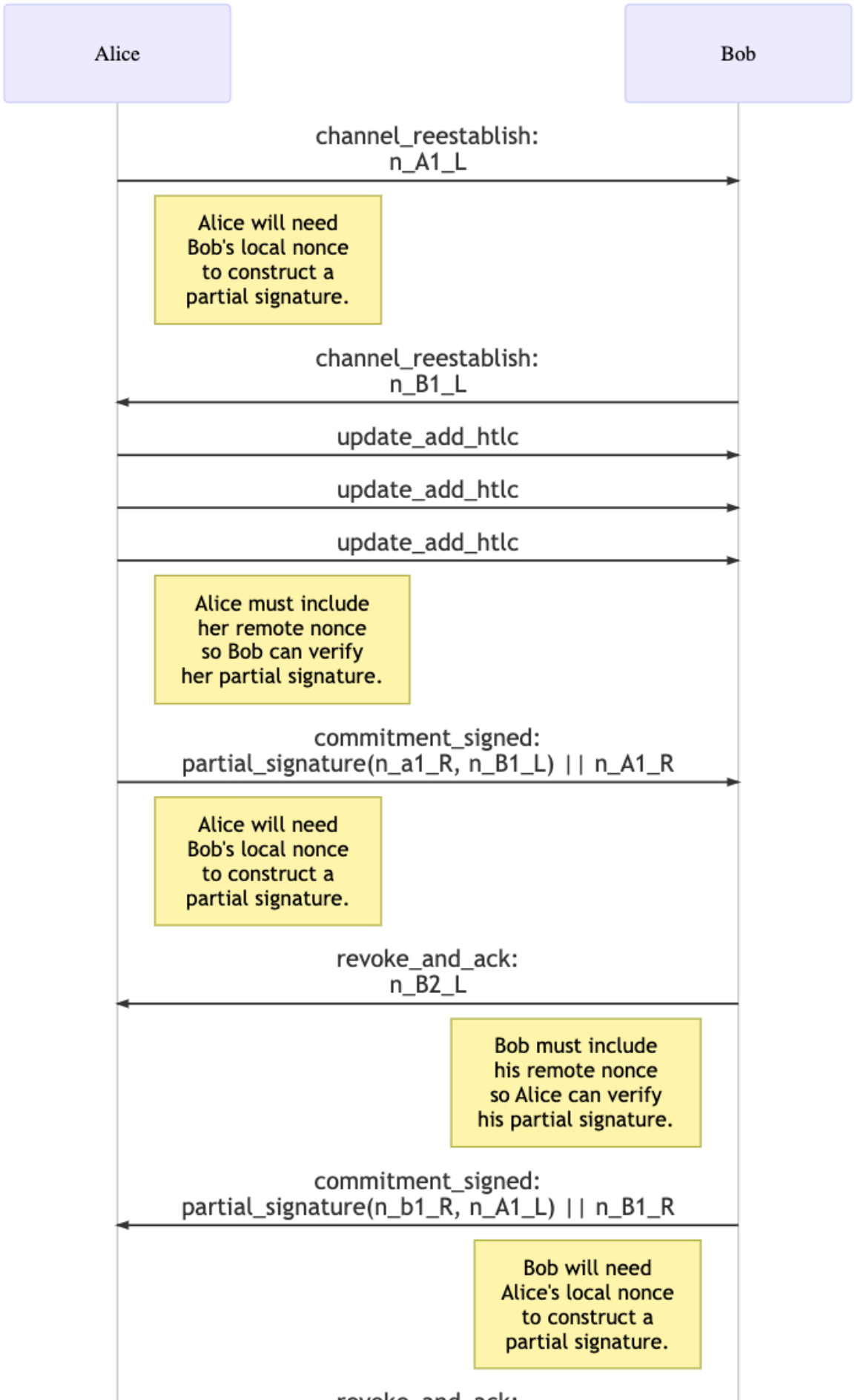
Security Considerations

- Nonces MUST be generated using cryptographically secure randomness
- Previously used nonces MUST NOT be reused for different closing transactions

Channel Operation

`n_a_L`: Alice's local secret nonce

`n_B_R`: Bob's remote public nonce


Alice
Bob
channel_reestablish:
n_A1_L
Alice will need
Bob's local nonce
to construct a
partial signature.
channel_reestablish:
n_B1_L
update_add_htlc
update_add_htlc
update_add_htlc
Alice must include
her remote nonce
so Bob can verify
her partial signature.
commitment_signed:
partial_signature(n_a1_R, n_B1_L) || n_A1_R
Alice will need
Bob's local nonce
to construct a
partial signature.
revoke_and_ack:
n_B2_L
Bob must include
his remote nonce
so Alice can verify
his partial signature.
commitment_signed:
partial_signature(n_b1_R, n_A1_L) || n_B1_R
Bob will need
Alice's local nonce
to construct a
partial signature.

`commitment_signed` Extensions

A new TLV stream is added to the `commitment_signed` message:

1. `tlv_stream`: `commitment_signed_tlvs`
2. types:
    1. type: 2 (`partial_signature_with_nonce`)
    2. data:
        - [`98*byte`: `partial_signature || public_nonce`]

Requirements

The sender:

- MUST set the original, non-TLV `signature` field to a 0-byte-array of length

  64.

- MUST retrieve the `musig2` public key previously aggregated from the sorted `funding_pubkeys` using the `KeyAgg` algorithm from `bip-musig2`.
- MUST generate a unique nonce to combine with the `next_local_nonce` previously received in the latest of `channel_ready`, `channel_reestablish`, or `revoke_and_ack` using `NonceAgg` from `bip-musig2`.
- MUST use the generated secret nonce and the calculated aggregate nonce to construct a `musig2` partial signature for the sender's remote commitment using the `Sign` algorithm from `bip-musig2`.
- MUST include the partial signature and the public counterpart of the generated nonce in the `partial_signature_with_nonce` field.
- MUST compute each HTLC signature according to BIP 342, as a BIP 340 Schnorr signature (non-partial).
    - Each HTLC signature must be packed as a 64-byte byte value within the existing `htlc_signature` field.

The recipient:

- MUST fail the channel if `signature` is non-zero.
- MUST fail the channel if `partial_signature_with_nonce` is absent.
- MUST compute the aggregate nonce from:
    - the `next_local_nonce` field the recipient previously sent in the latest of `channel_ready`, `channel_reestablish`, or `revoke_and_ack`
    - the `public_nonce` included as part of the `partial_signature_with_nonce` field
- MUST verify the `partial_signature_with_nonce` field using the `PartialSigVerifyInternal` algorithm of `bip-musig2`:
    - if the partial signature is invalid, MUST fail the channel

- MUST fail the channel if *any* of the received HTLC signatures is invalid according to BIP 342.

`revoke_and_ack` Extensions

A new TLV stream is added to the `revoke_and_ack` message:

1. `tlv_stream`: `revoke_and_ack_tlvs`
2. types:
    1. type: 22 (`next_local_nonces`)
    2. data:
        - `[...*nonce_entry: nonces]`

Similar to sending the `next_per_commitment_point`, we also send the *next* `musig2` nonces, after we revoke a state. Sending these nonces allows the remote party to propose another state transition as soon as the message is received.

Requirements

The sender:

- MUST use the `musig2.NonceGen` algorithm to generate unique nonces to send in the `next_local_nonces` field.
- MUST include one entry for each active commitment transaction, indexed by the commitment transaction's `funding_txid`.

The recipient:

- MUST fail the channel if `next_local_nonces` is absent.
- MUST fail the channel if an active commitment's `funding_txid` is missing in `next_local_nonces`.
- If the local nonce generation is non-deterministic and the recipient co-signs commitments only upon pending broadcast:
    - MUST **securely** store the local nonce.

Message Retransmission

`channel_reestablish` Extensions

We add a new TLV field to the `channel_reestablish` message:

1. `tlv_stream`: `channel_reestablish_tlvs`
2. types:
    1. type: 22 (`next_local_nonces`)
    2. data:
        - `[...*nonce_entry: nonces]`

Similar to the `next_per_commitment_point`, by sending the `next_local_nonces` value in this message, we ensure that the remote party has our public nonces, which are required to generate new commitment signatures.

Requirements

The sender:

- MUST set `next_local_nonces` to fresh, unique `musig2` nonces as specified by `bip-musig2`.
- MUST include one entry for each active commitment transaction, indexed by the commitment transaction's `funding_txid`.

The recipient:

- MUST fail the channel if `next_local_nonces` is absent, or cannot be parsed.
- MUST fail the channel if an active commitment's `funding_txid` is missing in `next_local_nonces`.

A node:

- If ALL of the following conditions apply:
    - It has previously sent a `commitment_signed` message
    - It never processed the corresponding `revoke_and_ack` message
    - It decides to retransmit the exact `update_` messages from the last sent `commitment_signed`
- THEN it must regenerate the partial signature using the newly received `next_local_nonces`

Splice Coordination

Splicing allows parties to modify the funding output of an existing channel without closing it. During splice operations, multiple commitment transactions may exist concurrently, each requiring its own MuSig2 nonce coordination. The `next_local_nonces` field enables this coordination by mapping transaction IDs to their respective nonces.

Splice Nonce Management

During splice negotiation:

- Each splice transaction MUST have a unique TXID as the key in the `next_local_nonces` map
- Parties MUST include nonces for all active commitment transactions in their `next_local_nonces` map, indexed by their `funding_txid`

Requirements for Splice Coordination

When a splice is initiated:

- The initiating party MUST generate a fresh nonce for the splice transaction
- Both parties MUST add the splice TXID and corresponding nonce to their `next_local_nonces` map
- The nonce MUST be communicated in the next `revoke_and_ack` or `channel_reestablish` message

When multiple splices are pending:

- Each splice MUST have a distinct TXID and nonce pair
- Nonces MUST NOT be reused across different splice transactions

- The `next_local_nonces` map MUST contain multiple entries during concurrent splice operations

Backward Compatibility

Nodes that do not support splicing will simply use a nonce map with a single entry indexed by the commitment transaction's `funding_txid`.

Funding Transactions

For our Simple Taproot Channels, `musig2` is used to generate a single aggregated public key.

The resulting funding output script takes the form: `OP_1 funding_key`

where:

- `funding_key = combined_funding_key + tagged_hash("TapTweak", combined_funding_key)*G`
- `combined_funding_key = musig2.KeyAgg(musig2.KeySort(pubkey1, pubkey2))`

The funding key is derived via the recommendation in BIP 341 that states: "If the spending conditions do not require a script path, the output key should commit to an unspendable script path instead of having no script path. This can be achieved by computing the output key point as `Q = P + int(hashTapTweak(bytes(P)))G`". We refer to BIP 86 for the specifics of this derivation.

By using the musig2 sorting routine, we inherit the lexicographical key ordering from the base segwit channels. Throughout the lifetime of a channel any keys aggregated via musig2 also are assumed to use the `KeySort` routine

Commitment Transactions

To Local Outputs

For the simple taproot commitment type, we use the same flow of revocation or delay, while re-using the NUMS point to ensure that the internal key is always revealed (script path always taken). This ensures that the anchor outputs can *always* be spent given chain revealed information. For the remote party, the `remotepubkey` will be revealed once the remote party sweeps. For the local party, we ensure that the `local_delayedpubkey` is revealed with an extra noop data push.

The new output has the following form:

- `OP_1 to_local_output_key`
- where:
    - `to_local_output_key = taproot_nums_point + tagged_hash("TapTweak", taproot_nums_point || to_delay_script_root)*G`
    - `to_delay_script_root = tapscript_root([to_delay_script, revoke_script])`
    - `to_delay_script` is the delay script: ``` OP_CHECKSIGVERIFY OP_CHECKSEQUENCEVERIFY
    - `revoke_script` is the revoke script: `<local_delayedpubkey> OP_DROP <revocation_pubkey> OP_CHECKSIG`

The parity (even or odd) of the y-coordinate of the derived `to_local_output_key` MUST be known in order to derive a valid control block.

The tapscript_root routine constructs a valid taproot commitment according to BIP 341+342. Namely, a leaf version of 0xc0 MUST be used.

In the case of a commitment breach, the to_delay_script_root can be used along side <revocationpubkey> to derive the private key needed to sweep the top-level key spend path. The control block can be crafted as such:

```
revoke_control_block = (output_key_y_parity | 0xc0) || taproot_nums_point || revoke_script
```

A valid witness is then:

```
<revoke_sig> <revoke_script> <revoke_control_block>
```

In the case of a routine force close, the script path must be revealed so the broadcaster can sweep their funds after a delay. The control block to spend is only 33 bytes, as it just includes the internal key (along with the y-parity bit and leaf version):

```
delay_control_block = (output_key_y_parity | 0xc0) || taproot_nums-point || to_delay_srcipt
```

A valid witness is then:

```
<local_delayedsig> <to_delay_script> <delay_control_block>
```

As with base channels, the nSequence field must be set to to_self_delay.

To Remote Outputs

As we inherit the anchor output, semantics we want to ensure that the remote party can unilaterally sweep their funds after the 1 block CSV delay. In order to achieve this property, we'll utilize a NUMS point (simple_taproot_nums) By using this point as the internal key, we ensure that the remote party isn't able to by pass the CSV delay.

Using a NUMS key has a key benefit: the static internal key allows the remote party to scan for their output on chain, which is useful for various recovery scenarios.

The to_remote output has the following form:

- OP_1 to_remote_output_key
- where:
    - taproot_nums_point = 0245b18183a06ee58228f07d9716f0f121cd194e4d924b037522503a7160432f15
    - to_remote_output_key = taproot_nums_point + tagged_hash("TapTweak", taproot_nums_point || to_remote_script_root)*G
    - to_remote_script_root = tapscript_root([to_remote_script])
    - to_remote_script is the remote script: <remotepubkey> OP_CHECKSIGVERIFY 1 OP_CHECKSEQUENCEVERIFY

This output can be swept by the remote party with the following witness:

```
<remote_sig> <to_remote_script> <to_remote_control_block>
```

where to_remote_control_block is:

```
(output_key_y_parity | 0xc0) || combined_funding_key
```

The sequence field of the input MUST also be set to 1.

Anchor Outputs

For simple taproot channels (option_simple_taproot) we'll maintain the same form as base segwit channels, but instead will utilize local_delayedpubkey and the remotepubkey rather than the multi-sig keys as that's no longer revealed due to musig2.

An anchor output has the following form:

- OP_1 anchor_output_key
- where:
    - anchor_internal_key = remotepubkey/local_delayedpubkey
    - anchor_output_key = anchor_internal_key + tagged_hash("TapTweak", anchor_internal_key || anchor_script_root)
    - anchor_script_root = tapscript_root([anchor_script])
    - anchor_script: OP_16 OP_CHECKSEQUENCEVERIFY

This output can be swept by anyone after 16 blocks with the following witness:

<anchor_script> <anchor_control_block>

where anchor_control_block is:

(output_key_y_parity | 0xc0) || anchor_internal_key

nSequence needs to be set to 16.

HTLC Scripts & Transactions

Offered HTLCs

We retain the same second-level HTLC mappings as base channels. The main modifications are using the revocation public key as the internal key, and splitting the timeout and success paths into individual script leaves.

An offered HTLC has the following form:

- OP_1 offered_htlc_key
- where:
    - offered_htlc_key = revocation_pubkey + tagged_hash("TapTweak", revocation_pubkey || htlc_script_root)
    - htlc_script_root = tapscript_root([htlc_timeout, htlc_success])
    - htlc_timeout: <local_htlcpubkey> OP_CHECKSIGVERIFY <remote_htlcpubkey> OP_CHECKSIG
    - htlc_success: OP_SIZE 32 OP_EQUALVERIFY OP_HASH160 <RIPEMD160(payment_hash)> OP_EQUALVERIFY <remote_htlcpubkey> OP_CHECKSIGVERIFY 1 OP_CHECKSEQUENCEVERIFY

In order to spend a offered HTLC, via either script path, an inclusion_proof must be specified along with the control block. This inclusion_proof is simply the tap_leaf hash of the path *not* taken.

Accepted HTLCs

Accepted HTLCs inherit a similar format:

- `OP_1 accepted_htlc_key`
- where:
    - `accepted_htlc_key = revocation_pubkey + tagged_hash("TapTweak", revocation_pubkey || htlc_script_root)`
    - `htlc_script_root = tapscript_root([htlc_timeout, htlc_success])`
    - `htlc_timeout: <remote_htlcpubkey> OP_CHECKSIGVERIFY 1 OP_CHECKSEQUENCEVERIFY OP_VERIFY <cltv_expiry> OP_CHECKLOCKTIMEVERIFY`
    - `htlc_success: OP_SIZE 32 OP_EQUALVERIFY OP_HASH160 <RIPEMD160(payment_hash)> OP_EQUALVERIFY <local_htlcpubkey> OP_CHECKSIGVERIFY <remote_htlcpubkey> OP_CHECKSIG`

Note that there is no `OP_CHECKSEQUENCEVERIFY` in the offered HTLC's timeout path nor in the accepted HTLC's success path. This is not needed here as these paths require a remote signature that commits to the sequence, which needs to be set at 1.

In order to spend an accepted HTLC, via either script path, an `inclusion_proof` must be specified along with the control block. This `inclusion_proof` is simply the `tap_leaf` hash of the path *not* taken.

HTLC Second Level Transactions

For the second level transactions, we retain the existing structure: a 2-of-2 multi-spend going to a CLTV delayed output. Like the normal HTLC transactions, we also place the revocation key as the internal key, allowing easy breach sweeps with each party only retaining the specific script root.

An HTLC-Success or HTLC-Timeout is a one-in-one-out transaction signed using the `SIGHASH_SINGLE|SIGHASH_ANYONECANPAY` flag. These transactions always have *zero* fees attached, forcing them to be aggregated with each other and a change input.

HTLC-Success Transactions

A HTLC-Success transaction has the following structure:

- version: 2
- locktime: 0
- input:
    - txid: commitment_tx
    - vout: htlc_index
    - sequence: 1
    - witness: `<remotehtlcsig> <localhtlcsig> <preimage> <htlc_success_script> <control_block>`
- output:
    - value: htlc_value
    - script:
        - OP_1 htlc_success_key

        - where:
            - `htlc_success_key = revocation_pubkey + tagged_hash("TapTweak", revocation_pubkey || htlc_script_root)`
            - `htlc_script_root = tapscript_root([htlc_success])`
            - `htlc_success`:

```
<local_delayedpubkey> OP_CHECKSIGVERIFY
<to_self_delay> OP_CHECKSEQUENCEVERIFY
```

HTLC-Timeout Transactions

A HTLC-Timeout transaction has the following structure:

- version: 2
- locktime: cltv_expiry
- input:
    - txid: commitment_tx
    - vout: htlc_index
    - sequence: 1
    - witness: `<remotehtlcsig> <localhtlcsig> <htlc_timeout_script> <control_block>`
- output:
    - value: htlc_value
    - script:
        - OP_1 htlc_timeout_key
        - where:
            - `htlc_timeout_key = revocation_pubkey + tagged_hash("TapTweak", revocation_pubkey || htlc_script_root)`
            - `htlc_script_root = tapscript_root([htlc_timeout])`
            - `htlc_timeout`:

```
<local_delayedpubkey> OP_CHECKSIGVERIFY
<to_self_delay> OP_CHECKSEQUENCEVERIFY
```

Appendix

Footnotes

Test Vectors

All test vectors are derived deterministically from a single 32-byte seed: `000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`. Private keys are produced via `SHA256(seed || label)` where `label` is a fixed ASCII string for each key (e.g. `"local-funding"`, `"remote-funding"`, etc.). The per-commitment point is derived by computing `SHA256(seed || "local-per-commit-secret")` and using the result as the per-commitment secret scalar, then multiplying by the generator to obtain the per-commitment point.

The vectors cover two areas:

1. **Script vectors**: the decomposed tapscript trees for every output type (funding, `to_local`, `to_remote`, anchors, offered/accepted HTLCs on both local and remote commits, and second-level HTLC transactions). Each entry includes the raw leaf scripts, leaf hashes, tapscript root, internal key, output key, and the resulting `pkScript`.

2. **Transaction vectors**: full serialized commitment transactions and their HTLC resolution transactions for three scenarios — a simple commitment with no HTLCs, a commitment with five untrimmed HTLCs, and a commitment with the same HTLCs at a higher fee rate causing some to be trimmed.

Channel parameters used: `funding_amount = 10,000,000 sat`, `dust_limit = 354 sat` (taproot dust), `csv_delay = 144 blocks`, `commit_height = 42`, `fee_per_kw` as specified per test case.

Key Derivation Labels

The following labels are used with `SHA256(seed || label)` to derive each private key:

Key	Label
`local_funding_privkey`	`"local-funding"`
`remote_funding_privkey`	`"remote-funding"`
`local_payment_basepoint_secret`	`"local-payment-basepoint"`
`remote_payment_basepoint_secret`	`"remote-payment-basepoint"`
`local_delayed_payment_basepoint_secret`	`"local-delayed-payment-basepoint"`
`remote_revocation_basepoint_secret`	`"remote-revocation-basepoint"`
`local_htlc_basepoint_secret`	`"local-htlc-basepoint"`
`remote_htlc_basepoint_secret`	`"remote-htlc-basepoint"`
`local_per_commit_secret`	`"local-per-commit-secret"`

MuSig2 Nonce Generation

MuSig2 partial signatures require deterministic nonces so that test vectors are reproducible across implementations. The signing randomness for each party's JIT nonces is derived as `SHA256(seed || label)` using:

Party	Label
Local signer	`"local-signing-rand"`
Remote signer	`"remote-signing-rand"`

The resulting 32-byte hash is used as the nonce randomness input to the MuSig2 signing session.

Because different MuSig2 implementations (e.g., libsecp256k1-zkp vs btcd) may produce different nonces from the same randomness input, the test vectors include both the raw secret nonces and the derived public nonces so that implementations can inject them directly rather than re-derive them.

Each transaction test case includes the following nonce fields:

Field	Size	Description
`local_nonce`	66 bytes	Local party's public nonce for this commitment tx
`remote_nonce`	66 bytes	Remote party's public nonce for this commitment tx
`local_sec_nonce`	97 bytes	Local party's secret nonce (two 32-byte scalars + 33-byte pubkey)
`remote_sec_nonce`	97 bytes	Remote party's secret nonce (two 32-byte scalars + 33-byte pubkey)

All nonces in a given test case correspond to the **same commitment transaction** (the local party's commitment, as produced by `ForceClose`). Specifically:

- `local_nonce` / `local_sec_nonce`: the verification nonce that the local party contributes to the MuSig2 session for their own commitment. This nonce is generated during the initial nonce exchange (or nonce rotation after a prior commitment update) and sent to the remote party.
- `remote_nonce` / `remote_sec_nonce`: the JIT signing nonce that the remote party generates when producing their partial signature on the local party's commitment transaction.

To replay the MuSig2 signing from the test vectors:

1. Aggregate the two public nonces using `NonceAgg` from BIP-327.
2. Use each party's secret nonce with `Sign` to reproduce their partial signature.
3. Combine both partial signatures to obtain the final 64-byte Schnorr signature, which should match the commitment transaction witness.

Commitment Transaction Signatures

The `remote_partial_sig` field in each transaction test case is a 32-byte MuSig2 partial signature scalar (hex-encoded). This is the remote party's contribution to the combined Schnorr signature on the commitment transaction. The commitment transaction witness contains the final aggregated 64-byte Schnorr signature produced by combining both parties' partial signatures.

HTLC Resolution Transactions

Each `htlc_descs` entry describes a second-level HTLC transaction that spends from one of the commitment transaction's HTLC outputs. The entries are ordered by their corresponding commitment transaction output index (BIP 69 ordering), which matches the order of `htlc_signature` messages exchanged during the commitment signing protocol.

The `remote_partial_sig_hex` for HTLC transactions is a 64-byte Schnorr signature (not a MuSig2 partial sig), since HTLC second-level transactions use a regular tapscript spend path rather than a keyspend MuSig2 path. These Schnorr signatures use BIP-340 deterministic nonce derivation with zero `auxrand` (32 zero bytes). Implementations must use the same nonce derivation to reproduce matching signatures — passing no auxiliary randomness (or explicitly zero bytes) to the BIP-340 `Sign` algorithm.

The `resolution_tx_hex` contains the fully serialized second-level transaction with a complete witness stack. For HTLC-success transactions, the witness layout is:

```
<remote_htlcsig> <local_htlcsig> <payment_preimage> <htlc_success_script> <control_block>
```

For HTLC-timeout transactions, the witness layout is:

```
<remote_htlcsig> <local_htlcsig> <htlc_timeout_script> <control_block>
```

HTLC Trimming

Taproot channels use zero-fee HTLC transactions, meaning the HTLC output value on the commitment transaction equals the HTLC amount directly (no fee is deducted from the output). As a result, HTLC trimming depends solely on whether the HTLC amount falls below the `dust_limit_satoshis`, not on the fee rate. The "commitment tx with some HTLCs trimmed" test case uses `dust_limit = 2500 sat` to trim the three smallest test HTLCs (1000, 2000, and 2000 sats), leaving only the 3000 and 4000 sat HTLCs on the commitment transaction.

Implementations SHOULD regenerate these vectors from the seed using the derivation method described above and verify that all intermediate values (keys, scripts, leaf hashes, transactions) match.


```
{
  "params": {
    "seed": "000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f",
    "funding_amount_satoshis": 10000000,
    "dust_limit_satoshis": 354,
    "csv_delay": 144,
    "commit_height": 42,
    "nums_point": "02dca094751109d0bd055d03565874e8276dd53e926b44e3bd1bb6bf4bc130a279",
    "keys": {
      "local_funding_privkey":
        "20ae2d254ab29afd3dcbf8744a5b88d06070f55a4bd5532483a093ac4db91277",
      "local_funding_pubkey":
        "03b7203dec7c13896b6ff1f58b24f84458c441720a12b5a57426397e22f0a8c78b",
      "remote_funding_privkey":
        "f0c5500a9dbd7cdcd46ced7bdeb937d4dcbf90f9b9357626e7ee54ab024c3df0",
      "remote_funding_pubkey":
        "02956e6845a6f346f97c5e028c0f8ab38a76b0124fd7184deab60f682b3e657fdb",
      "local_payment_basepoint_secret":
        "277975b5b081a9cbc4834e066d7bb494e4fde4f7637257dd3d312a0ae7cb7754",
      "local_payment_basepoint":
        "03955b6085296cbd2447a1dde0f7e273e19b83e83de1814993b1517aaf193b7f33",
      "remote_payment_basepoint_secret":
        "f1cd3a5ca44b52baf4eacb849fbf06e75aace97477b8bfe31d2b814dbbb562b1",
      "remote_payment_basepoint":
        "03595f2ef2a51d2250a21077dbea4a7fc3ce550f10676996bf63719e2a71d1f4c9",
      "local_delayed_payment_basepoint_secret":
        "83ccf0b638c514db5ebefdc6cbf901505e2bb20edb2bb7248ce1a51523325f9b",
      "local_delayed_payment_basepoint":
        "02ae68d8ff4c59864c03a42bbff6c07f9ae18047e0daa9bc40d07c410f9a0f7899",
      "remote_revocation_basepoint_secret":
        "36c4175b91cff9731a63d1472b5b1c4cf3e7b688e87d5fb806b2e8350484e68d",
      "remote_revocation_basepoint":
        "02c354121ef71922b5cb32fa685c08ac0014b558f96e28f383c45eb28b7da264c3",
      "local_htlc_basepoint_secret":
        "786eb5024e4851bea3ddc6e40036c81b1efcf50eeed440eedefe5245bde6fc14",
```

```
    "local_htlc_basepoint":
      "033ce88bf3c8333e242996964ac91ee7cd945bfe4c49668ea10f3211f3d418fbc8",
    "remote_htlc_basepoint_secret":
      "51c9b6cf8279def85e3925bc8f16fc0ff100ee7b03ce7c954149ca29c834b684",
    "remote_htlc_basepoint":
      "02932dfbf6737001e3c516696ae3dcd323fd91a01ce7898f7f91ab98eebacc323e",
    "local_per_commit_secret":
      "037b507180b3985cea6396d6a70987cea11ccd05fde49e943a3ea0fe56ee33ed",
    "local_per_commit_point":
      "02a0f5a09017c1dec2d30dd54a25dc4037fc5a2aa3832ee3c7b58f3a88a0836287",
    "derived_local_delayed_pubkey":
      "0315ec0138eb42f1ab4603042123988d53c854e89d1d87aa4dbb97a57482029c05",
    "derived_revocation_pubkey":
      "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
    "derived_local_htlc_pubkey":
      "0271e82ef65d5c667159036bfcf662cac2f6c41e38323d148bbbd00fdcd923739e",
    "derived_remote_htlc_pubkey":
      "032deba21cf03c42362c9f912094f62ba045a040a2060882ba1ed3abf1f664a47d",
    "derived_remote_payment_pubkey":
      "03595f2ef2a51d2250a21077dbea4a7fc3ce550f10676996bf63719e2a71d1f4c9"
  }
},
"scripts": {
  "funding": {
    "funding_tx_hex":
      "020000000100000000000000000000000000000000000000000000000000000000000000000000000000ffffffff0180969

    "combined_key": "d0ebb4909d563a7ae1213fddede4ae54132fba0ef0b97ee3f8469191fecd348e",
    "pkscript": "5120d0ebb4909d563a7ae1213fddede4ae54132fba0ef0b97ee3f8469191fecd348e"
  },
  "to_local": {
    "scripts": {
      "revocation":
      "2015ec0138eb42f1ab4603042123988d53c854e89d1d87aa4dbb97a57482029c057520d4c77088d346bce67c13bbbf82ca1

      "settle": "2015ec0138eb42f1ab4603042123988d53c854e89d1d87aa4dbb97a57482029c05ad029000b2"
    },
    "leaf_hashes": {
      "revocation": "8fcd64d212bbbf1bcec2360bbf229963240d05992fc2efb482fe6dca85b9469a",
      "settle": "dbf0400e9c7c57f30b6ad0b0677e396b5a002cbf050d873c8925b966048e6a62"
    },
    "tapscript_root": "b8b76c2e893ca785072f0d7393e35d5bd72adf8b7ff2a53538aa664378a38a36",
    "internal_key": "02dca094751109d0bd055d03565874e8276dd53e926b44e3bd1bb6bf4bc130a279",
    "output_key": "023e1fcbbd06c8a7414704612c72be9834a75d86ed85b29f0ef0c52e1950afaff3",
    "pkscript": "51203e1fcbbd06c8a7414704612c72be9834a75d86ed85b29f0ef0c52e1950afaff3"
  },
  "to_remote": {
    "scripts": {
      "settle": "20595f2ef2a51d2250a21077dbea4a7fc3ce550f10676996bf63719e2a71d1f4c9ad51b2"
    },
    "leaf_hashes": {
      "settle": "63ce35b16eb8f8687293d5a88c1d8ada3236843b79ca315fe9dd7c47f30f2bc9"
    },
    "tapscript_root": "63ce35b16eb8f8687293d5a88c1d8ada3236843b79ca315fe9dd7c47f30f2bc9",
    "internal_key": "02dca094751109d0bd055d03565874e8276dd53e926b44e3bd1bb6bf4bc130a279",
    "output_key": "023609bb705034e5629aa6ec05c5ca906ac89ac08b34c4583c259521ec30174408",
    "pkscript": "51203609bb705034e5629aa6ec05c5ca906ac89ac08b34c4583c259521ec30174408"
```

```
},
"local_anchor": {
  "scripts": {
    "sweep": "60b2"
  },
  "leaf_hashes": {
    "sweep": "2b88a8f3f52386d61d5b3f2d822df659c35214d7360ed05352ad7ddc1ab03912"
  },
  "tapscript_root": "2b88a8f3f52386d61d5b3f2d822df659c35214d7360ed05352ad7ddc1ab03912",
  "internal_key": "0315ec0138eb42f1ab4603042123988d53c854e89d1d87aa4dbb97a57482029c05",
  "output_key": "02f67ab012701705f3203d132f909a6810ef18c5da4c11d986cb50818803b8344e",
  "pkscript": "5120f67ab012701705f3203d132f909a6810ef18c5da4c11d986cb50818803b8344e"
},
"remote_anchor": {
  "scripts": {
    "sweep": "60b2"
  },
  "leaf_hashes": {
    "sweep": "2b88a8f3f52386d61d5b3f2d822df659c35214d7360ed05352ad7ddc1ab03912"
  },
  "tapscript_root": "2b88a8f3f52386d61d5b3f2d822df659c35214d7360ed05352ad7ddc1ab03912",
  "internal_key": "03595f2ef2a51d2250a21077dbea4a7fc3ce550f10676996bf63719e2a71d1f4c9",
  "output_key": "021249c50576fdf914caa14f9221370b986df520bdbc73f57d5056a86ee03e5ac4",
  "pkscript": "51201249c50576fdf914caa14f9221370b986df520bdbc73f57d5056a86ee03e5ac4"
},
"offered_htlc_local_commit": {
  "scripts": {
    "success":
    "82012088a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688202deba21cf03c42362c9f912094f62ba045a040a2060

    "timeout":
    "2071e82ef65d5c667159036bfcf662cac2f6c41e38323d148bbbd00fdcd923739ead202deba21cf03c42362c9f912094f62

  },
  "leaf_hashes": {
    "success": "cd4b7ba74d132998f2bcea85f76082f5018e614c86f27f2631b6569c4914320f",
    "timeout": "dd0bd08b3df902c399f5493a682f6c50c476c89e233ba454e89a234d2d16ffe3"
  },
  "tapscript_root": "f36c8bd45002c5264cfce9944211e7bc6ea974a6b90cf99a87812d18acf28a2a",
  "internal_key": "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
  "output_key": "033e5c3be9f4ce7ae07c28ad5e0eb0ab617c06eeb82b8d6ef10a5bf561848df5f0",
  "pkscript": "51203e5c3be9f4ce7ae07c28ad5e0eb0ab617c06eeb82b8d6ef10a5bf561848df5f0"
},
"offered_htlc_remote_commit": {
  "scripts": {
    "success":
    "82012088a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688202deba21cf03c42362c9f912094f62ba045a040a2060

    "timeout":
    "2071e82ef65d5c667159036bfcf662cac2f6c41e38323d148bbbd00fdcd923739ead202deba21cf03c42362c9f912094f62

  },
  "leaf_hashes": {
    "success": "cd4b7ba74d132998f2bcea85f76082f5018e614c86f27f2631b6569c4914320f",
    "timeout": "dd0bd08b3df902c399f5493a682f6c50c476c89e233ba454e89a234d2d16ffe3"
  },
```

```
  "tapscript_root": "f36c8bd45002c5264cfce9944211e7bc6ea974a6b90cf99a87812d18acf28a2a",
  "internal_key": "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
  "output_key": "033e5c3be9f4ce7ae07c28ad5e0eb0ab617c06eeb82b8d6ef10a5bf561848df5f0",
  "pkscript": "51203e5c3be9f4ce7ae07c28ad5e0eb0ab617c06eeb82b8d6ef10a5bf561848df5f0"
},
"accepted_htlc_local_commit": {
  "scripts": {
    "success":
    "82012088a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688202deba21cf03c42362c9f912094f62ba045a040a2060

    "timeout":
    "2071e82ef65d5c667159036bfcf662cac2f6c41e38323d148bbbd00fdcd923739ead51b26902f401b1"
  },
  "leaf_hashes": {
    "success": "69192ca730d4480044ade8741b8bd0845a32880aebaf58bc6f9186f8d2be8cbf",
    "timeout": "4da43c795365bf757ed1e9656d12ea744b4cf52b01719a3ea94e6569115623f0"
  },
  "tapscript_root": "1a990caa4bb0ed41ceb19e7466fcea5d9b31e3da968f348f6223201c5831d0a3",
  "internal_key": "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
  "output_key": "029aadbdd9aff986e5ea086cf53ae062972d33d0a5c7f5fb986dafec7fa6d7e6ea",
  "pkscript": "51209aadbdd9aff986e5ea086cf53ae062972d33d0a5c7f5fb986dafec7fa6d7e6ea"
},
"accepted_htlc_remote_commit": {
  "scripts": {
    "success":
    "82012088a914b8bcb07f6344b42ab04250c86a6e8b75d3fdbbc688202deba21cf03c42362c9f912094f62ba045a040a2060

    "timeout":
    "2071e82ef65d5c667159036bfcf662cac2f6c41e38323d148bbbd00fdcd923739ead51b26902f401b1"
  },
  "leaf_hashes": {
    "success": "69192ca730d4480044ade8741b8bd0845a32880aebaf58bc6f9186f8d2be8cbf",
    "timeout": "4da43c795365bf757ed1e9656d12ea744b4cf52b01719a3ea94e6569115623f0"
  },
  "tapscript_root": "1a990caa4bb0ed41ceb19e7466fcea5d9b31e3da968f348f6223201c5831d0a3",
  "internal_key": "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
  "output_key": "029aadbdd9aff986e5ea086cf53ae062972d33d0a5c7f5fb986dafec7fa6d7e6ea",
  "pkscript": "51209aadbdd9aff986e5ea086cf53ae062972d33d0a5c7f5fb986dafec7fa6d7e6ea"
},
"second_level_htlc_success": {
  "scripts": {
    "success":
    "2015ec0138eb42f1ab4603042123988d53c854e89d1d87aa4dbb97a57482029c05ad029000b2"
  },
  "leaf_hashes": {
    "success": "dbf0400e9c7c57f30b6ad0b0677e396b5a002cbf050d873c8925b966048e6a62"
  },
  "tapscript_root": "dbf0400e9c7c57f30b6ad0b0677e396b5a002cbf050d873c8925b966048e6a62",
  "internal_key": "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
  "output_key": "02df20bcec43daa75161f7d013254e401812e0fee8bc3369220b6a33672fc18ba0",
  "pkscript": "5120df20bcec43daa75161f7d013254e401812e0fee8bc3369220b6a33672fc18ba0"
},
"second_level_htlc_timeout": {
  "scripts": {
    "success":
    "2015ec0138eb42f1ab4603042123988d53c854e89d1d87aa4dbb97a57482029c05ad029000b2"
```

```
    },
    "leaf_hashes": {
      "success": "dbf0400e9c7c57f30b6ad0b0677e396b5a002cbf050d873c8925b966048e6a62"
    },
    "tapscript_root": "dbf0400e9c7c57f30b6ad0b0677e396b5a002cbf050d873c8925b966048e6a62",
    "internal_key": "03d4c77088d346bce67c13bbbf82ca112588f4b1c9595a1f8af3be9b2f95a109a0",
    "output_key": "02df20bcec43daa75161f7d013254e401812e0fee8bc3369220b6a33672fc18ba0",
    "pkscript": "5120df20bcec43daa75161f7d013254e401812e0fee8bc3369220b6a33672fc18ba0"
  }
},
"transactions": [
  {
    "name": "simple commitment tx with no HTLCs",
    "local_balance_msat": 7000000000,
    "remote_balance_msat": 3000000000,
    "fee_per_kw": 15000,
    "htlcs": null,
    "local_sec_nonce":
      "22a453171ba4a634da1addcf660d63d8e23fb63169a2a7206f4e23290e4cc59bb3c3011fe3c31cb4f1192a2df56c2e52350

    "remote_sec_nonce":
      "ccdaea6955c7bd9fcb082dc67e32809a5bdd3a3edf770cbb990116c45f4b006e4b56aa81f8e555d6bd1906b159af16d0ac3

    "local_nonce":
      "025f2272ea289c5fe9d52d411f5a50a6d4882341bf0ecb201d5675850a2ba0b09d025bc489cf67752134ba81f8d7f1146d7

    "remote_nonce":
      "02d324627074522af8cf4287caf1e073a3493550b99aed2697e58f476ec402e272039c25fc616207e15917b7145cefcb4c9

    "remote_partial_sig": "3fa93659d4c2d590eadbd422595a37597ed58607e026a91f4e6e19329134a931",
    "expected_commitment_tx_hex":
      "020000000001015474cba49124ab0c4327c244bb2907059585c4af3fa5f3469701534120fec0170000000000c5fe178004

    "htlc_descs": null
  },
  {
    "name": "commitment tx with five HTLCs untrimmed",
    "local_balance_msat": 6988000000,
    "remote_balance_msat": 3000000000,
    "fee_per_kw": 644,
    "htlcs": [
      {
        "incoming": true,
        "amount_msat": 1000000,
        "expiry": 500,
        "preimage": "0000000000000000000000000000000000000000000000000000000000000000"
      },
      {
        "incoming": true,
        "amount_msat": 2000000,
        "expiry": 501,
        "preimage": "0101010101010101010101010101010101010101010101010101010101010101"
      },
      {
        "incoming": false,
        "amount_msat": 2000000,
```

```
    "expiry": 502,
    "preimage": "0202020202020202020202020202020202020202020202020202020202020202"
  },
  {
    "incoming": false,
    "amount_msat": 3000000,
    "expiry": 503,
    "preimage": "0303030303030303030303030303030303030303030303030303030303030303"
  },
  {
    "incoming": true,
    "amount_msat": 4000000,
    "expiry": 504,
    "preimage": "0404040404040404040404040404040404040404040404040404040404040404"
  }
],
"local_sec_nonce":
  "22a453171ba4a634da1addcf660d63d8e23fb63169a2a7206f4e23290e4cc59bb3c3011fe3c31cb4f1192a2df56c2e52350

"remote_sec_nonce":
  "8c40a8ab26f1bd7d58010e991c72b0f102df6d0f050df28f56cefdafb9d06479958ea0a72180184e338b1b77fdf20690d5c

"local_nonce":
  "025f2272ea289c5fe9d52d411f5a50a6d4882341bf0ecb201d5675850a2ba0b09d025bc489cf67752134ba81f8d7f1146d7

"remote_nonce":
  "038a4018a074b5ddfc1424551e871bd739259c4e209f47177eb67c70cfbdb1e57c03c75684ad3a42d8c86a3eddb83b8160c

"remote_partial_sig": "46efde50f08c128aa6472bbd50ea156fb9bde7b013f5e042f700729c94053613",
"expected_commitment_tx_hex":
  "020000000001015474cba49124ab0c4327c244bb2907059585c4af3fa5f3469701534120fec0170000000000c5fe178009

"htlc_descs": [
  {
    "remote_partial_sig_hex":
  "fc97f7cfb97e1e48792b0ed174704cd98c886d368ede5adeb6288f0b350c8f88d076488973d0656da72d072e03eb9eb8b31

    "resolution_tx_hex":
  "02000000000101ec4c0a34c981864f9badcb8383bbe42ec6b32e68c2aa1a7c7c2e8422adde673f020000000001000000001e

  },
  {
    "remote_partial_sig_hex":
  "c99d8d1ca721d1d4b9796cde49698fd46bb3faddd2bc7215de291ae9bd85cd9b2ab9bf40e36c752b698098b0986abe92b3c

    "resolution_tx_hex":
  "02000000000101ec4c0a34c981864f9badcb8383bbe42ec6b32e68c2aa1a7c7c2e8422adde673f030000000001000000001c

  },
  {
    "remote_partial_sig_hex":
  "b361ed8b70a09f4128fe610db649abcaafbc9b8600136e6f1b9735b4781cd65681faa32c29fef2a1485cc569c655b31b8eb

    "resolution_tx_hex":
  "02000000000101ec4c0a34c981864f9badcb8383bbe42ec6b32e68c2aa1a7c7c2e8422adde673f040000000001000000001c

  },
```

```
    {
      "remote_partial_sig_hex":
    "9d1193d1bae8793ec502aa705a43a140e9f4c46607b4b8b414f76923c746a81cc320ffef6575f4c1ba55795770b41885dc9

      "resolution_tx_hex":
    "02000000000101ec4c0a34c981864f9badcb8383bbe42ec6b32e68c2aa1a7c7c2e8422adde673f050000000001000000001b

    },
    {
      "remote_partial_sig_hex":
    "58338a2d50a03ea615f0e0e295b12413308e6381412c14d276c4b39a2f1d17193a689cbe2ad0d33a63344cff36daa2edc94

      "resolution_tx_hex":
    "02000000000101ec4c0a34c981864f9badcb8383bbe42ec6b32e68c2aa1a7c7c2e8422adde673f060000000001000000001a

    }
  ]
},
{
  "name": "commitment tx with some HTLCs trimmed",
  "local_balance_msat": 6988000000,
  "remote_balance_msat": 3000000000,
  "fee_per_kw": 644,
  "dust_limit_satoshis": 2500,
  "htlcs": [
    {
      "incoming": true,
      "amount_msat": 1000000,
      "expiry": 500,
      "preimage": "0000000000000000000000000000000000000000000000000000000000000000"
    },
    {
      "incoming": true,
      "amount_msat": 2000000,
      "expiry": 501,
      "preimage": "0101010101010101010101010101010101010101010101010101010101010101"
    },
    {
      "incoming": false,
      "amount_msat": 2000000,
      "expiry": 502,
      "preimage": "0202020202020202020202020202020202020202020202020202020202020202"
    },
    {
      "incoming": false,
      "amount_msat": 3000000,
      "expiry": 503,
      "preimage": "0303030303030303030303030303030303030303030303030303030303030303"
    },
    {
      "incoming": true,
      "amount_msat": 4000000,
      "expiry": 504,
      "preimage": "0404040404040404040404040404040404040404040404040404040404040404"
    }
  ],
```

```
      "local_sec_nonce":
        "22a453171ba4a634da1addcf660d63d8e23fb63169a2a7206f4e23290e4cc59bb3c3011fe3c31cb4f1192a2df56c2e52350

      "remote_sec_nonce":
        "8c5bd39820b3e20d65bf72e530078bf5e8ba057c1654d0a15778a0d3225e0233eb8d7b1d3574e40e57a2cd12dc3b33193c0

      "local_nonce":
        "025f2272ea289c5fe9d52d411f5a50a6d4882341bf0ecb201d5675850a2ba0b09d025bc489cf67752134ba81f8d7f1146d7

      "remote_nonce":
        "03fd9fa808377737b105f7df362ed513e3946f2bb49dfbca5c2ce2be138ff0607502e4c73701eae82afa7a01993f6232164

      "remote_partial_sig": "3e454598e0661188da0e4cf1b806b13c627adb7ab38bab27418cc976769771c3",
      "expected_commitment_tx_hex":
        "020000000001015474cba49124ab0c4327c244bb2907059585c4af3fa5f3469701534120fec0170000000000c5fe178006

      "htlc_descs": [
        {
          "remote_partial_sig_hex":
        "7058caa9a075344e095d95736bb6a00c5b7259f439e60e7e1c6cd641a20d6a25c13d2930ebe51a69ffea78daa12b0723717

          "resolution_tx_hex":
        "020000000001018c47e10e0d210da9e50d73b1adda7a598f79ced25806dd0fa6807bb78780dbbb020000000001000000001b

        },
        {
          "remote_partial_sig_hex":
        "818132325cc01e441876615f30c3df5c27df1722db49005dff3856226a41f34c776f98a35dd74fe2f863ea23ac611e06d0a

          "resolution_tx_hex":
        "020000000001018c47e10e0d210da9e50d73b1adda7a598f79ced25806dd0fa6807bb78780dbbb030000000001000000001a

        }
      ]
    }
  ]
}
```


Acknowledgements

The commitment and HTLC scripts are heavily based off of t-bast's Lightning Is Getting Taprooty Scriptless-Scripty document.

AJ Towns proposed a more ambitious "one shot" upgrade which includes PTLCs and a modified revocation scheme which this proposal takes inspiration from.

Arik and Wilmer Paulino for suggesting the "JIT nonce" approach used in this specification when sending musig2 partial signatures.

Conclusion

Thank you for reading the Lightning Book of BOLTs. The specs will keep moving — new feature bits, new scripts, new ways to pay — and the canonical text lives in lightning/bolts, not in this compilation.

I didn't write the protocol. The credit for every message type, every script, and every hard-won MUST belongs to the original BOLT authors and the people who still review pull requests there. This book is a reading order and a set of signposts.

If you implement something, or you find a contradiction, take it upstream. That's how Lightning stays one network instead of a pile of dialects.

See you on the next commit.